

SYSTEM DESIGN SERIES

60 Days of System Design Questions

A question-a-day workbook — scenario, four options, and the reasoning behind the right call.

AUTHOR Joud Awad

CONTENTS 60 questions · 240 explanations

FORMAT Attempt the question, turn the page for the breakdown

FOLLOW [LinkedIn](#) · [YouTube](#) · [Medium](#) · [Substack](#)

Contents

Day 01 Decoupling Mobile from Backend Services

Day 03 Rate Limiting Without Boundary Bursts

Day 05 Choosing a Database Sharding Strategy

Day 07 Event Ordering in a Message Queue

Day 09 Splitting Read and Write Data Models

Day 11 Handling Webhook Retries Idempotently

Day 13 Managing a Shared Connection Pool

Day 15 Fast “Has the User Seen This?” Checks

Day 17 Backpressure on an Overwhelmed Consumer

Day 19 Read-Your-Writes with Read Replicas

Day 21 Choosing a Real-Time Streaming Transport

Day 23 Feed Fanout for Celebrity Accounts

Day 25 Queue Backpressure Under a Traffic Spike

Day 27 Keeping an LLM’s Answers Up to Date

Day 29 Coordinating a Multi-Agent Workflow

Day 31 Cutting Cross-Region Latency

Day 33 Reconstructing State with Event Sourcing

Day 35 Geospatial “Find Nearby Drivers” at Scale

Day 37 Multi-Writer Conflicts in Collaborative Editing

Day 39 Preventing Concurrent Balance Overspend

Day 41 Moving from Batch to Real-Time Streaming

Day 43 Reliable CDN Cache Invalidation on Deploy

Day 45 Scaling Full-Text Search

Day 47 Incrementally Strangling a Monolith

Day 02 Killing the N+1 Query Problem

Day 04 Preventing Duplicate Payment Charges

Day 06 Safe Distributed Locks for Cron Jobs

Day 08 Keeping Cache and Database in Sync

Day 10 Distributed Transactions Across Services

Day 12 Indexing a High-Ingest Table

Day 14 Safely Rolling Out a Risky Change

Day 16 Taming a Hot Partition Key

Day 18 Surviving a Cache Stampede

Day 20 Containing a Failing Downstream Dependency

Day 22 Reliable Messaging Across Services

Day 24 Paginating Large Result Sets Efficiently

Day 26 Write-Path Cache Consistency

Day 28 Choosing a Vector Store for Semantic Search

Day 30 Choosing a File Storage Backend

Day 32 Managing Secrets and Credentials

Day 34 Improving LLM Classification Accuracy

Day 36 Choosing HTTP/3 vs HTTP/2 at the Edge

Day 38 Fitting a Large Document into an LLM

Day 40 Offloading Heavy Work Off the Main Thread

Day 42 Designing Memory for an AI Agent

Day 44 Syncing Offline Edits Without Data Loss

Day 46 Enforcing Structured Output from an LLM

Day 48 How an Agent Chooses Which Tools to Call

Day 49 Controlling Data-Warehouse Query Costs

Day 51 Isolating Noisy Tenants in Multi-Tenant SaaS

Day 53 Zero-Downtime Schema Migrations

Day 55 Parallel Agents with Shared State

Day 57 Consistency for Payment-Confirmation Reads

Day 59 Designing Idempotency Keys for Payments

Day 50 Serving ML Models at High Throughput

Day 52 Versioning an API Without Breaking Clients

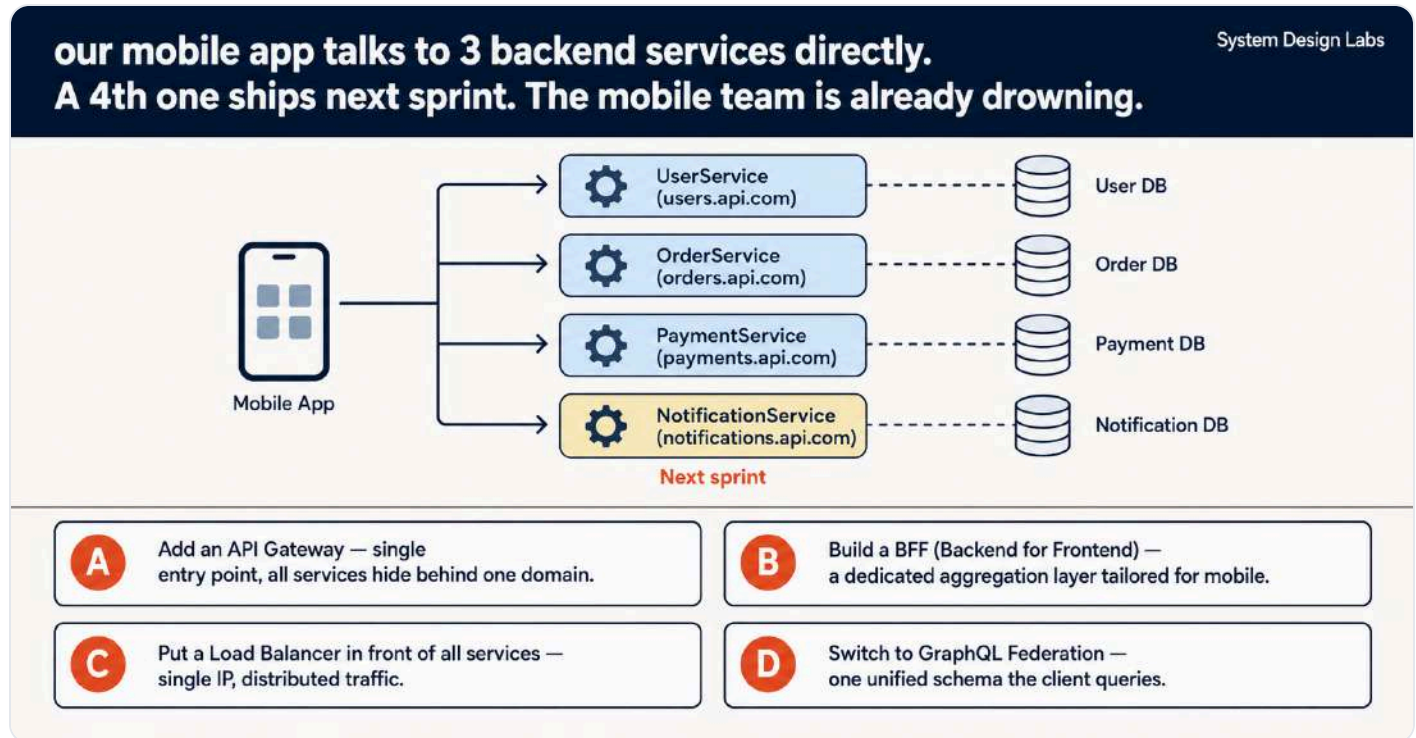
Day 54 Handling Embedding Drift in RAG

Day 56 Tracking Long-Running Async Jobs

Day 58 Observability for LLM Agent Pipelines

Day 60 Architecting a SaaS Platform from Scratch

Decoupling Mobile from Backend Services



our mobile app talks to 3 backend services directly.

A 4th one ships next sprint. The mobile team is already drowning.

Every new service means a new domain to whitelist, a new auth scheme to wire, and a new error shape to parse. You're asked to reduce coupling before NotificationService lands.

Here's the setup:

Mobile → UserService (users.api.com)

Mobile → OrderService (orders.api.com)

Mobile → PaymentService (payments.api.com)

...and NotificationService next sprint.

The client is doing routing the backend should be doing. What do you do?

A Add an API Gateway — single entry point, all services hide behind one domain.

B Build a BFF (Backend for Frontend) — a dedicated aggregation layer tailored for mobile.

C Put a Load Balancer in front of all services — single IP, distributed traffic.

D Switch to GraphQL Federation — one unified schema the client queries.

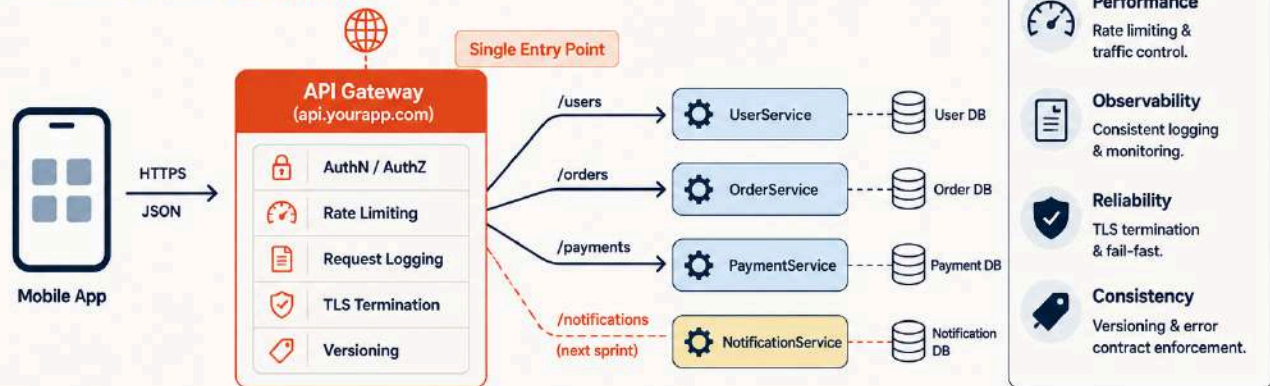
Answer 1

✓ CORRECT ANSWER

Why A wins (API Gateway)

One domain. One auth flow. One error contract. The client stops caring that UserService, OrderService, and PaymentService live on different hosts — the gateway hides them behind **api.yourapp.com/users, /orders, /payments**.

When NotificationService ships next sprint, you add one route.
The mobile app ships **zero changes**.



That's what "future-proof" actually means at the transport layer:
new services cost nothing on the client side.

Battle-tested at scale:
Netflix, Stripe, and every
mature fintech run some
form of this.

NETFLIX
stripe

Why A wins (API Gateway):

One domain. One auth flow. One error contract. The client stops caring that UserService, OrderService, and PaymentService live on different hosts — the gateway hides them behind **api.yourapp.com/users, /orders, /payments**. When NotificationService ships next sprint, you add one route. The mobile app ships **zero changes**. That's what "future-proof" actually means at the transport layer: new services cost nothing on the client side.

It also centralizes the things you don't want scattered across 4 codebases: authN/authZ, rate limiting, request logging, TLS termination, versioning. Netflix, Stripe, and every mature fintech run some form of this.

Answer 2

Why B is the trap answer (BFF):

BFF is a legitimate pattern — and it's close enough to fool a lot of senior engineers. But BFF solves a different problem: *data shape per client type*. Mobile needs a slim payload, web needs a richer one. BFF gives each client its own backend that aggregates and reshapes data.

In this scenario, the pain isn't payload shape — it's domain sprawl and coupling. Putting a BFF here trades 4 URLs for 1 URL + 1 extra service you own end-to-end. That's the wrong tradeoff.

Answer 3

Why C is wrong (Load Balancer):

A load balancer distributes traffic across **identical instances of the same service**. It doesn't route between *different* services on different domains. Putting an LB in front of UserService + OrderService doesn't make sense — it has no concept of "send /orders here and /users there." That's application-layer routing, which is exactly what an API Gateway does.

Answer 4

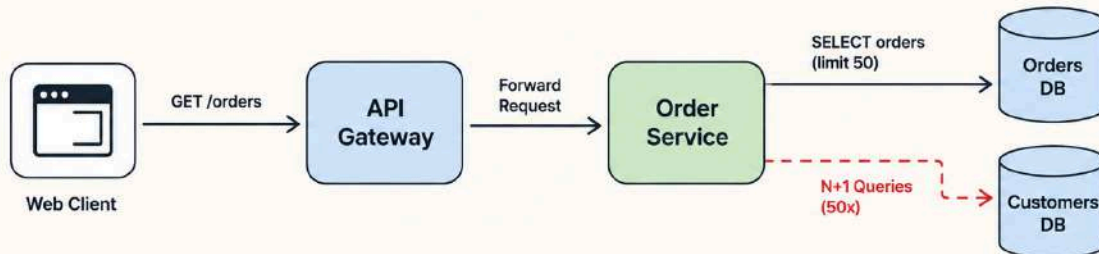
Why D is wrong here (GraphQL Federation):

GraphQL Federation solves **schema unification**, not transport-layer coupling. You'd still need a gateway in front of the federation, and you'd rewrite every backend service to expose a GraphQL subgraph. That's a multi-quarter migration to solve a problem you can fix in a sprint with a gateway.

Killing the N+1 Query Problem

**Your /orders endpoint loads 50 orders on a page.
P95 is 2.4s. The DB's fine. The app server's fine. Nothing's on fire.**

System Design Labs



A Eager-load the relation — include: { customer: true } on the Prisma query. One JOIN, done.

B Add a DataLoader in front of the customer lookup — batches the 50 IDs into one WHERE id IN (...) behind the scenes.

C Cache the customer by ID in Redis — every lookup hits cache first, DB only on miss.

D Denormalize customer_name onto the orders table — read it straight from the orders row, no join, no second query.

Your /orders endpoint loads 50 orders on a page.

P95 is 2.4s. The DB's fine. The app server's fine. Nothing's on fire.

Then you open the query log.

51 queries per request. One SELECT for the orders list. Then 50 more — one per order — to fetch the customer. The ORM is doing lazy-load on order.customer inside your map.

Classic N+1. You've seen it before. The fix is "obvious" — until the team meeting, where three engineers propose three different things and everyone thinks they're right.

Here's what's on the table:

A Eager-load the relation — include: { customer: true } on the Prisma query. One JOIN, done.

B Add a DataLoader in front of the customer lookup — batches the 50 IDs into one WHERE id IN (...) behind the scenes.

C Cache the customer by ID in Redis — every lookup hits cache first, DB only on miss.



Denormalize customer_name onto the orders table — read it straight from the orders row, no join, no second query.

Answer 1

✓ CORRECT ANSWER

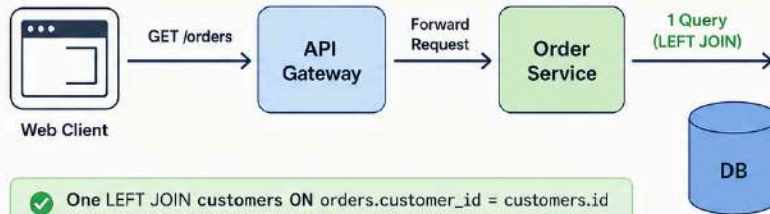


Why A wins:

Eager-load the relation — include: { customer: true }
on the Prisma query. **One JOIN, done.**

A

✓ What changes



SQL shape (conceptual)

```

SELECT o.id, o.created_at, o.total,
       c.id AS customer_id, c.name AS customer_name,
       c.email AS customer_email
FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.id
WHERE o.user_id = ?
ORDER BY o.created_at DESC
LIMIT 50 OFFSET 0;
  
```



Performance

P95 drops from 2.4s
to ~80ms.



Simplicity

No new infrastructure.
No new failure modes.



Reliability

Fewer moving parts.
Fewer things to break.



The real lesson

The N+1 problem in an ORM almost always has the same root cause: a **lazy relation** the developer didn't realize was lazy. Prisma's include, Sequelize's include, TypeORM's relations — **every ORM has this**. Reach for fancier patterns only when JOIN genuinely can't solve it.

Why A wins: One LEFT JOIN customers ON orders.customer_id = [customers.id](#) returns the whole page in a single round trip. P95 drops from 2.4s to ~80ms. No new infrastructure. No new failure modes. The N+1 problem in an ORM almost always has the same root cause: a lazy relation the developer didn't realize was lazy. Prisma's include, Sequelize's include, TypeORM's relations — every ORM has this. Reach for fancier patterns only when JOIN genuinely can't solve it.

Answer 2

Why B is the trap (DataLoader): Beautiful pattern — but earns its keep in GraphQL where the resolver graph fans out in ways a single JOIN can't express. Here you have one list endpoint with one relation. A JOIN is strictly simpler and strictly faster. You're solving a problem you don't have and paying the complexity tax forever.

Answer 3

Why C is wrong (Redis): Cache-first fixes the symptom, not the query pattern. You still issue 50 cache GETs per request, now own a cache invalidation problem every time a customer updates, and cold-hit every ID after a deploy. Cache is a scaling tool, not an N+1 tool.

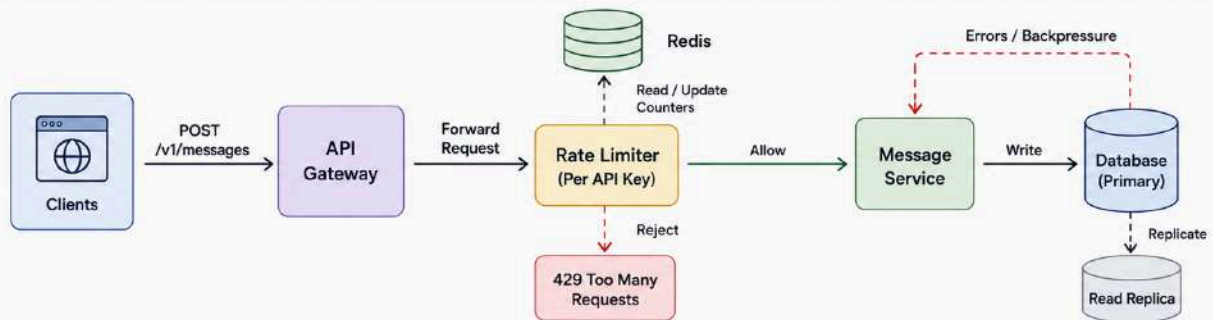
Answer 4

Why D is the career-long argument (denormalize): Legitimate at Instagram scale. Also how you end up with a customer's maiden name three years after they got married because nobody wrote the update path. Denormalization is a write-amplification decision masquerading as a read-optimization. Do it when the JOIN genuinely can't hit your latency SLO at your scale — not before.

Rate Limiting Without Boundary Bursts

You run a SaaS API. POST /v1/messages is the money endpoint. Customers keep getting 429s at minute boundaries.

System Design Labs



A

Fixed Window — bucket per (api_key, minute). Increment, check, reject. Simple, fast, one Redis key per user.

B

Sliding Window Log — store every request timestamp in Redis. Count requests in the last 60s on every call. Exact to the millisecond.

C

Token Bucket — 100 tokens per key, refill at 100/60 per second. Each request takes a token. Bursts up to 100, then shaped.

D

Leaky Bucket — queue incoming requests, dispatch at a constant 1.66/sec. Excess requests wait or get dropped.

You run a SaaS API. POST /v1/messages is the money endpoint.

Plan limit: 100 requests/minute per API key. Customers keep getting 429s at minute boundaries even though their average RPS is well under the cap.

Here's what's happening in prod:

- 12:59:58 → customer sends 90 requests (batched job kicked off)
- 13:00:02 → same customer sends 90 more (next tick)
- Your counter resets at 13:00:00
- Both bursts pass. 180 requests in 4 seconds. Your downstream database starts crying.

Now Support is on your back about 429s, and SRE is on your back about traffic spikes. You need to replace the limiter this week.

FOUR OPTIONS ON THE TABLE:

A

Fixed Window — bucket per (api_key, minute). Increment, check, reject. Simple, fast, one Redis key per user.

B Sliding Window Log — store every request timestamp in Redis. Count requests in the last 60s on every call. Exact to the millisecond.

C Token Bucket — 100 tokens per key, refill at 100/60 per second. Each request takes a token. Bursts up to 100, then shaped.

D Leaky Bucket — queue incoming requests, dispatch at a constant 1.66/sec. Excess requests wait or get dropped.

Answer 1 ✓ CORRECT ANSWER



Answer: C — Token Bucket ✓

System Design Labs

Token Bucket allows bursts up to the limit, then enforces a steady rate.

No boundary resets. Smooth traffic. Happy customers. Happy DB.



Why C wins (Token Bucket):

- ✓ Token Bucket is what AWS API Gateway, Stripe, GitHub, and basically every mature public API runs. Each API key gets a bucket of 100 tokens, refilling at ~1.66 tokens/sec. Every request takes a token. Empty bucket = 429.
- ✓ This fixes the boundary-burst bug exactly: no window resets. The 12:59:58 burst drains the bucket, and 13:00:02 finds an almost-empty bucket — not a shiny new 100-request allowance.
- ✓ O(1) memory per key, O(1) check, trivially distributed.

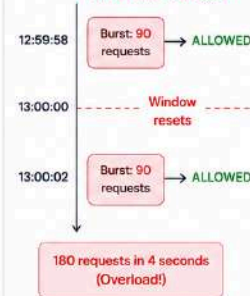
How Token Bucket works

Refill rate: $100/60 \approx 1.66$ tokens/sec

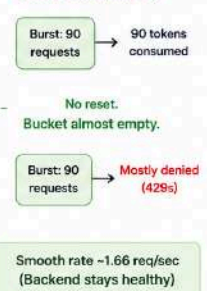
Each request takes 1 token.
No token available? Return 429.

Why it fixes the boundary-burst

Fixed Window (buggy)



Token Bucket (correct)



Why the other three trip up good engineers



Fixed Window

- ✗ Resets at boundaries → bursts
- ✗ Allows 2x limit in a short time
- ✗ Unfair and spiky traffic

Good for simple, non-critical limits.
Not for money endpoints.



Sliding Window Log

- ✓ Accurate to the millisecond
- ✗ Stores every request timestamp
- ✗ High memory & CPU at scale
- ✗ Expensive in Redis

Overkill cost for little extra benefit.



Token Bucket

- ✓ Handles bursts up to limit
- ✓ Smooths traffic over time
- ✓ O(1) memory, O(1) check
- ✓ Easy to distribute
- ✓ Industry standard

Right balance of fairness, performance, and simplicity.



Leaky Bucket

- ✓ Smooth output rate
- ✗ Adds queuing delay
- ✗ Increases tail latency
- ✗ Complex under load

Good for shaping internal traffic, not for API rate limits.

Answer: C — Token Bucket ✓

Here's why, and why the other three trip up good engineers:

Why C wins (Token Bucket):

Token Bucket is what AWS API Gateway, Stripe, GitHub, and basically every mature public API runs. Each API key gets a bucket of 100 tokens, refilling at ~1.66 tokens/sec. Every request takes a token. Empty bucket = 429.

This fixes the boundary-burst bug exactly: no window resets. The 12:59:58 burst drains the bucket, and 13:00:02 finds an almost-empty bucket — not a shiny new 100-request allowance. O(1) memory per key, O(1) check, trivially distributed.

Answer 2

Why B is the trap (Sliding Window Log):

Mathematically perfect — but stores one Redis entry *per request per key*. At 100k active keys you're looking at real memory pressure and p99 latency spikes. Smart engineers pick it without thinking about the memory cost. That's the tell.

Answer 3

Why A is wrong (Fixed Window):

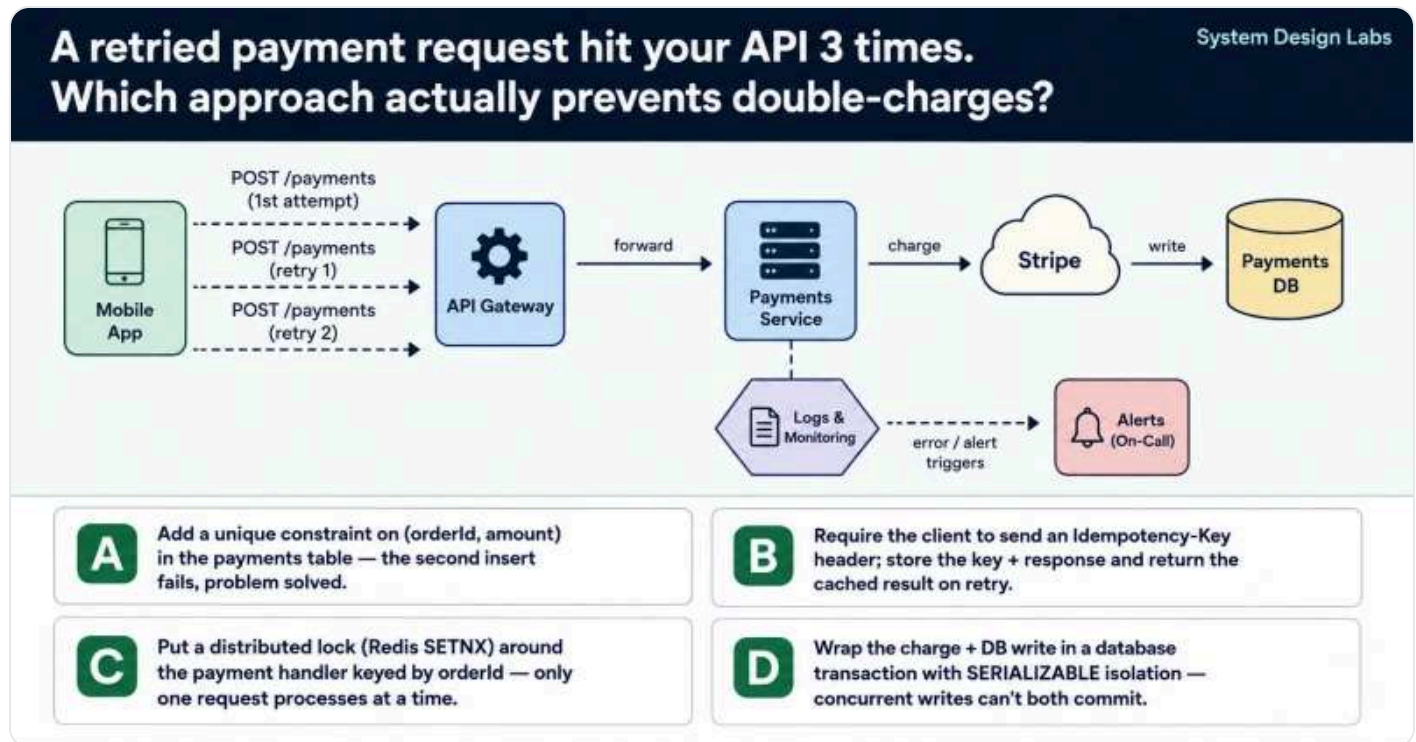
Literally the bug in the post. Counter resets at the minute boundary → customers can burst 2x their limit in a 2-second window. Still the most common production mistake.

Answer 4

Why D is wrong here (Leaky Bucket):

Solves a different problem — shapes *output* by queuing requests at a constant dispatch rate. Great for protecting a fragile downstream. Wrong for a customer-facing API because it adds latency to legitimate bursts. Token Bucket rejects excess; Leaky Bucket delays it. For a public API, reject > delay.

Preventing Duplicate Payment Charges



A user taps "Pay \$499" on checkout.

The spinner hangs. They tap it again. And once more.

Three requests hit your Payments API. Two of them actually succeed. The customer is charged \$1,497 and opens a support ticket before your on-call even sees the alert.

Here's the setup:

Mobile → POST /payments { orderId: "ord_8821", amount: 499 }

Network stalls on the first request → client retries twice

Your service receives 3 near-identical POSTs within 4 seconds

Stripe gets charged 3 times. Your DB writes 3 payment rows.

You have one sprint to make sure a retried request never double-charges again. What do you do?

- A** Add a unique constraint on (orderId, amount) in the payments table — the second insert fails, problem solved.

- B** Require the client to send an Idempotency-Key header; store the key + response and return the cached result on retry.
- C** Put a distributed lock (Redis SETNX) around the payment handler keyed by orderId — only one request processes at a time.
- D** Wrap the charge + DB write in a database transaction with SERIALIZABLE isolation — concurrent writes can't both commit.

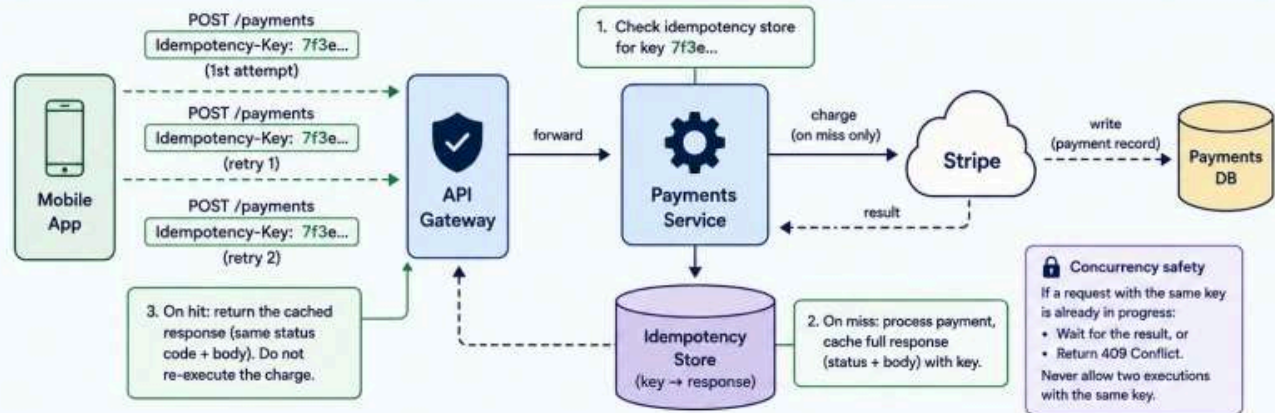
Answer 1

✓ CORRECT ANSWER

Answer: B — Idempotency-Key header ✓

System Design Labs

The server does the work once, stores {key → response}, and returns the same response on retries.



Answer: B — Idempotency-Key header ✓

Here's why, and why the other three are the exact mistakes I've seen in real post-mortems:

Why B wins (Idempotency-Key):

The client generates a UUID *before* the first request and attaches it as Idempotency-Key: 7f3e... on every retry of that same logical operation. Your server does the work once, stores {key → response} in a durable store (Postgres, DynamoDB, Redis with persistence — not plain Redis), and on any retry with the same key, it returns the *original* response without re-executing the charge.

Critical detail most people miss: you cache the **full response** (status code + body), not just "I saw this key." If retry #2 arrives while retry #1 is still processing, you make it wait or return a 409 — never let two executions race with the same key.

This is exactly what Stripe (Idempotency-Key), PayPal (PayPal-Request-Id), Shopify, and AWS (ClientRequestToken on half their APIs) all ship. It's the only approach that handles the full failure matrix: network retries, client-triggered retries, load balancer retries, and "the request succeeded but the response got lost" — which is the nastiest case and the one the other three options silently fail on.

Answer 2

Why A is the trap answer (unique constraint on orderId):

This is the answer I've seen in 4 different production codebases, and it's wrong for a subtle reason.

A unique constraint stops the *second insert* from committing — good. But what about the *first charge to Stripe*? In most implementations, you call Stripe's API first, then insert the payment row. So request #1 charges Stripe, DB insert succeeds. Request #2 charges Stripe *again*, then the DB insert fails with a constraint violation. The customer is now charged twice. Your DB looks clean. Your logs say "duplicate order prevented."

Even if you flip the order (insert first, charge second), you've just moved the race condition — now retry #2 fails at the DB, but if retry #1 crashes between insert and charge, you have a payment row with no actual charge. Worse than double-charging. Silent money loss.

A unique constraint is a great *defense-in-depth* layer. It is not the primary mechanism.

Answer 3

Why C is wrong (distributed lock):

A Redis lock on orderId serializes the 3 retries so only one runs at a time. Sounds clean. But it only solves the *concurrent* case. It doesn't help with sequential retries that arrive after the lock has already been released and the response was lost in transit.

Request #1 acquires lock, charges Stripe, releases lock, returns 200 — but the 200 never reaches the client because the connection died. Client retries 30 seconds later. The lock is long gone. You charge again.

Distributed locks are for preventing concurrent execution. Idempotency is for preventing *repeated* execution across time. Different problems.

Also: distributed locks are famously hard to get right. Martin Kleppmann's "How to do distributed locking" post exists for a reason. Don't reach for one unless you actually need the concurrency guarantee.

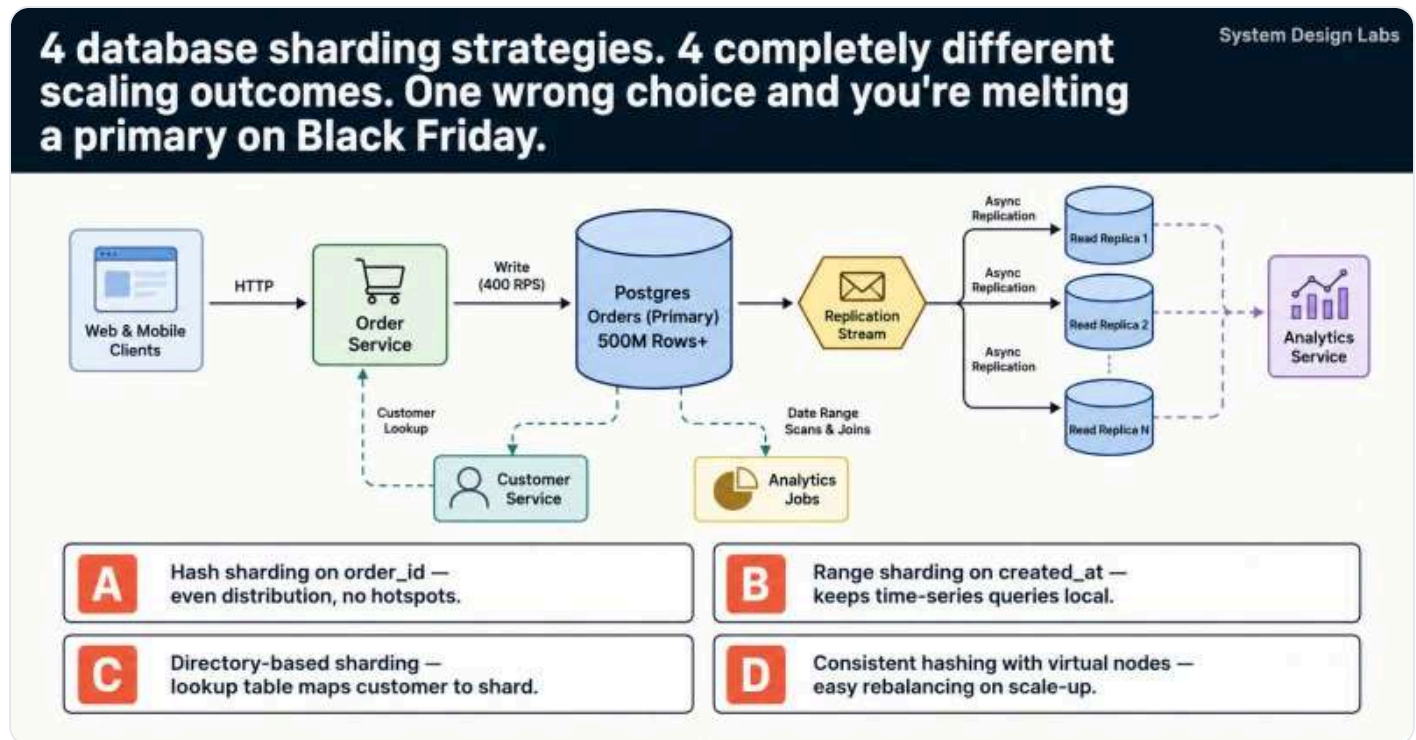
Answer 4

Why D is wrong (SERIALIZABLE transaction):

SERIALIZABLE isolation is about protecting the *database* from concurrent-write anomalies. It has nothing to say about the *external* side effect — the charge to Stripe. Your transaction can roll back and your DB can be pristine, but the HTTP POST to Stripe already went out, and that money is gone.

This is the most common conceptual bug in the "make payments safe" category: treating the database as the boundary of correctness when the real boundary is the external side effect. The moment you call Stripe, the universe has changed. Your DB transaction can't un-change it.

Choosing a Database Sharding Strategy



4 database sharding strategies. 4 completely different scaling outcomes. One wrong choice and you're melting a primary on Black Friday.

Your Postgres orders table just crossed 500M rows.

Range scans that used to take 40ms are creeping past 800ms.

Vertical scaling is dead. You need to shard.

Here is the workload:

- orders = 500M rows, growing 3M/week
- 80% reads = "customer X's last 30 days of orders"
- 15% reads = analytics joins on date ranges
- 5% writes = new orders (steady 400 RPS, 2x on sale days)

Which strategy do you pick?

A Hash sharding on `order_id` — even distribution, no hotspots.

B Range sharding on `created_at` — keeps time-series queries local.

C Directory-based sharding — lookup table maps customer to shard.

D Consistent hashing with virtual nodes — easy rebalancing on scale-up.

Answer 1

✓ CORRECT ANSWER

Answer: C — Directory-based sharding ✓

Here is why the other three trick smart engineers:

Why C wins (Directory-based):

80% of reads are "customer X's orders." You want every order for a customer on exactly one shard. A customer-scoped read hits one node, no scatter-gather fan-out.

You keep a `customer_shard_map` table (cached in Redis). If shard 3 gets hot, you migrate specific heavy customers (whales) without rehashing anything. Targeted rebalancing is the superpower. Figma, Notion, and Slack do this.

Answer 2

Why A is the trap (Hash on `order_id`):

The classic tutorial answer: "Hash the primary key!" It wrecks you here. A single customer's 200 orders scatter across all shards. Every read becomes a 4-shard fan-out gated by your slowest node. P99 latency goes up.

Answer 3

Why B is wrong (Range on `created_at`):

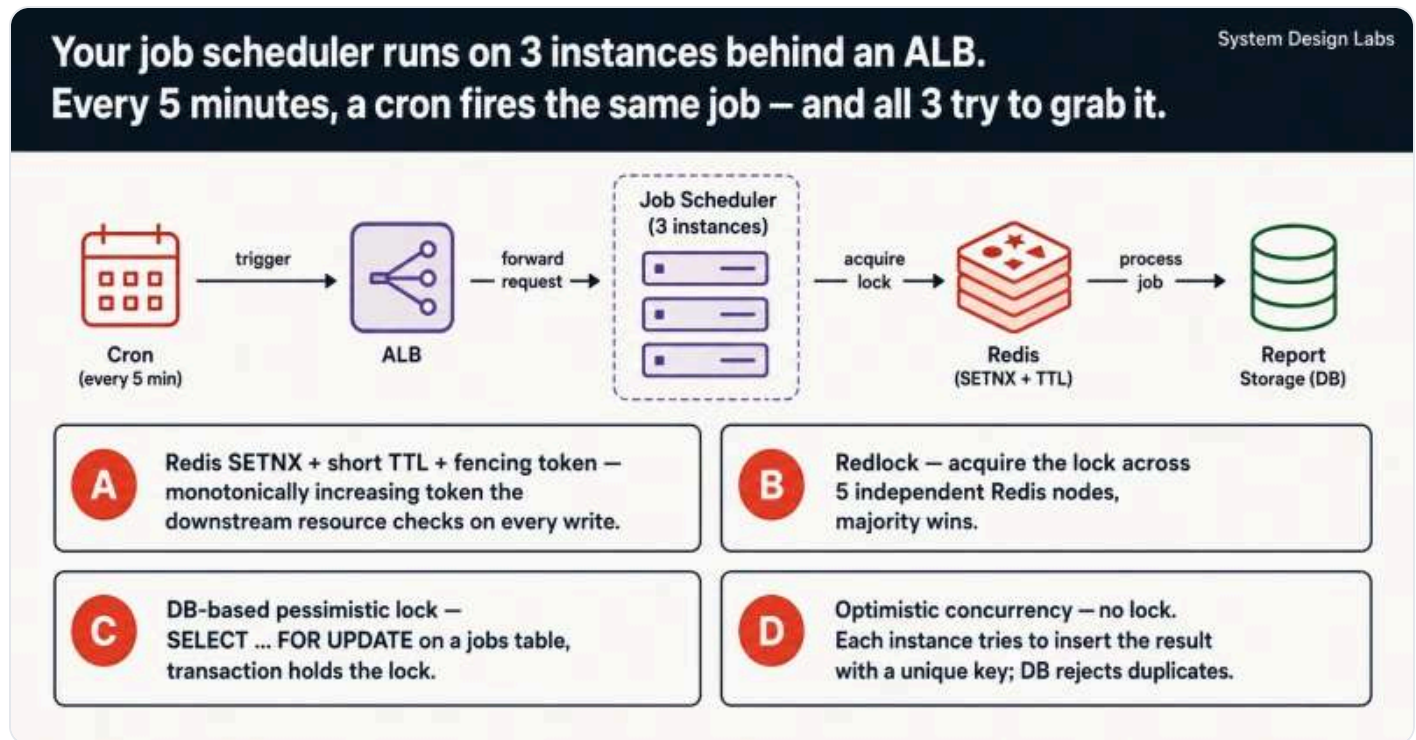
Creates a hot shard by design. Every new order writes to the same shard. Your newest shard absorbs 100% of writes and 70% of reads. On Black Friday, this shard melts while others sit idle.

Answer 4

Why D is wrong (Consistent hashing):

Great for uniform data, but you can't migrate specific heavy tenants. If four whales land on the same shard, you are stuck.

Safe Distributed Locks for Cron Jobs



Your job scheduler runs on 3 instances behind an ALB.

Every 5 minutes, a cron fires the same "generate-daily-report" job — and all 3 instances try to grab it.

You reach for the classic fix: a Redis SETNX lock.

SET job:daily-report locked NX EX 300 → only one instance wins. Done.

Works perfectly for a month. Until 2am on a Saturday.

Instance B wins the lock. Starts the job. 60 seconds in, the pod gets OOM-killed. The lock is still there. TTL says 300 seconds. The report runs once every 5 minutes.

Nothing processes that report for 4 more minutes. Then instance A grabs it — but instance B's pod restarted and it's ALSO trying to run it now. Both instances processing the same job.

Worse: a slow GC pause on instance A makes it hold the lock past TTL. Redis quietly hands the lock to instance C. Now A thinks it owns the lock. C also thinks it owns the lock. Both write to the same output.

What's the right pattern here?

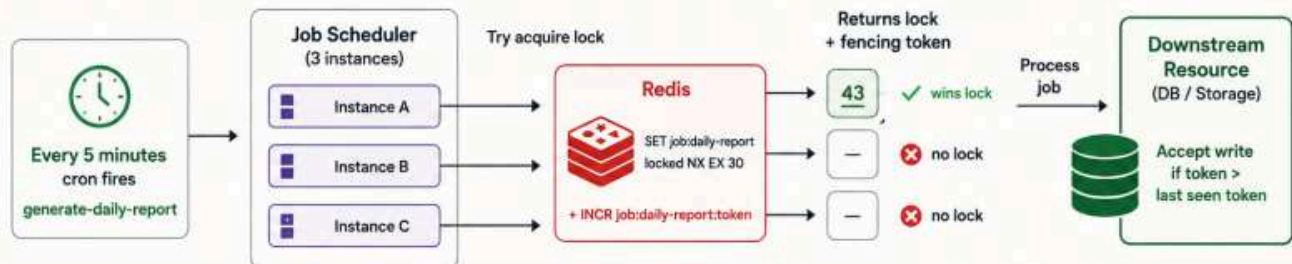
- A** Redis SETNX + short TTL + fencing token — monotonically increasing token the downstream resource checks on every write.
- B** Redlock — acquire the lock across 5 independent Redis nodes, majority wins.
- C** DB-based pessimistic lock — SELECT ... FOR UPDATE on a jobs table, transaction holds the lock.
- D** Optimistic concurrency — no lock. Each instance tries to insert the result with a unique key; DB rejects duplicates.

Answer 1

✓ CORRECT ANSWER

A — SETNX + short TTL + fencing token ✓

System Design Labs

Monotonically increasing token that the downstream resource checks on every write.**Why this survives failures**

- 1 If the lock holder dies or is paused, the TTL expires.
- 2 Another instance acquires the lock and gets a higher token (e.g., 44).
- 3 If the old instance wakes up and tries to write with token 43, the resource rejects it. Only the highest token wins.

Key guarantee

The downstream resource is the source of truth, not the lock service.

Why the others fail in the scenario**B****Redlock**

✗

- Still time-based and unsafe under process pauses and partitions.
- Martin Kleppmann showed how it can grant the lock to multiple clients.

C**DB-based pessimistic lock**

✗

- If the process dies, the DB lock is released — good.
- But a long GC pause can still overlap work, causing duplicate side effects. No fencing = resource can't tell.

D**Optimistic concurrency**

✗

- Prevents duplicate final results, but all instances still do the work.
- Wastes resources and can overwhelm downstream systems.



Only A (SETNX + short TTL + fencing token) provides correctness under pauses, failures, and partitions.

The fencing token makes the downstream resource the final authority.

Answer: A — SETNX + short TTL + fencing token ✓

Here's why, and why the other three trick smart engineers:

Why A wins (SETNX + TTL + fencing token):

A lock alone can't protect a distributed resource. You need to assume the lock holder might be dead, paused, or partitioned — and the lock might get reassigned while they still think they own it. This is exactly the scenario above: GC pause, TTL expires, Redis gives the lock to someone else, original owner wakes up and keeps writing.

The fencing token fixes it. Every time a lock is acquired, Redis (or the lock service) hands out a monotonically increasing number — 42, 43, 44. The downstream resource (the DB, the storage system, the output queue) remembers the highest token it's seen and rejects any write with a lower token.

So when instance A wakes up from its GC pause holding token 42, and instance C is already writing with token 43, A's write gets rejected by the DB itself. The resource is the source of truth — not the lock service.

This is what Google's Chubby does. It's what Martin Kleppmann recommends in "How to do distributed locking." It's the only pattern on this list that's actually safe against process pauses.

Answer 2

Why B is the trap answer (Redlock):

Redlock is the pattern most senior engineers reach for. It looks rigorous. Martin Kleppmann wrote a now-famous post (2016) showing Redlock is unsafe in the exact scenario above: any process can pause long enough to lose its lock without noticing.

More importantly — Redlock without a fencing token has the same fundamental flaw as SETNX. If your resource can't verify a token on write, Redlock doesn't save you. You're maintaining 5 Redis clusters for a problem a fencing token solves on 1.

Answer 3

Why C is wrong (DB pessimistic lock):

SELECT ... FOR UPDATE inside a transaction works — for tiny critical sections. The lock auto-releases when the transaction ends or the connection dies.

But the lock lives inside a single DB transaction. If your job is "generate a daily report" that takes 60 seconds of S3 writes and external API calls, you're holding a row lock on your primary DB for 60 seconds per job. Connection pool starved. Every other query blocked behind it.

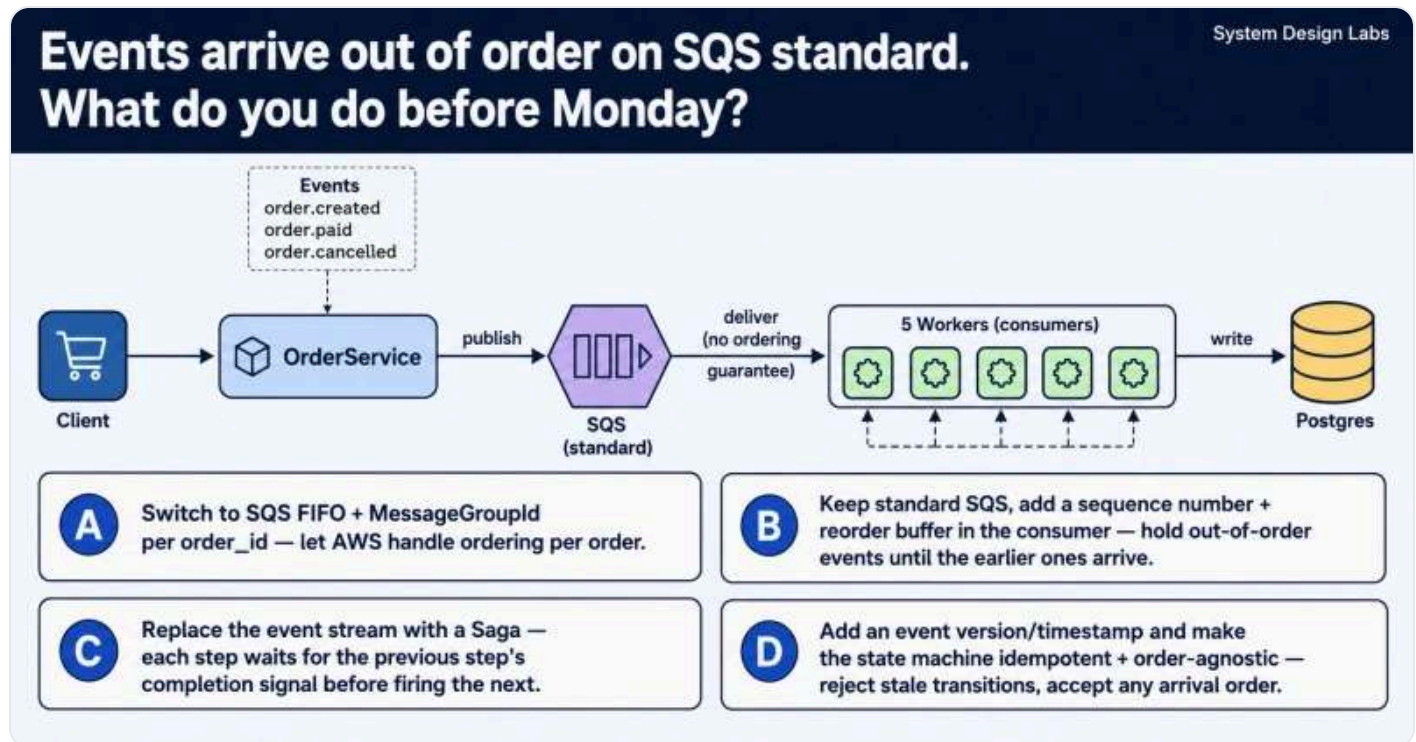
Answer 4

Why D is wrong here (optimistic concurrency):

Optimistic concurrency is beautiful when it fits — no lock service, no lock holder, no stale-lock problem. Each instance tries the operation; the DB unique constraint rejects losers.

It breaks when the work isn't idempotent or isn't a single write. Three instances racing to do all of that means triple the S3 writes, triple the Slack pings, triple the compute cost.

Event Ordering in a Message Queue



Your order service publishes 3 events per order: created, paid, cancelled.

Standard SQS queue, 5 consumers, ~2K orders/minute at peak.

Last night, prod alerted: 47 orders stuck in "cancelled" state with no "created" record. Customers got refunded for orders that were never marked as placed. Finance is not happy.

You dig in. The order was placed, paid, and cancelled within 400ms. Three messages hit SQS in order. Three consumers picked them up in parallel. "cancelled" got processed first. State machine rejected "created" as invalid. Data is now wrong.

Here's the setup:

OrderService → SQS (standard) → 5 workers → Postgres

Events: order.created → order.paid → order.cancelled

Problem: workers process in parallel, SQS standard doesn't guarantee order, downstream state machine breaks when events arrive out of sequence.

You have until Monday. What do you do?

- A** Switch to SQS FIFO + MessageGroupId per order_id — let AWS handle ordering per order.
- B** Keep standard SQS, add a sequence number + reorder buffer in the consumer — hold out-of-order events until the earlier ones arrive.
- C** Replace the event stream with a Saga — each step waits for the previous step's completion signal before firing the next.
- D** Add an event version/timestamp and make the state machine idempotent + order-agnostic — reject stale transitions, accept any arrival order.

Answer 1

✓ CORRECT ANSWER

Answer: A — SQS FIFO + MessageGroupId per order_id

Per-order ordering guaranteed. High throughput. Minimal changes. Ship it Monday.



✓ Answer: A — SQS FIFO + MessageGroupId per order_id

Why A wins:

You don't need global ordering — you need *per-order* ordering. Set `MessageGroupId = order_id`. FIFO guarantees messages with the same group ID are delivered in exact send order, with only one consumer processing that group at a time. Your 2K orders/min still fan out across 5 workers — throughput is unchanged. Order #12345's `created` → `paid` → `cancelled` always lands in sequence. Cost: a few ms latency. Ship it Monday.

Answer 2

Why B is the trap (reorder buffer):

Fools senior engineers on the whiteboard. You'd end up building: in-memory buffer per `order_id` + TTL logic + dead-letter path for gaps + crash recovery + stuck buffer monitoring. You just rebuilt FIFO in app code. With more bugs. And you own every edge case. B is valid only when you *can't* use FIFO (e.g. Kafka with a taken partition key). That's not this situation.

Answer 3

Why C is wrong (Saga):

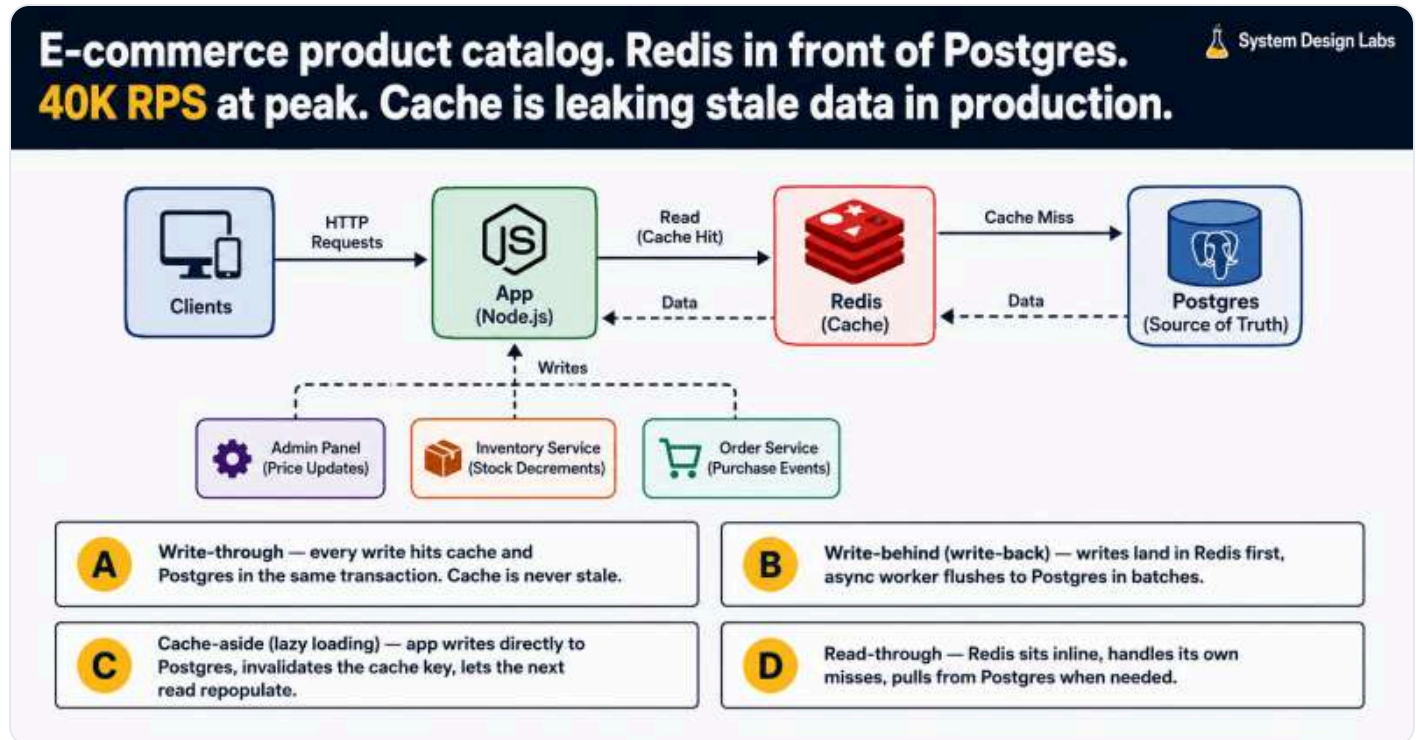
Saga is for long-running distributed transactions with compensating actions (book flight → charge card → reserve hotel). Your problem is a *state stream for one entity* — not a multi-service transaction. Forcing it into a Saga turns async events into sync RPC, increases coupling, and still doesn't fix ordering.

Answer 4

Why D is wrong (event versioning):

Partially works — but you'd *drop* legitimate events. "cancelled" arrives first, moves state forward, then "created" gets rejected. DB says order is cancelled with no creation record. To make D work correctly you need full event sourcing (6-month rewrite, not a Monday fix).

Keeping Cache and Database in Sync



E-commerce product catalog. Redis in front of Postgres. 40K RPS at peak.

In staging, cache hit ratio sits at 94%. Response times under 20ms. Everything looks clean.

Ship to prod. Within 48 hours, support tickets start rolling in. Customers seeing wrong prices. Old stock counts. Products showing "available" that shipped out three hours ago.

Here's the setup:

App (Node.js) → Redis (cache) → Postgres (source of truth)

Writes come from: admin panel (price updates), inventory service (stock decrements), order service (purchase events).

Reads come from: product pages, search results, checkout flow.

The cache is leaking stale data in production. Your team is debating the invalidation strategy. What do you ship?

- A** Write-through — every write hits cache and Postgres in the same transaction. Cache is never stale.

B Write-behind (write-back) — writes land in Redis first, async worker flushes to Postgres in batches.

C Cache-aside (lazy loading) — app writes directly to Postgres, invalidates the cache key, lets the next read repopulate.

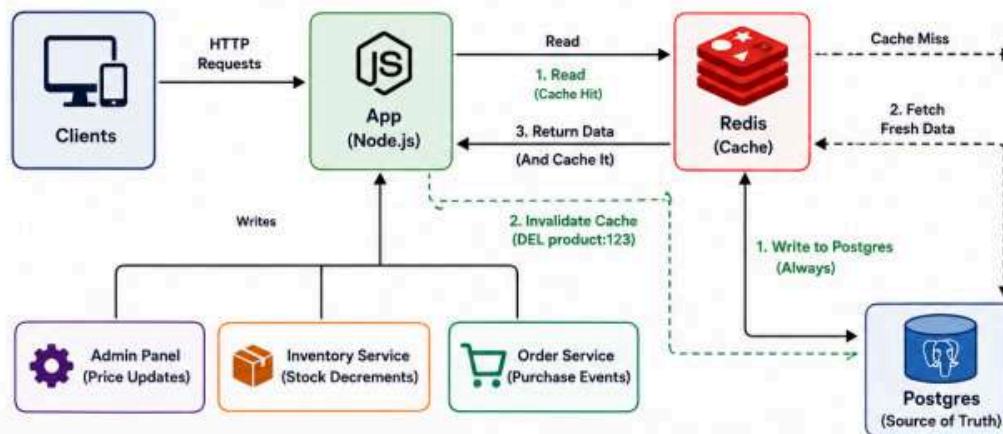
D Read-through — Redis sits inline, handles its own misses, pulls from Postgres when needed.

Answer 1

✓ CORRECT ANSWER

ANSWER: C — Cache-aside (lazy loading)

App writes to Postgres → invalidates Redis → next read repopulates.

**Why C Wins**

- ✓ Multiple writers update Postgres independently.
- ✓ Each writer invalidates Redis. Next read repopulates fresh data.
- ✓ Postgres remains the single source of truth.
- ✓ Redis is disposable. If it dies, the app still works (correct, just slower).
- ✓ Battle-tested by Shopify, Etsy, and AWS reference architectures.

✓ Answer: C — Cache-aside

Why C wins:

Multiple write paths (admin panel, inventory service, order service) all hit Postgres independently. Cache-aside fits perfectly: each writer updates Postgres → invalidates the cache key (DEL product:123) → next read misses → fetches fresh data → repopulates Redis. Postgres stays the uncontested source of truth. Redis is just a disposable read accelerator. If Redis dies, the app still works — slower, but correct. This is what Shopify, Etsy, and most AWS reference architectures run in prod.

Answer 2**Why A is the trap (Write-through):**

"We write to both atomically" — except there's no distributed transaction between Redis and Postgres. You write to Postgres, then write to Redis. If the Redis write fails (network blip, timeout), you have consistent Postgres + stale Redis. The exact bug you were trying to fix. Worse, write-through couples every service to Redis — now your inventory service and order service both need Redis credentials. One Redis outage takes down your entire write path. Works great for single-service/single-writer systems. Falls apart with multiple writers.

Answer 3

Why B is wrong (Write-behind):

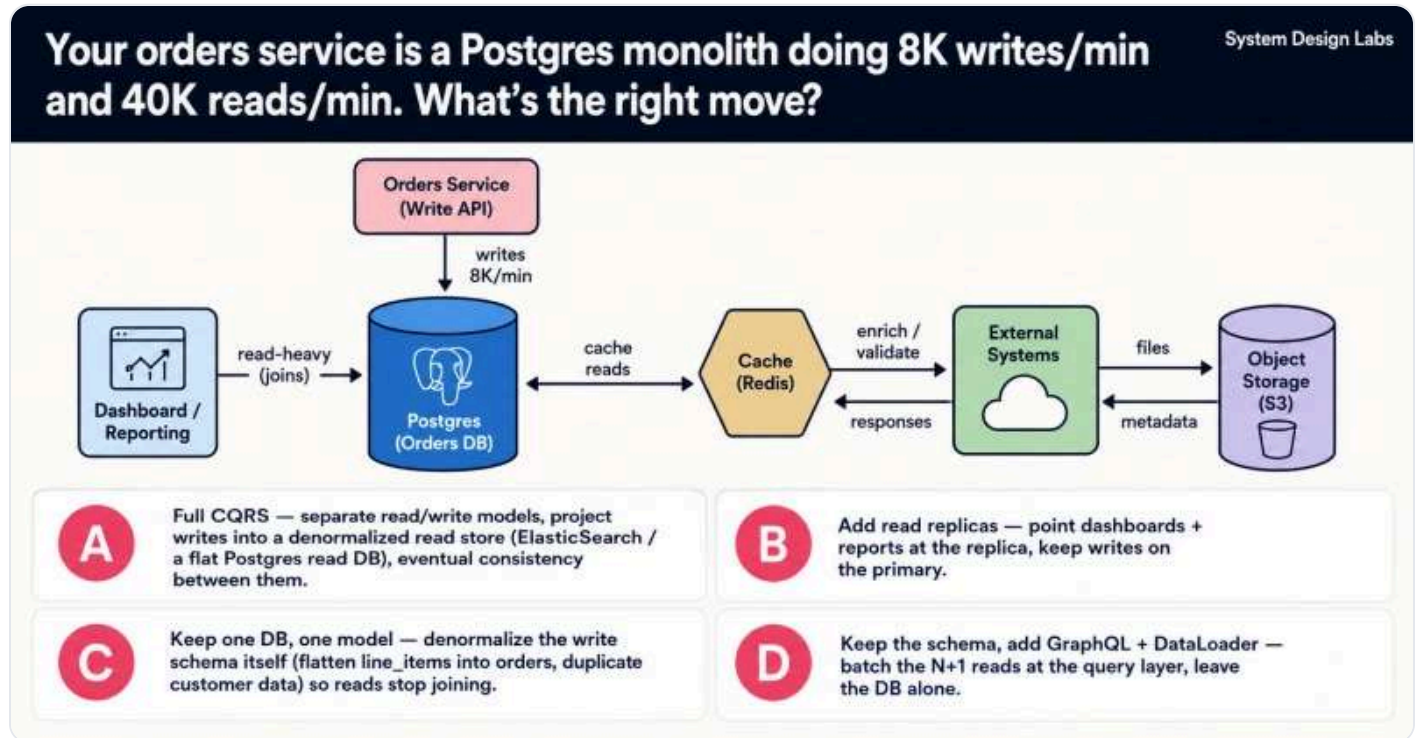
Redis becomes your source of truth. Redis loses data during failovers, OOM kills, AOF lag. "We might lose the last few price updates" is not defensible in a postmortem when real money is involved. Write-behind belongs in metrics pipelines and analytics counters — not e-commerce write paths.

Answer 4

Why D is wrong (Read-through):

Only solves the read side. Your writes are still going through 3 different services with nothing telling the cache. Read-through doesn't fix stale data from multiple writers — you'd still need invalidation on top, which is just cache-aside with extra steps.

Splitting Read and Write Data Models



Your orders service is a Postgres monolith doing 8K writes/min and 40K reads/min.

Same tables. Same schema. Everything's grinding.

The write side needs a normalized schema — orders, line_items, payments, shipments, addresses. Clean FKs, ACID, no duplicated data. The read side wants the opposite — the dashboard joins 7 tables to render one order card, and the reporting queries are lighting up the CPU at 85% every morning at 9am.

You've already tuned indexes. You've already added caching. The fundamental problem isn't hardware — the read model and the write model want **different shapes of the same data**, and you've been pretending they're the same thing for two years.

Here's the decision on the table:

- A** Full CQRS — separate read/write models, project writes into a denormalized read store (ElasticSearch / a flat Postgres read DB), eventual consistency between them.
- B** Add read replicas — point dashboards + reports at the replica, keep writes on the primary.

C Keep one DB, one model — denormalize the write schema itself (flatten line_items into orders, duplicate customer data) so reads stop joining.

D Keep the schema, add GraphQL + DataLoader — batch the N+1 reads at the query layer, leave the DB alone.

Answer 1

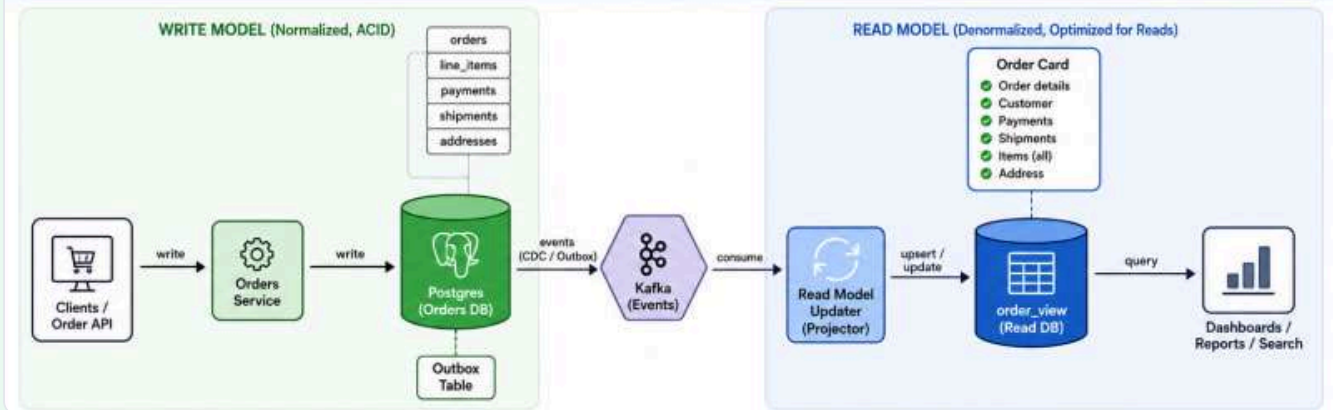
✓ CORRECT ANSWER

A

Full CQRS — Separate Read/Write Models

System Design Labs

Project writes into a denormalized read store. Eventual consistency.



✓ Answer: A — Full CQRS

Why A wins:

The pain isn't "reads are slow" — it's that the write side wants a normalized schema and the read side wants a denormalized one, and you're forcing them to share. No amount of hardware fixes a schema shape mismatch. Write side stays clean: orders, line_items, payments, all normalized, ACID, FK-enforced. Read side is a projection: a flat order_view table (or Elasticsearch index) where every order card renders from one row, zero joins. Projector keeps them in sync (outbox → Kafka → read-model updater, or CDC). You eat eventual consistency — the dashboard might lag by a few hundred ms. For 95% of reads (dashboards, reports, search), that's invisible. For the 5% that needs strong consistency (user's own order confirmation), read from the write side directly.

Answer 2

Why B is the trap (Read replicas):

B fixes read *load*. Not read *shape*. The 7-table join is still a 7-table join — it just runs on different hardware. You've moved the CPU bill, not removed it. Latency drops for 6 months while traffic grows, then you're back where you started with 3 replicas all running the same expensive joins. B is right when the joins are structurally fine but the primary is overwhelmed. Wrong when the joins themselves are the problem.

Answer 3

Why C is wrong (Denormalize the write model):

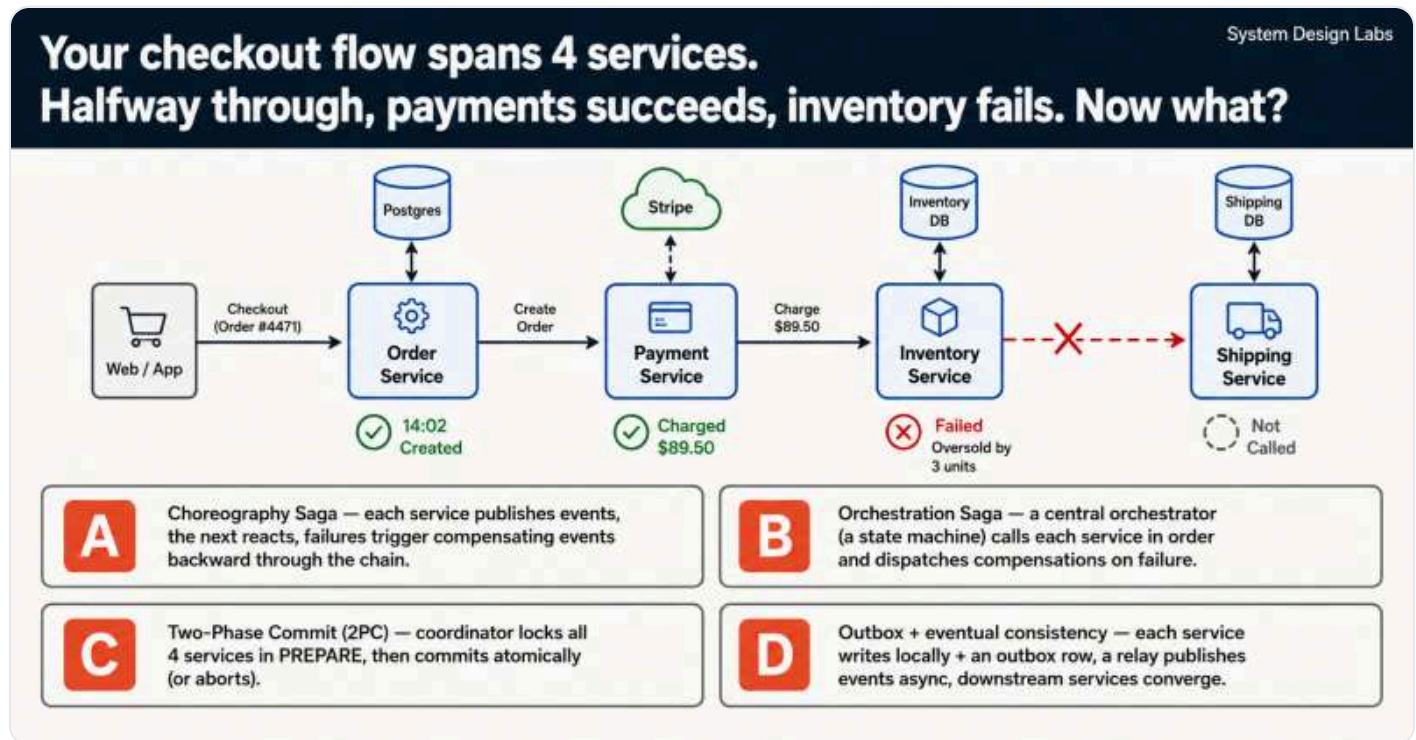
Every line-item change now rewrites the order row. Every address update cascades through every order ever placed. Write amplification compounds at 8K writes/min. Schema ossifies — adding a field to line_items means migrating millions of denormalized rows. Classic "avoid the architecture conversation" move. Buys a quarter, mortgages two.

Answer 4

Why D is wrong (GraphQL + DataLoader):

DataLoader batches API calls, not joins. The 7-table join happens in the database, not in the application. 3 months of GraphQL migration later, reads are exactly as slow as before. Right answer when clients are the problem. Wrong answer when the DB schema is.

Distributed Transactions Across Services



Order #4471 just hit your system at 14:02. The flow:

1. OrderService creates the order (Postgres) ✓
2. PaymentService charges \$89.50 to Stripe ✓
3. InventoryService tries to reserve the SKU → fails (oversold by 3 units)
4. ShippingService never gets called

You're now holding the customer's money, no inventory to ship, and an order stuck in PENDING. In a monolith you'd wrap all 4 in a transaction and ROLLBACK. You can't do that across 4 services with 4 databases.

What do you reach for?

- A** Choreography Saga — each service publishes events, the next reacts, failures trigger compensating events backward through the chain.
- B** Orchestration Saga — a central orchestrator (a state machine) calls each service in order and dispatches compensations on failure.

C Two-Phase Commit (2PC) — coordinator locks all 4 services in PREPARE, then commits atomically (or aborts).

D Outbox + eventual consistency — each service writes locally + an outbox row, a relay publishes events async, downstream services converge.

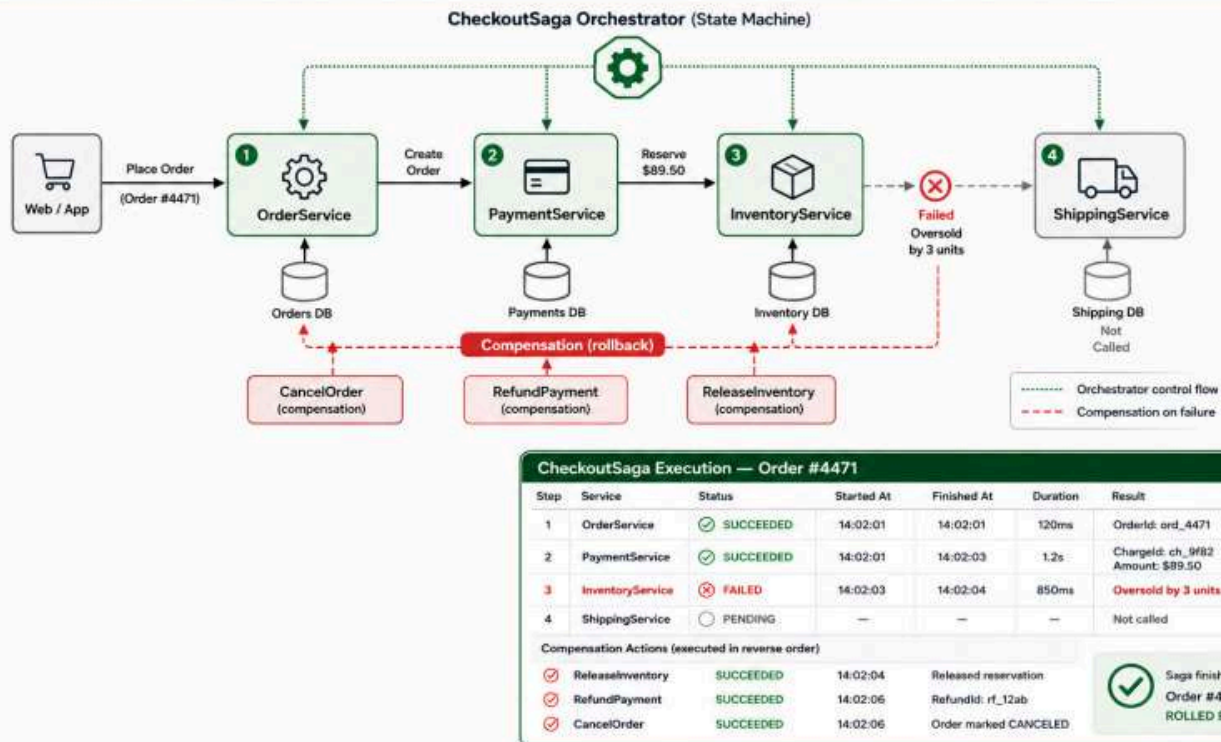
Answer 1

✓ CORRECT ANSWER

ANSWER: B Orchestration Saga

System Design Labs

Central orchestrator calls each service in order and dispatches compensations on failure.



✓ Answer: B — Orchestration Saga

Why B wins:

4-step checkout with strict ordering + per-step compensations = exactly what an orchestrator is built for. Stand up a CheckoutSaga state machine (Temporal, AWS Step Functions, Camunda, or hand-rolled). It calls each service in order: OrderService → PaymentService → InventoryService → ShippingService. Each step has a registered compensation: CancelOrder, RefundPayment, ReleaseInventory. When Inventory fails, the orchestrator walks backward and refunds automatically. State is durable, observable, replayable. At 3am you open the orchestrator UI and see exactly which step failed with full inputs/outputs.

Answer 2

Why A is the trap (Choreography):

Elegant on paper. Falls apart at 4+ steps with branching compensations. The business logic for "what happens when inventory fails after payment succeeded" gets smeared across 3 services that all need to know each other's failure modes. Nobody owns the flow. No single place to see "what is the state of order #4471?" At 3am you're piecing it together from events across 4 Kafka topics. Choreography is right for loose, fire-and-forget side effects — not sequential checkouts with specific compensations.

Answer 3

Why C is wrong (2PC):

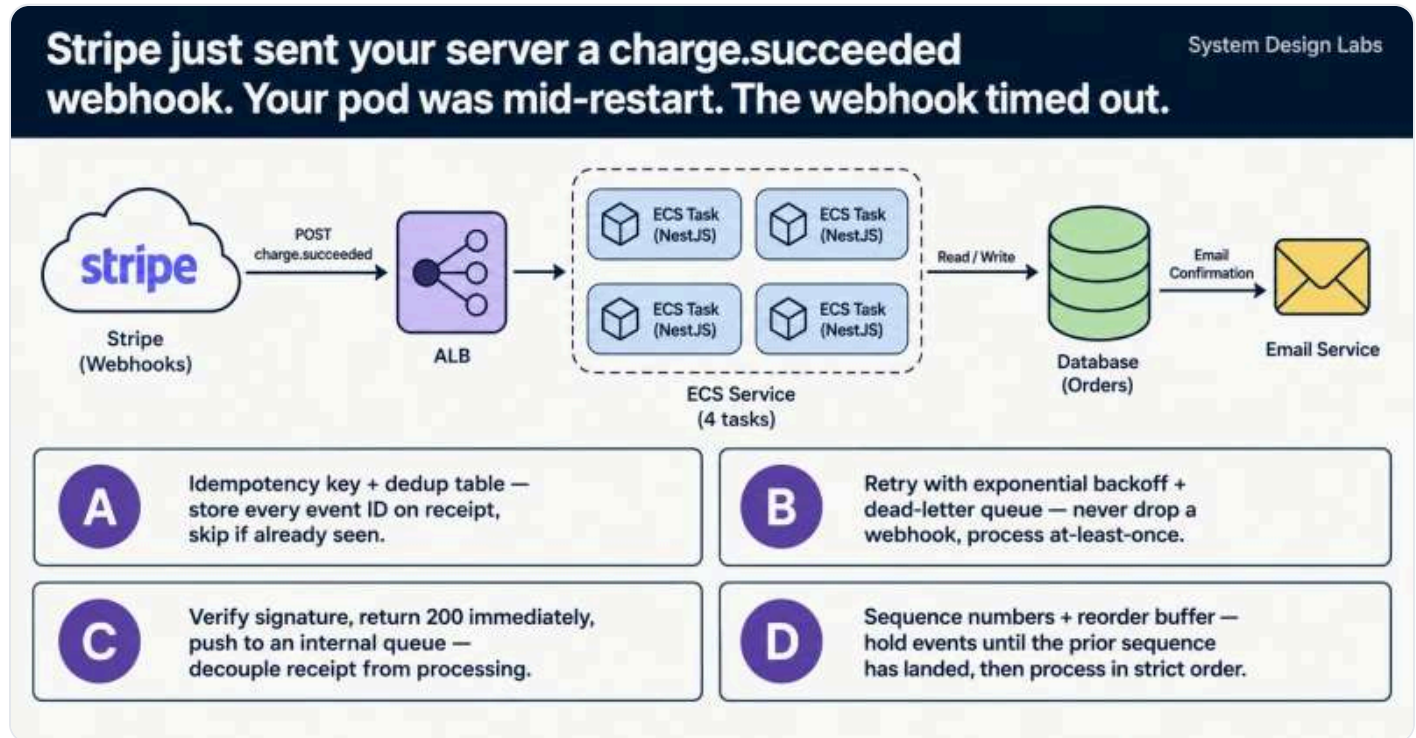
Stripe doesn't speak 2PC. Neither does any shipping carrier API. Beyond that, 2PC holds locks during PREPARE — one slow service stalls everything. At checkout volumes, the lock contention alone takes you down. Textbook answer that dies on contact with reality.

Answer 4

Why D is wrong (Outbox):

Outbox is infrastructure, not a saga. It solves how to reliably publish events after a local DB commit. It doesn't define step ordering, doesn't define compensations, doesn't answer what happens when Inventory fails. You'd still need a saga on top of it.

Handling Webhook Retries Idempotently



Stripe just sent your server a charge.succeeded webhook.

Your pod was mid-restart. The webhook timed out. Stripe will retry — but now you're racing.

Here's the setup:

NestJS API on ECS, 4 tasks behind an ALB. Stripe POSTs charge.succeeded events to /webhooks/stripe. You have ~10s to return a 200 before Stripe times out and queues a retry. On Black Friday you're handling ~80 webhooks/sec.

Three failure modes are already happening in prod:

- Pod restarts mid-deploy → webhook missed → Stripe retries → you process it, but a second pod also processed the original → **double charge confirmation email sent**
- DB is slow at peak → handler takes 12s → Stripe times out → retries → now you're processing the same event twice while the first one is still running
- Stripe sends charge.succeeded then charge.refunded 200ms apart → your queue delivers them out of order → you mark the order paid AFTER it was already refunded

The handler is a 30-line function that does the right thing 99% of the time. The 1% is killing you.

What do you do?

- A** Idempotency key + dedup table — store every event ID on receipt, skip if already seen.
- B** Retry with exponential backoff + dead-letter queue — never drop a webhook, process at-least-once.
- C** Verify signature, return 200 immediately, push to an internal queue — decouple receipt from processing.
- D** Sequence numbers + reorder buffer — hold events until the prior sequence has landed, then process in strict order.

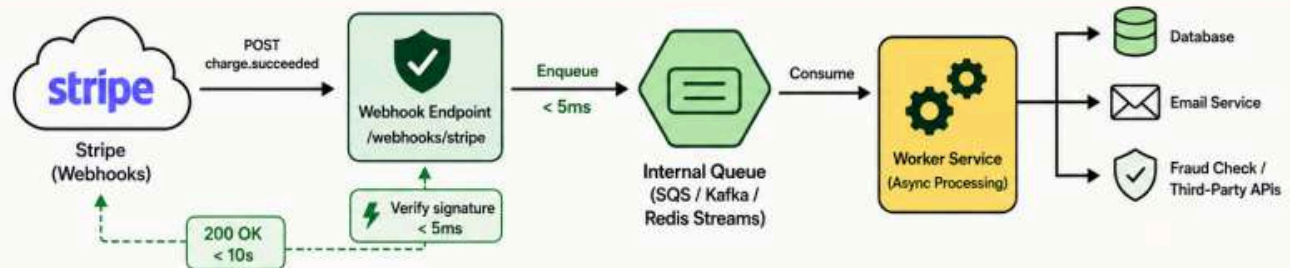
Answer 1

✓ CORRECT ANSWER

**Answer: C**

Verify signature, return 200 immediately, push to an internal queue.

System Design Labs

**C**

Verify signature, return 200 immediately, push to an internal queue.

- ✓ ACK to Stripe in < 10s (usually < 10ms).
- ✓ Decouples receipt from processing.
- ✓ Survives pod restarts and DB slowness.
- ✓ Events are durable; workers handle retries and scale independently.

A

Idempotency key + dedup table — store every event ID on receipt, skip if already seen.

- Great for preventing duplicates, but does not solve slow processing or timeouts.
- Still keeps heavy work in the webhook request path.

B

Retry with exponential backoff + dead-letter queue — never drop a webhook, process at-least-once.

- Useful, but it's still processing inside the webhook path.
- Doesn't address 10s timeout, pod restarts, or ordering problems.

D

Sequence numbers + reorder buffer — hold events until the prior sequence has landed, then process in strict order.

- Helps ordering, but increases latency and complexity.
- Does not solve timeouts, retries, or duplicate execution.

Answer: C — Verify signature, return 200 immediately, push to an internal queue ✓

Here's why, and why the other three trick smart engineers:

Why C wins (Sync receipt + async processing):

The fundamental problem is that Stripe's clock and your handler's clock are decoupled. Stripe gives you ~10 seconds to ACK. Your DB write, your fraud check, your email send — none of that should live inside that 10-second window. Ever.

The pattern: validate the HMAC signature in <5ms, write the raw payload to SQS / Kafka / Redis Streams, return 200. Now your handler is bounded by network + signature verification, not by whatever your downstream services are doing today. Pod restart mid-processing? The message is already durable in the queue — a worker picks it up. DB slow? The webhook endpoint doesn't care — it returned 200 in 8ms. This is what Stripe, Shopify, and GitHub literally tell you to do in their own webhook docs.

Answer 2

Why A is the trap answer (Idempotency key + dedup table):

This is the answer that fools senior engineers in interviews — and it's not wrong, it's just incomplete.

Yes, you absolutely need idempotency. Stripe will deliver the same event ID more than once. A dedup table keyed on [event.id](#) with a unique constraint is table stakes. But it doesn't solve the original pain: the handler is still synchronous, still on the critical 10-second path, still getting killed by pod restarts mid-write.

If you ship A alone, you've prevented double-processing — but you're still timing out, still triggering retries, still creating the conditions for the race. Idempotency is a *property your handler must have*, not an *architecture*. C is the architecture; A lives inside it. The senior engineer trap is treating idempotency as the whole answer when it's one of three layers.

Answer 3

Why B is wrong as the answer (Retry + DLQ):

Stripe is *already doing* the retry with exponential backoff. That's their job, not yours. Stripe retries failed webhooks for up to 3 days with backoff. You don't need to build that for inbound webhooks — you'd be duplicating provider behavior.

Retry + DLQ is the right pattern for *your internal queue* (the one C creates), not for the webhook endpoint itself. If your worker fails to process a queued event, sure, retry it and DLQ after N attempts. But that's a downstream concern. Putting "retry logic" at the webhook receiver level confuses inbound and outbound retry semantics.

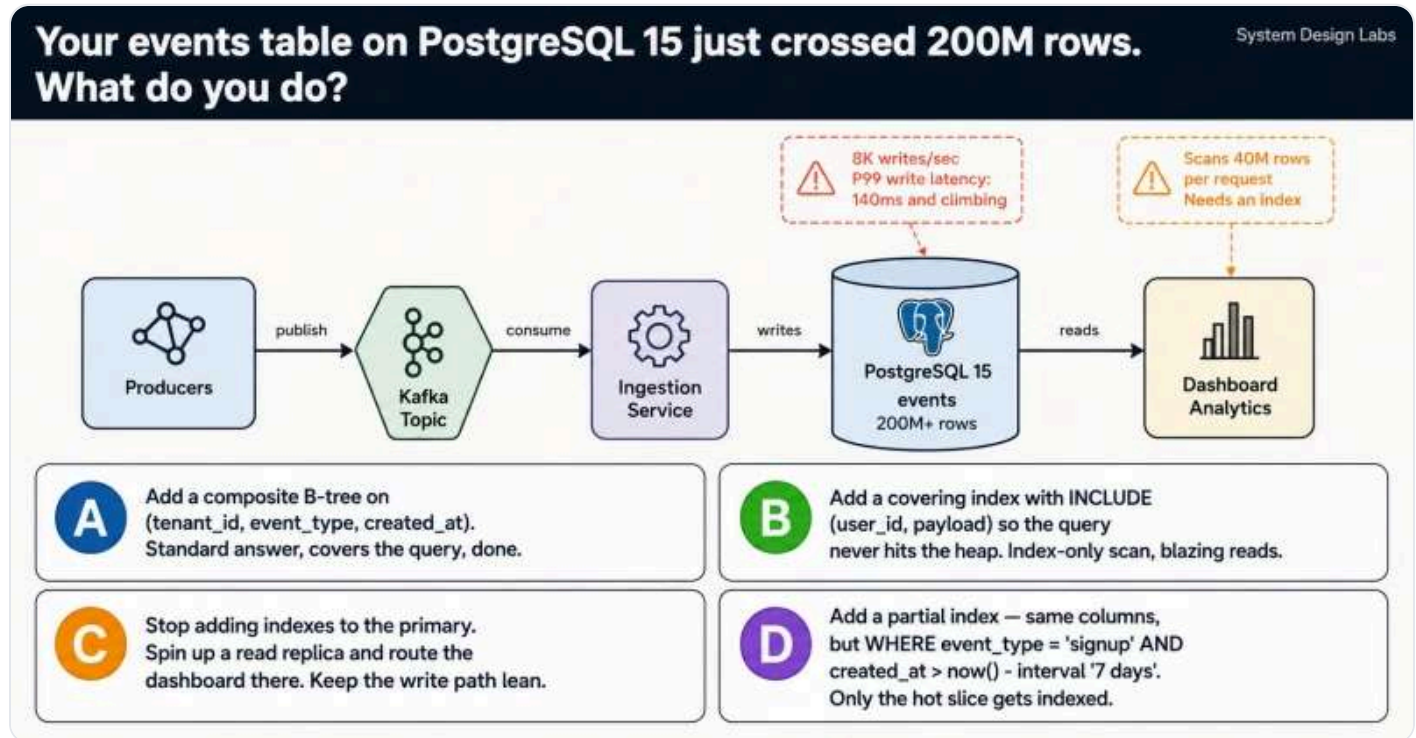
Answer 4

Why D is wrong (Sequence numbers + reorder buffer):

This sounds rigorous and it's the answer engineers reach for after they've been burned by ordering once. The problem: most webhook providers don't give you sequence numbers. Stripe doesn't. GitHub doesn't. Even when they do (Shopify's X-Shopify-Webhook-Id), the gaps aren't dense — you can't tell "missing event 47" from "47 was filtered."

More importantly: ordering at the receiver is the wrong layer. You handle ordering at the *domain level* — when you process `charge.refunded`, you check the current state of the order before transitioning. If the order is already in refunded, you no-op. State machines beat reorder buffers because they handle the case where events arrive out of order *AND* the case where one of them never arrives at all. A reorder buffer just means you stall forever when an event gets dropped upstream.

Indexing a High-Ingest Table



Your events table on PostgreSQL 15 just crossed 200M rows.

Ingestion runs at 8K writes/sec from a Kafka consumer. P99 write latency was 12ms — now it's 140ms and climbing.

The dashboard team is screaming. Their query filters on (tenant_id, event_type, created_at) and scans 40M rows per request. They want an index. Yesterday.

You check `pg_stat_user_indexes`. The table already has 4 indexes. Each new write touches every one of them. Add a fifth and your ingestion lag becomes ingestion failure.

But 92% of dashboard queries hit a single tenant's last 7 days of signup events. A tiny slice of a huge table.

What do you do?

- A) Add a composite B-tree on (tenant_id, event_type, created_at). Standard answer, covers the query, done.
- B) Add a covering index with INCLUDE (user_id, payload) so the query never hits the heap. Index-only scan.
- C) Stop adding indexes to the primary. Spin up a read replica and route the dashboard there. Keep writes lean.

D) Add a partial index — same columns, but WHERE event_type = 'signup' AND created_at > now() - interval '7 days'. Only the hot slice gets indexed.

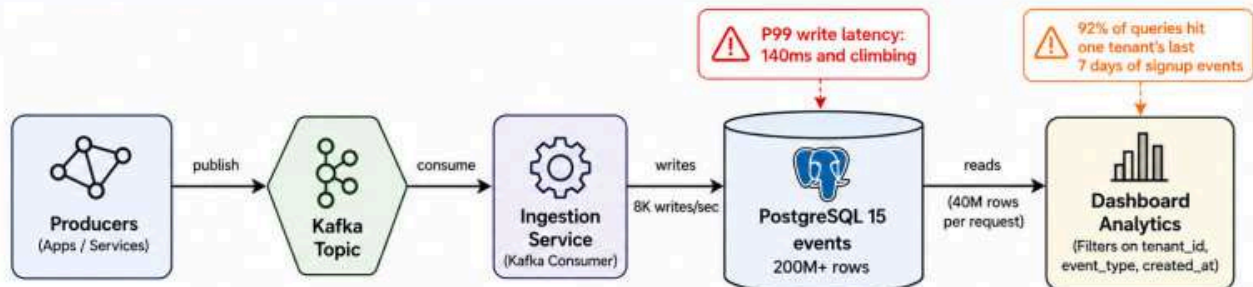
Answer 1

✓ CORRECT ANSWER

D**Correct Answer**

**Add a partial index — same columns,
WHERE event_type = 'signup' AND created_at > now() - interval '7 days'.**

System Design Labs

**D) Partial Index (Correct)**

Best Balance



Only indexes rows where event_type = 'signup' AND created_at > now() - interval '7 days'.

- ✓ Indexes only the hot slice (~0.2% of table)
- ✓ Inserts outside the predicate pay zero cost
- ✓ Planner uses it for the dashboard query
- ✓ Fast reads, minimal write amplification



Trade-off: if the dashboard needs 30 days instead of 7, you rebuild the index. Known cost, not a hidden one.

A**A) Composite B-tree Index**

- ✗ Indexes all rows (200M+)
- ✗ Every write updates it
- ✗ High write amplification
- ✗ Will worsen write latency

B**B) Covering Index INCLUDE (user_id, payload)**

- ✗ Even more writes per insert
- ✗ Much larger index
- ✗ Higher memory & I/O
- ✗ Write path takes the hit

C**C) Read Replica**

- ✗ Does not fix slow writes on primary
- ✗ Still need the index somewhere for fast reads
- ✗ Replication lag won't help real-time dashboard
- ✗ Adds infrastructure and ops complexity

Answer: D — Partial Index ✓

Here's why, and why the other three trick smart engineers.

Why D wins (Partial Index)

92% of your queries hit one tenant's last 7 days of signup events. A partial index only indexes rows that match its WHERE clause — everything else writes to the table without touching this index at all.

The math: if signup events are ~4% of the stream and the 7-day window covers ~5% of the table, your partial index covers roughly 0.2% of rows. Inserts that don't match the predicate cost you nothing on this index. The write amplification is almost entirely gone.

PostgreSQL's planner uses the partial index for any query whose WHERE clause is a subset of the index predicate. Your dashboard query gets full coverage. Reads stay fast, writes stay alive.

The catch: when the dashboard team wants 30 days instead of 7, you rebuild. That's a known cost, not a hidden one.

Answer 2

Why B is the trap (Covering Index with INCLUDE)

This fools senior engineers more than any other option. An index-only scan looks gorgeous in EXPLAIN. The problem is what you stuffed into INCLUDE.

That payload column is probably a JSONB blob averaging a few KB. Every insert now writes the row to the heap *and* writes a fat index entry containing the full payload. Every UPDATE to user_id or payload rewrites the index entry. You didn't reduce write cost — you multiplied it.

At 8K writes/sec on a table that's already index-heavy, a covering index with JSONB inside is how you turn 140ms p99 into 800ms p99. INCLUDE is the right call for narrow columns on read-heavy tables. This is neither.

Answer 3

Why A is wrong (Full composite B-tree)

A (tenant_id, event_type, created_at) index will make the dashboard fast. It will also index every single one of your 8K writes/sec across all tenants and all event types. You're paying full write cost to serve a query that hits 0.2% of the data. That's the definition of a bad tradeoff.

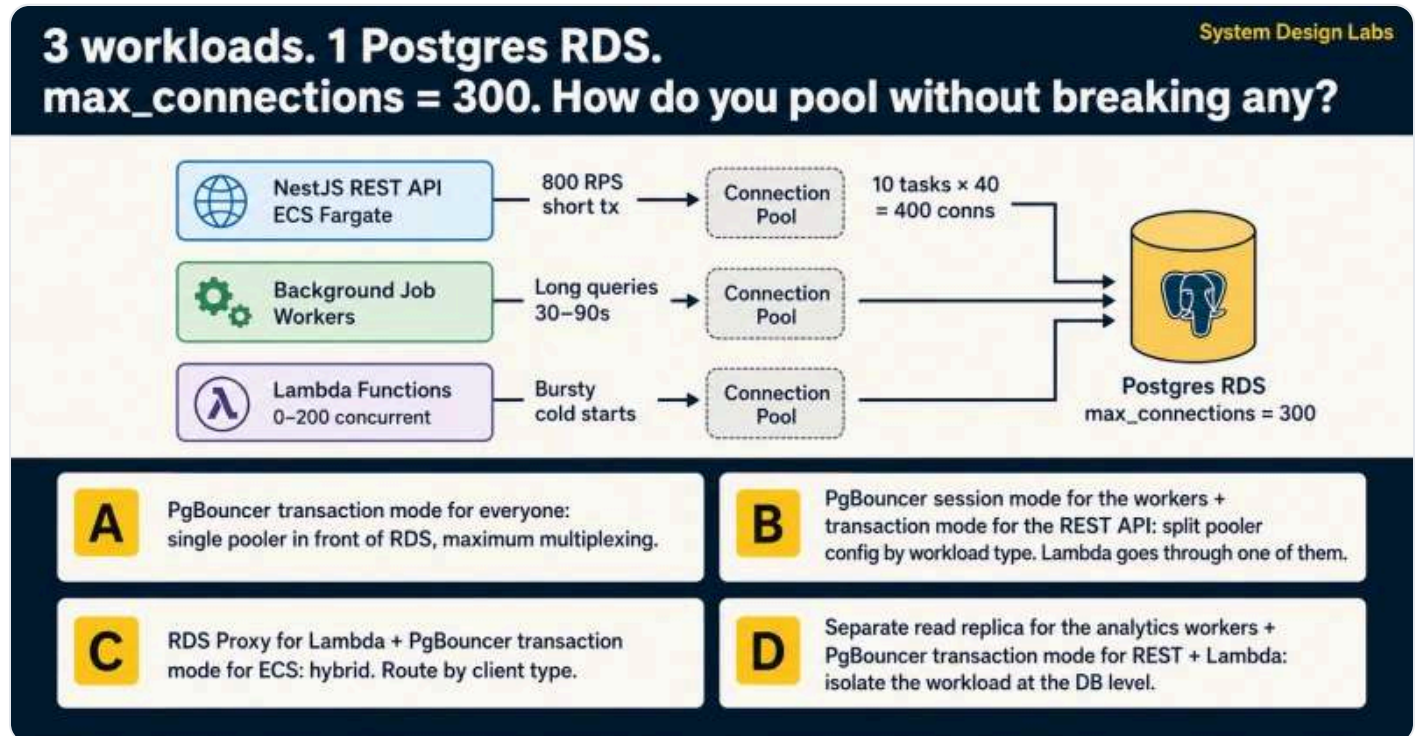
Answer 4

Why C is wrong (Read replica)

The indexes still have to exist somewhere. Put them on the replica and it falls behind replaying WAL while maintaining indexes. Put them on the primary and you're back to the original problem.

Read replicas solve "my primary can't serve all the reads." They don't solve "my write path is bottlenecked by index maintenance."

Managing a Shared Connection Pool



You're running 3 workloads against the same Postgres RDS instance. max_connections = 300. They all need a connection pool. None of them want to share.

Here's the setup:

→ NestJS REST API on ECS Fargate. Short transactions, 800 RPS, 10 tasks × 40 pool = **400 connections requested**.

→ Background job workers. Long-running analytics queries. Hold a connection for **30–90 seconds** each.

→ Lambda functions. Bursty, cold-start heavy, **0–200 concurrent invocations**.

Three clients. One database. 300 max connections. Math doesn't math.

You need to pool without breaking any of them. Pick your poison:

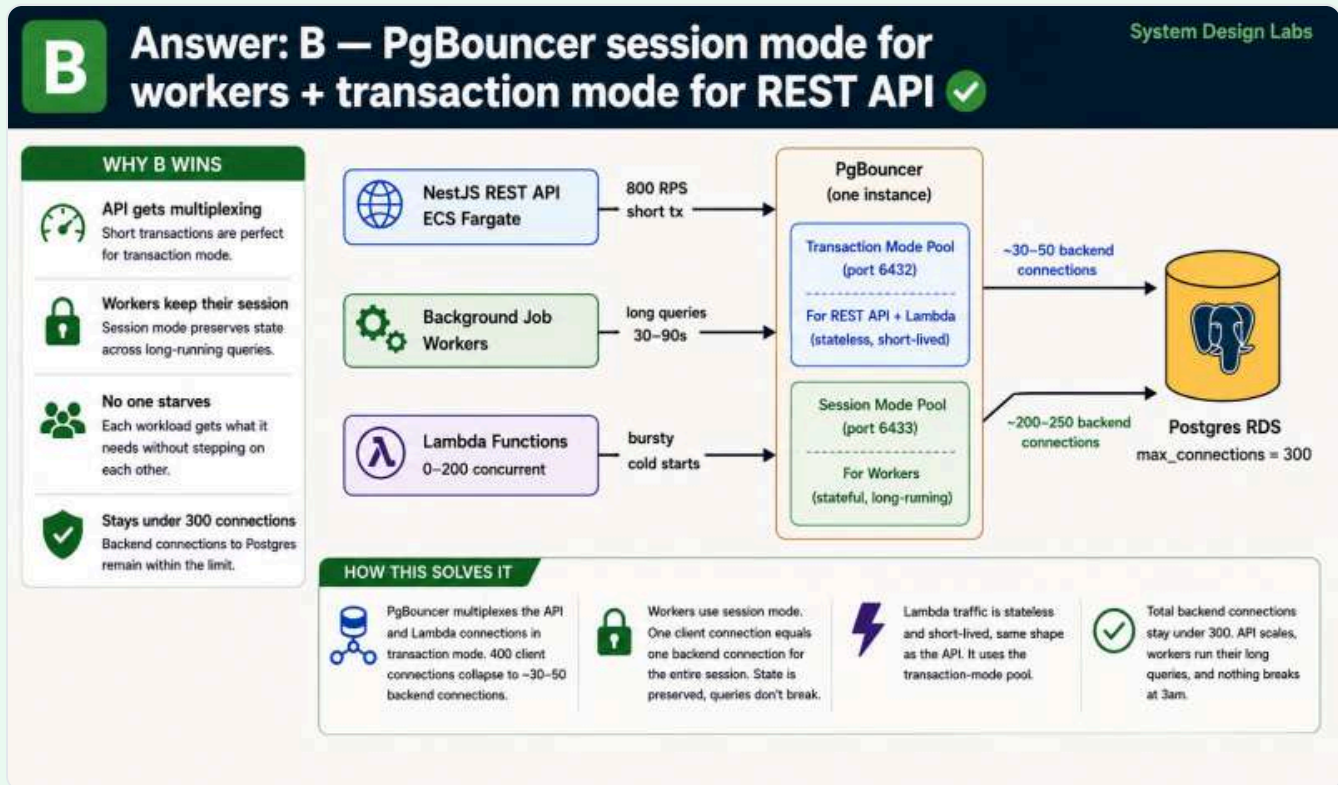
A PgBouncer transaction mode for everyone: single pooler in front of RDS, maximum multiplexing.

B PgBouncer session mode for the workers + transaction mode for the REST API: split pooler config by workload type. Lambda goes through one of them.

C RDS Proxy for Lambda + PgBouncer transaction mode for ECS: hybrid. Route by client type.

D Separate read replica for the analytics workers + PgBouncer transaction mode for REST + Lambda: isolate the workload at the DB level.

Answer 1 ✓ CORRECT ANSWER



Answer: B — PgBouncer session mode for workers + transaction mode for REST API ✓

This one separates engineers who've actually run PgBouncer in prod from ones who've only read the docs. Here's the breakdown:

Why B wins (split pooler config by workload type):

The whole point of PgBouncer is multiplexing — one backend Postgres connection serves many client connections, but **only when it's safe to hand the connection back between transactions**. That's transaction mode.

Transaction mode is *perfect* for the NestJS REST API: short transactions, 800 RPS, no session state, no SET LOCAL, no advisory locks held across queries, no prepared statements that need to survive. The REST API's 400 requested client connections collapse down to ~30-50 actual backend connections to Postgres. Math suddenly maths.

Transaction mode is *catastrophic* for the analytics workers. Their queries run 30-90 seconds and span multiple transactions — some hold session state, some use temp tables, some use SET search_path, some use cursors. Hand the connection back between transactions and that state is gone. Worse: at 800 RPS from the API side, the worker's chance of getting the *same* backend connection back on the next transaction is basically zero. The worker silently breaks.

So you give the workers their own PgBouncer pool in **session mode** — one client connection = one backend connection for the whole session. They get fewer connections, but they hold them, which is what they actually need.

The Lambda traffic goes through the REST API's transaction-mode pool (it's stateless, short-lived — same shape as the API). One pooler instance, two pool configs, two listening ports. PgBouncer supports this natively.

End state: API gets multiplexing, workers don't starve, total backend connections to Postgres stay under 300, nobody breaks. That's the boring correct answer that wins on call.

Answer 2

...Why A is the trap (PgBouncer transaction mode for everyone):

This is the answer most engineers reach for because "transaction mode = best multiplexing." It works beautifully for the API. It silently destroys the workers.

The moment an analytics query needs anything that doesn't fit in one transaction — a temp table, a session variable, a SET LOCAL, a prepared statement, a cursor that outlives a transaction — PgBouncer hands the connection back to the pool. Next query? Different backend connection. State gone. Query fails or worse, returns wrong results.

The failure mode is the cruelest kind: it works in dev (low concurrency, same connection by luck), it works in staging (light load), it dies in prod when the API is hammering the pool and the workers can't get a stable connection. You'll find this at 3am during a financial close.

Answer 3

Why C is wrong (RDS Proxy for Lambda + PgBouncer for ECS):

Defensible on paper. RDS Proxy is genuinely good for Lambda — it handles the cold-start connection storm, manages IAM auth, integrates with the VPC. If the question were just "Lambda + Postgres," C is right.

But C doesn't fix the workers. They're still long-running, they still need session state, and you've now introduced a second pooling product to operate, monitor, and pay for. RDS Proxy is also *more expensive than PgBouncer* per connection-hour, and it doesn't support session pinning across transactions the way session-mode PgBouncer does.

C solves the easiest of the three problems and leaves the hardest one untouched.

Answer 4

Why D is the senior-engineer trap (read replica for workers):

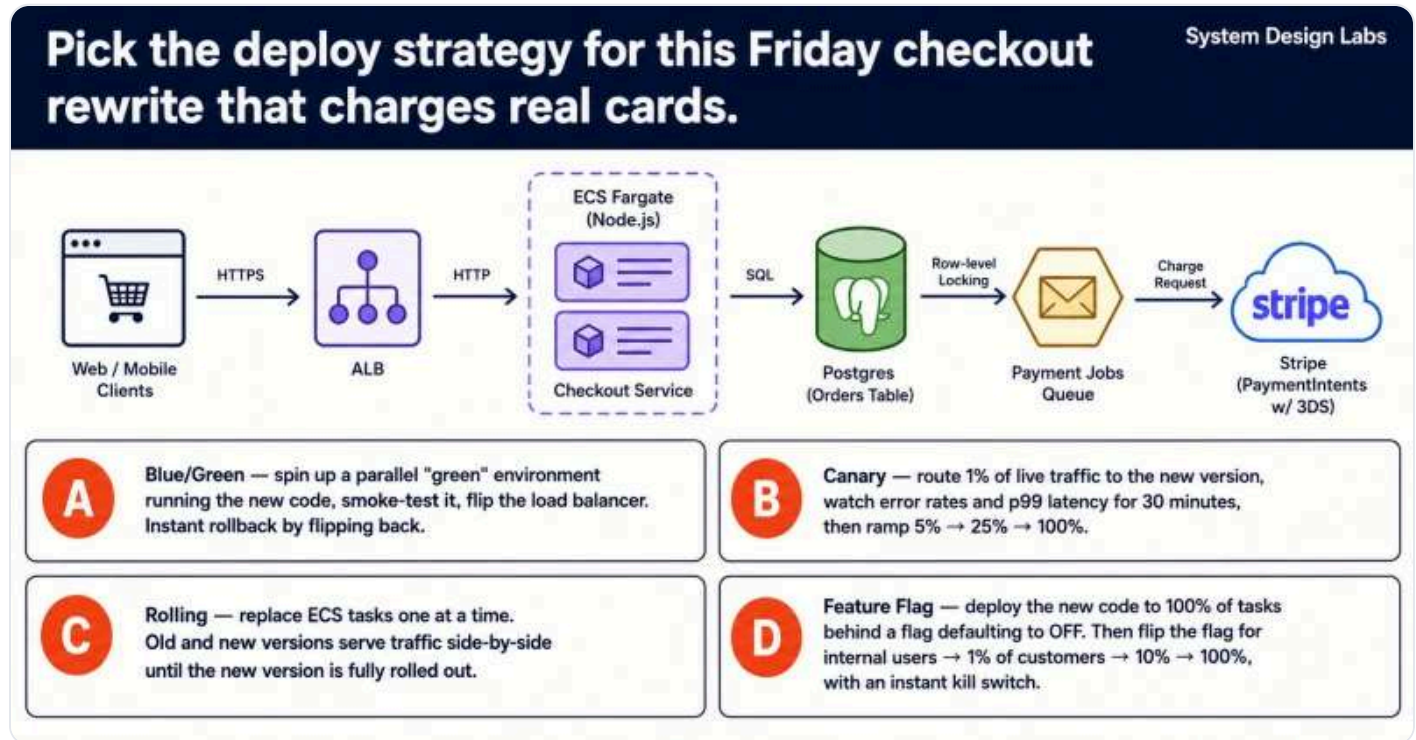
This is the option that fools experienced engineers, because it pattern-matches to "isolate the workload." And isolating analytics from OLTP at the DB level is genuinely a great idea — for a *different* problem.

D solves **read scaling** and **noisy neighbor on query plans / cache / IO**. It does **not** solve the connection math. Your read replica still has its own max_connections ceiling. Your workers still hold connections for 30–90 seconds. You've just moved the connection exhaustion problem from the primary to the replica — and now you're paying for an extra RDS instance.

If the problem statement was "analytics queries are starving the OLTP workload," D would be right. The problem statement is "we don't have enough connections." Different pain. Different pattern.

D is what you do *after* you have B working and you realize analytics is also competing for IOPS and buffer cache.

Safely Rolling Out a Risky Change



You're shipping a rewrite of your checkout write path on Friday.

Node.js on ECS Fargate. ~3,000 RPS at peak. Postgres with row-level locking on the orders table. The change touches the line of code that actually charges the customer's card — old path uses Stripe Charges API, new path uses PaymentIntents with 3DS.

The QA env is green. Load test passed. Your staff engineer signs off.

But it's *checkout*. If this breaks, money breaks. And you can't roll back a card that already got charged.

Pick the deploy strategy:

- A** Blue/Green — spin up a parallel "green" environment running the new code, smoke-test it, flip the load balancer. Instant rollback by flipping back.
- B** Canary — route 1% of live traffic to the new version, watch error rates and p99 latency for 30 minutes, then ramp 5% → 25% → 100%.

C Rolling — replace ECS tasks one at a time. Old and new versions serve traffic side-by-side until the new version is fully rolled out.

D Feature Flag — deploy the new code to 100% of tasks behind a flag defaulting to OFF. Then flip the flag for internal users → 1% of customers → 10% → 100%, with an instant kill switch.

Answer 1

✓ CORRECT ANSWER

ANSWER: D — FEATURE FLAG ✓

System Design Labs

**Why D wins (Feature Flag):**

Feature flags decouple **deploy** from **release**. The new code ships to 100% of your ECS tasks — but it's dormant. Nothing executes until you flip the flag.



Rollback is a config push, not a deploy.

Sub-second.
No ECS restarts,
no LB drain,
no DNS TTL.



You target who sees the new path, not what % of random traffic.

\$0.99 customer and \$40K B2B customer shouldn't get coin-flipped equally — that's not a strategy, that's hope.

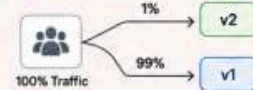


You can keep the old code path warm for weeks.

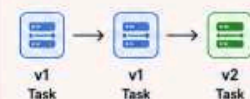
3DS edge case on a specific issuing bank in Brazil? Flip those users back while keeping everyone else on v2.

⚠ Why the other options look right — but quietly fail when the change touches money**A Blue/Green**

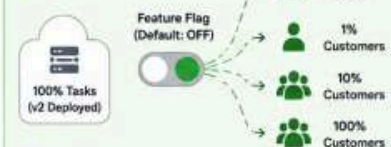
Big bang. You flip everyone at once. If the new payment flow has a flaw that only appears under real money conditions, you learn about it at 3AM with 100% impact.

B Canary

Random % is the wrong targeting. High-value customers, specific banks, or regions might be in the 1% — and the blast radius is still real money.

C Rolling

Old and new code run side-by-side. You can't control which customers hit which version. Shared DB + money writes = inconsistent behavior and hard-to-debug issues.

D Feature Flag

- ✓ Deploy once. Release safely.
- ✓ Target precisely. Roll back instantly.
- ✓ Keep old path ready as a safety net.

Answer: D — Feature Flag ✓

Here's why, and why the other three look right but quietly fail when the change touches money:

Why D wins (Feature Flag):

Feature flags decouple **deploy** from **release**. The new code ships to 100% of your ECS tasks — but it's dormant. Nothing executes until you flip the flag.

- **Rollback is a config push, not a deploy.** Sub-second. No ECS restarts, no LB drain, no DNS TTL.
- *You target who sees the new path, not what % of random traffic.* \$0.99 customer and \$40K B2B customer shouldn't get coin-flipped equally — that's not a strategy, that's hope.
- **You can keep the old code path warm for weeks.** 3DS edge case on a specific issuing bank in Brazil? Flip *those* users back while keeping everyone else on v2.

Answer 2

Why B is the trap (Canary):

1. Canary routes by request, not by customer. A B2B billing job hits your 1% canary 50 times in an hour. Bug on their card type? You've charged them 50 times before dashboards catch it.
2. Payment failures don't surface as 5xx errors — they surface as successful 200s that charged the wrong amount. Your error-rate alarm never fires.

Answer 3

Why A is wrong (Blue/Green):

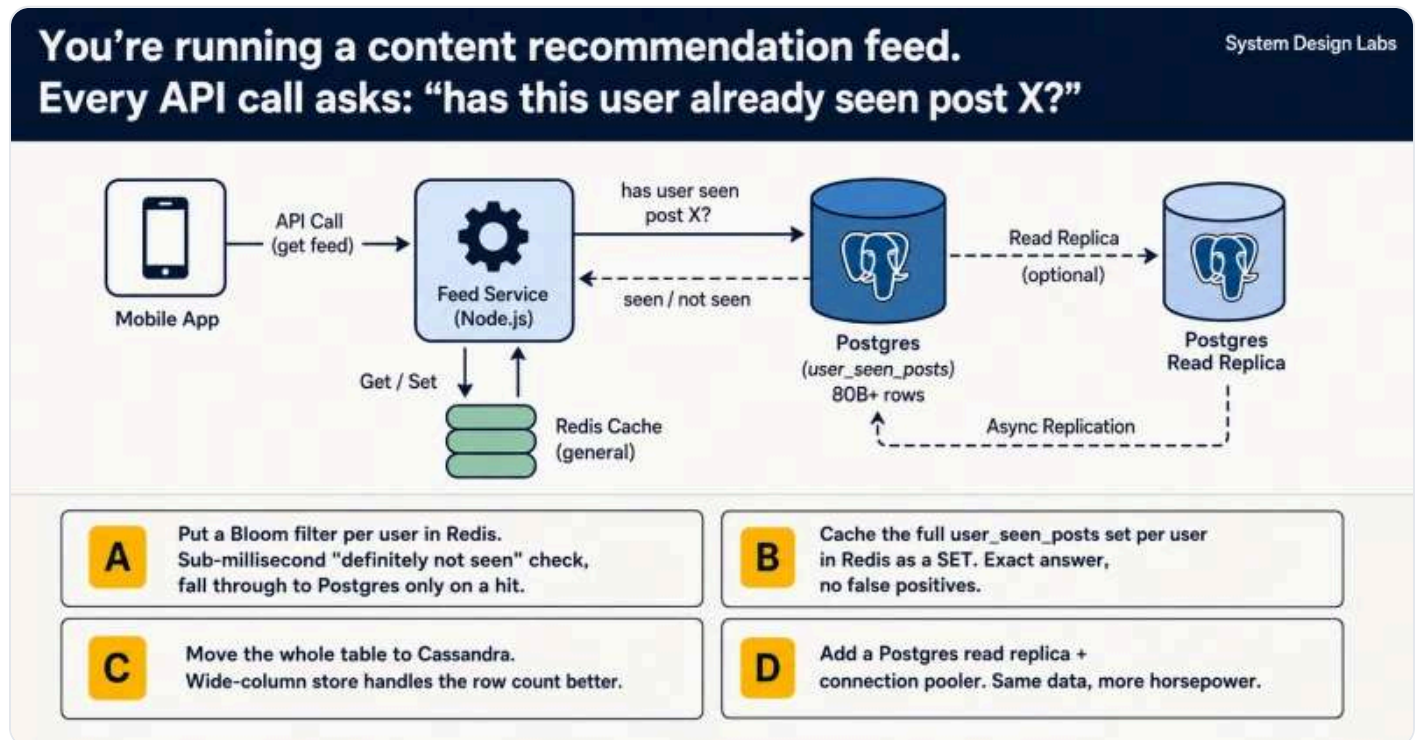
"Instant cutover" is the problem. The moment you flip the LB, 100% of live traffic hits new code — including Stripe webhooks confirming PaymentIntents *created* by the old code. Cross-version race condition on real money. Rollback doesn't undo a charge.

Answer 4

Why C is wrong (Rolling):

For 5–10 minutes, old and new tasks serve checkout simultaneously. "Create order" hits v1. "Confirm payment" webhook 800ms later hits v2. Inconsistent state written by code that was never tested together.

Fast “Has the User Seen This?” Checks



You're running a content recommendation feed. 50M users.

Every API call asks: "has this user already seen post X?"

Right now you're hitting Postgres on every check. The user_seen_posts table has 80B rows and growing. p99 latency on the feed endpoint just crossed 600ms and your DBA is sending you screenshots of CPU graphs at 2am.

Here's the setup:

- Feed service: Node.js, ~120K RPS at peak
- "Already seen" check: SELECT against Postgres on every recommendation
- Table: user_seen_posts(user_id, post_id, seen_at) (partitioned, indexed, still slow)
- Cache hit ratio is fine. The problem is the long tail of cold lookups
- False positives on "already seen" are tolerable. False negatives (showing the same post twice) are not great but survivable

Product wants p99 under 100ms. You have one sprint. What do you do?

A Put a Bloom filter per user in Redis. Sub-millisecond "definitely not seen" check, fall through to Postgres only on a hit.

B Cache the full user_seen_posts set per user in Redis as a SET. Exact answer, no false positives.

C Move the whole table to Cassandra. Wide-column store handles the row count better.

D Add a Postgres read replica + connection pooler. Same data, more horsepower.

Answer 1

✓ CORRECT ANSWER

■ Author's diagram unavailable (source image returned 404)

A — Bloom Filter per user in Redis (THE RIGHT ANSWER)

Bloom filters give one ironclad guarantee: "*this item is DEFINITELY NOT in the set.*" If the filter says no → skip Postgres entirely. If it says maybe → fall through to confirm.

Why it wins here:

- ~97% of feed checks are for posts the user has *never* seen → sub-millisecond "definitely no" from Redis, Postgres never touched
- **Memory math:** 10K posts per user in a Bloom filter at 1% false-positive = **12 KB**. Same data as a Redis SET = **80 KB+**, and grows linearly
- At 50M users: Bloom filter = ~600 GB. Redis SET = ~4 TB. That's the difference
- Real-world users: Medium's "have you read this", Cassandra's SSTable short-circuit, Bitcoin's SPV wallets

The 1% false positive here means you *occasionally* skip showing a post the user hasn't seen. Product said that's tolerable. Bloom filters never produce false negatives (showing the same post twice) — which would be the dealbreaker.

Answer 2**B — Redis SET per user (SENIOR ENGINEER TRAP)**

Looks strictly better on paper — same $O(1)$ speed, *plus* perfect accuracy, no false positives. SISMEMBER is battle-tested.

The trap: **memory explodes on power users**. A user with 50K posts seen = 3–4 MB in a Redis SET. Your top 5% users blow up the Redis cluster. You'll either over-provision (expensive) or start evicting hot users (defeats the whole point).

Rule of thumb: Redis SET when cardinality is small (<1K per key) or false positives are unacceptable. Bloom filter when cardinality is large and "probably not" is good enough.

Answer 3

C — Move to Cassandra (RIGHT PATTERN, WRONG PROBLEM)

Cassandra handles wide-column data and massive row counts fine. But the problem isn't *storage* — Postgres with proper partitioning handles 80B rows. The problem is **read latency under load**.

Switching storage engines doesn't fix the read pattern. You're still doing a point lookup per recommendation. Now you've added a multi-quarter migration + new ops skillset to solve what's fixable in a sprint with 12 KB per user in Redis.

Cassandra is the answer when the bottleneck is write throughput or wide-row scans. Not for 120K point lookups per second on a hot path.

Answer 4

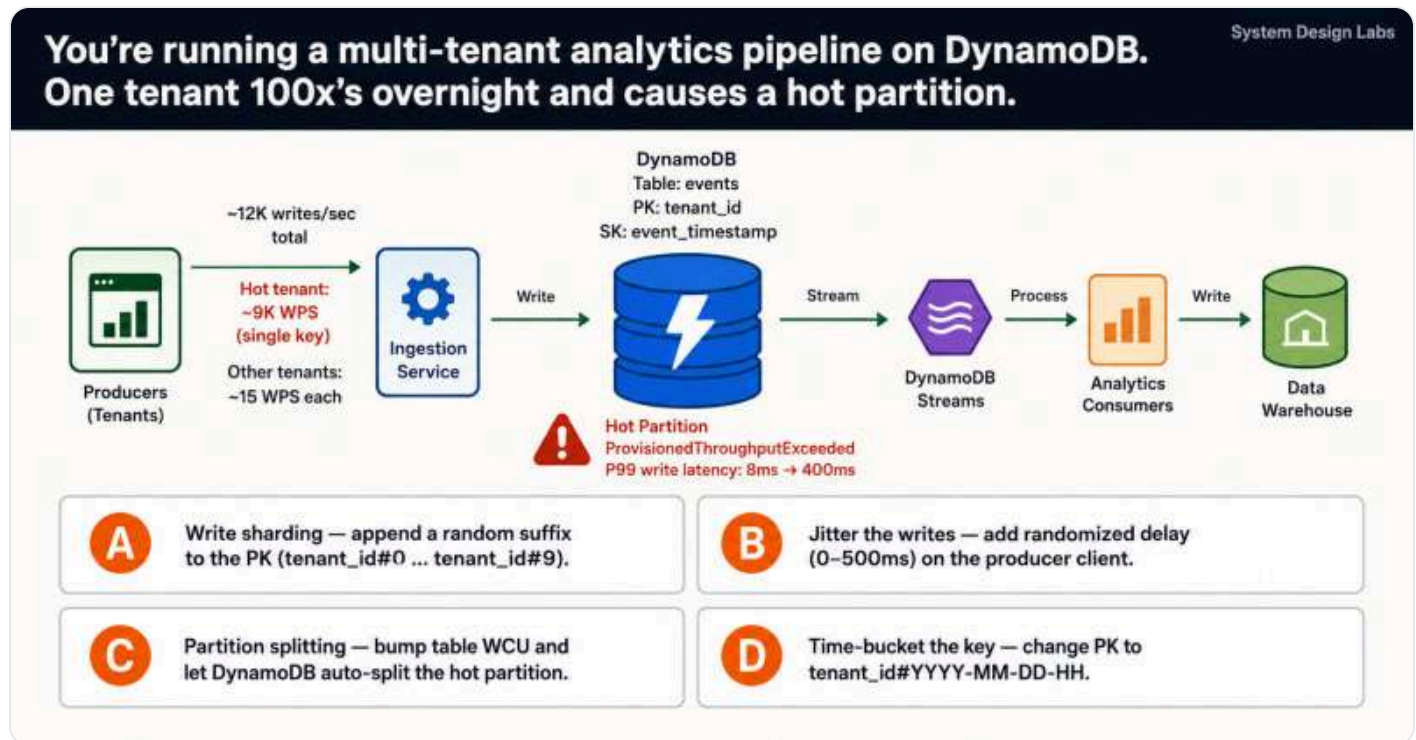
D — Postgres read replica + connection pooler (CAPACITY ≠ COST)

Replicas buy horsepower, not efficiency. You're still hitting disk on every check, still doing a B-tree lookup, still paying round-trip per request. 3 replicas → maybe 2.5x read throughput before replication lag and connection limits bite.

To hit p99 <100ms at 120K RPS, you don't need *more database* — you need to **not hit the database** for the 97% of requests where the answer is "no."

This is the option chosen by teams who treat every performance problem as a capacity problem.

Taming a Hot Partition Key



You're running a multi-tenant analytics pipeline on DynamoDB. 200 tenants, 12K writes/sec total. Everything is fine — until it isn't.

One tenant onboards a massive customer overnight. Their event volume 100x's. Now that one tenant is hitting 9K writes/sec on a single partition key. The other 199 tenants sit idle.

ProvisionedThroughputExceeded errors start firing. P99 write latency spikes from 8ms to 400ms. Your on-call gets paged. Everyone is getting throttled because of one key.

Here's the setup:

- Table: events (DynamoDB, on-demand capacity)
- PK: tenant_id · SK: event_timestamp
- Hot tenant: ~9K WPS · All other tenants: ~15 WPS each
- Single partition key absorbing all 9K writes — DynamoDB's per-partition limit is the wall

Classic hot partition. What do you do?

A Write sharding — append a random suffix to the PK (tenant_id#0 ... tenant_id#9).

B Jitter the writes — add randomized delay (0–500ms) on the producer client.

C Partition splitting — bump table WCU and let DynamoDB auto-split the hot partition.

D Time-bucket the key — change PK to tenant_id#YYYY-MM-DD-HH.

Answer 1

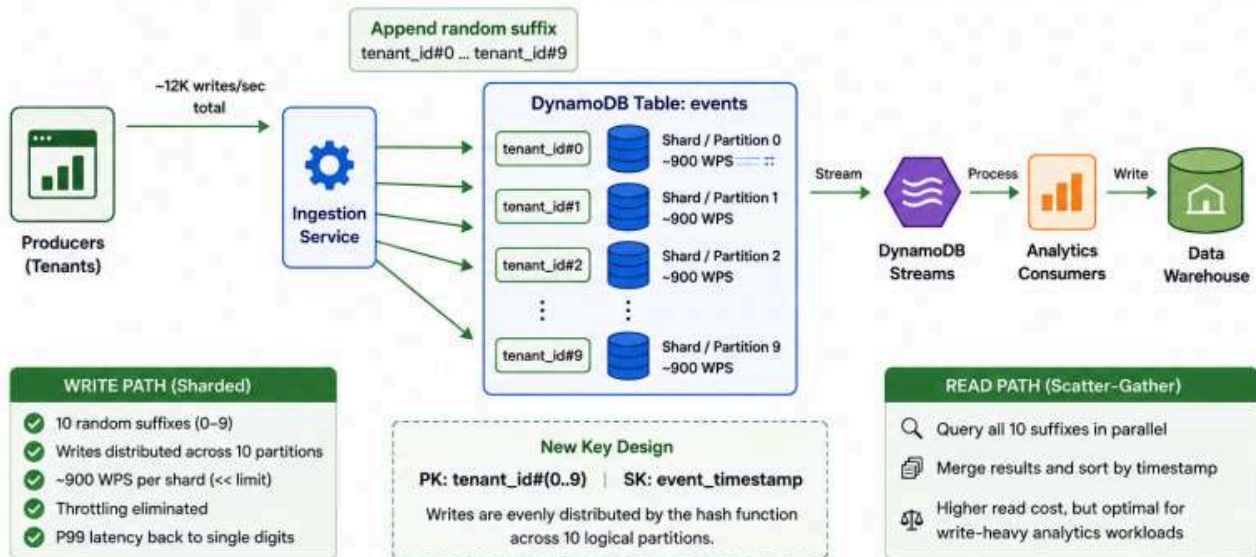
✓ CORRECT ANSWER

ANSWER: A — WRITE SHARDING (THE RIGHT ANSWER)

System Design Labs

**Append a random suffix to the PK to spread writes across multiple partitions.**

10 shards ≈ 900 WPS each | Throttling stops | P99 latency back to single digits



This is the canonical fix for hot partitions — and the pattern AWS documents.

A — Write Sharding (THE RIGHT ANSWER)

DynamoDB hashes the partition key to decide which physical shard handles the write. One PK value = one shard = one wall. When that tenant hits 9K WPS, the shard caps out.

The fix: append a random suffix at write time — tenant_id#0 through tenant_id#9. Now the hash function sees 10 different keys and distributes writes across 10 logical partitions (~900 WPS each, well under the limit). Throttling stops, P99 drops back to single digits.

The read cost: scatter-gather across all 10 suffixes to query that tenant's data. For a write-heavy analytics pipeline, that's the right tradeoff.

This is literally the pattern AWS documents as the canonical fix for hot partitions.

Answer 2

C — Partition Splitting (SENIOR ENGINEER TRAP)

DynamoDB *does* auto-split partitions — so this sounds right. But the split is based on **throughput**, not **key cardinality**. If one PK value generates 9K WPS, all those writes still hash to the same destination after the split. You can't split a single key across two physical shards.

You'll see the partition split in CloudWatch, assume the problem is fixed, and then get paged again 10 minutes later. It's the answer that survives the design doc and dies in the war room.

Answer 3

B — Jitter the writes

Jitter is a real pattern — it solves the **thundering herd** problem, where N clients fire requests at the same millisecond and create a spike. Spreading those over 500ms smooths the burst.

But a hot partition is a **sustained** 9K WPS, not a burst. Adding delay doesn't reduce the rate — DynamoDB still sees 9K WPS on the same partition. You just made every write slower for zero benefit.

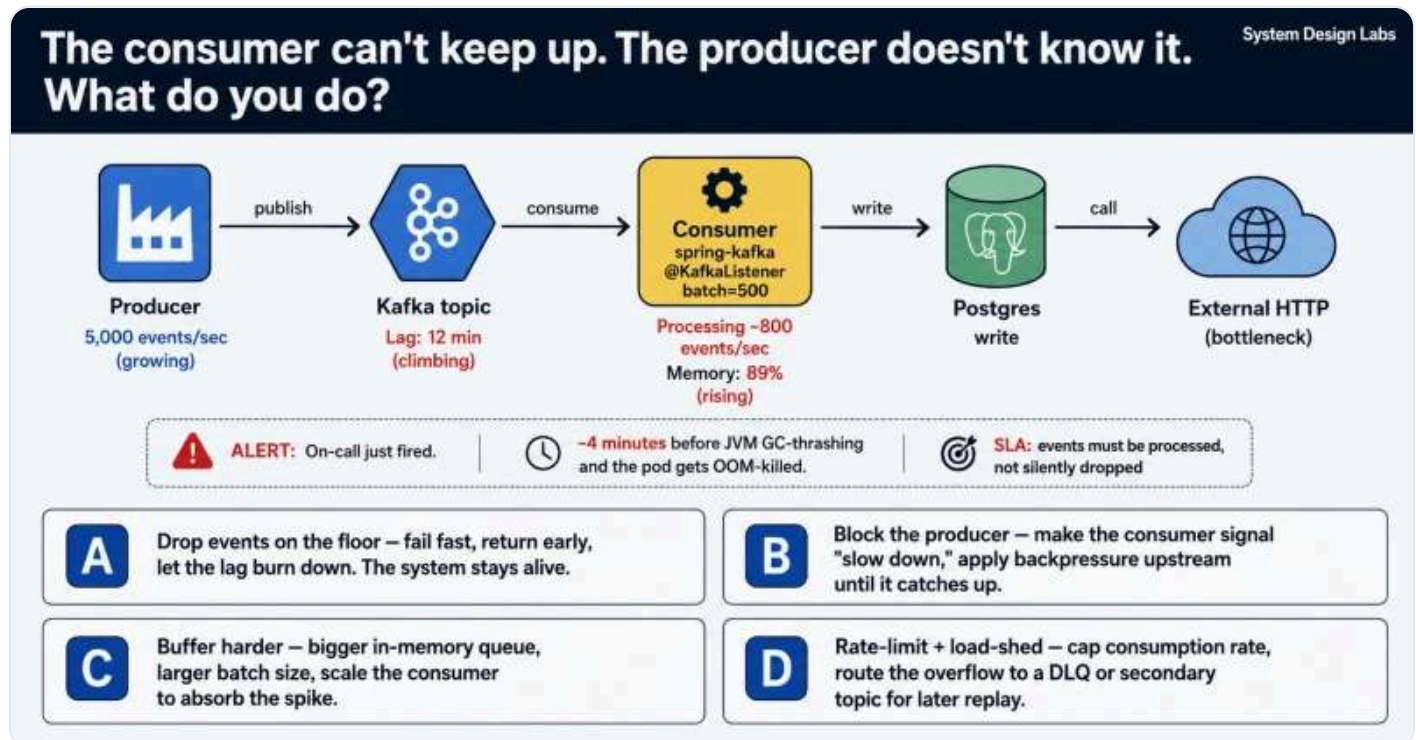
Answer 4

D — Time-bucket the key

`tenant_id#2026-05-21-14` is a real pattern for time-series data — it keeps per-bucket size manageable and enables efficient TTL expiry.

But at any given hour, you *still* have one partition key absorbing all 9K WPS for that tenant. You renamed the hot partition to a hot time bucket. Same problem, new name. Plus now your reads have to query 24 different PKs for a day's worth of data.

Backpressure on an Overwhelmed Consumer



Your Kafka consumer is processing 800 events/sec.

The producer just hit 5,000 events/sec and it's not slowing down.

Lag chart: 12 minutes behind and climbing. Consumer memory: 89% and rising. The on-call alert just fired. You have ~4 minutes before the JVM starts GC-thrashing and the pod gets OOM-killed.

Here's the setup:

Producer → Kafka topic (5K events/sec, growing)

Consumer → spring-kafka @KafkaListener, batch=500, processes ~800 events/sec

Downstream → Postgres write + external HTTP call (the real bottleneck)

SLA → events must be processed, not silently dropped

The consumer can't keep up. The producer doesn't know it. What do you do?

- A** Drop events on the floor — fail fast, return early, let the lag burn down. The system stays alive.

B Block the producer — make the consumer signal "slow down," apply backpressure upstream until it catches up.

C Buffer harder — bigger in-memory queue, larger batch size, scale the consumer to absorb the spike.

D Rate-limit + load-shed — cap consumption rate, route the overflow to a DLQ or secondary topic for later replay.

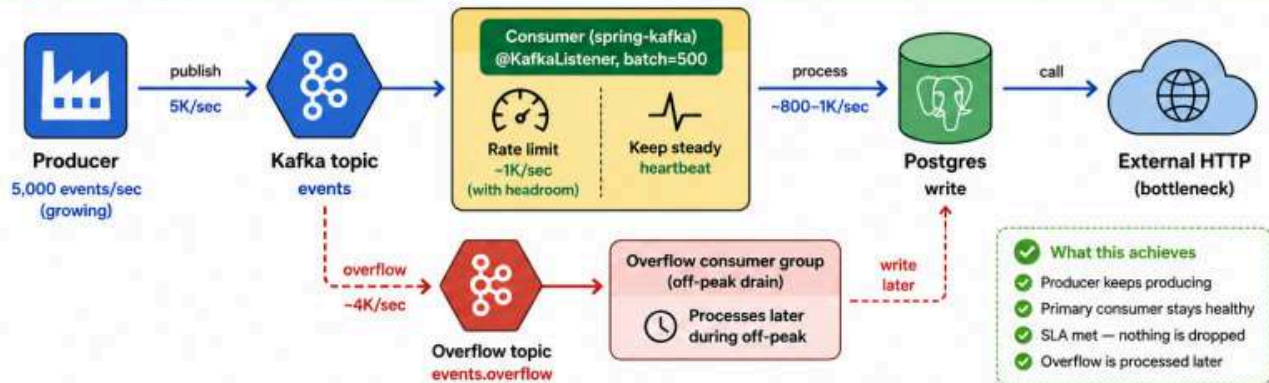
Answer 1

✓ CORRECT ANSWER

✓ ANSWER: D — Rate-limit + load-shed

System Design Labs

🎯 Goal: Keep the system healthy, meet the SLA, and process overflow later — not drop, not block.



★ Why D wins:

The SLA says events **must be processed** — not dropped, not blocked. That kills **A** and **B** immediately.

Cap the consumer's intake at ~1K/sec with headroom, route overflow to a durable secondary topic (**events.overflow**) with its own consumer group that drains during off-peak.

Producer keeps producing. Primary consumer keeps a steady heartbeat.

Overflow gets processed later. That's **graceful degradation**.



Stripe and Shopify publish architecture posts on this exact pattern.

Why others lose:

A

Drop events on the floor

Violates the SLA. Events are lost forever.

B

Block the producer

Kafka doesn't provide consumer → producer backpressure. You'd need custom acks/transactions — not this stack.

C

Buffer harder

More memory = faster OOM. Larger batches increase latency and worsen recovery time.

Answer: D — Rate-limit + load-shed ✓

Why D wins:

The SLA says events must be processed — not dropped, not blocked. That kills A and B immediately. Cap the consumer's intake at ~1K/sec with headroom, route overflow to a durable secondary topic (**events.overflow**) with its own consumer group that drains during off-peak. Producer keeps producing. Primary consumer keeps a steady heartbeat. Overflow gets processed later. That's graceful degradation. Stripe and Shopify publish architecture posts on this exact pattern.

Answer 2

Why B is the trap answer:

This is what fools senior engineers. Backpressure-as-blocking works in Reactor, RxJava, Akka — because the protocol carries the signal. Kafka is pull-based and decoupled. The producer doesn't know your consumer exists. There's no socket-level backpressure signal flowing upstream. Engineers from gRPC streaming or RxJava reach for B because it worked there. In a brokered, decoupled, log-based system — it doesn't apply.

Answer 3

Why A is wrong:

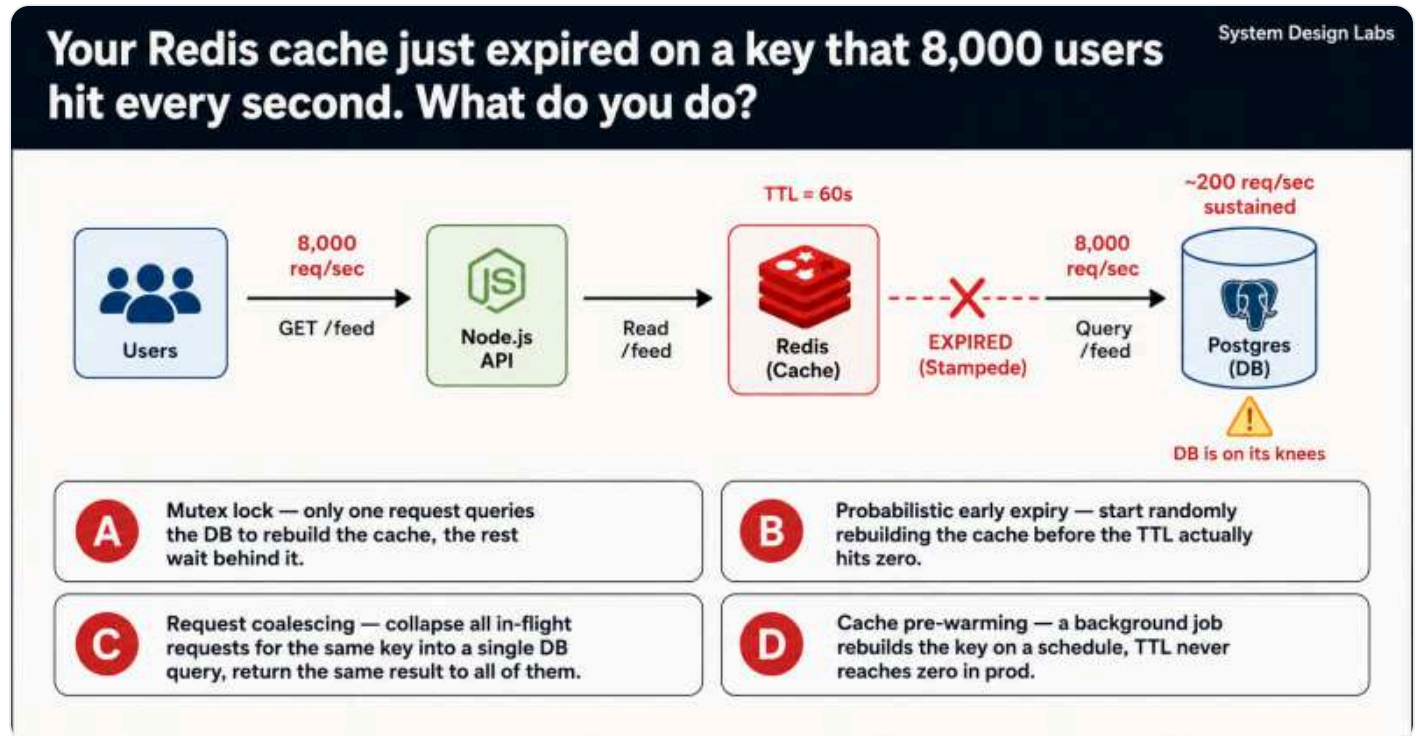
Drops events. SLA says "must be processed, not silently dropped." Done.

Answer 4

Why C is wrong:

Buffering is a delay tactic. A bigger buffer absorbs a 30-second spike — not a sustained 5K/sec input on an 800/sec consumer. The math: you're 4,200 events behind per second. A 1M-event buffer buys ~4 minutes. Then you're back where you started, but now with a memory exhaustion problem. And the real bottleneck is the external HTTP call — scaling consumers 5x doesn't help if downstream rate-limits you at request #51.

Surviving a Cache Stampede



Your Redis cache just expired on a key that 8,000 users hit every second.

Every single one of those requests is now flying straight at your database.

This is the thundering herd. You didn't have a traffic problem — you had a cache problem. Now you have both.

Here's the setup:

Service → Node.js API, 8,000 req/sec on the /feed endpoint

Cache → Redis, TTL = 60s on the feed key

DB → Postgres, comfortable at ~200 req/sec sustained

What happened → TTL expired at peak traffic, all 8,000 req/sec hit Postgres simultaneously

The DB is on its knees. You have minutes before it falls over. And the next TTL expiry is in 60 seconds.

What do you do?

A Mutex lock — only one request queries the DB to rebuild the cache, the rest wait behind it.

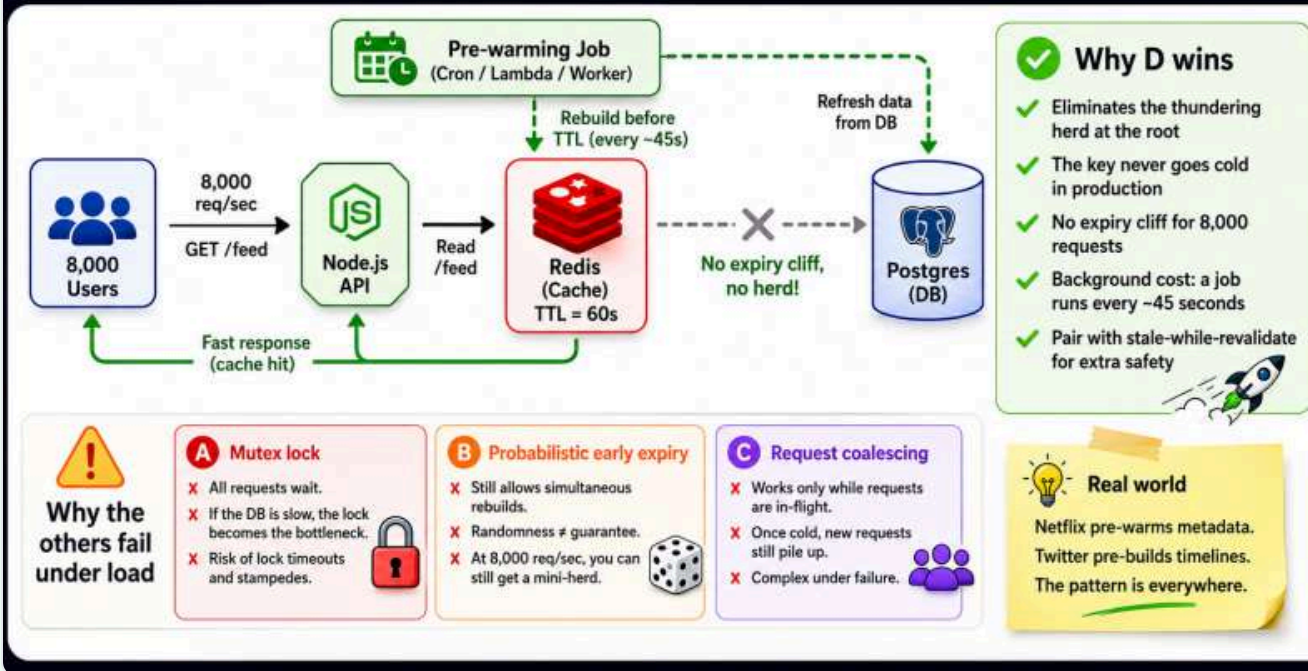
- B** Probabilistic early expiry — start randomly rebuilding the cache before the TTL actually hits zero.
- C** Request coalescing — collapse all in-flight requests for the same key into a single DB query, return the same result to all of them.
- D** Cache pre-warming — a background job rebuilds the key on a schedule, TTL never reaches zero in prod.

Answer 1

✓ CORRECT ANSWER

D Answer: D — Cache pre-warming ✓Pre-warm before TTL expires. **No cold key, no herd.**

System Design Labs

**Answer: D — Cache pre-warming** ✓

Here's why, and why the other three trick smart engineers:

Why D wins (Cache pre-warming):

Pre-warming eliminates the thundering herd at the root. A background job — a cron, a scheduled Lambda, a Sidekiq worker — rebuilds the cache key on a fixed schedule that's shorter than the TTL. The key never goes cold in production. There's no expiry cliff for 8,000 requests to fall off.

You know the data is hot. You know when it expires. You have the compute to rebuild it proactively. The cost is a background job running every ~45 seconds; the benefit is your database never sees the spike. Netflix pre-warms content metadata. Twitter pre-builds timelines for high-follower accounts. The pattern is everywhere — it's just not glamorous enough to make it into most architecture posts.

One detail that matters: pair it with stale-while-revalidate. Serve the stale value while the background job refreshes, so a slightly-late rebuild never causes a miss.

Answer 2

Why C is the trap answer (Request coalescing):

Coalescing sounds most sophisticated — collapse all concurrent requests for the same key into one DB query, return the same result to all of them. No stampede, no lock contention. Elegant.

Here's where it breaks: coalescing requires in-process coordination. It works beautifully on a single server. On 50 Node.js instances behind a load balancer, each instance coalesces independently — so now you have 50 concurrent DB queries instead of 8,000. Better, but still a 50x spike at every TTL expiry. To coalesce across instances you need a distributed coordination layer, and now you've built something nearly as complex as a mutex without the simplicity.

Right answer for a single-process server. Partial solution at scale that engineers convince themselves is complete.

Answer 3

Why A is wrong (Mutex lock):

Mutex looks clean: one request gets the lock, rebuilds the cache, everyone else waits. No duplicate DB queries.

The problem is "everyone else waits." Under 8,000 req/sec, waiting means API threads blocked, request queue filling up, p99 latency spiking to seconds. You've traded a DB overload for an application-layer backup. The database survives; the user experience dies. And if the lock holder is slow — DB is struggling, rebuild takes 500ms — every request in the system stalls for that window.

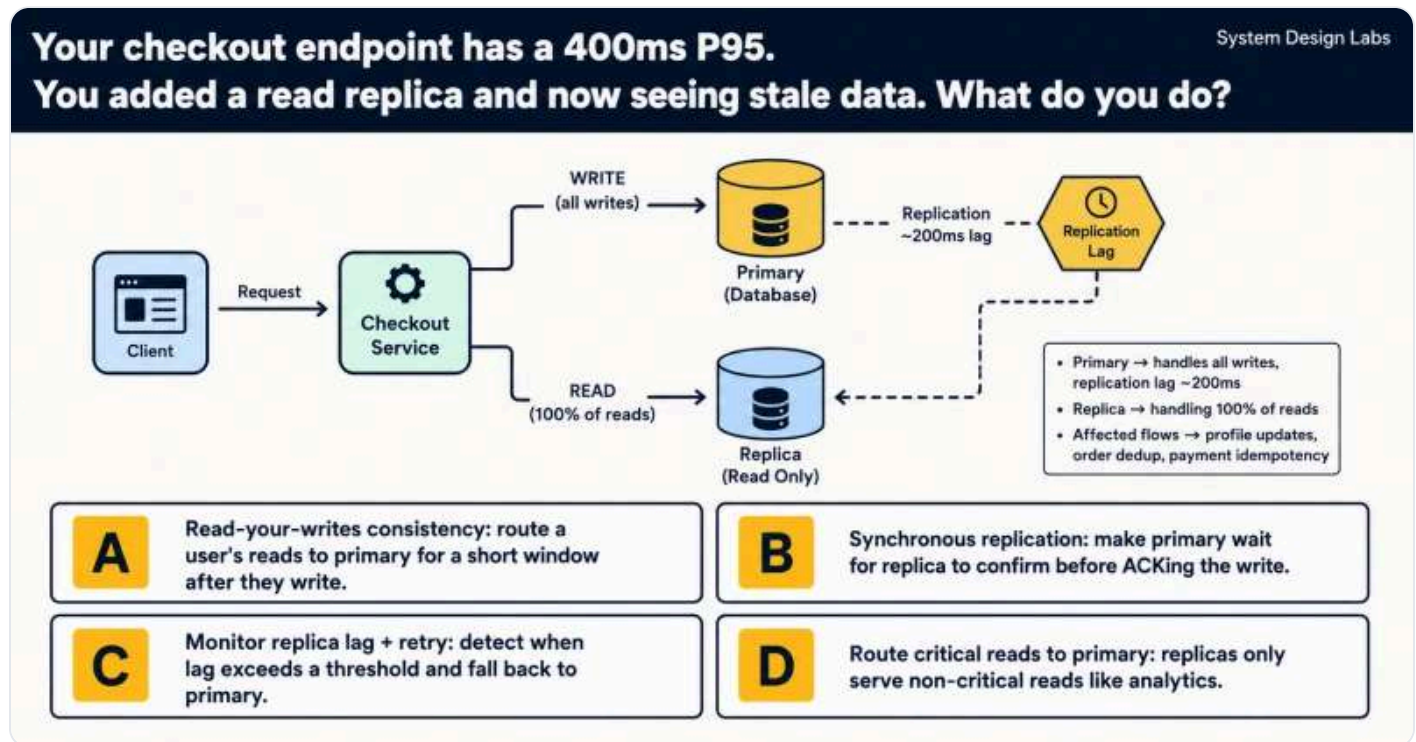
Answer 4

Why B is wrong (Probabilistic early expiry):

XFetch / jitter-based expiry is clever: as the TTL counts down, each cache read has a small random chance of triggering a rebuild early. No lock, no coordination layer. Elegant math.

The problem is it's probabilistic, not guaranteed. Under the exact wrong timing (low traffic right before TTL, then a sudden spike) the key still expires cold. You've reduced the probability of a thundering herd without eliminating it. For an endpoint running at 8,000 req/sec 24/7, you want a deterministic guarantee — not good odds.

Read-Your-Writes with Read Replicas



Your checkout endpoint has a 400ms P95. Profiling shows 70% of that is DB reads.

You add a read replica and point all SELECT queries at it. P95 drops to 90ms. The team celebrates.

Two hours later, support tickets flood in. Customers update their shipping address but see the old one on the confirmation screen. One customer gets charged twice because the "order already exists" check read stale data and missed the duplicate.

Here's the setup:

- Primary → handles all writes, replication lag ~200ms
- Replica → handling 100% of reads
- Affected flows → profile updates, order dedup, payment idempotency

The replica is working exactly as designed. That's the problem.

What do you do?

- A** Read-your-writes consistency: route a user's reads to primary for a short window after they write.

B Synchronous replication: make primary wait for replica to confirm before ACKing the write.

C Monitor replica lag + retry: detect when lag exceeds a threshold and fall back to primary.

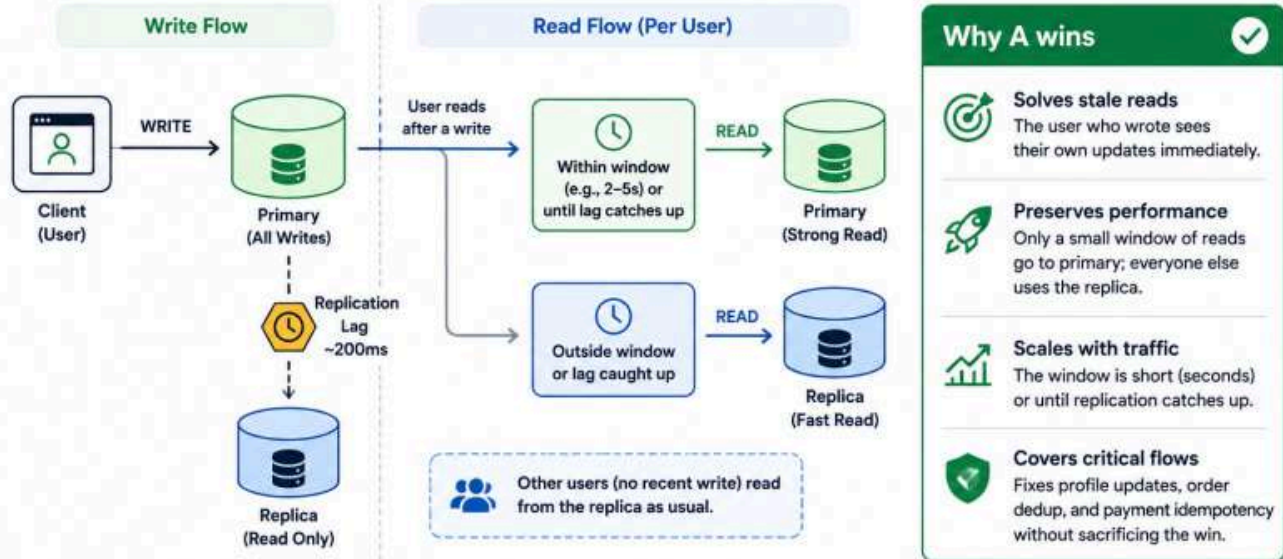
D Route critical reads to primary: replicas only serve non-critical reads like analytics.

Answer 1

✓ CORRECT ANSWER

ANSWER: A — Read-your-writes consistency:
route a user's reads to primary for a short window after they write.

System Design Labs



Key idea: After a user performs a write, their subsequent reads are routed to the primary for a short window: typically a few seconds, or until replica lag catches up. Everyone else reads from the replica. The performance win is preserved for the vast majority of traffic.

Why A wins:

After a user performs a write, their subsequent reads are routed to the primary for a short window: typically a few seconds, or until replica lag catches up. Everyone else reads from the replica. The performance win is preserved for the vast majority of traffic.

Answer 2

Why D is the trap:

"Critical" is not a stable category. The primary read list quietly shrinks, stale read bugs come back, and you spend the next quarter playing whack-a-mole with consistency bugs. You've traded a systematic solution for a per-feature judgment call.

Answer 3

Why B is wrong:

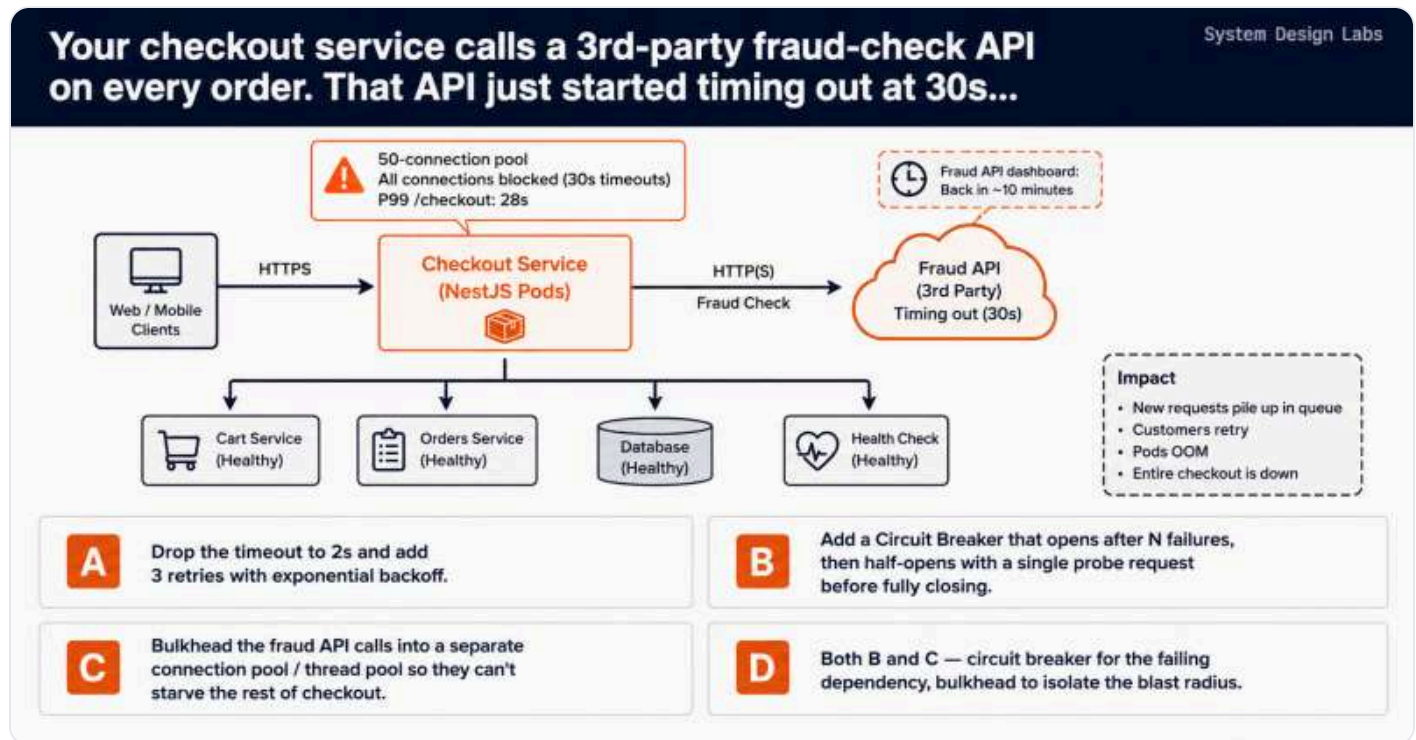
Synchronous replication means the primary waits for the replica to confirm every write. Replica lag goes to zero, but your write P95 goes from 20ms to 80–120ms. Worse, you've coupled your primary's availability to your replica's health.

Answer 4

Why C is wrong:

Average lag isn't your problem. Your problem is that a user reads their own write within 200ms of making it. The lag metric looks fine (200ms is "normal"), but the data the user needs is still in transit.

Containing a Failing Downstream Dependency



Your checkout service calls a 3rd-party fraud-check API on every order.

That API just started timing out at 30s instead of its usual 200ms.

Your Node.js checkout pods have a 50-connection pool. Within 90 seconds, every connection is parked waiting on the fraud API. New checkout requests pile up in the queue. P99 latency on /checkout goes from 300ms to 28s. Customers retry. Pods OOM. The fraud API is degraded — your entire checkout is down.

Here's the setup:

- Checkout (NestJS) → Fraud API (3rd party) — 30s timeouts
- Same pods also handle /cart, /orders, /health — all healthy dependencies
- Fraud API's own dashboard says it'll be back in ~10 minutes
- Your SLO budget for the quarter is about to evaporate

You need to stop the bleeding without losing the rest of checkout. What do you do?

A Drop the timeout to 2s and add 3 retries with exponential backoff.

B Add a Circuit Breaker that opens after N failures, then half-opens with a single probe request before fully closing.

C Bulkhead the fraud API calls into a separate connection pool / thread pool so they can't starve the rest of checkout.

D Both B and C — circuit breaker for the failing dependency, bulkhead to isolate the blast radius.

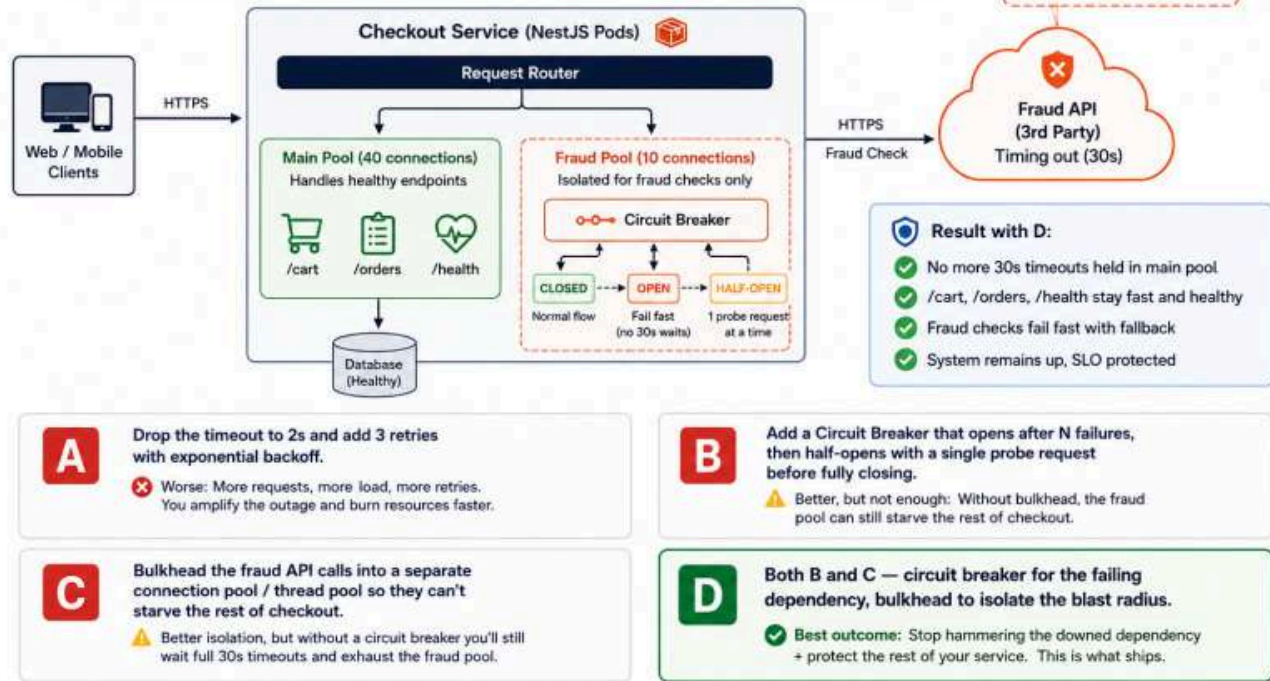
Answer 1 ✓ CORRECT ANSWER

Correct Answer: D — Circuit Breaker + Bulkhead, together ✓

System Design Labs



Why D wins: Circuit Breaker stops the bleeding. Bulkhead contains the blast radius. Together, they keep the rest of checkout healthy while the fraud API is degraded.



Answer: D — Circuit Breaker + Bulkhead, together ✓

Here's why, and why the other three trick smart engineers:

Why D wins (Circuit Breaker + Bulkhead):

These two patterns solve two *different* failure modes, and you need both.

The **Circuit Breaker** stops you from hammering a dead dependency. After N consecutive failures (or a failure rate threshold over a rolling window), it flips to OPEN — every subsequent call to the fraud API fails instantly with a fallback. No 30s wait. No connection held hostage. After a cooldown (say 30s), it goes HALF-OPEN: it allows exactly *one* probe request through. If that probe succeeds, the breaker closes and traffic resumes. If it fails, back to OPEN for another cooldown. Half-open is the part most tutorials gloss over — it's what prevents the thundering herd from re-killing a service that's just coming back up.

The **Bulkhead** is the part most engineers forget exists until they get burned. It isolates resource pools. Your fraud API gets its own dedicated pool of, say, 10 connections — separate from the 40 connections that serve /cart, /orders, /health. When the fraud API hangs, it can saturate its 10 connections, but

the other 40 stay free. /cart keeps working. /health keeps working. The blast radius stops at the fraud feature. The ship doesn't sink because one compartment flooded — that's literally where the name comes from.

Resilience4j, Polly (.NET), and Hystrix (legacy) all ship both. AWS App Mesh + Envoy give you bulkheads at the proxy layer. Netflix wrote Hystrix specifically because they learned this lesson the hard way: a single slow dependency cascading through shared thread pools took down recommendations during peak hours.

Answer 2

Why B alone is the staff-engineer trap answer:

A circuit breaker without a bulkhead is the answer that *sounds* complete in an interview. It's the most-mentioned pattern in resilience talks. And it does fix the specific failure described — once the breaker opens, fraud calls fail fast and stop tying up connections.

But here's the production reality: between failure #1 and the breaker tripping (failure #N), every one of those N requests is still holding a connection from the shared pool. If your threshold is 20 failures over 10 seconds, you can still saturate a 50-connection pool before the breaker even notices. You've reduced the outage window, not eliminated the cascade.

Bulkheads make the breaker's threshold *latency-tolerant*. They give you slack to detect and react without the rest of the system already being on fire.

This is the trap because B is right *and* incomplete. Senior engineers stop at B in design reviews because it sounds sufficient. Staff engineers ship D because they've seen B fail in production at 3am.

Answer 3

Why C alone is partial:

Bulkhead alone is better than nothing — it contains the damage. But your 10 dedicated fraud connections will still spend 30 seconds each waiting on doomed calls. You're paying full latency cost on every checkout, just on a smaller pool. Customers still see slow checkouts; you just don't lose /cart with them.

Bulkhead without a breaker = "the leak is contained, but the room is still flooding."

Answer 4

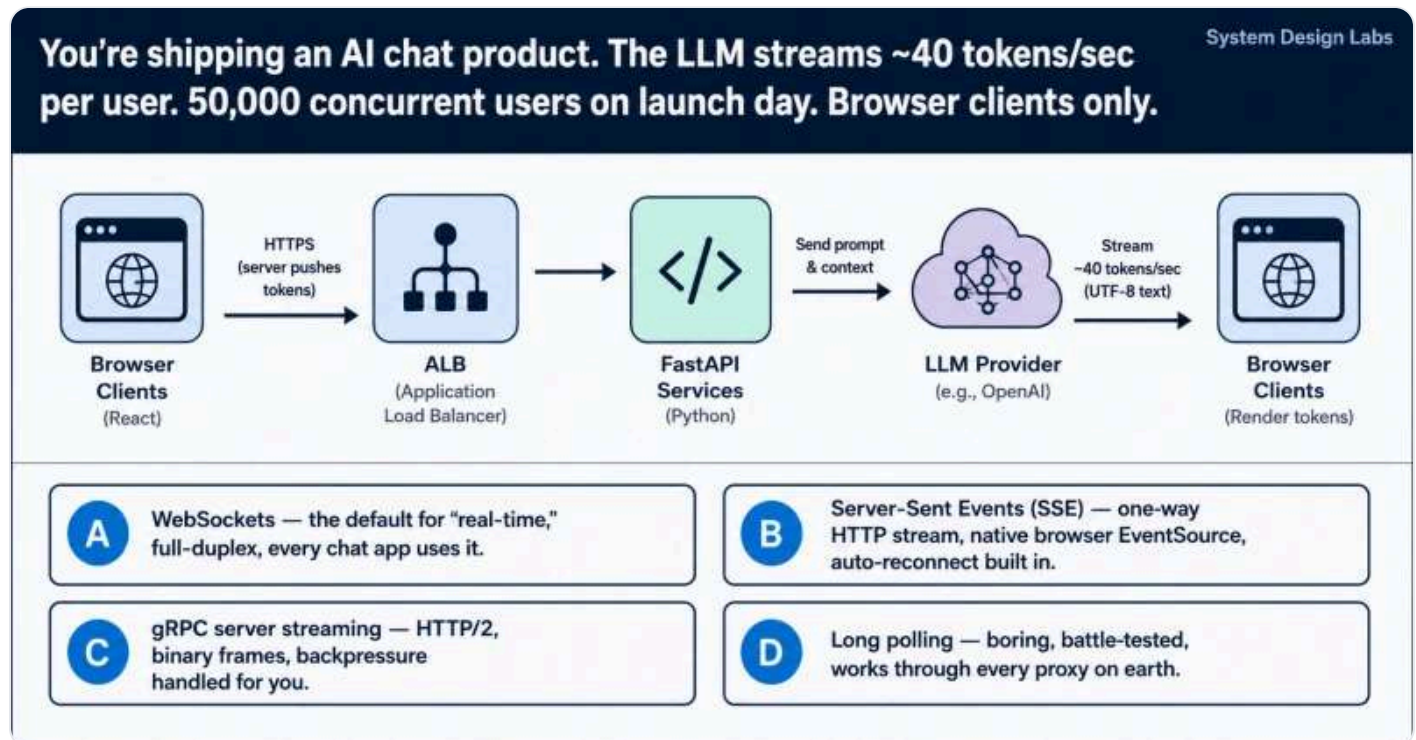
Why A is wrong (and dangerous):

Lower timeout + retries with backoff is the single worst thing you can do to a degraded dependency. It's the canonical anti-pattern.

The fraud API is already struggling. You just decided to send it 3x the traffic. Every retry storm from every checkout pod converges on a service that needs *less* load to recover. This is how a partial outage becomes a total outage — and how a 10-minute incident becomes a 4-hour incident. Retries belong on transient errors (a single 503, a connection reset), not on systemic degradation. And they always need a circuit breaker upstream of them to cap the blast.

If you've ever read a postmortem with the phrase "retry storm contributed to extended recovery time" — this is what they meant.

Choosing a Real-Time Streaming Transport



You're shipping an AI chat product. The LLM streams ~40 tokens/sec per user.

50,000 concurrent users on launch day. Browser clients only. Tokens flow one way: server → user.

Your team meets to pick the transport. Everyone shows up with a strong opinion.

Here's the setup:

- Frontend: React in the browser
- Backend: Python (FastAPI) behind an ALB
- Payload: UTF-8 text tokens, ~5–20 bytes each
- Direction: server pushes, client just renders
- Reconnects must be invisible (mobile networks drop constantly)

The team lead says "WebSockets, obviously." The platform engineer pushes back. What do you ship?

A WebSockets — the default for “real-time,” full-duplex, every chat app uses it.

B Server-Sent Events (SSE) — one-way HTTP stream, native browser EventSource, auto-reconnect built in.

C gRPC server streaming — HTTP/2, binary frames, backpressure handled for you.

D Long polling — boring, battle-tested, works through every proxy on earth.

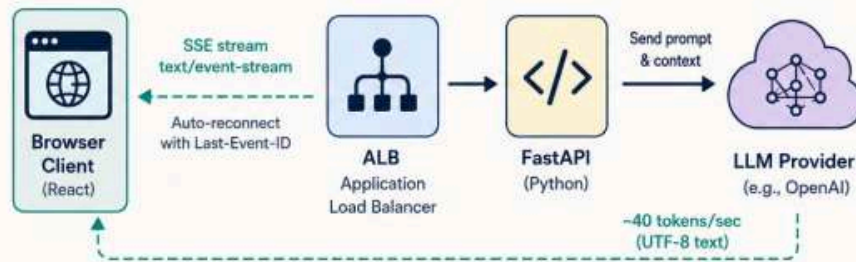
Answer 1

✓ CORRECT ANSWER

RIGHT ANSWER: B) SSE

System Design Labs

Server-Sent Events (SSE) — one-way HTTP stream, native browser EventSource, auto-reconnect built in.



- ✓ **One-way traffic.** Server pushes tokens, client renders. Perfect match for SSE.
- 🔄 **Built-in auto-reconnect.** EventSource reconnects and sends Last-Event-ID so the server can resume from there.
- 🛡️ **Simpler and more reliable.** No protocol upgrade. No new framing. No new auth path. Less code. Fewer bugs.

A **WebSockets**
the default for "real-time," full-duplex, every chat app uses it.

- ✗ Overkill for one-way traffic
- ✗ Manual reconnect & resume logic
- ✗ Higher infra and operational cost

B **Server-Sent Events (SSE)**
one-way HTTP stream, native browser EventSource, auto-reconnect built in.

- ✓ Built for one-way server push
- ✓ Automatic reconnect with Last-Event-ID
- ✓ Simple, efficient, and reliable

C **gRPC server streaming**
HTTP/2, binary frames, backpressure handled for you.

- ✗ Browser support is poor
- ✗ Extra complexity for text tokens
- ✗ Harder to operate at scale

D **Long polling**
boring, battle-tested, works through every proxy on earth.

- ✗ High overhead and latency
- ✗ Inefficient at scale
- ✗ Not real-time enough

Why B wins (SSE):

The traffic is one-way. Server pushes tokens, client renders. That's it. The client never sends a token back mid-stream — user input goes through a separate POST. So you're paying for full-duplex on a half-duplex problem.

SSE is built for exactly this shape. One long-lived HTTP connection, text/event-stream, server writes, browser's native EventSource reads. No protocol upgrade. No new framing. No new auth path.

The killer feature: **automatic reconnect with Last-Event-ID**. Browser drops, mobile switches from WiFi to LTE — EventSource reconnects on its own and tells the server the last event it saw. You replay from there. With WebSockets, you write that logic yourself, and you write it wrong the first three times.

OpenAI's streaming API? SSE. Anthropic's streaming API? SSE.

Answer 2

Why A is the trap (WebSockets):

The word "chat" tricks people. User-to-user chat (Slack, WhatsApp) is bidirectional. An AI chat app is a unidirectional token stream with a separate POST for the prompt. Different shape, different transport.

The cost at 50K connections:

- Sticky sessions on the ALB
- Hand-rolled reconnect + replay logic
- Heartbeats and ping/pong to detect dead connections
- Higher per-connection memory for buffers you'll never use

Every one of those turns into an on-call page.

Answer 3

Why C is wrong (gRPC streaming):

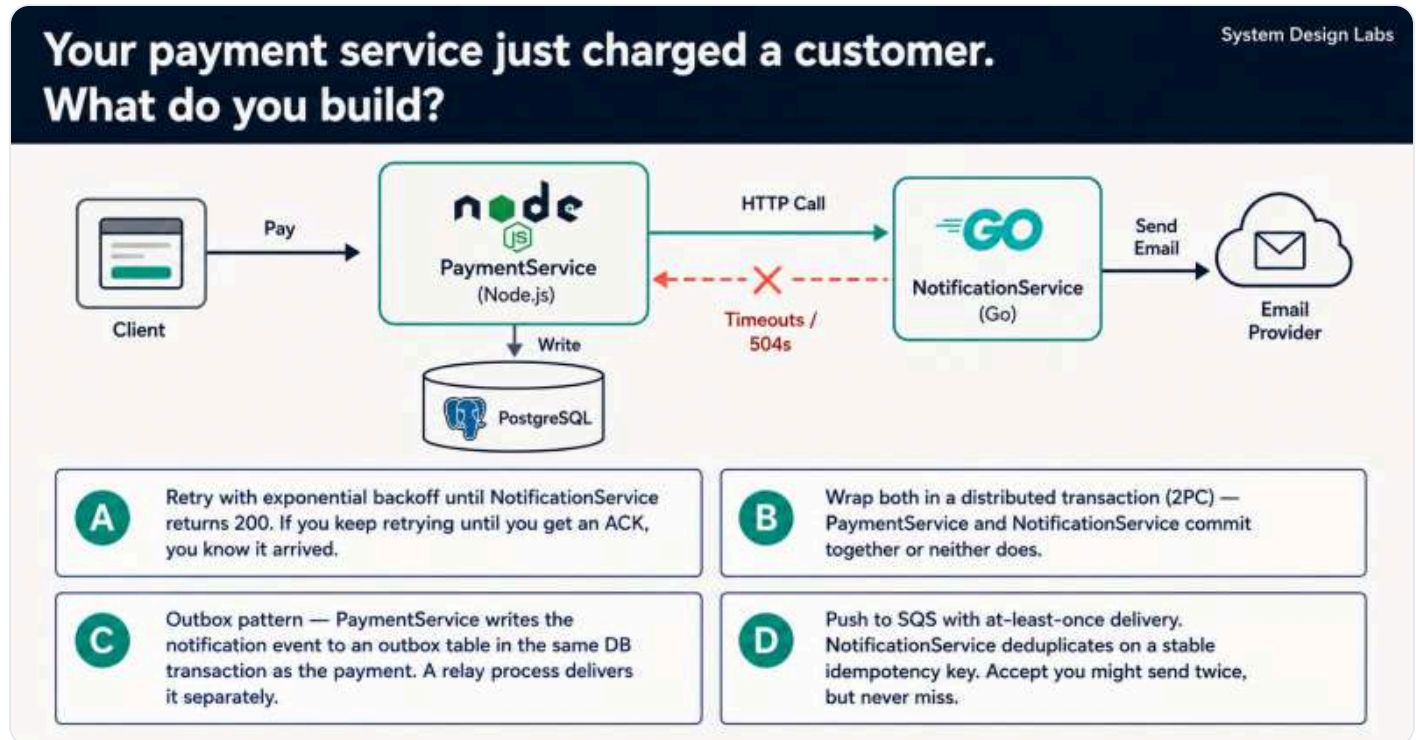
Browsers don't speak gRPC. You'd need Envoy + gRPC-Web, which downgrades the streaming model and adds a proxy hop. You're now operating Envoy and debugging Protobuf frames in the network tab — to ship UTF-8 tokens. Great for service-to-service. Wrong tool for browsers.

Answer 4

Why D is wrong (Long polling):

At 40 tokens/sec, every poll cycle burns a full HTTP request. $50K \text{ users} \times 40 \text{ req/sec} = \text{2M RPS just to render text}$. It's a valid fallback when SSE/WS are blocked. It's not your primary transport in 2026.

Reliable Messaging Across Services



Your payment service just charged a customer.

It writes to the DB. Now it needs to tell the notification service: "Send the confirmation email."

The HTTP call times out. Did it arrive? You don't know. You retry. Customer gets two emails.

You've just hit the Two Generals Problem. It's not a bug. It's a proof.

No protocol over an unreliable channel can guarantee both sides agree on the final message. Not HTTP. Not TCP. Not your retry loop. The uncertainty is mathematically irreducible.

Here's the setup:

PaymentService (Node.js, PostgreSQL) → NotificationService (Go)

~40ms p99 latency, occasional 504s under load.

You need to send exactly one confirmation email per payment — no double-sends, no missed sends.

What do you build?

A) Retry with exponential backoff until NotificationService returns 200. If you keep retrying until you get an ACK, you know it arrived.

B) Wrap both in a distributed transaction (2PC) — PaymentService and NotificationService commit together or neither does.

C) Outbox pattern — PaymentService writes the notification event to an outbox table in the same DB transaction as the payment. A relay process delivers it separately.

D) Push to SQS with at-least-once delivery. NotificationService deduplicates on a stable idempotency key. Accept you might send twice, but never miss.

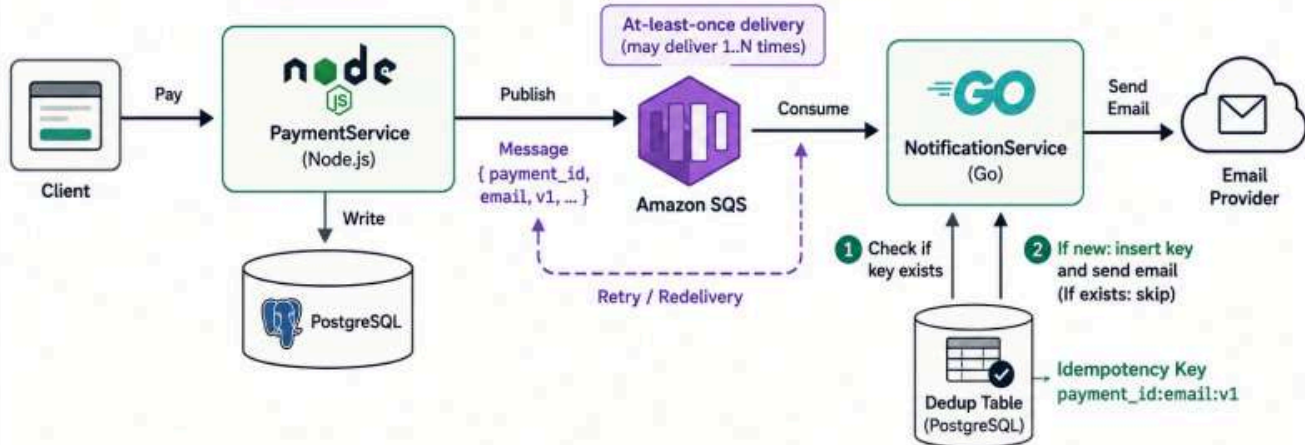
Answer 1

✓ CORRECT ANSWER

D

Push to SQS with at-least-once delivery.
NotificationService deduplicates on a stable idempotency key.
Accept you might send twice, but never miss.

✓ CORRECT

**Accept the reality**

Network + systems are unreliable. Messages will be delivered — maybe more than once.

**Queue absorbs uncertainty**

SQS durably stores the message and guarantees at-least-once delivery.

**Idempotency absorbs duplicates**

Dedup table with key (payment_id:email:v1) makes replays a no-op.

**Outcome**

No missed sends.
No double-sends in practice.

★ This is how Stripe and AWS handle it: Queue + Idempotency = Exactly-once effects in practice.

D — SQS + at-least-once + idempotency key (CORRECT)

The only answer that accepts the impossibility and designs around it. At-least-once means the message will arrive — maybe twice. Idempotency on the consumer side (payment_id:email:v1 in a dedup table) makes the second delivery a no-op. No missed sends, no double-sends in practice. This is exactly how Stripe and AWS handle it. The queue absorbs the uncertainty. Idempotency absorbs the duplicates.

Answer 2**A — Retry until ACK (SENIOR ENGINEER TRAP)**

Feels airtight. It's not. What if the 200 response is what timed out — not the request?

NotificationService sent the email AND returned 200, but you never saw the response. So you retry. Second email sent. Now you need to confirm your ACK arrived too... which is the Two Generals recursion. No finite number of retries closes this loop. More handshakes = more latency and more failure surfaces, not more certainty.

Answer 3

B — 2PC (WRONG)

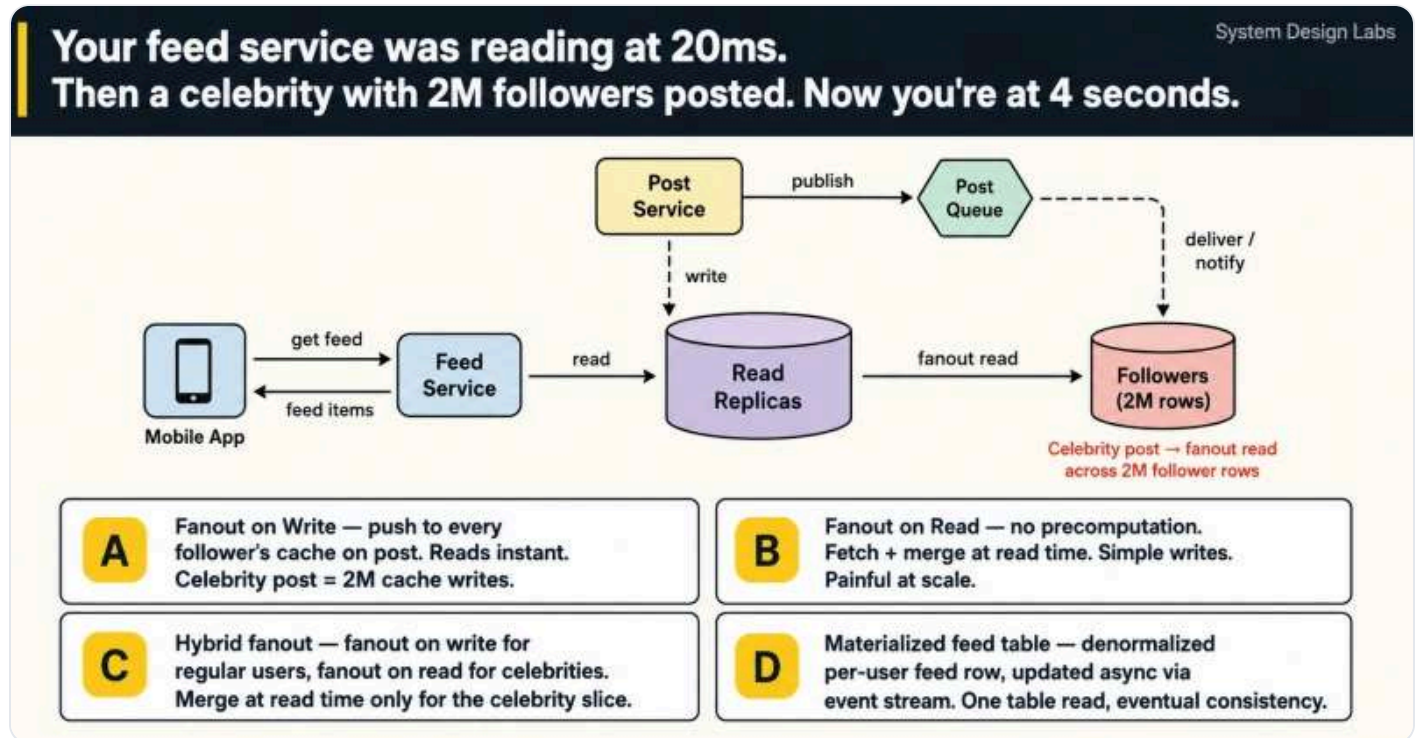
The coordinator is a single point of failure. If it crashes between Phase 1 (prepare) and Phase 2 (commit), both services are stuck with locks held, waiting for a decision that never comes. You've traded message uncertainty for coordinator-failure uncertainty — same problem, more complexity.

Answer 4

C — Outbox Pattern (CORRECT but heavier)

Actually solid — atomicity at the DB level, event and payment always in sync. But you're now running a relay process + CDC pipeline + outbox cleanup. For most teams with SQS already in place, D gets 95% of the same guarantees with a fraction of the infrastructure. C shines at scale or when strict ordering matters. D is the right answer for most teams today.

Feed Fanout for Celebrity Accounts



Your feed service was reading at 20ms.

Then a celebrity with 2M followers posted.

Now you're at 4 seconds. P99 is on fire.

Here's the setup:

10M users. ~50k posts/day. One account: 2M followers.

Every time that account posts, your feed query fans out across 2M follower rows, sorts by timestamp, and buries your read replicas.

Your team is debating how to fix the delivery model — not the query, the architecture.

A) Fanout on Write — push to every follower's cache on post. Reads instant. Celebrity post = 2M cache writes.

B) Fanout on Read — no precomputation. Fetch + merge at read time. Simple writes. Painful at scale.

C) Hybrid fanout — fanout on write for regular users, fanout on read for celebrities. Merge at read time only for the celebrity slice.

D) Materialized feed table — denormalized per-user feed row, updated async via event stream. One table read, eventual consistency.

Answer 1

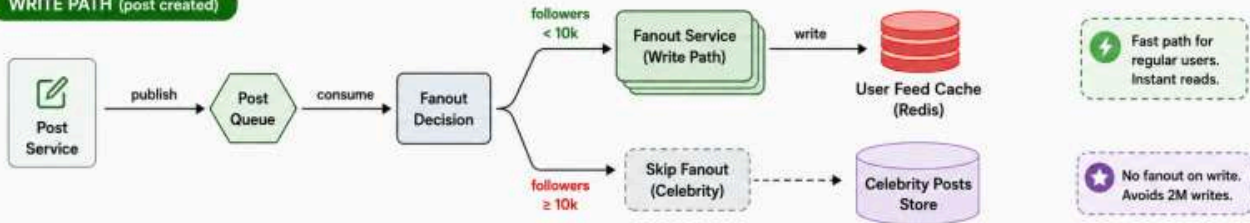
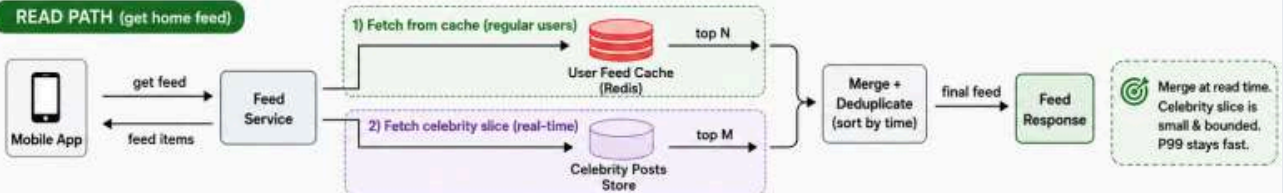
✓ CORRECT ANSWER

ANSWER: C — Hybrid fanout ✓

Fanout on write for regular users, fanout on read for celebrities.
Merge at read time only for the celebrity slice.

Why C wins

- ✓ Avoids 2M synchronous writes
- ✓ Keeps reads fast (P99)
- ✓ Bounded fanout at read time
- ✓ Used by Twitter at scale

WRITE PATH (post created)**READ PATH (get home feed)**

Hybrid is honest about both failure modes:
Fanout-on-write is great for most users.
Fanout-on-read is reserved for the few heavy hitters.



< 10k followers
Fanout on write
(instant reads)



≥ 10k followers
Fanout on read
(bounded cost)



Result
Fast P50 / P99
at scale

Answer: C — Hybrid fanout**Why C wins:**

This is how Twitter actually solved it — they published the architecture. The insight: fanout-on-write is fast for readers but catastrophic for high-follower accounts. Fanout-on-read is simple but destroys latency at scale. Hybrid is honest about both failure modes.

When a post is created → check follower count. Under 10k? Fan out immediately to all followers' Redis feed caches. Over 10k (celebrity)? Skip the fanout. At read time, fetch from two sources: precomputed cache (regular accounts) + real-time query for celebrity accounts. Merge + deduplicate. The celebrity slice is small and bounded. Result: P99 stays fast, no 2M synchronous writes on celebrity posts.

Answer 2

Why A is the trap (Fanout on Write):

Looks perfect. Reads are instant. Until Cristiano Ronaldo posts and you're writing to 500M caches in parallel. At 2M followers, even 1ms per write = 2,000 seconds of write work. Cache cluster saturates. Write queue backs up. Thundering herd on every celebrity post.

Teams implement this, it works great at 1M users, then hits the celebrity problem when it's already in production. That's the trap.

Answer 3

Why B is wrong (Fanout on Read):

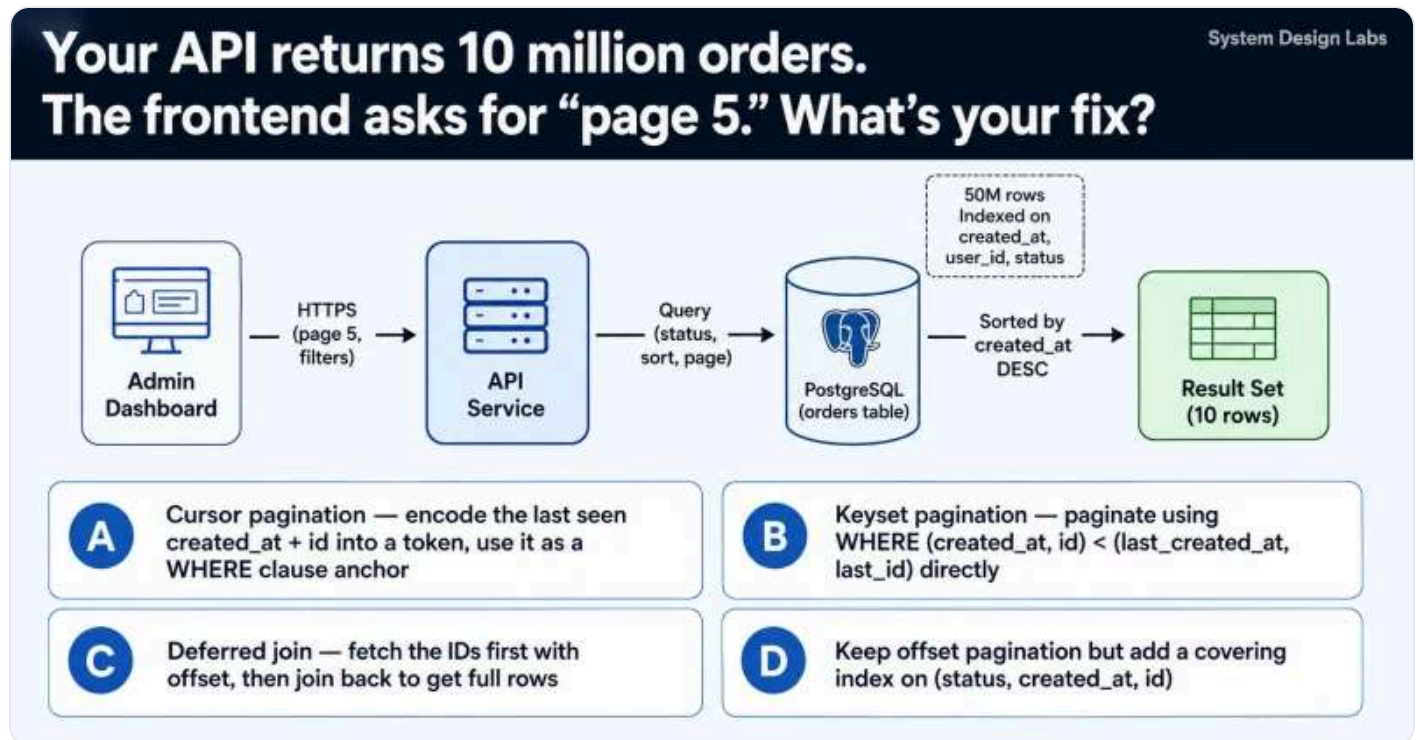
500 accounts followed = 500 queries (or one giant JOIN), sorted in memory, before returning anything. You can cache the result — but now you're doing expensive computation to fill a cache that expires in 30s. That's just a slow fanout-on-write with extra latency. Doesn't survive 10M users.

Answer 4

Why D is wrong (Materialized feed table):

Sounds clean — denormalized row per user, updated async via CDC/event stream. But a celebrity post still triggers 2M row updates, just asynchronously. You haven't solved write amplification, you've deferred it. Plus: operational complexity of an event stream + idempotent consumers + reconciliation logic. More moving parts for the same problem.

Paginating Large Result Sets Efficiently



Your API returns 10 million orders.

The frontend asks for "page 5."

Offset 40, limit 10. Simple enough — until your DBA messages you at 2 AM.

Query time: 4.2 seconds. Table scans climbing. Prod is lagging.

Here's the stack: PostgreSQL, 50M rows in orders, sorted by created_at DESC. Users can filter by status. You're paginating an admin dashboard used by 200 concurrent support agents.

Offset pagination worked fine at 10K rows. At 50M, it's reading and discarding 40 million rows just to return 10.

You need to fix this before the next sprint ships the customer-facing version — which will have 10x the traffic.

Here's your setup:

- Table: orders — 50M rows, indexed on created_at, user_id, status
- Query: sorted by created_at DESC, filtered by status
- Client: needs "previous/next" navigation + jump-to-page

- SLA: p99 < 200ms

Four engineers propose four different approaches:

- A** Cursor pagination — encode the last seen `created_at` + `id` into a token, use it as a `WHERE` clause anchor
- B** Keyset pagination — paginate using `WHERE (created_at, id) < (last_created_at, last_id)` directly
- C** Deferred join — fetch the IDs first with offset, then join back to get full rows
- D** Keep offset pagination but add a covering index on `(status, created_at, id)`

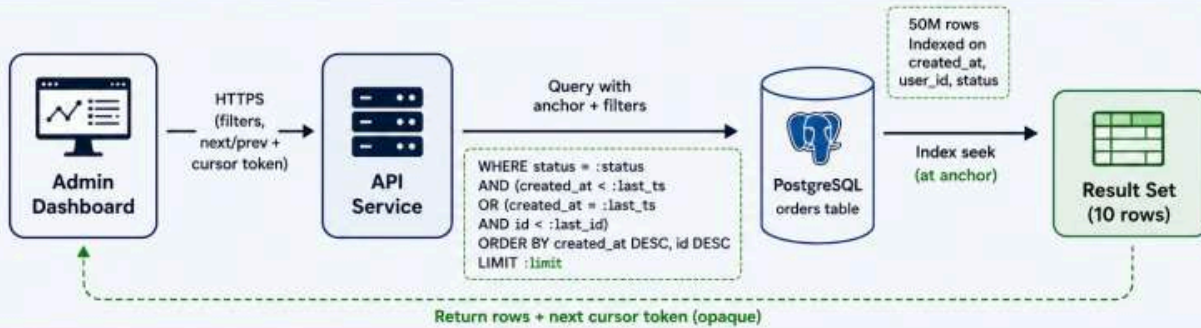
Answer 1

✓ CORRECT ANSWER

A**ANSWER: CURSOR PAGINATION**

Encode the last seen (created_at, id) into a token.
Use it as an anchor for the next query.

System Design Labs

**WHY THIS WINS HERE**

Query always hits the index at the anchor point — no scan, no discard.



p99 stays flat whether you're on page 1 or page 50,000.



Works with the status filter:

```
WHERE status = 'pending' AND (created_at, id) < (anchor)
```



The cursor is position-stable: new rows inserted before your cursor don't shift your page.

HOW THE CURSOR TOKEN WORKS

Last row in current page

```
created_at  
2024-05-01 10:15:30  
id  
879123456
```

Encode

Opaque token

```
eyJjcmVhadGVdX2F0IjozMjAy  
NC0wNS0wMVQxMDoxNToz  
MCIsImlkIjojODc5MTIzNDU2  
In0.K8xJ...
```

Decode

Used in next request

```
:last_ts  
2024-05-01 10:15:30  
:last_id  
879123456
```

**UX TRADEOFF**

No jump-to-page.
Only previous/next.
But this is the right tradeoff for the dashboard. Support agents navigate sequentially. Nobody needs "jump to page 3,847."

**THE BOTTOM LINE**

At 50M rows with concurrent filters, cursor pagination is the only approach that hits p99 < 200ms reliably.

p99 < 200ms**Option A — Cursor pagination ✓ CORRECT**

Encode the last seen (created_at, id) into an opaque token. On the next request, decode it and use `WHERE created_at < :last_ts OR (created_at = :last_ts AND id < :last_id)` as your anchor.

Why this wins here:

- Query always hits the index at the anchor point — no scan, no discard
- p99 stays flat whether you're on page 1 or page 50,000
- Works with the status filter: `WHERE status = 'pending' AND (created_at, id) < (anchor)`
- The cursor is position-stable: new rows inserted before your cursor don't shift your page

The UX tradeoff: no jump-to-page. Only previous/next. **But this is the right tradeoff for the dashboard.** Support agents navigate sequentially. Nobody needs "jump to page 3,847."

At 50M rows with concurrent filters, cursor pagination is the only approach that hits p99 < 200ms reliably.

Answer 2

Option B — Keyset pagination

Keyset looks identical to cursor on the surface. The difference is subtle: keyset is a *query pattern*, cursor is a *product decision*.

Keyset uses `WHERE (created_at, id) < (val1, val2)` as a SQL construct. Cursor wraps that into an encoded token the client can't inspect or manipulate.

Here's where it falls apart: **the status filter breaks row-tuple comparison in most query planners.** `WHERE status = 'pending' AND (created_at, id) < (anchor)` doesn't use a composite index efficiently in PostgreSQL when the equality column (status) isn't part of the tuple comparison. You end up with a partial index scan that degrades under high cardinality filters.

Cursor pagination handles this by encoding the full WHERE context, not just the positional anchor. That's the gap. It's subtle, and it's the thing that bites you in production three months after you ship it...

Answer 3

Option C — Deferred join

```
SELECT o.* FROM orders o

JOIN (

    SELECT id FROM orders

    WHERE status = 'pending'

    ORDER BY created_at DESC

    OFFSET 500000 LIMIT 10

) ids ON o.id = ids.id
```

This is a legitimate optimization for offset pagination. The inner query fetches only IDs (fits in index, much faster), then the outer join fetches full rows for just 10 records.

It's real. It works. Shopify uses a version of this.

But: you still pay $O(\text{offset})$ cost on the inner query at deep pages. It helps — it does not solve. At page 50,000, you're still scanning 500,000 index entries. You shaved 80% of the cost and kept 20% of the problem.

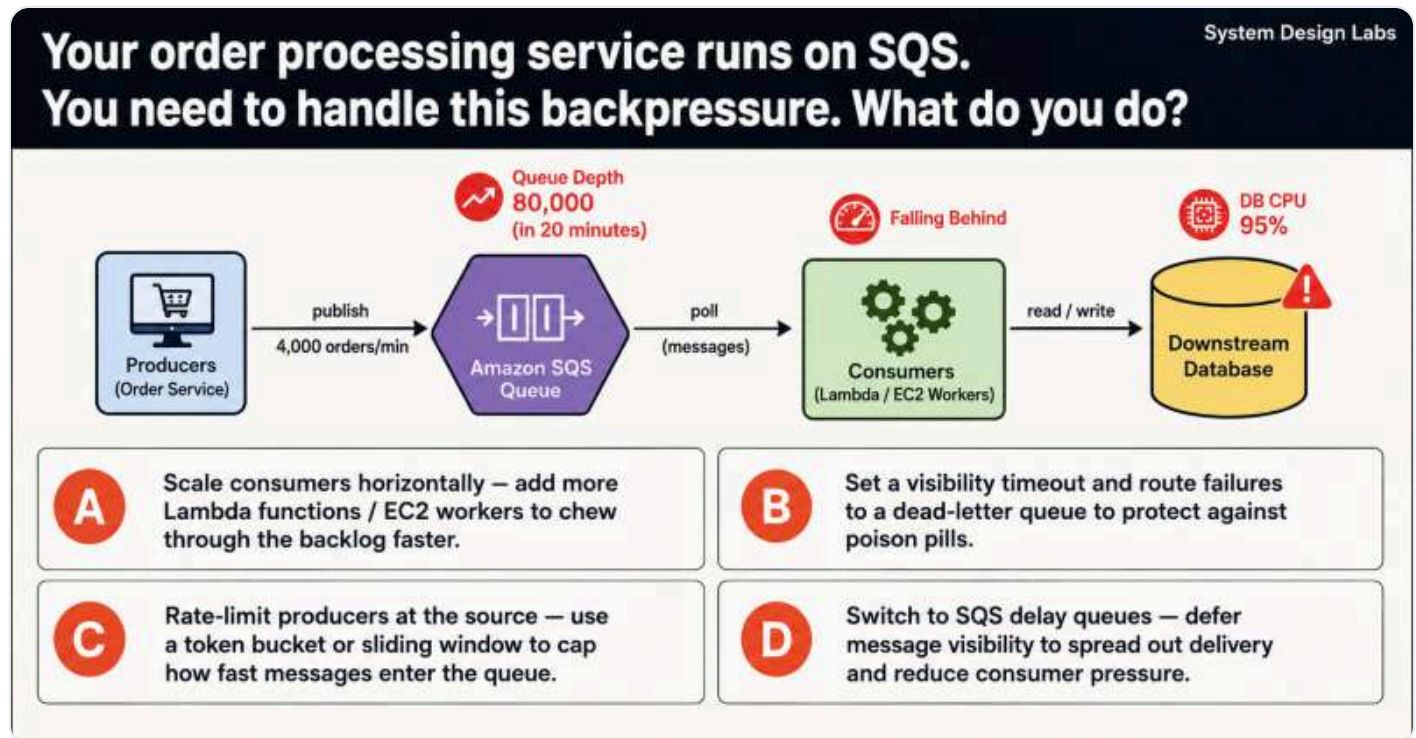
Answer 4

Option D — Covering index

Adding `CREATE INDEX ON orders(status, created_at DESC, id DESC)` is a good idea regardless of which approach you pick. It removes heap fetches for status-filtered queries.

But an index doesn't change the algorithmic complexity of `OFFSET`. It makes your table scans faster, not shorter. This is the "throw hardware at it" answer. It buys you time, not a solution.

Queue Backpressure Under a Traffic Spike



Your order processing service runs on SQS.

Normal load: 200 orders/min. Consumers keep up fine.

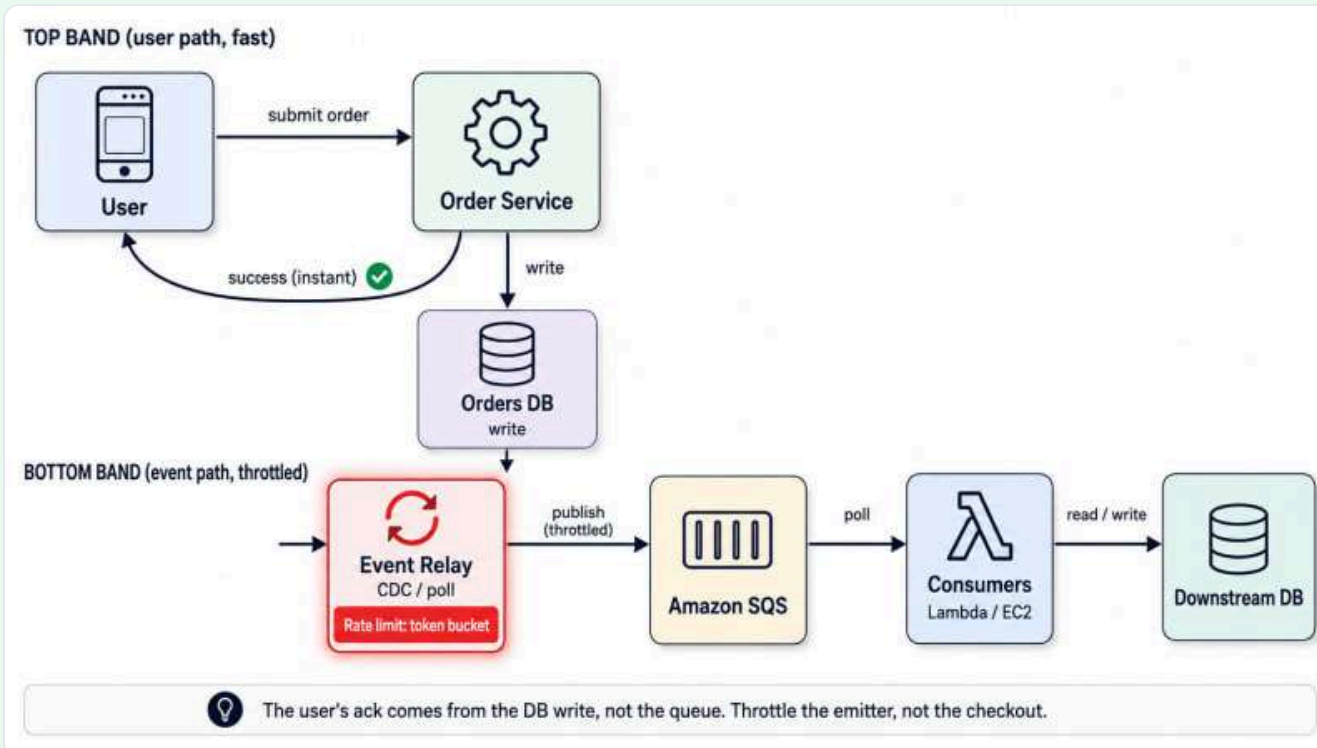
Then Black Friday hits. Producers start pushing 4,000 orders/min. Queue depth climbs to 80,000 messages in 20 minutes. Your downstream DB is at 95% CPU. Consumers are falling behind and you're watching the queue grow in real time.

You need to handle this backpressure. What do you do?

- A) Scale consumers horizontally — add more Lambda functions / EC2 workers to chew through the backlog faster.
- B) Set a visibility timeout and route failures to a dead-letter queue to protect against poison pills.
- C) Rate-limit producers at the source — use a token bucket or sliding window to cap how fast messages enter the queue.
- D) Switch to SQS delay queues — defer message visibility to spread out delivery and reduce consumer pressure.

Answer 1

✓ CORRECT ANSWER



Why C wins, and why it doesn't wreck UX

The queue grows because the producer is winning the race. 4,000 in, 200 out. You can scale consumers all day and that gap won't close on its own, especially with the DB already sitting at 95% CPU.

The fix is upstream. Rate-limit the producer with a token bucket or sliding window. In AWS that's API Gateway usage plans, Lambda reserved concurrency, or throttle middleware inside the service. Queue depth stabilizes, consumers catch up at their natural pace, and the DB stops cooking.

The mental model: slow the tap, don't just widen the drain.

Now the part the diagram glossed over: the "producer" isn't your user.

In most real systems the order flow is decoupled. A user submits an order, it's written to the DB, and they get their success response right away. A separate processor then reads those events (polling or CDC) and emits them to the queue. That emitter is the producer you throttle.

So rate-limiting never touches checkout. The user already has their ack from the write. You're capping how fast events flow into the queue, not how fast users can order. UX stays intact; the queue just stops outrunning your consumers.

That's what backpressure actually is: the downstream system telling the upstream to slow down. The queue depth is the signal. You react to it upstream.

Answer 2

Why A is the trap answer (Scale consumers):

This is the instinct answer, and it's close enough to fool senior engineers.

Yes, more consumers means more throughput. But you're treating a **rate mismatch** as a **capacity problem**. If your producer can always generate faster than your consumers can process — and on Black Friday it can — you're in an arms race you can't win. You spin up 10x Lambda functions, hit DynamoDB write throughput limits, hammer your RDS connection pool, and the queue still grows. Just slower, and now your infra bill doubled.

Consumer scaling works when your bottleneck is compute. It doesn't work when the problem is unbounded producer rate.

Answer 3

Why B is wrong (Visibility timeout + DLQ):

This is failure handling, not backpressure.

DLQ catches poison pills — messages that fail processing repeatedly and would otherwise loop forever. Visibility timeout controls how long a message stays invisible after a consumer picks it up (preventing double-processing). Neither of these slows the producer or reduces queue depth growth.

Mixing up backpressure with retry/error handling is one of the most common architecture confusions I see. They solve completely different problems.

Answer 4

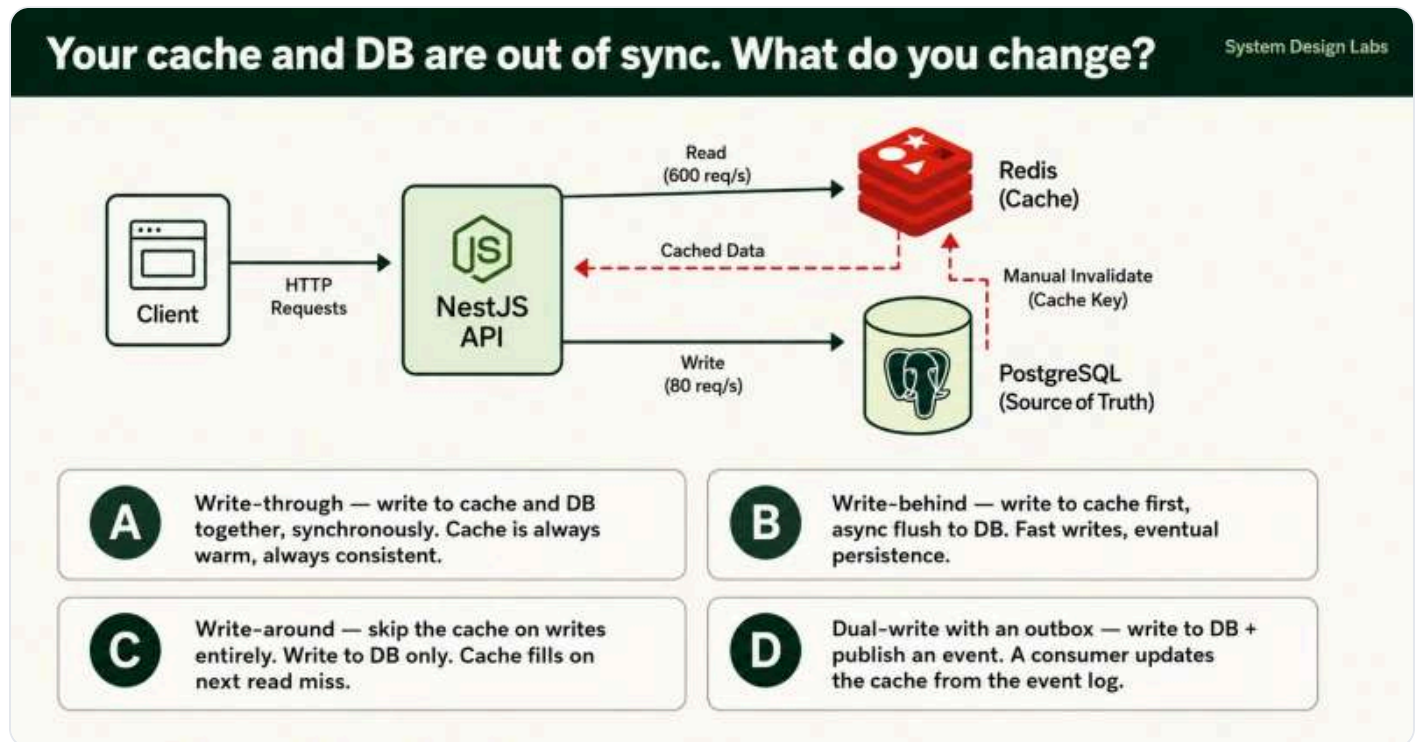
Why D is wrong (SQS delay queues):

Delay queues defer message *visibility*, not message *creation*. Your producer still pushes 4,000 msg/min. Messages pile up invisibly, then become visible in bursts when the

delay expires. You haven't reduced backpressure — you've deferred it and made the delivery pattern spikier.

This is like covering a flooding sink with a lid and calling it fixed. The water is still rising....

Write-Path Cache Consistency



Your cache and DB are out of sync. Again.

A user updates their profile. The cache still serves the old name for the next 10 minutes. Support gets a ticket. You patch it with a cache flush. It happens again next week.

You're asked to fix write consistency before it becomes a customer-facing incident.

Here's the setup:

NestJS API → PostgreSQL (source of truth) + Redis (cache)

~600 req/s reads, ~80 req/s writes at peak

Current pattern: write to DB, manually invalidate cache key on success

3 incidents this month — all traced back to stale cache after writes

You need a strategy that survives race conditions, retries, and partial failures

What do you change?

- A** **Write-through** — write to cache and DB together, synchronously. Cache is always warm, always consistent.

B Write-behind — write to cache first, async flush to DB. Fast writes, eventual persistence.

C Write-around — skip the cache on writes entirely. Write to DB only. Cache fills on next read miss.

D Dual-write with an outbox — write to DB + publish an event. A consumer updates the cache from the event log.

Answer 1

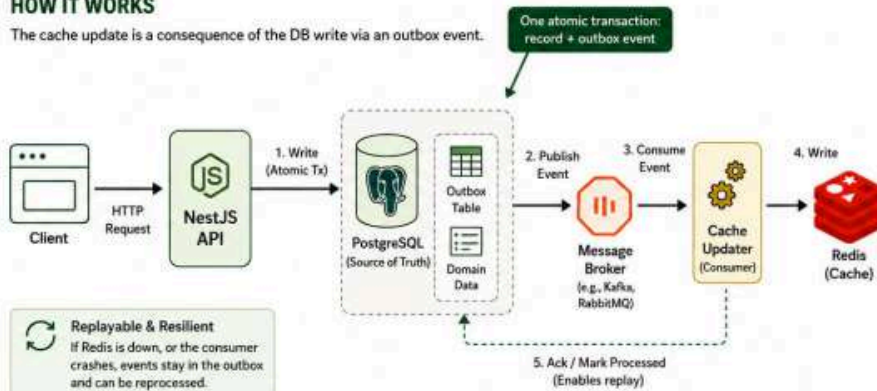
✓ CORRECT ANSWER

ANSWER: D — DUAL-WRITE WITH AN OUTBOX ✓

System Design Labs

HOW IT WORKS

The cache update is a consequence of the DB write via an outbox event.



Replayable & Resilient
If Redis is down, or the consumer crashes, events stay in the outbox and can be reprocessed.

WHY D WINS

- ✓ Eliminates the dual-write race condition: DB write and outbox event are in the same atomic transaction.
- ✓ The cache update becomes a consequence of the DB write, not a sibling operation.
- ✓ Survives crashes, retries, and partial failures.
- ✓ Provides replay: if Redis is down and comes back, re-process the outbox and catch up.

A WRITE-THROUGH

Write to cache and DB together, synchronously.



- ✗ Two synchronous I/Os on every write.
- ✗ If Redis is slow, your write API is slow.
- ✗ If Redis is down, do you block the user? You've made Redis a hard dependency of your write path.

B WRITE-BEHIND

Write to cache first, async flush to DB.



- ✗ Fast, but dangerous.
- ✗ If the async worker crashes before DB flush, that write is gone.
- ✗ Acceptable for analytics counters. Never acceptable for a source of truth.

C WRITE-AROUND

Skip the cache on writes. Write to DB only.



- ✗ Solves stale cache by not writing to cache at all.
- ✗ Read-after-write is broken under replica lag.
- ✗ High-write workloads tank your cache hit rate.

D DUAL-WRITE WITH AN OUTBOX

Write to DB + publish an event. A consumer updates the cache from the event log.



- ✓ DB write + outbox event are atomic.
- ✓ Cache update is a consequence of the DB write.
- ✓ Survives crashes, retries, and partial failures.
- ✓ Replayable and eventually consistent.

Answer: D — Dual-write with an outbox

The core problem isn't "which order to write in" — it's "what happens when one write succeeds and the other fails?"

Why D wins: Every naive dual-write has the same race condition: write to DB, write to cache, crash between the two → stale cache indefinitely. The outbox makes the cache update a *consequence* of the DB write, not a sibling operation. One atomic DB transaction: record + outbox event. A consumer updates Redis from the event. Also gives you replay — if Redis goes down and comes back, re-process the outbox.

Answer 2

Why A fails (Write-through): Looks safest. Is the most dangerous at scale. Two synchronous I/Os on every write. If Redis is slow, your write API is slow. If Redis is down, do you block the user? You've made Redis a hard dependency of your write path.

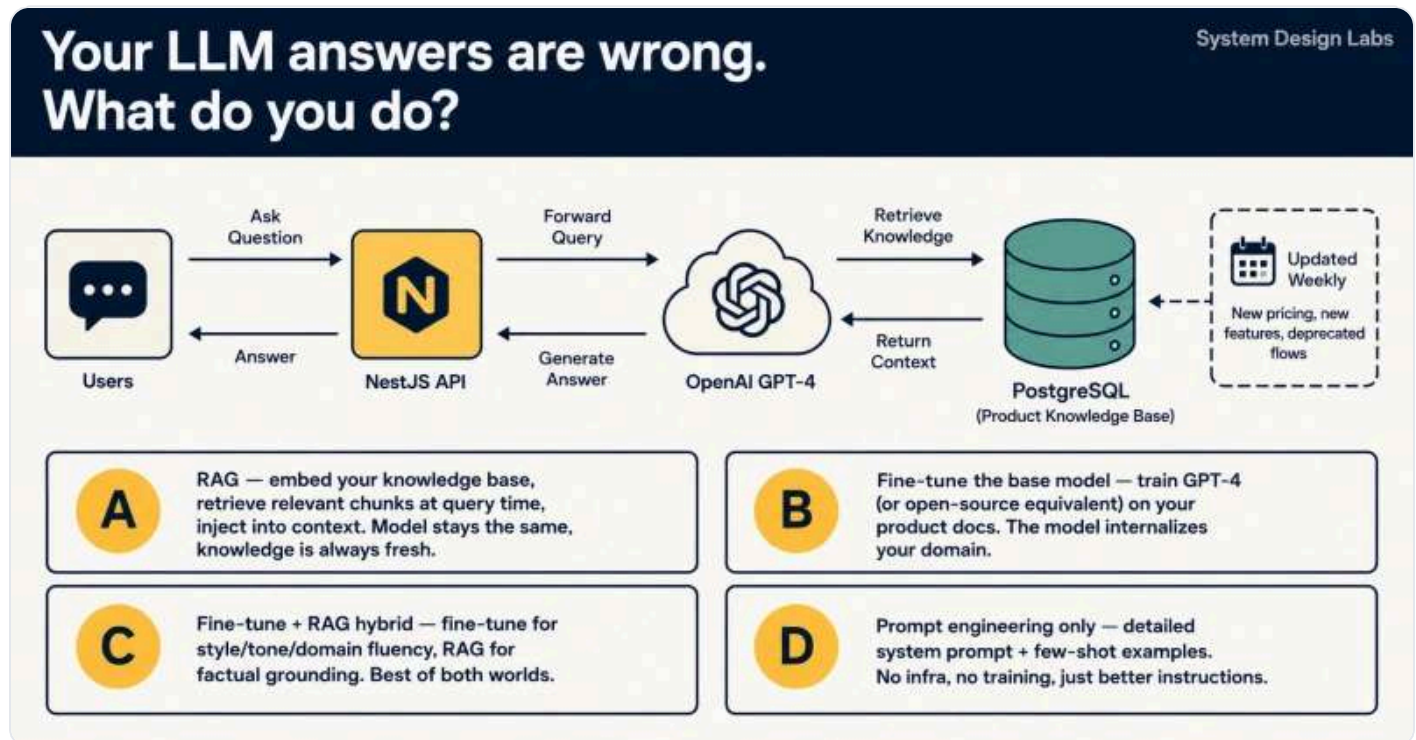
Answer 3

Why B fails (Write-behind): Fast, but: if the async worker crashes before DB flush, that write is gone. Acceptable for analytics counters. Never acceptable for a source of truth.

Answer 4

Why C is a partial fix (Write-around): Solves stale cache by not writing to cache at all. Clean — but read-after-write is broken under replica lag, and high-write workloads tank your cache hit rate.

Keeping an LLM's Answers Up to Date



Your LLM answers are wrong. Not hallucination-wrong — outdated-wrong.

You shipped a customer support bot on GPT-4. It's trained through early 2024. Your product changed 14 times since then. Every week, users get answers that were accurate 8 months ago and are flat-out wrong today.

The team is debating the fix.

Here's the setup:

NestJS API → OpenAI GPT-4 + PostgreSQL (product knowledge base)

~2,000 support queries/day, 15% return wrong answers tied to stale knowledge

Knowledge base updates weekly — new pricing, new features, deprecated flows

Budget: mid-size startup, not training custom models from scratch

You need accurate, up-to-date answers without re-training on every product update

What do you do?

A) RAG — embed your knowledge base, retrieve relevant chunks at query time, inject into context. Model stays the same, knowledge is always fresh.

- B)** Fine-tune the base model — train GPT-4 (or open-source equivalent) on your product docs. The model internalizes your domain.
- C)** Fine-tune + RAG hybrid — fine-tune for style/tone/domain fluency, RAG for factual grounding. Best of both worlds.
- D)** Prompt engineering only — detailed system prompt + few-shot examples. No infra, no training, just better instructions.

Answer 1

✓ CORRECT ANSWER

Answer: A — RAG

System Design Labs

Separate knowledge from reasoning. Keep the model frozen.**Solves the real problem**

The issue is freshness, not understanding. RAG gives the model up-to-date facts.

**No retraining cost**

Model stays frozen. No fine-tuning cycles, no redeployment.

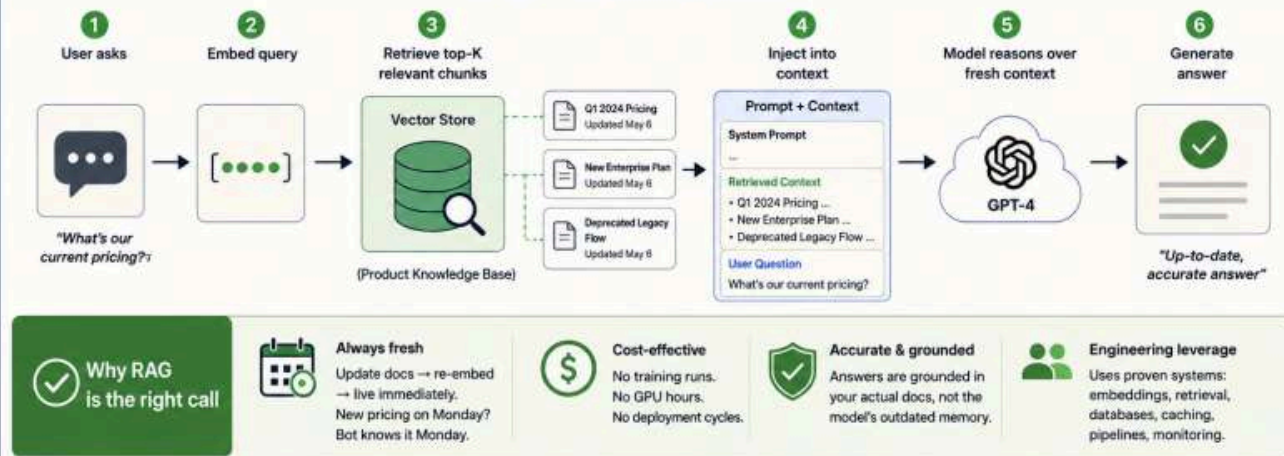
**Knowledge is decoupled**

Your knowledge lives in a vector store. Update, re-embed, done. Bot knows it immediately.

**Pipeline, not model problem**

You've turned this into a data pipeline problem — something every team already knows.

How RAG Works at Runtime

**Bottom line:** RAG gives you up-to-date, accurate answers without retraining.**Fix freshness at the data layer.** Let the model do what it does best: reason.**Answer: A — RAG**

The problem is freshness — the model doesn't know what changed, not that it doesn't understand your domain.

RAG separates knowledge from reasoning. The model stays frozen (no retraining cost, no deployment cycle). Your knowledge base lives in a vector store — update a doc, re-embed it, done. New pricing ships Monday, your bot knows it by Monday.

At runtime: user asks → embed query → retrieve top-K relevant chunks → inject into context → model reasons over fresh, accurate content.

You've turned a model problem into a data pipeline problem. Every engineering team already knows how to solve data pipelines.

Answer 2

B — Fine-tuning (the trap answer)

Feels like the obvious upgrade. Makes the problem worse.

Fine-tuning bakes knowledge into weights. Those weights are static until you retrain. Weekly product updates = weekly retraining at real cost (time + money) + real risk of catastrophic forgetting (new training can degrade recall of old content if not carefully managed).

Fine-tuning is the right answer for *behavioral* changes — tone, format, domain fluency, internal jargon. It teaches the model *how to behave*, not *what to know*. Wrong tool for a freshness problem.

Answer 3

C — Fine-tune + RAG hybrid (overkill)

Genuinely powerful architecture — but it's step 3, not step 1.

You can't productively fine-tune for style until your factual grounding is solid first. The hybrid is what you reach for *after* you've shipped RAG cleanly and hit the ceiling of what retrieval alone can do.

For a startup with a 15% wrong-answer rate due to stale knowledge, shipping a hybrid is a 3-month project when a 2-week RAG implementation fixes the actual problem.

Answer 4

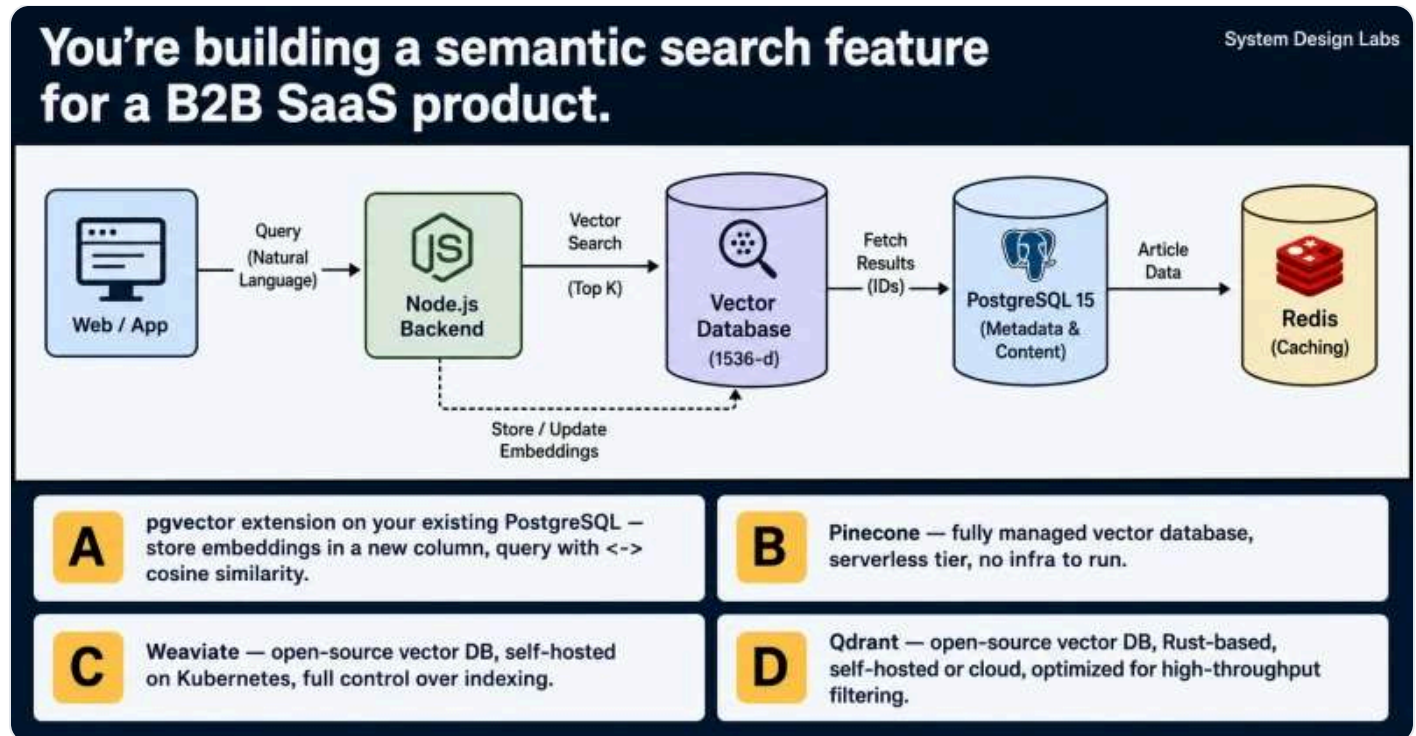
D — Prompt engineering only (hits the ceiling fast)

Fastest to ship, first to break.

A detailed system prompt can set tone and format. It cannot inject 400 pages of updated product docs without hitting token limits — and manually picking which docs to include is just RAG without the retrieval infrastructure.

Prompt engineering works when the model already knows what it needs to know. It can't fix a knowledge gap.

Choosing a Vector Store for Semantic Search



You're building a semantic search feature for a B2B SaaS product.

The corpus: 4 million support articles, docs, and user-generated tickets. Users type natural language queries. They expect Google-quality results — not keyword matching.

Your current stack: PostgreSQL 15, Redis, and a Node.js backend. The search team says ILIKE and pg_trgm aren't cutting it. Embeddings are the answer. Now you need a place to store and query 1536-dimensional vectors (OpenAI ada-002) at <100ms p99.

4 million rows. ~24GB of raw embeddings. Query volume: 300 req/s with weekend spikes to 900 req/s.

Where do you store and query those vectors?

- A** pgvector extension on your existing PostgreSQL — store embeddings in a new column, query with <-> cosine similarity.
- B** Pinecone — fully managed vector database, serverless tier, no infra to run.
- C** Weaviate — open-source vector DB, self-hosted on Kubernetes, full control over indexing.



Qdrant — open-source vector DB, Rust-based, self-hosted or cloud, optimized for high-throughput filtering.

Answer 1

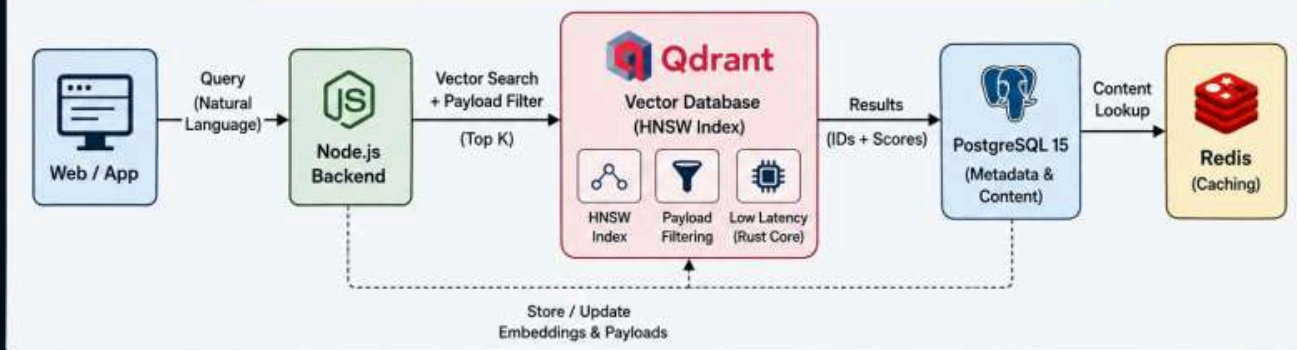
✓ CORRECT ANSWER

Why D wins: Qdrant is the right fit

System Design Labs



Qdrant handles vector search + metadata filter in one pass



Built for Scale & Low Latency

Rust core = low latency and predictable memory under load. At 300 req/s you need an index that handles concurrent ANN queries without degrading — Qdrant's HNSW implementation is tuned for this.



Killer Feature: Payload Filtering

In a B2B product, users search within a workspace, tenant, or product line — not globally. Qdrant handles vector search + metadata filter in one pass. Every other option forces *post-filtering*, which blows up *recall* and adds round-trips.



Full Control, Future Proof

Self-hosted gives you full control over HNSW params (*m*, *ef_construction*), memory mapping, and on-disk indexing — critical when your corpus grows past RAM.

Why D wins (Qdrant):

Qdrant is purpose-built for exactly this workload. Rust core = low latency and predictable memory under load. At 300 req/s you need an index that handles concurrent ANN queries without degrading — Qdrant's HNSW implementation is tuned for this.

The killer feature here is **payload filtering**. In a B2B product, users search within a workspace, tenant, or product line — not globally. Qdrant handles vector search + metadata filter in one pass. Every other option forces *post-filtering*, which blows up *recall* and adds round-trips.

Self-hosted gives you full control over HNSW params (*m*, *ef_construction*), memory mapping, and on-disk indexing — critical when your corpus grows past RAM.

Answer 2

Why A is the trap (pgvector):

pgvector is great for getting started — under ~500k vectors it's often the right call because you already own PostgreSQL. But at 4M rows and 300 req/s, the HNSW index runs inside Postgres's buffer pool, competing with your transactional workload. Latency goes non-linear under concurrency. VACUUM on a large embeddings table is painful.

Rule of thumb: pgvector under 500k vectors + low concurrency = fine. Above that, you're borrowing against future pain.

Answer 3

Why B loses (Pinecone):

Pinecone works and teams ship with it fast. But the cost model bites hard at 300 req/s sustained — serverless tier prices per query unit, you're looking at thousands of dollars/month. There's also proprietary lock-in: no standard wire protocol, so migrating means re-implementing your ingestion pipeline from scratch.

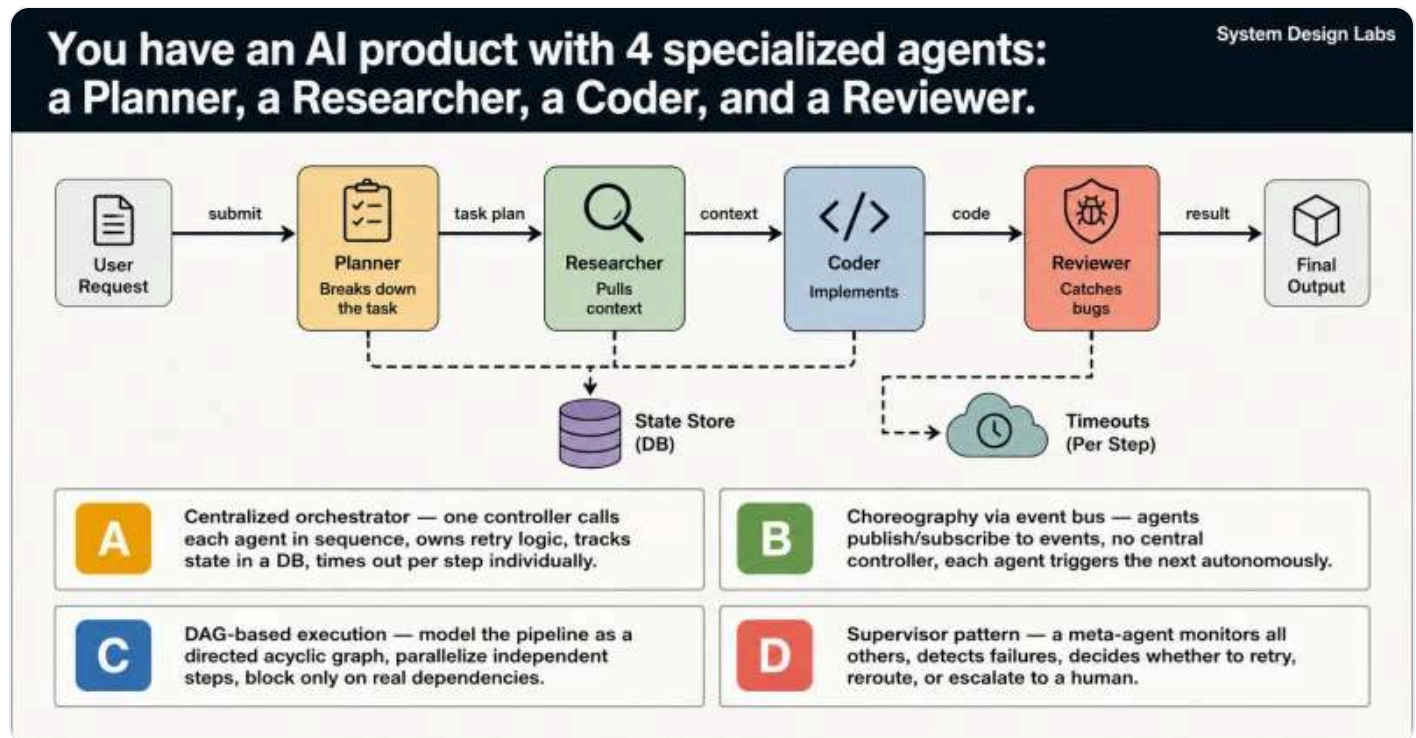
Right tool when: speed to market matters more than cost, and query volume is under ~50 req/s.

Answer 4

Why C loses (Weaviate):

Weaviate is solid with good hybrid search (BM25 + vector) and multi-modal support. But the Kubernetes footprint is heavier than Qdrant for this use case. For a team not already running K8s for their vector layer, the operational overhead is real. Weaviate shines for multi-modal search or semantic graph relationships — for pure dense vector search with filtering, Qdrant is leaner.

Coordinating a Multi-Agent Workflow



You have an AI product with 4 specialized agents: a Planner, a Researcher, a Coder, and a Reviewer.

The Planner breaks down the task. The Researcher pulls context. The Coder implements. The Reviewer catches bugs.

Simple on paper. In production, it's falling apart.

Here's what's happening:

- The Researcher sometimes returns before the Planner finishes → Coder gets incomplete context
- The Reviewer flags issues → but there's no retry loop, so bugs ship anyway
- One agent timeout hangs the entire pipeline for 40 seconds
- You have no visibility into which agent failed or why

You need to redesign the orchestration layer. What do you do?

A) Centralized orchestrator — one controller calls each agent in sequence, owns retry logic, tracks state in a DB, times out per step individually.

B) Choreography via event bus — agents publish/subscribe to events, no central controller, each agent triggers the next autonomously.

C) DAG-based execution — model the pipeline as a directed acyclic graph, parallelize independent steps, block only on real dependencies.

D) Supervisor pattern — a meta-agent monitors all others, detects failures, decides whether to retry, reroute, or escalate to a human.

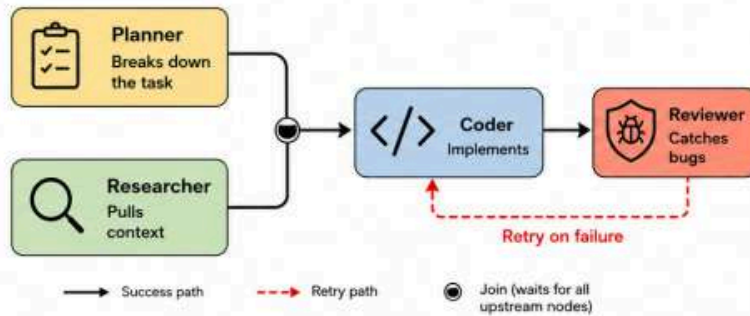
Answer 1

✓ CORRECT ANSWER

Correct Answer: C — DAG-based execution

System Design Labs

Your pipeline has hard dependencies (Coder must wait for BOTH Planner AND Researcher) AND needs retry loops (Reviewer → Coder on failure).

**What this solves directly:****Race condition**

DAG blocks Coder until both upstream nodes complete. No manual checks.

**Timeout hanging pipeline**

Each node has its own deadline. One agent timeout doesn't freeze everything.

**No retry loop**

Retry is a first-class edge in the graph, not bolted-on logic.

**No visibility**

DAG engines give you a full execution trace per run.

A

Centralized orchestrator — one controller calls each agent in sequence, owns retry logic, tracks state in a DB, times out per step individually.

B

Choreography via event bus — agents publish/subscribe to events, no central controller, each agent triggers the next autonomously.

C

DAG-based execution — model the pipeline as a directed acyclic graph, parallelize independent steps, block only on real dependencies.

D

Supervisor pattern — a meta-agent monitors all others, detects failures, decides whether to retry, reroute, or escalate to a human.



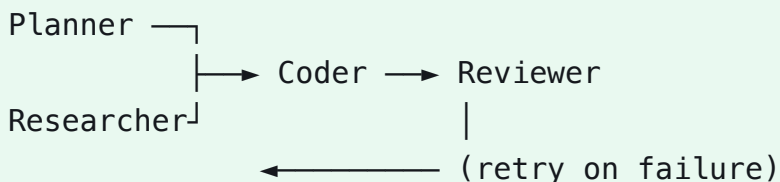
Real implementations: LangGraph, Temporal, AWS Step Functions, Prefect, Dagster.

They all converge on this model because it makes dependencies visible, failures local, and execution testable.

Correct Answer: C — DAG-based execution

Your pipeline has hard dependencies (Coder must wait for BOTH Planner AND Researcher) AND needs retry loops (Reviewer → Coder on failure).

A DAG models this explicitly:



What this solves directly:

- **Race condition** → DAG blocks Coder until both upstream nodes complete. No manual checks.
- **Timeout hanging pipeline** → each node has its own deadline. One agent timeout doesn't freeze everything.
- **No retry loop** → retry is a first-class edge in the graph, not bolted-on logic.
- **No visibility** → DAG engines give you a full execution trace per run.

Real implementations: LangGraph, Temporal, AWS Step Functions, Prefect, Dagster. They all converge on this model because it makes dependencies visible, failures local, and execution testable.

Answer 2

A — Centralized orchestrator (senior engineer trap)

This is the closest wrong answer. A central controller with per-step timeouts and DB state *feels* right — it's structured, it has retry logic, it has visibility.

The problem: it's **sequential by default**. Planner and Researcher still run one after the other unless you explicitly add parallel execution logic. You're basically hand-building a DAG imperatively, and you'll do it badly. Every new dependency becomes a code change.

Why it traps senior engineers: it looks like "proper architecture" but it's just imperative orchestration with extra ceremony.

Answer 3

B — Choreography via event bus

Works well for loosely coupled, fire-and-forget pipelines (order placed → invoice generated → email sent). But your pipeline has **tight dependencies** — Coder literally cannot start without both Planner AND Researcher finishing.

With pure choreography, there's no natural "wait for these two" primitive. You'd have to build coordination logic *inside the agents themselves*. Now your Coder is doing orchestration work. That's worse than your current problem.

Rule of thumb: choreography for independence, orchestration for dependency.

Answer 4

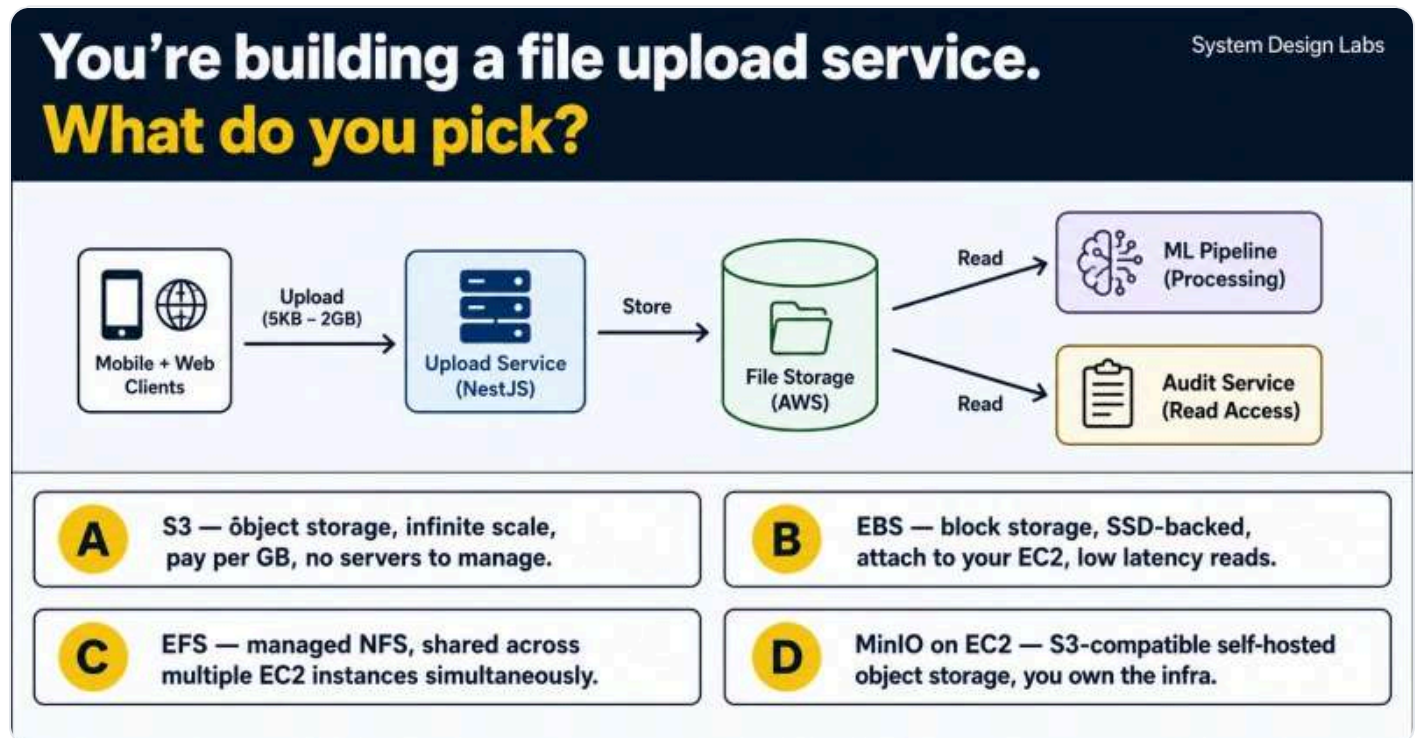
D — Supervisor pattern

A supervisor meta-agent that monitors, detects failures, and reroutes is a great **layer on top** of C — for human escalation, anomaly detection, or dynamic rerouting when the DAG can't handle it.

As a *base* orchestration model? It adds latency (every decision goes through the supervisor), a single point of failure, and doesn't solve the dependency problem at all. The supervisor still needs to know the execution order — which is just an implicit DAG inside the supervisor's brain.

Use it as resilience on top of C. Not instead of C.

Choosing a File Storage Backend



You're building a file upload service. 10TB of user files today. 100TB in 12 months.

Your team is having a fight.

The backend lead says: "Just use S3. Done."

The DevOps engineer says: "Mount an EBS volume. Simpler, faster."

The platform architect says: "We need EFS — multiple services need to read the same files."

The startup CTO says: "We can't afford cloud storage at scale. Self-host with MinIO."

All four have shipped this in production. All four have opinions backed by scars.

Here's the setup:

- Upload service (NestJS) receives files from mobile + web clients
- ML pipeline needs to read uploaded images for processing
- Audit service needs read access to the same files
- Files range from 5KB profile pics to 2GB video exports
- You're on AWS

What do you pick?

- A) S3 — object storage, infinite scale, pay per GB, no servers to manage.
- B) EBS — block storage, SSD-backed, attach to your EC2, low latency reads.
- C) EFS — managed NFS, shared across multiple EC2 instances simultaneously.
- D) MinIO on EC2 — S3-compatible self-hosted object storage, you own the infra.

Answer 1

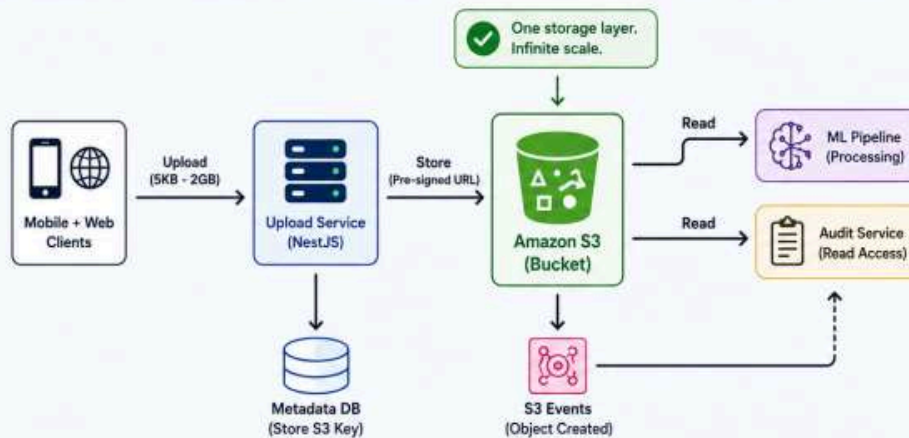
✓ CORRECT ANSWER



A WINS: S3 — Object Storage

The right choice for this file upload service.

System Design Labs



Massive Scale, No Limits

Handles 10TB today, 100TB+ tomorrow without any changes.



Cost Efficient at Scale

At 100TB → ~\$2,300/month. Pay only for what you use.



Lifecycle to Glacier

Automatically move cold files to Glacier. Save up to 90%.



Event Driven

S3 events trigger your ML pipeline the moment a file lands.



Built-in Security & Compliance

Versioning, encryption, audit logs — all built in, zero extra work.



For greenfield cloud services, this is the default.

Only deviate when you have a specific reason not to.

A

S3 — object storage, infinite scale, pay per GB, no servers to manage.

B

EBS — block storage, SSD-backed, attach to your EC2, low latency reads.

C

EFS — managed NFS, shared across multiple EC2 instances simultaneously.

D

MinIO on EC2 — S3-compatible self-hosted object storage, you own the infra.

Why A wins (S3):

One storage layer. Zero capacity planning. Every service reads from the same bucket using the S3 key stored in your DB.

At 100TB → ~\$2,300/month. Lifecycle policies move cold files to Glacier automatically. S3 events trigger your ML pipeline the moment a file lands. Versioning, encryption, audit logs — all built in, zero extra work.

For greenfield cloud services, this is the default. You only deviate when you have a specific reason not to.

Answer 2

Why B destroys your architecture quietly (EBS):

EBS is a disk. A fast, reliable disk — attached to **one EC2 instance at a time**.

The moment your ML pipeline runs on a different instance than your upload service, it can't read those files. Add auto-scaling? Each new instance gets a blank disk. Deploy across two AZs? Same problem.

Works perfectly in dev. Silently breaks the moment you scale horizontally. The team that picks EBS here usually discovers this 6 months later when they're doing a painful migration under pressure.

Answer 3

Why C is great — but not here (EFS):

EFS is genuinely powerful. Shared NFS, multiple EC2 instances mount it simultaneously, full POSIX semantics.

The problem: **\$0.30/GB/month**. At 100TB that's **\$30,000/month** vs S3's **\$2,300/month**. A 13x cost difference.

EFS is the right answer when your legacy app is hardcoded to `open()` / `read()` / `write()` filesystem calls and can't be refactored. For a greenfield NestJS service with an S3 SDK? You're paying 13x for filesystem semantics you don't need.

Answer 4

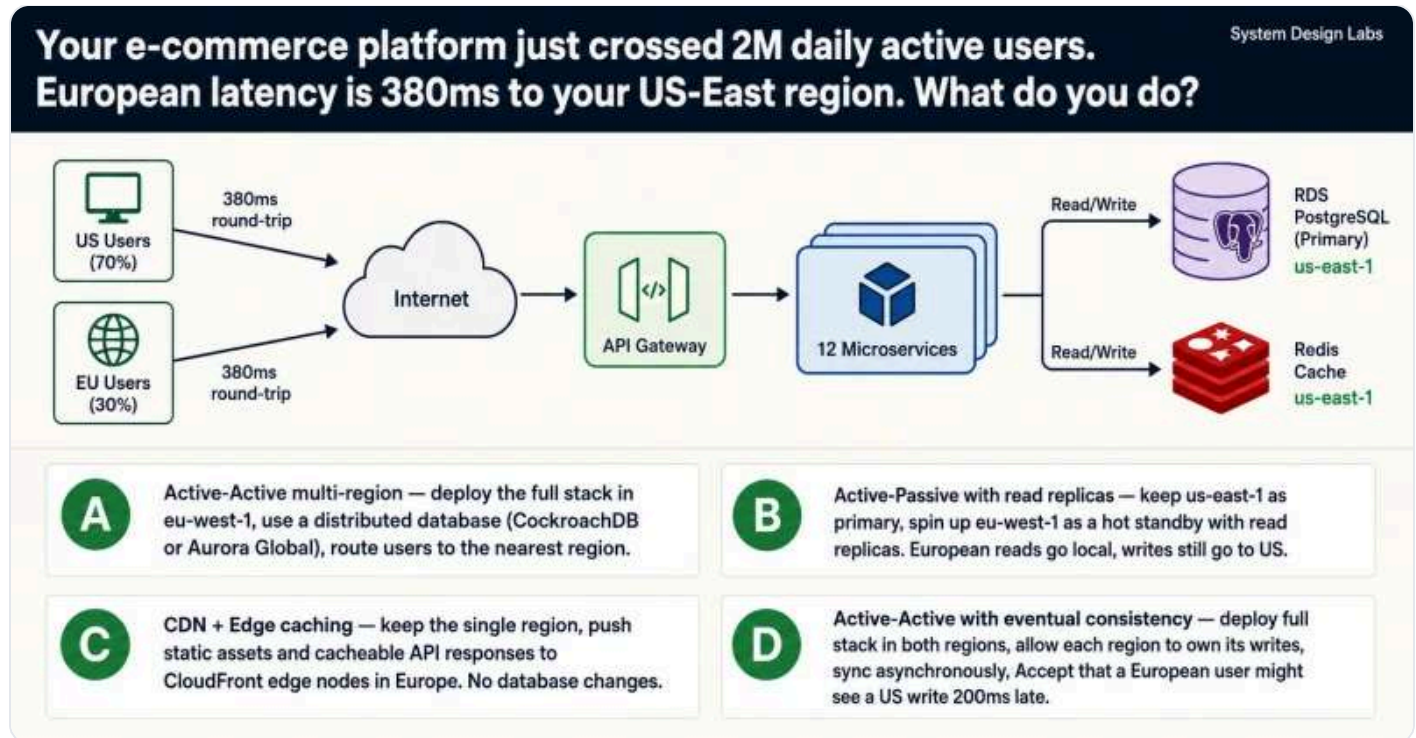
Why D is a cost-savings trap (MinIO):

MinIO is a real product used at hyperscale. S3-compatible API, high throughput, runs on your own hardware.

But self-hosted means **you own availability, disk failures, backups, capacity planning, and 3am incidents**. At 10–100TB, the engineering cost of running MinIO reliably exceeds S3 spend by a wide margin. You're not saving money — you're converting cloud spend into engineering hours, which cost more.

MinIO makes sense at petabyte scale with dedicated infra teams and painful egress bills. Not here.

Cutting Cross-Region Latency



Your e-commerce platform just crossed 2M daily active users. 70% are in the US, 30% in Europe.

Latency complaints are piling up from European users — 380ms average round-trip to your US-East region. Support tickets are up 40%. Black Friday is in 6 weeks.

Your infrastructure: single AWS us-east-1 region, RDS PostgreSQL (primary), Redis cache, 12 microservices behind an API Gateway.

You need to get European latency under 80ms. The engineering team is debating four approaches.

Here's your constraint: you cannot afford a full database rewrite, and you need this shipped before Black Friday.

A) Active-Active multi-region — deploy the full stack in eu-west-1, use a distributed database (CockroachDB or Aurora Global), route users to the nearest region. Writes go to both regions simultaneously.

B) Active-Passive with read replicas — keep us-east-1 as primary, spin up eu-west-1 as a hot standby with read replicas. European reads go local, writes still go to US. Failover in minutes if US goes down.

C) CDN + Edge caching — keep the single region, push static assets and cacheable API responses to CloudFront edge nodes in Europe. No database changes.

D) Active-Active with eventual consistency — deploy full stack in both regions, allow each region to own its writes, sync asynchronously. Accept that a European user might see a US write 200ms late.

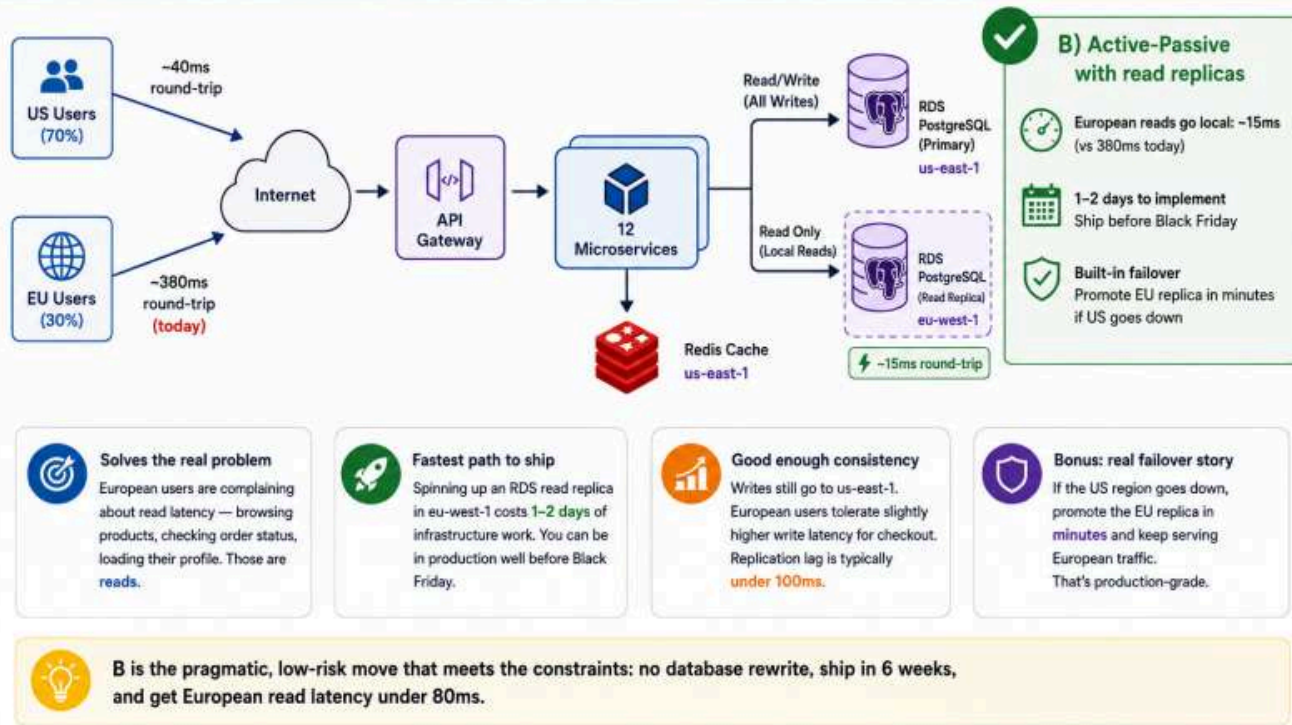
Answer 1

✓ CORRECT ANSWER

Why B wins: Active-Passive with read replicas

System Design Labs

Fastest way to get European read latency under 80ms — without rewriting the database.



Why B wins:

The constraint is the answer. You can't rewrite the database, and you have 6 weeks.

European users are complaining about *read* latency — browsing products, checking order status, loading their profile. Those are reads. An RDS read replica in eu-west-1 costs you 1-2 days of infra work and immediately moves European reads to ~15ms instead of 380ms. Writes still go to us-east-1 (acceptable — users tolerate slightly higher write latency for checkout). Replication lag is typically under 100ms.

Active-passive also gives you a real failover story: US goes down → promote EU replica in minutes.

Answer 2

Why A is the trap (Active-Active + distributed DB):

This is the "correct at scale" answer that kills teams before Black Friday. Migrating from RDS PostgreSQL to CockroachDB or Aurora Global under a 6-week deadline is a multi-quarter project — distributed transactions, global clock skew, new failure modes to test. Right architecture for the long term. Wrong answer for the problem as stated.

Answer 3

Why C is wrong (CDN + Edge caching):

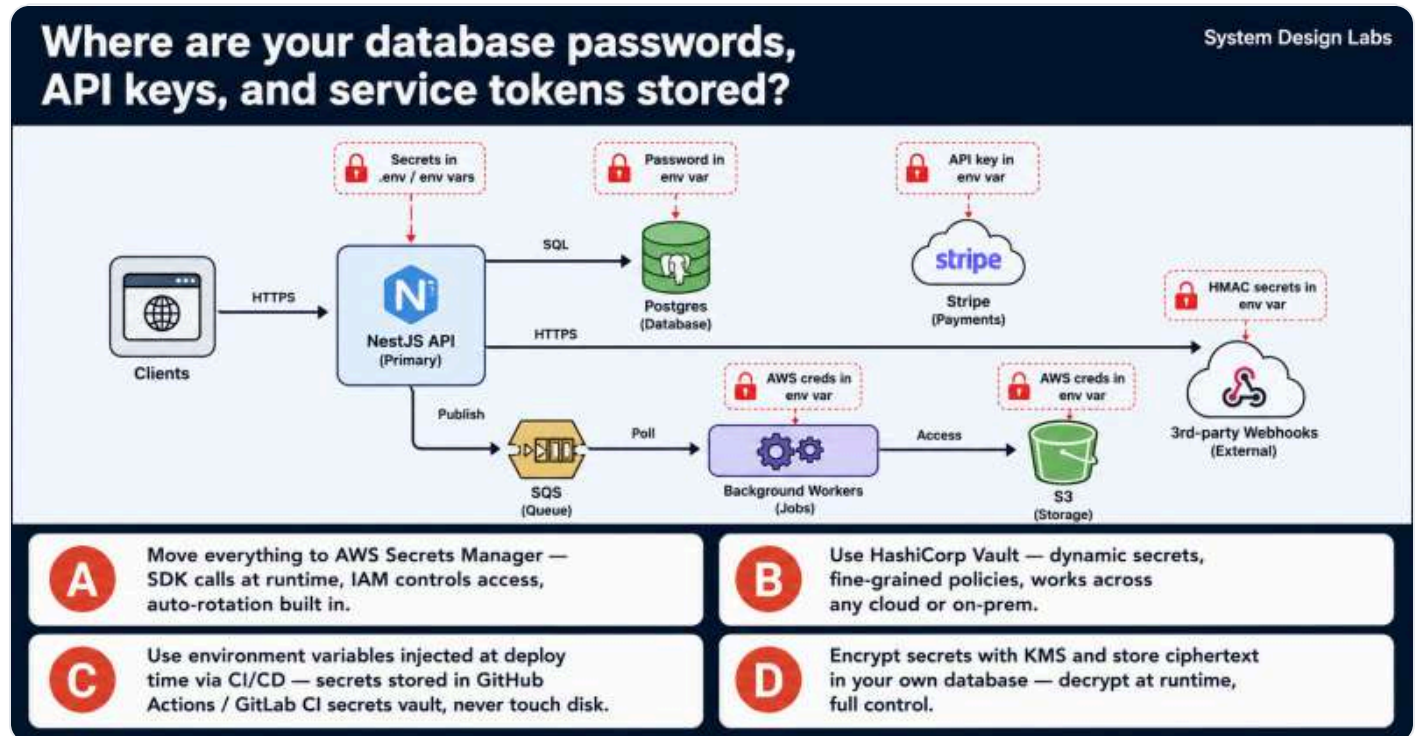
CDN solves static asset latency — images, JS, CSS. It does nothing for dynamic reads (user cart, order history, account data) and zero for write latency. It's a layer you add *on top of* a real solution, not instead of one. Partial credit, not the answer.

Answer 4

Why D is dangerous (Active-Active + eventual consistency):

Most honest version of multi-region — and the most dangerous to deploy without prep. Eventual consistency across regions means a European user places an order and your US region sees a different inventory count for 200ms. For limited stock = oversell. For checkout = duplicate charges. Conflict resolution is a product problem disguised as an infra problem.

Managing Secrets and Credentials



Your startup just got its first SOC 2 audit.

The auditor asks: "Where are your database passwords, API keys, and service tokens stored?"

Your senior engineer goes quiet.

Turns out half of them are in .env files committed to git 18 months ago. Three are hardcoded in Lambda environment variables. One is in a Slack message from 2023.

You have 6 services in production, 4 environments, and zero rotation policy.

Here's the setup:

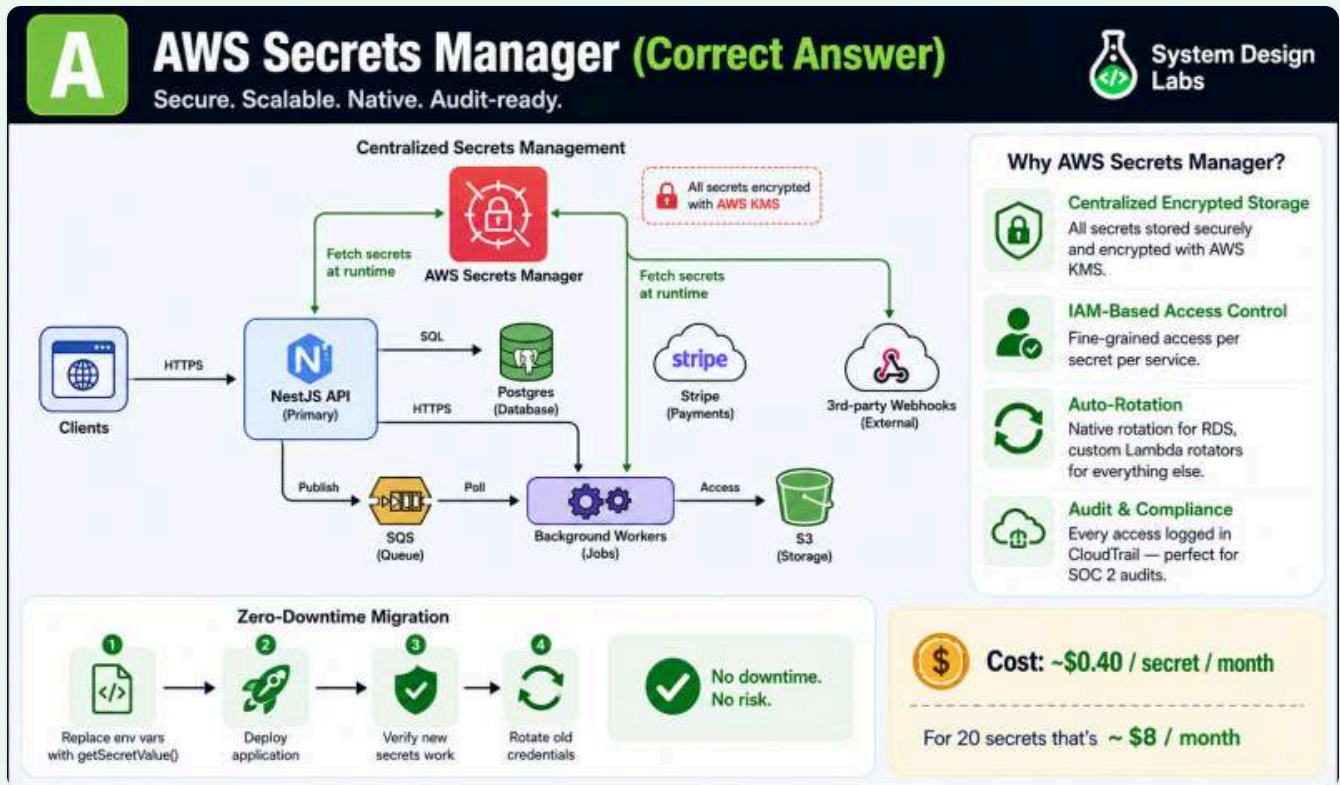
- NestJS API → Postgres (password in env var)
- NestJS API → Stripe (API key in env var)
- Background workers → SQS, S3 (AWS credentials in env var)
- 3rd-party webhooks → HMAC secrets in env var
- Zero rotation. Zero audit trail. Zero centralized access control.

You need to fix this. And you can't take downtime.

- A)** Move everything to AWS Secrets Manager — SDK calls at runtime, IAM controls access, auto-rotation built in.
- B)** Use HashiCorp Vault — dynamic secrets, fine-grained policies, works across any cloud or on-prem.
- C)** Use environment variables injected at deploy time via CI/CD — secrets stored in GitHub Actions / GitLab CI secrets vault, never touch disk.
- D)** Encrypt secrets with KMS and store ciphertext in your own database — decrypt at runtime, full control.

Answer 1

✓ CORRECT ANSWER



A — AWS Secrets Manager (correct)

You're already AWS-native (Lambda, SQS, S3). Secrets Manager gives you:

- Centralized encrypted storage (KMS-backed)
- IAM-based access control per secret per service
- Native auto-rotation for RDS + custom Lambda rotators for everything else
- Every access logged in CloudTrail → exactly what your SOC 2 auditor wants

Migration is zero-downtime: swap env vars for `getSecretValue()` calls, deploy, rotate the old creds after confirming the new ones work. Cost: ~\$0.40/secret/month. For 20 secrets that's \$8/month.

Answer 2

B — HashiCorp Vault (senior engineer trap)

Vault is *more powerful* — dynamic secrets (a fresh Postgres credential per-request that expires in 1h) is one of the most elegant patterns in production infra.

But Vault is infrastructure you own and operate. You need a Raft cluster (3+ nodes), a storage backend, unsealing procedures, DR planning, and an ops team that knows Vault. If Vault goes down, secrets become inaccessible — services fail.

When Vault wins: Multi-cloud, on-prem requirements, dynamic secrets at scale, or you have a dedicated infra team. If you're AWS-native under 50 engineers — Secrets Manager first, always.

Answer 3

C — CI/CD Injected Env Vars (partially right, wrong scope)

GitHub Actions / GitLab CI secrets are the right place for *build-time* and *deploy-time* secrets — Docker Hub creds, Terraform tokens, deploy keys.

They're wrong for *runtime* secrets — the DB password your app needs at 3 AM, the Stripe key for a live webhook. Those get injected as env vars into ECS task definitions or Lambda config — harder to rotate atomically, no audit trail on runtime access.

C complements A or B. It doesn't replace them.

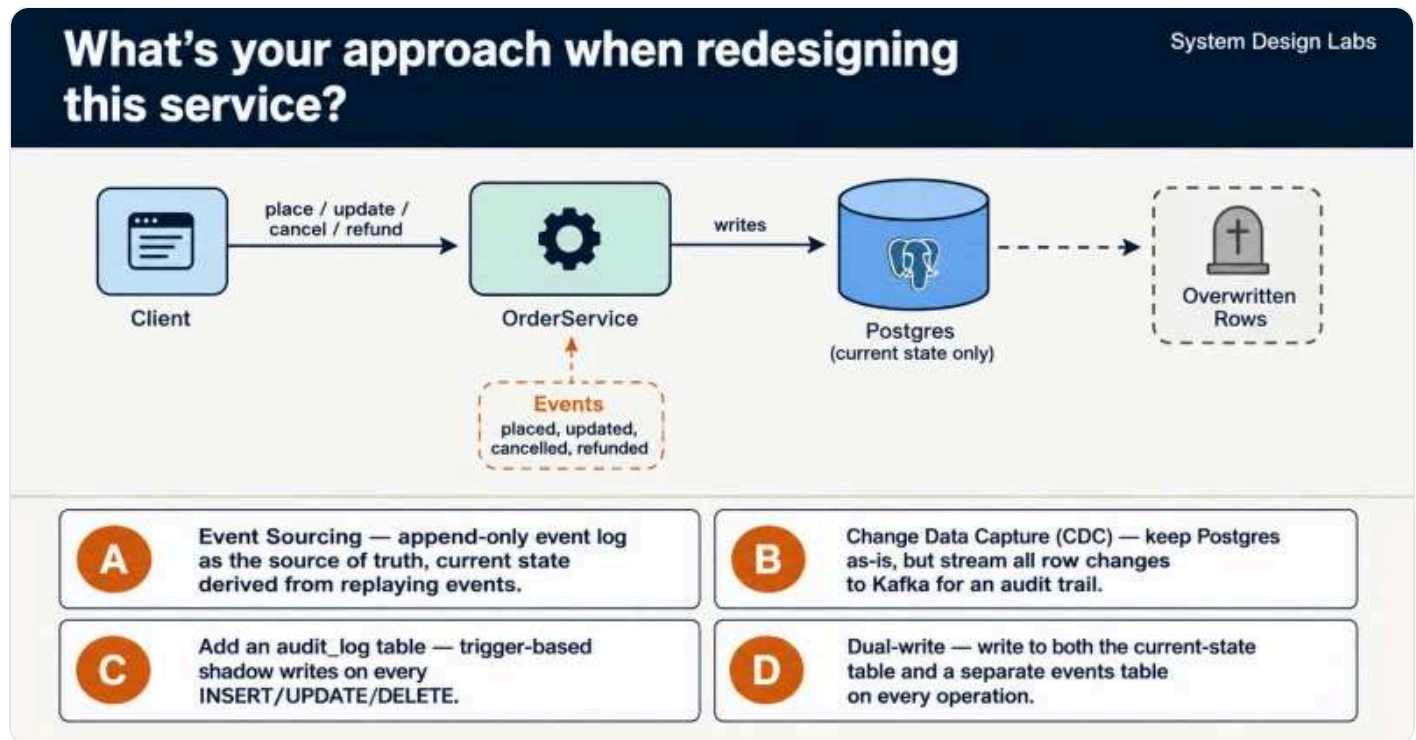
Answer 4

D — DIY KMS + own database (wrong)

This is reinventing Secrets Manager badly. You now own: encryption key management, rotation logic, access control, audit logging, and a database that itself needs credentials to connect to.

The meta-problem: what secret protects the secret store? You end up with a bootstrapping problem that managed secret stores are specifically designed to solve. This is what teams built in 2012 before managed options existed.

Reconstructing State with Event Sourcing



Your order service takes 200 writes/sec at peak.

You audit 6 months of data. Something's off — two orders show the same ID, different totals.

You have the current state. You don't have how it got there.

Your DB is a graveyard of overwritten rows.

Here's the system:

- OrderService → Postgres (current state only)
- Events: placed, updated, cancelled, refunded
- Every UPDATE overwrites the previous row
- No audit log. No event history. No replay.

A billing dispute just landed. You need to reconstruct exactly what happened to Order #8471. You can't.

Instead of storing the current state, you store the sequence of events that produced it.

What's your approach when redesigning this service?

- A** Event Sourcing — append-only event log as the source of truth, current state derived from replaying events.
- B** Change Data Capture (CDC) — keep Postgres as-is, but stream all row changes to Kafka for an audit trail.
- C** Add an audit_log table — trigger-based shadow writes on every INSERT/UPDATE/DELETE.
- D** Dual-write — write to both the current-state table and a separate events table on every operation.

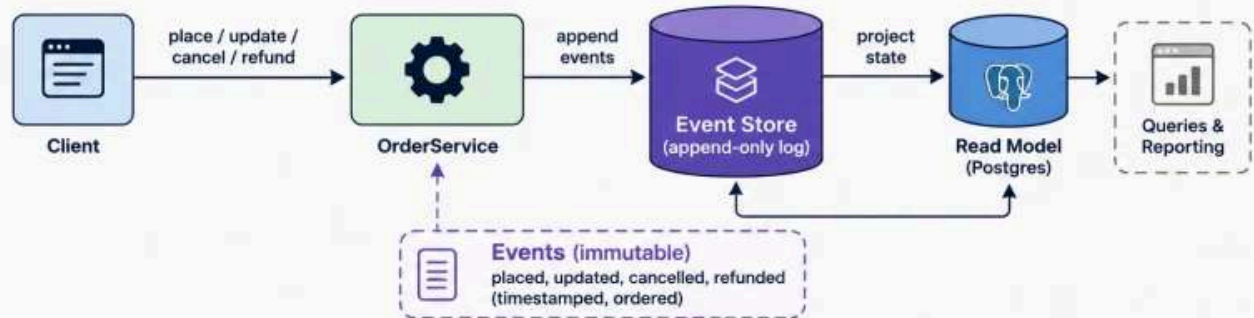
Answer 1

✓ CORRECT ANSWER

ANSWER: A — EVENT SOURCING ✓

System Design Labs

Append-only event log as the source of truth; current state derived from replaying events.

**A EVENT SOURCING**

Append-only event log as the source of truth; current state derived from replaying events.

- ✓ Full replay & time travel
- ✓ Projection flexibility
- ✓ Immutable & auditable
- ✓ True source of truth

B CHANGE DATA CAPTURE (CDC)

Keep Postgres as-is, but stream all row changes to Kafka for an audit trail.

- ✗ Misses in-app context and intent
- ✗ Hard to reconstruct business events
- ✗ Lag, ordering & schema evolution challenges
- ✗ It's still a side channel, not the source of truth

C ADD AN AUDIT_LOG TABLE

Trigger-based shadow writes on every INSERT/UPDATE/DELETE.

- ✗ Easy to bypass or break
- ✗ Adds write overhead to hot path
- ✗ Captures data changes, not business events
- ✗ Schema coupling & maintenance burden

D DUAL-WRITE

Write to both the current-state table and a separate events table on every operation.

- ✗ Dual-write inconsistency risk
- ✗ Requires distributed transactions or outbox
- ✗ Complex failure modes
- ✗ Not guaranteed sequence or completeness

Answer: A — Event Sourcing ✓

Here's why, and why the other three look reasonable but miss the point:

Why A wins (Event Sourcing):

Event Sourcing flips the model entirely. Instead of storing the *latest state* and losing how you got there, you store every event that ever happened. The current state is a projection — computed by replaying events from the beginning (or from a snapshot).

For Order #8471: you don't query a row. You replay OrderPlaced, OrderUpdated, PaymentCaptured, RefundInitiated. Every mutation is preserved, timestamped, immutable. You can reconstruct state at any point in time. You can build new projections (e.g., "total revenue by SKU last 90 days") without changing the core model — just replay and project differently.

This is what Axon, EventStoreDB, and the event-sourced layers in most serious fintech/e-commerce systems do. It's not a log — it's the primary data model.

Answer 2

Why B is the trap (CDC):

CDC is powerful and often underrated. Tools like Debezium + Kafka let you stream every Postgres row change downstream without touching your app. It looks like Event Sourcing from the outside — you have a sequence of changes.

But here's the trap: CDC captures *state deltas*, not *business events*. You get total changed from 89.99 to 79.99 — not DiscountApplied by coupon SAVE10. The semantic meaning is lost. You can reconstruct what changed, not *why* it changed. That distinction kills you during disputes and compliance audits.

CDC is a great complement to Event Sourcing for propagation. It's not a substitute for it.

Answer 3

Why C is wrong (audit_log table):

Trigger-based audit logs are the classic band-aid. The problem: you're treating the row as source of truth and logging changes as a side effect.

Side effects break. Triggers get disabled during migrations. Bulk imports skip them. Schema changes orphan the trigger. Six months later, the audit_log has gaps — exactly when you need it most. It's a shadow, not a foundation.

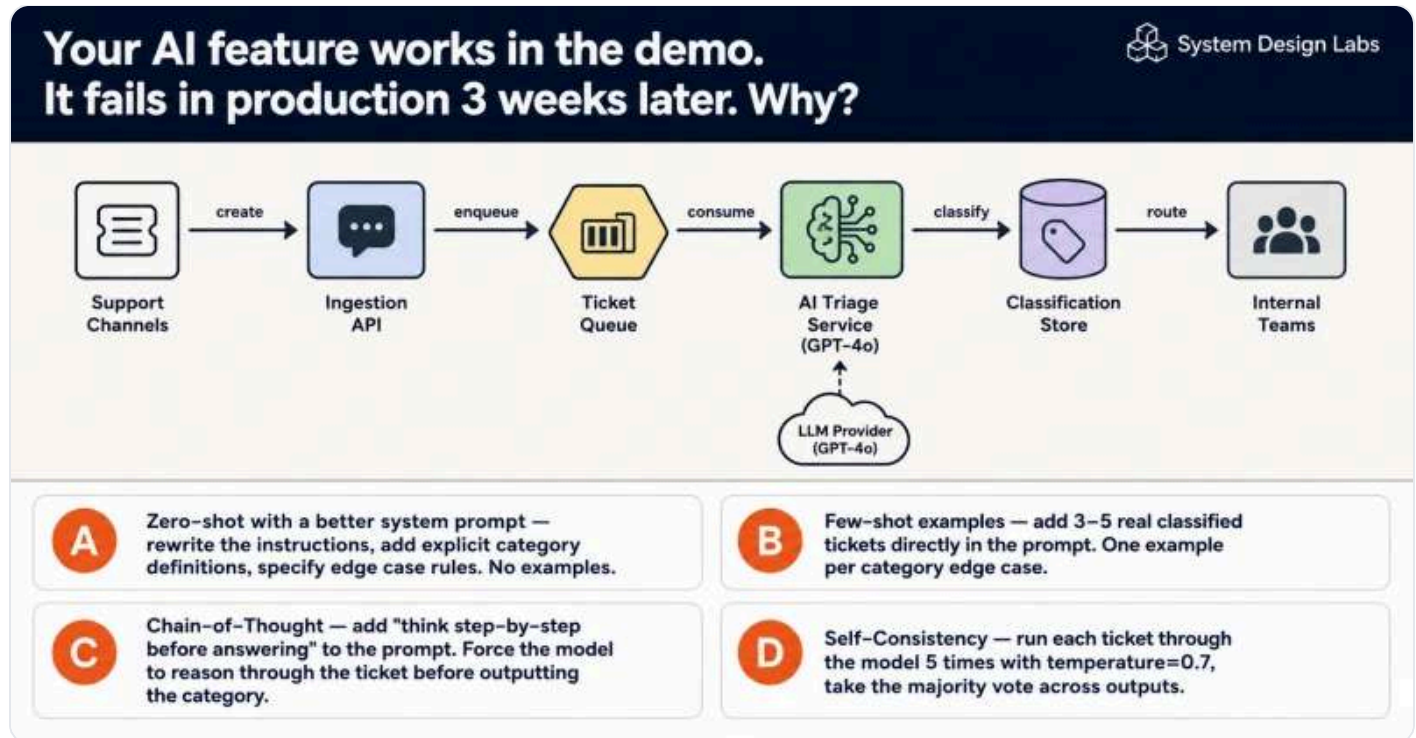
Answer 4

Why D is wrong (Dual-write):

Dual-write seems reasonable. But you've created a distributed consistency problem inside your own service. If the state write succeeds and the events write fails — network hiccup, crash, OOM — you've got state with no corresponding event. Audit trail unreliable by design.

This is the exact problem the Outbox Pattern was invented to solve. But if you're going that far, you're halfway to proper Event Sourcing anyway.

Improving LLM Classification Accuracy



Your AI feature works in the demo.

It fails in production 3 weeks later. Nobody touched the model. Nobody changed the code.

The only thing that changed: the inputs got messier.

Here's the setup:

You're at a SaaS company. 50,000 support tickets a week. Your team builds an AI triage system — GPT-4o classifies each ticket into 6 categories (billing, bug, feature request, account access, security, other) so the right team gets it instantly.

In dev, it nails 71% accuracy. You need 90%+ to cut manual review.

The model is locked. The budget for inference isn't unlimited. You need to close the 19-point gap.

Here are your four options:

A) Zero-shot with a better system prompt — rewrite the instructions, add explicit category definitions, specify edge case rules. No examples.

B) Few-shot examples — add 3–5 real classified tickets directly in the prompt. One example per category edge case.

C) Chain-of-Thought — add "think step-by-step before answering" to the prompt. Force the model to reason through the ticket before outputting the category.

D) Self-Consistency — run each ticket through the model 5 times with temperature=0.7, take the majority vote across outputs.

Answer 1

✓ CORRECT ANSWER

B — Few-shot Best answer here

Accuracy: 88–93%. Cost increase: near-zero.

Few-shot shifts the model from *reading rules* to *pattern-matching against real examples*. Fundamentally stronger signal.

The part most teams get wrong: they pick the obvious examples. A crystal-clear billing ticket. A textbook bug report. Those teach the model nothing it didn't already know.

You want the *ambiguous* ones. The ticket that looks like billing but is account access. The feature request that reads like a bug. Those are where the model currently fails — and those are the examples that close the gap.

Get the selection right, you close most of the 19 points without touching anything else.

Answer 2**A — Zero-shot with a better system prompt**

Accuracy ceiling: ~78–82%.

This is where every team starts. You iterate the prompt, add category definitions, enumerate edge cases. It feels like progress. And for a while it is.

The hard wall: instructions *describe* categories. They can't *show* the model where the ambiguous cases land.

Real example: you define "billing" as anything about charges or invoices. Then you get this ticket — "*I can't log in and I was charged twice last month.*" Billing? Account access? Both?

No amount of rewriting tells the model which bucket that gets. It guesses wrong 20–25% of the time on edge cases. And your dataset is full of edge cases. The ceiling is real.

Answer 3

C — Chain-of-Thought

Accuracy delta: $\pm 2\%$. Latency: +200–400ms.

CoT is the most misapplied technique in production AI right now. Teams add "think step-by-step" to everything. For some tasks it works. Not this one.

Why it struggles here: the model generates a reasoning chain, then commits to a label. On ambiguous tickets, that chain becomes a liability. "*Mentions a charge... but also a login issue... charge is mentioned first... probably billing.*" It picks billing. Right answer is account access.

CoT helps when the answer *emerges from reasoning* — math, debugging, planning. For classification, the answer emerges from calibration. Reasoning doesn't substitute for calibration.

Layer it on top of few-shot if you're still missing edge cases. Don't reach for it first.

Answer 4

D — Self-Consistency

Accuracy: 93–96%. Cost: 5x. Latency: 5x.

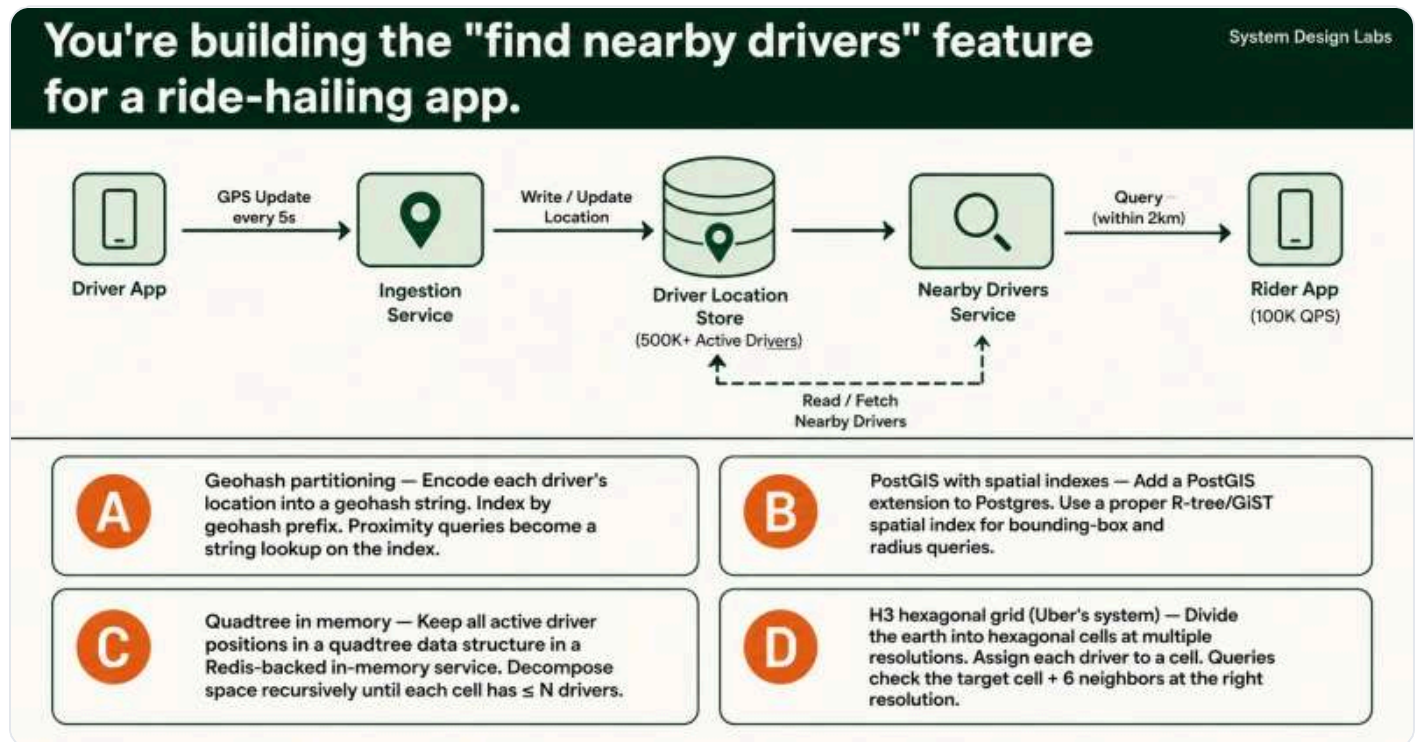
Run the same ticket 5–10x at temperature=0.7, take the majority vote. Works because each run samples a slightly different path — noise cancels out across runs.

The math is good. The cost is brutal.

50K tickets/week $\times$ \$0.002 $\times$ 5 runs = **\$500/week** vs **\$100/week** with few-shot. You're paying \$400/week extra for a 3–5% gain on the hardest cases.

Worth it when being wrong is expensive: fraud detection (a missed flag costs hundreds), medical triage (wrong category = real harm). For support routing, a misclassified ticket costs ~\$0.10 of human rerouting time. Use B.

Geospatial “Find Nearby Drivers” at Scale



You're building the "find nearby drivers" feature for a ride-hailing app.

At peak, you have 500,000 active drivers updating their GPS location every 5 seconds. Riders query for drivers within 2km. At scale, you're doing $\sim 100,000$ proximity queries per second.

Your naive implementation does this:

```
SELECT * FROM drivers
```

```
WHERE lat BETWEEN ? AND ?
```

```
AND lng BETWEEN ? AND ?
```

It works fine at 1,000 drivers. At 500,000, it's a full table scan on every query. Latency hits 800ms. Riders see a spinner. Drivers miss trips.

Your team proposes 4 approaches to fix this:

A) Geohash partitioning — Encode each driver's location into a geohash string. Index by geohash prefix. Proximity queries become a string lookup on the index.

B) PostGIS with spatial indexes — Add a PostGIS extension to Postgres. Use a proper R-tree/GiST spatial index for bounding-box and radius queries.

C) Quadtree in memory — Keep all active driver positions in a quadtree data structure in a Redis-backed in-memory service. Decompose space recursively until each cell has $\leq N$ drivers.

D) H3 hexagonal grid (Uber's system) — Divide the earth into hexagonal cells at multiple resolutions. Assign each driver to a cell. Queries check the target cell + 6 neighbors at the right resolution.

You need sub-50ms p99 latency, real-time updates, and it has to stay accurate at cell boundaries.

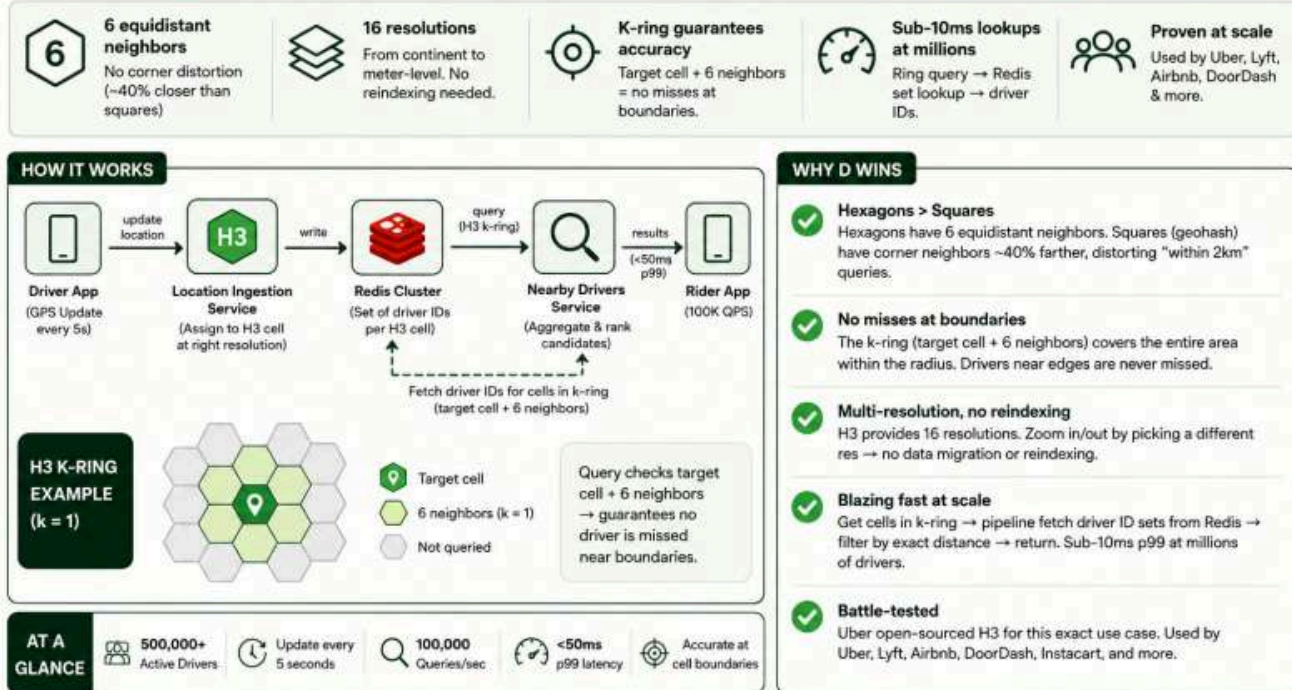
Every option here ships with its own diagram, so no single image marks the answer; the author's pick is **D — H3 Hexagonal Grid** (Answer 1 below).

Answer 1 ✓ CORRECT ANSWER

ANSWER: D — H3 Hexagonal Grid

System Design Labs

Best for accurate, low-latency "find nearby drivers" at scale.



★ **BOTTOM LINE:** H3 gives accurate, boundary-safe, multi-resolution proximity search with sub-10ms latency at massive scale.

Answer: D — H3 Hexagonal Grid

Why D wins:

Uber open-sourced H3 for exactly this problem. Hexagons have 6 neighbors, all equidistant from the center. Squares (geohash) have corner neighbors that are ~40% farther — this distorts "within 2km" queries. H3 also has 16 resolution levels (no reindexing on zoom changes). The killer feature: the k-ring query (target cell + 6 neighbors) guarantees no driver gets missed near a boundary. Uber does sub-10ms lookups at millions of drivers: get cells in ring → fetch IDs from Redis set keyed by cell. Lyft, Airbnb, DoorDash all use variants.

Answer 2

ANSWER: A is the trap — Geohash

System Design Labs

Production-grade, widely used — and wrong for real-time, sub-km proximity.



Boundary misses

Drivers 1m apart can land in completely different prefixes.



Z-order curve problem

Geohash uses interleaved bits (Z-order). Adjacent prefix strings ≠ adjacent geography.



Systematic miss rate

At scale (100K QPS), boundary misses become a real, measurable problem.



Good for partitioning, not proximity

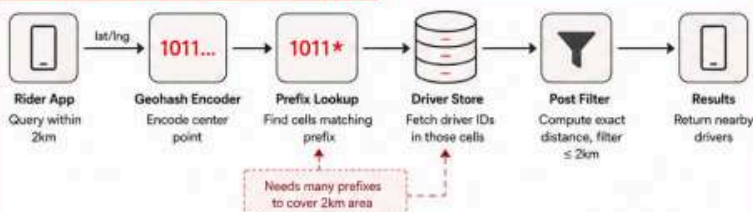
Great for coarse sharding by region. Wrong for real-time sub-km lookup.



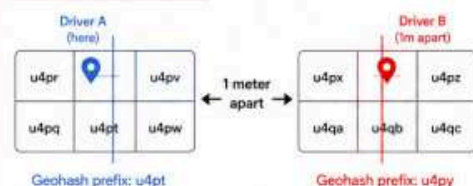
Latency suffers

More cells to cover + post-filtering = higher latency and cost.

HOW GEOHASH LOOKUP WORKS (AND FAILS)



BOUNDARY MISS EXAMPLE



Same prefix lookup won't return both.

One driver gets missed.
This happens everywhere at cell boundaries.

WHY A LOSES TO OTHER APPROACHES



Not boundary-safe

Z-order (geohash) doesn't align with circular distance. Edges and corners cause misses.



Unpredictable coverage

To cover 2km, you need a variable number of prefixes (4 to 40+). More cells = more reads + filtering.



Higher latency at scale

More prefixes → more data read → more post-filtering. Hard to hit sub-50ms p99.



No built-in neighbor guarantee

You must manually compute which adjacent prefixes to include. Easy to get wrong, still not perfect.



Wrong tool for the job

Great for coarse partitioning / sharding / caching by region. Wrong for real-time, accurate, sub-km proximity search.



Bottom line: Geohash is the trap. It looks simple, works at low scale, but guarantees misses at the boundaries. At 100K QPS with sub-2km accuracy requirements, it will hurt riders and drivers.



Use a boundary-safe approach like H3 (hex grid) instead.

Why A is the trap (Geohash):

Production-grade, widely used — and wrong here. Geohash uses a Z-order curve (interleaved bits) that doesn't guarantee geographic adjacency for adjacent prefix strings. Two drivers 1 meter apart can have entirely different prefix strings if they straddle a cell boundary. This becomes a systematic miss rate at 100k queries/sec. Geohash is right for coarse partitioning (sharding by region), wrong for real-time sub-km proximity.

Answer 3

ANSWER: B fails at scale — PostGIS

System Design Labs

PostGIS is correct — just not for 500k drivers updating every 5 seconds.



Write-heavy workload
100,000 writes/sec
(500k drivers × 12 updates/min)



GiST index is write-expensive
Every write updates multiple index pages.



Database burns capacity on index maintenance
Less headroom for queries and growth.

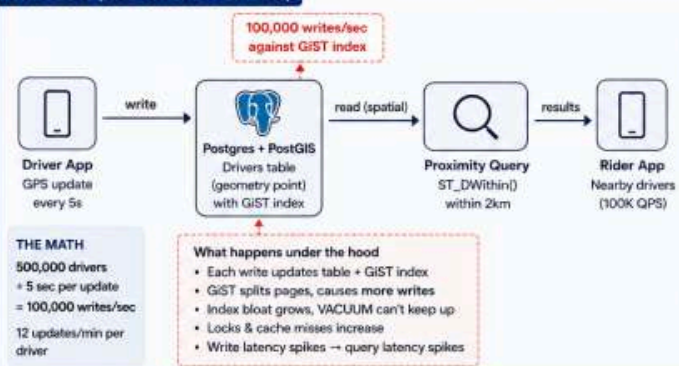


Hard to scale writes
Postgres/PostGIS doesn't scale writes horizontally.



Latency suffers under load
Contention, VACUUM, bloat, and lock amplification.

THE FLOW (AND WHERE IT BREAKS)



WHY THIS DOESN'T SCALE

- ❌ **GiST index write amplification**
GiST isn't optimized for high write rates. Each insert/update touches multiple index pages.
- ❌ **Database becomes write-bound**
At 100k writes/sec, most CPU, IOPS, and memory go to index maintenance, not serving queries.
- ❌ **Vertical scaling hits a wall**
Postgres/PostGIS scales up, not out. Bigger box = temporary relief, not a solution.
- ❌ **Operational overhead grows**
VACUUM, REINDEX, bloat management, autovac tuning — constant firefighting.
- ❌ **Not built for real-time, global, massive scale**
Great for thousands to tens of thousands. Not for 500k drivers updating every 5 seconds with 100k QPS queries.

GOOD FOR THIS (NOT THIS)

- | | |
|---|---|
| <p>✓ Good fit</p> <ul style="list-style-type: none">• Fleet dashboard• 100s to 5,000 assets• Ad-hoc spatial analytics• Internal tools | <p>✗ Not a fit</p> <ul style="list-style-type: none">• 500k active drivers• 100k writes/sec• 100k QPS proximity queries• Sub-50ms p99 requirement |
|---|---|

POSTGIS IS THE STARTING POINT, NOT THE DESTINATION



- BOTTOM LINE**
- PostGIS is the right tool for correctness and small/medium scale. But at 500k drivers with 100k writes/sec, it becomes the bottleneck. You'll spend more time fighting the database than building features.
- Use PostGIS to validate. Use H3 + Redis to scale.**

Why B fails at scale (PostGIS):

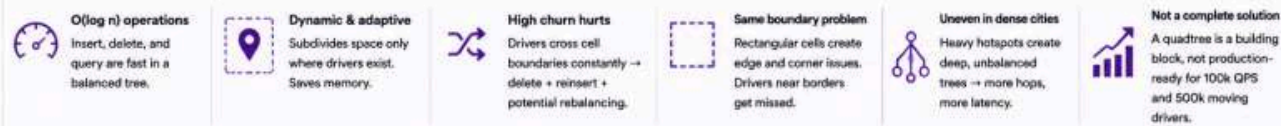
PostGIS is correct — it's the right answer for 5,000 trucks on a fleet dashboard. Not for 500k drivers updating every 5s. The math: $500k \times 12 \text{ updates/min} = 100k \text{ writes/sec}$ against a GiST index. GiST indexes are write-expensive. Your DB burns half its capacity on index maintenance. And you can't easily scale Postgres writes horizontally. PostGIS is the starting point — H3 + Redis is where you land when it breaks.

Answer 4

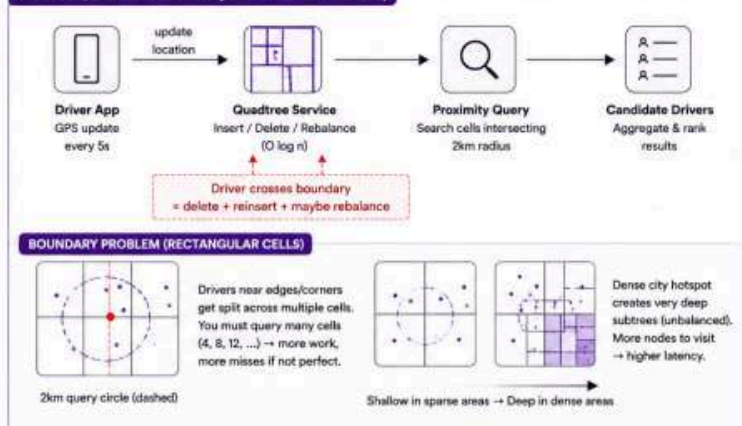
ANSWER: C almost works — Quadtree

Great in theory ($O(\log n)$). Breaks under ride-hailing scale and real-world density.

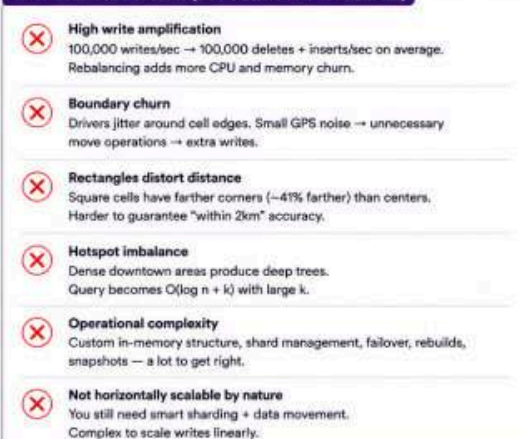
System Design Labs



HOW A QUADTREE WORKS (AND WHERE IT HURTS)



WHY C ALMOST WORKS (BUT DOESN'T AT THIS SCALE)



GOOD FOR

- Low to medium scale
- Static or low-churn data
- Games, simulations, robotics
- Spatial databases (read-heavy)

NOT GOOD FOR

- 500k moving drivers
- 100k writes/sec
- Strict sub-50ms p99 SLA
- Real-time proximity at scale

THE MATH

500,000 drivers × 12 updates/min
 = **100,000 writes/sec**

Delete + Insert + (Maybe) Rebalance
 → Millions of node ops/sec
 → CPU + memory + GC pressure

BOTTOM LINE

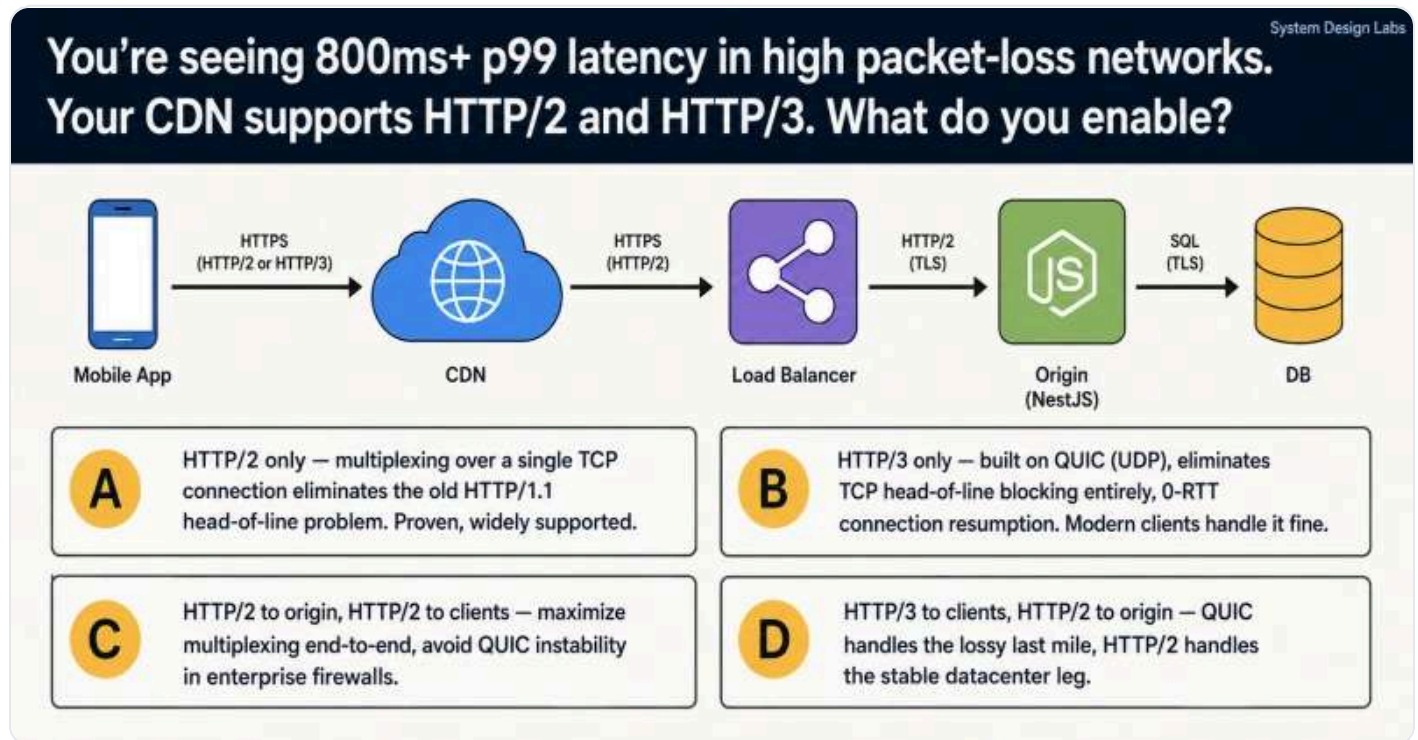
Quadtree has the right idea: adaptive spatial partitioning. But at ride-hailing scale, write churn, boundary issues, and imbalance make it too expensive and complex.

It's a building block — not the full solution. H3 + Redis wins.

Why C almost works (Quadtree):

$O(\log n)$ insert/query, great in theory. Breaks at ride-hailing density: driver crossing a cell boundary = delete + reinsert + potential rebalancing, 100k times per second. Same rectangular boundary problem as geohash. Dense cities create deep uneven trees. A quadtree is a building block — not a complete solution at this scale.

Choosing HTTP/3 vs HTTP/2 at the Edge



Your API response went from 320ms to 95ms after a CDN switch.

Nothing changed on the server. Same origin. Same payload.

The only difference: the CDN started speaking HTTP/3 to your clients.

Here's the setup:

Mobile app → Load Balancer → Origin (NestJS) → DB

Every request goes through the CDN first. Latency is acceptable — until high packet-loss environments (mobile 4G → 5G transitions, flaky WiFi, Asia-Pacific routes). You're seeing 800ms+ tail latency for the p99. The CDN supports HTTP/2 and HTTP/3. What do you enable?

- A** HTTP/2 only — multiplexing over a single TCP connection eliminates the old HTTP/1.1 head-of-line problem. Proven, widely supported.
- B** HTTP/3 only — built on QUIC (UDP), eliminates TCP head-of-line blocking entirely, 0-RTT connection resumption. Modern clients handle it fine.

C HTTP/2 to origin, HTTP/2 to clients — maximize multiplexing end-to-end, avoid QUIC instability in enterprise firewalls.

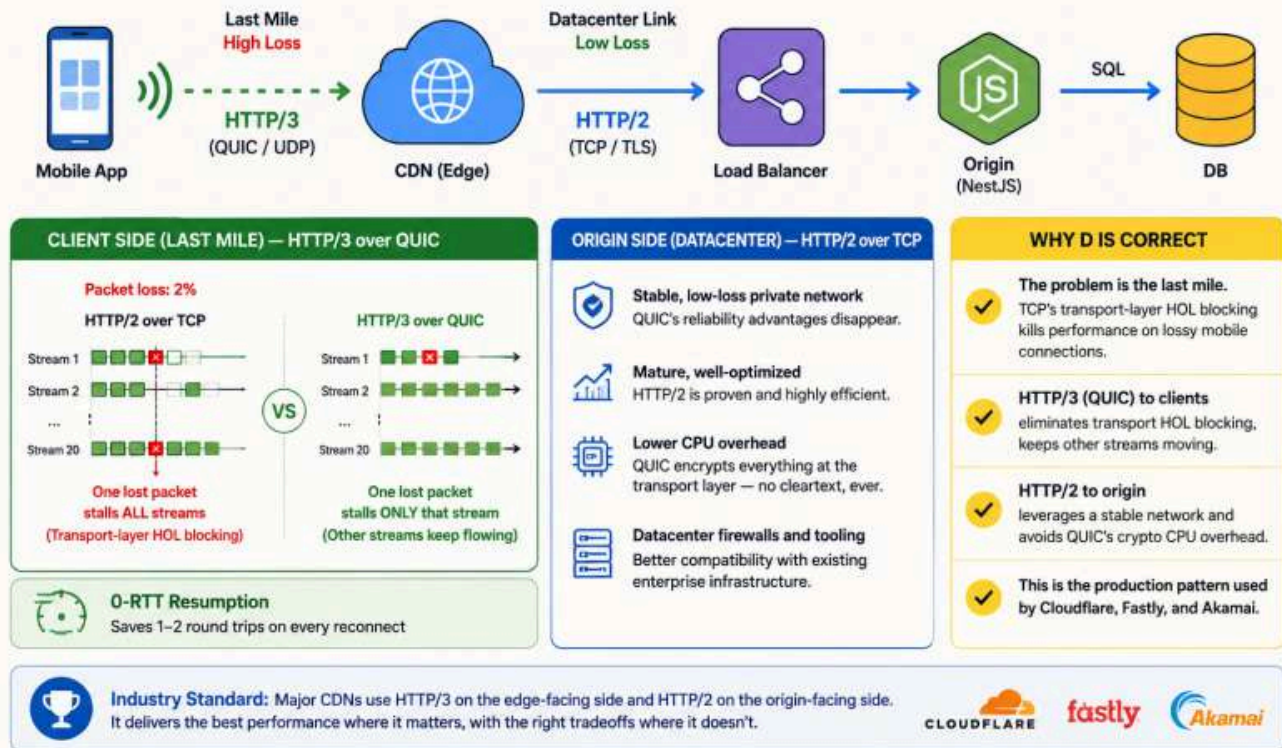
D HTTP/3 to clients, HTTP/2 to origin — QUIC handles the lossy last mile, HTTP/2 handles the stable datacenter leg.

Answer 1

✓ CORRECT ANSWER

ANSWER: D — HTTP/3 to clients, HTTP/2 to origin ✓

System Design Labs



Answer: D — HTTP/3 to clients, HTTP/2 to origin

The problem is the last mile. Mobile clients on lossy connections are where TCP falls apart.

HTTP/2 fixed HTTP/1.1's application-layer head-of-line blocking by multiplexing streams over one TCP connection. But TCP itself still has head-of-line blocking at the transport layer. A single lost packet stalls ALL streams until it's retransmitted. On a 4G connection dropping 2% of packets, your 20-stream HTTP/2 connection grinds to a halt.

HTTP/3 runs on QUIC (UDP). Each stream is independent. A dropped packet stalls only that stream, the other 19 keep flowing. Add 0-RTT resumption (subsequent connections skip the handshake entirely) and you shave 1-2 round trips on every reconnect. That's where the latency gains come from.

But your datacenter leg (CDN to origin) is a different story. It's a stable, low-loss private network. QUIC's benefits disappear on reliable connections. HTTP/2 multiplexing is mature, well-optimized, and doesn't carry QUIC's CPU overhead for cryptography (QUIC encrypts everything at the transport layer, no cleartext, ever).

This is exactly how Cloudflare, Fastly, and Akamai operate: HTTP/3 on the edge-facing side, HTTP/2 (or even HTTP/1.1 with keepalive) on the origin-facing side.

Answer 2

Why A falls short (HTTP/2 only):

HTTP/2 multiplexing is a genuine improvement over HTTP/1.1, but it doesn't solve TCP head-of-line blocking. On lossy mobile connections, you're still at the mercy of the TCP retransmission window. For p99 tail latency on bad networks, HTTP/2 won't move the needle.

Answer 3

Why B is wrong (HTTP/3 only):

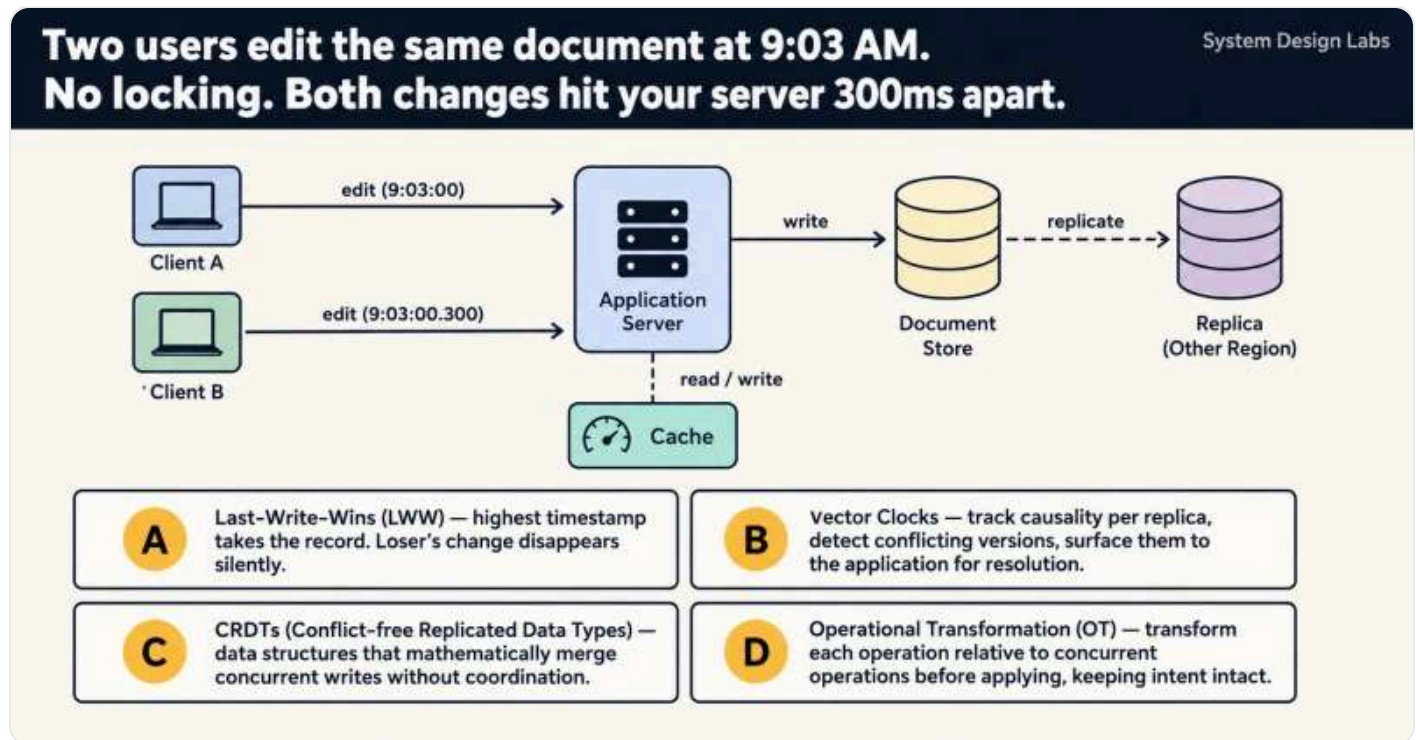
HTTP/3 to origin introduces real operational risk. Enterprise firewalls and some cloud load balancers block UDP port 443 or rate-limit it. Your origin infra isn't the bottleneck, the last mile is. Fixing the origin leg with QUIC doesn't help the problem; it just adds complexity and potential breakage.

Answer 4

Why C is wrong (HTTP/2 end-to-end):

Same problem as A. You've done nothing about the last mile. Consistent HTTP/2 end-to-end sounds clean, but "clean" doesn't show up in your p99.

Multi-Writer Conflicts in Collaborative Editing



Two users edit the same document at 9:03 AM.

No locking. No coordination. Just two clients writing to the same record.

Both changes hit your server 300ms apart. One lands first. The other overwrites it.

User A's work is gone. They never get an error. They just... lose their change.

This is the multi-writer conflict problem. Every collaborative system hits it eventually. Your choices:

A) Last-Write-Wins (LWW) — highest timestamp takes the record. Loser's change disappears silently.

B) Vector Clocks — track causality per replica, detect conflicting versions, surface them to the application for resolution.

C) CRDTs (Conflict-free Replicated Data Types) — data structures that mathematically merge concurrent writes without coordination.

D) Operational Transformation (OT) — transform each operation relative to concurrent operations before applying, keeping intent intact.

Answer 1

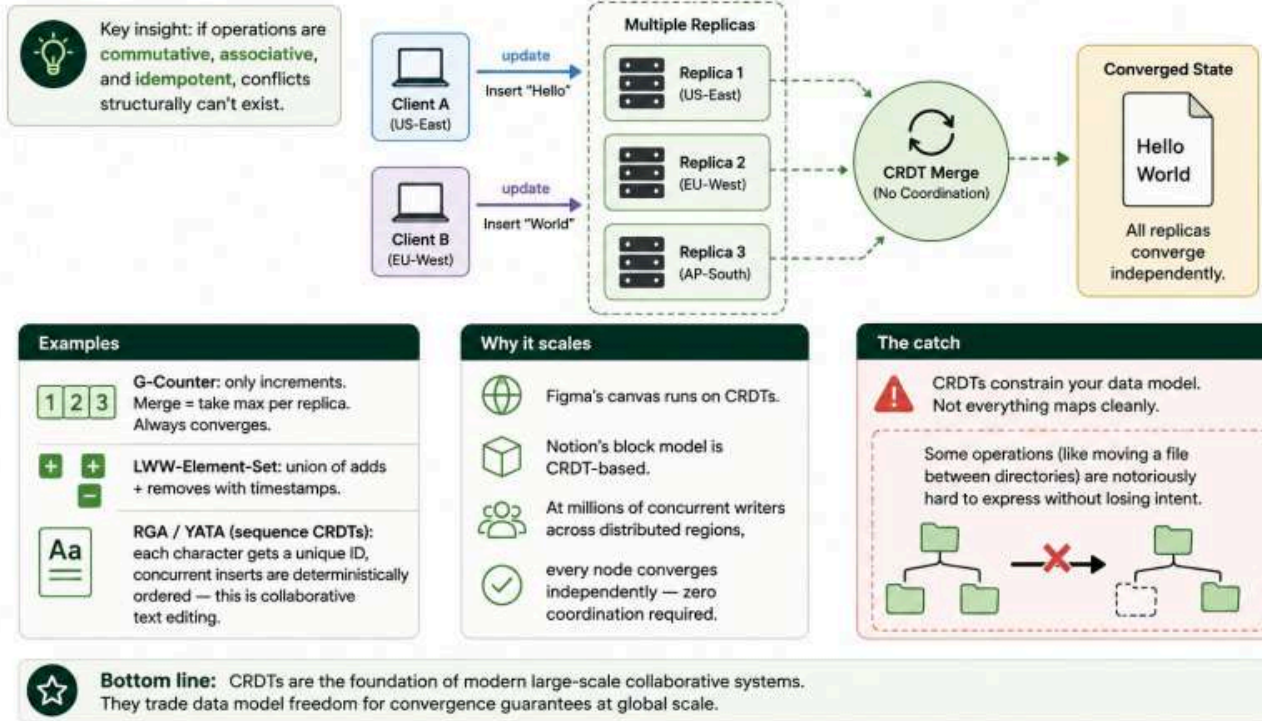
✓ CORRECT ANSWER

C

CRDTs wins at scale

Any two replicas can always be merged — mathematically, without coordination.

System Design Labs



Answer: C — CRDTs (but the real answer depends on your conflict model — full breakdown below)

C — CRDTs wins at scale

CRDTs are data structures designed so any two replicas can always be merged — mathematically, without coordination. The key insight: if operations are commutative, associative, and idempotent, conflicts structurally *can't* exist.

Examples:

- **G-Counter:** only increments. Merge = take max per replica. Always converges.
- **LWW-Element-Set:** union of adds + removes with timestamps.
- **RG / YATA (sequence CRDTs):** each character gets a unique ID, concurrent inserts are deterministically ordered — this is collaborative text editing.

Figma's canvas runs on CRDTs. Notion's block model is CRDT-based. At millions of concurrent writers across distributed regions, every node converges independently — zero coordination required.

The catch: CRDTs constrain your data model. Not everything maps cleanly. Some operations (like moving a file between directories) are notoriously hard to express without losing intent.

Answer 2

D — Operational Transformation (OT) ⚠️ impressive but hard

OT was Google Docs' original approach. When two operations arrive concurrently, you transform each one relative to the other before applying it. User A inserts at position 5, User B deletes at position 3 simultaneously — OT adjusts A's insert position to account for B's delete. Intent preserved.

That's why Google Docs felt magical in 2009.

But OT is operationally brutal. The transformation functions are notoriously hard to get right — the original OT papers had bugs. Every new operation type adds transformation pairs that grow quadratically. OT also requires a centralized server to sequence operations; it doesn't distribute cleanly.

Modern Google Docs has moved toward a hybrid incorporating CRDT-like ideas. OT alone doesn't scale to full distribution.

Answer 3

B — Vector Clocks ⚠️ honest but incomplete

Vector clocks track causality. Each write carries a version vector — one counter per replica. When two writes have no causal relationship (neither "happened before" the other), you know you have a conflict. Both versions surface to the application layer, which resolves them.

Amazon's internal Dynamo used this. It's correct and transparent. The problem: conflict resolution is now your job. You write merge logic for every data type. For shopping carts it's manageable (union the items). For arbitrary documents? It breaks down fast.

Vector clocks tell you *where* conflicts are. They don't tell you *how* to fix them.

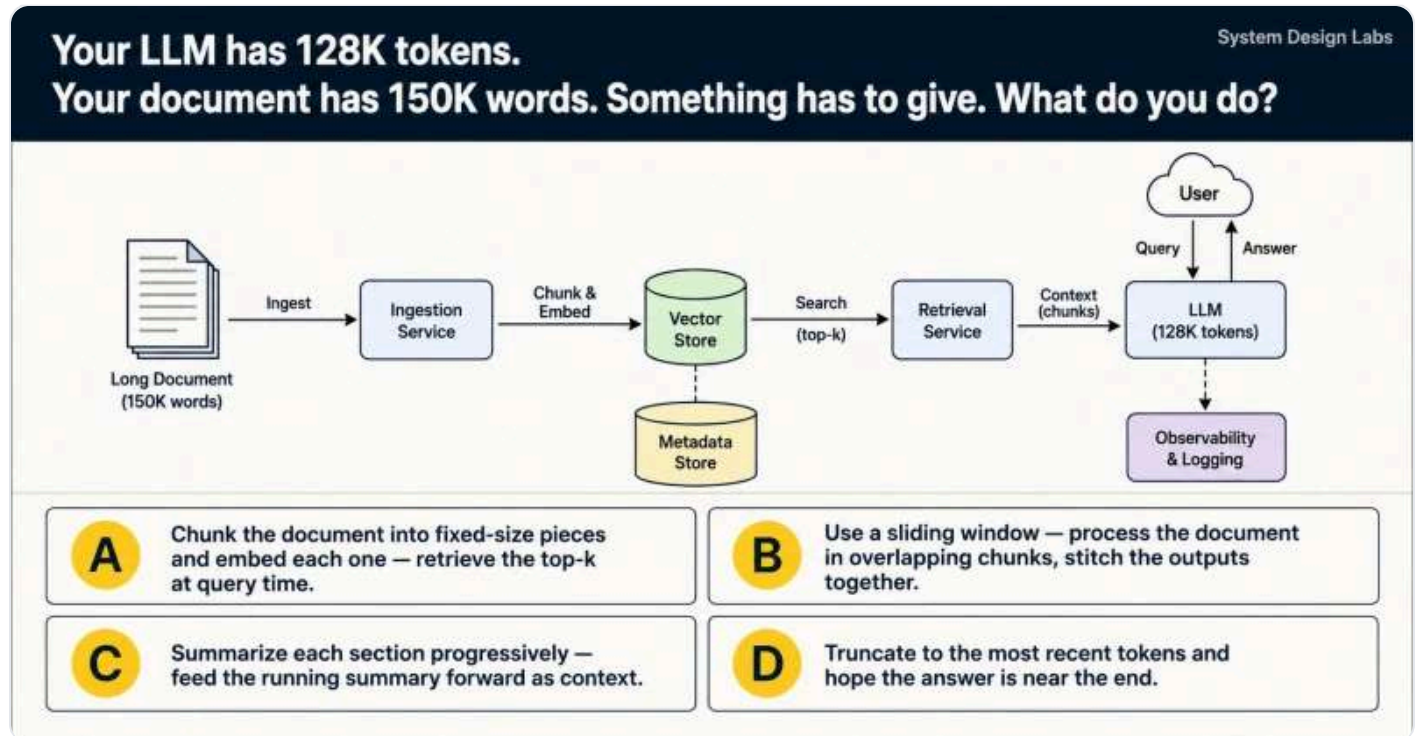
Answer 4

A — Last-Write-Wins (LWW) ❌ dangerous

Simple. Brutally simple. Highest timestamp wins. The loser's change vanishes — no error, no notification, just gone.

Cassandra defaults to this. For append-only workloads (metrics, event logs, telemetry) it's totally fine. For anything a human edited? It's silent data loss with a timestamp attached. The treacherous part: users never get an error. Their work just stops being there.

Fitting a Large Document into an LLM



Your LLM has 128K tokens.

Your document has 150K words.

Something has to give. What do you do?

- A** Chunk the document into fixed-size pieces and embed each one — retrieve the top-k at query time.
- B** Use a sliding window — process the document in overlapping chunks, stitch the outputs together.
- C** Summarize each section progressively — feed the running summary forward as context.
- D** Truncate to the most recent tokens and hope the answer is near the end.

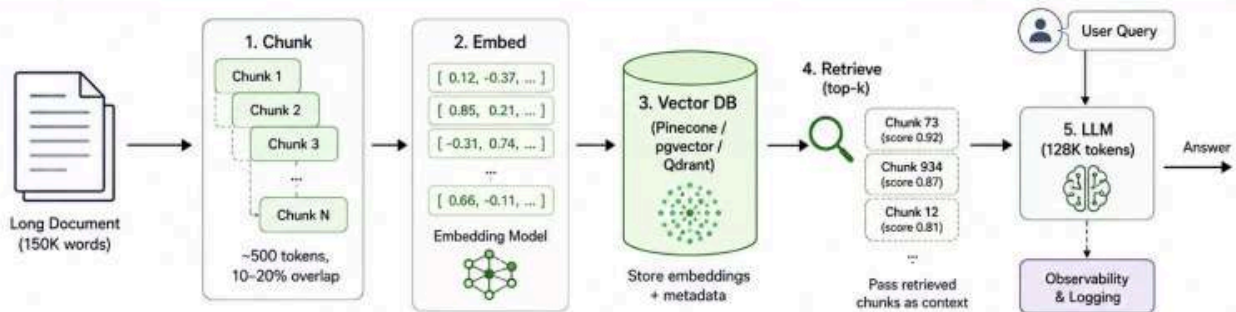
Answer 1

✓ CORRECT ANSWER

WHY A WINS: RAG WITH CHUNKED EMBEDDINGS

System Design Labs

✓ Split the document into overlapping chunks, embed each one, store in a vector DB, retrieve top-k at query time.



SCALES TO ANY SIZE

- LLM only sees retrieved chunks at inference.
- Retrieval is a fast ANN lookup — not a sequential scan.
- Keeps latency low and stays within context limits.



THE CATCH: BOUNDARIES HURT

- Chunk boundaries break semantic coherence.
- If a clause starts on page 12 and resolves on page 13, neither chunk retrieves well.
- Overlap and chunk size tuning matter a lot.



PRODUCTION FIX

- Chunk at sentence or paragraph boundaries, not character counts.
- Preserves meaning and improves retrieval quality.



HYBRID RETRIEVAL IS KEY

- Combine keyword (BM25) + semantic search.
- Covers edge cases semantic search alone misses.
- Most teams underestimate this.

Why A wins (RAG with chunked embeddings):

You split the document into overlapping chunks (~500 tokens, ~10-20% overlap), embed each one, store in a vector DB (Pinecone, pgvector, Qdrant), and retrieve top-k at query time.

This scales to any document size. Your LLM context at inference only sees the retrieved chunks — staying well inside the context window. Latency stays low because retrieval is a fast ANN lookup, not a sequential scan.

The catch: **chunking boundaries destroy semantic coherence**. If a clause starts on page 12 and resolves on page 13, a hard chunk boundary between them means neither chunk retrieves well for that question. This is why overlap and chunk size tuning matter so much. Most teams underestimate this.

Production fix: chunk at sentence or paragraph boundaries, not character counts. Hybrid retrieval (keyword + semantic) handles the edge cases.

Answer 2

Why B is the trap answer (sliding window):

Sliding window looks elegant — process the document in overlapping passes, accumulate the answer. But it doesn't actually fix the context window problem. You still have to process every chunk sequentially, and the answer to a question spanning chunk 1 and chunk 47 gets lost. You're doing $O(n)$ LLM calls per query. At 150K words and a 3s/call LLM, that's minutes per query. Nobody ships this in production for QA on large documents.

It works fine for **summarization** tasks where the output from chunk N feeds into chunk N+1. For retrieval? Wrong tool.

Answer 3

Why C sometimes works (progressive summarization):

For summarizing a 200-page doc? Solid. You map-reduce the document — summarize sections, then summarize the summaries. GPT-4o does this well.

For answering a specific question about clause 38(b) on page 97? It doesn't. The intermediate summary will have compressed away the specific detail you needed. Lossy compression applied to information retrieval is a category error.

Use progressive summarization when you want a high-level understanding. Don't use it when specific facts need to survive to the end of the pipeline.

Answer 4

Why D is obviously wrong (and also surprisingly common):

Truncating to the last N tokens is the default behavior of most LLM API wrappers when you exceed the context limit and don't handle it explicitly. Many teams ship this by accident.

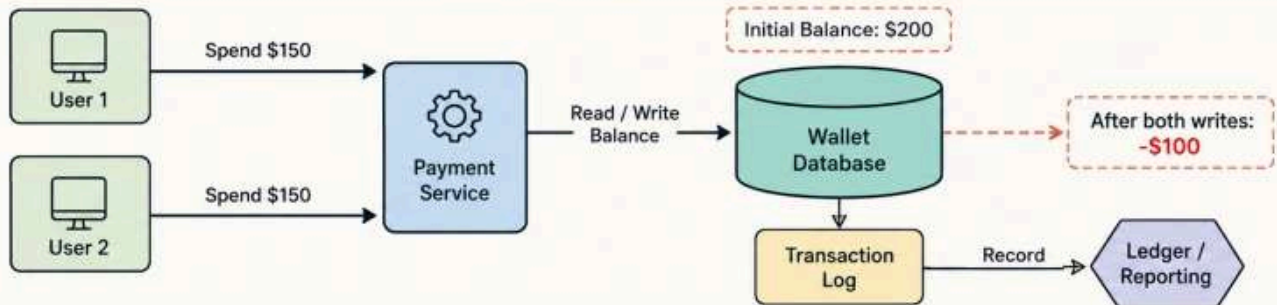
The problem beyond the obvious data loss: LLMs have a "**lost in the middle**" **failure mode** (Liu et al., 2023). Performance on retrieval tasks is highest when the answer is at the start or end of the context window — and degrades significantly when it's buried in the middle. So even if you fit everything in context, position matters.

Truncation compounds this — you're not just losing data, you're losing data unpredictably.

Preventing Concurrent Balance Overspend



Two users read \$200 from the same wallet at the same time. Both spend \$150. Wallet ends up at -\$100. How do you stop this?



A Pessimistic locking — `SELECT FOR UPDATE` on the wallet row. One transaction blocks until the other commits.

B Optimistic locking — read the row with a version number, only write if version hasn't changed since you read it. Retry on conflict.

C MVCC — let both reads see a consistent snapshot, rely on the database to detect write conflicts at commit time.

D Serializable isolation — set the transaction isolation level to `SERIALIZABLE` and let the database handle it.

You have a payment system. Two users try to spend from the same wallet balance at the same time.

Both read \$200. Both want to spend \$150. Both see enough balance. Both write the deduction.

The wallet is now at -\$100. How do you stop this?

A Pessimistic locking — `SELECT FOR UPDATE` on the wallet row. One transaction blocks until the other commits.

B Optimistic locking — read the row with a version number, only write if version hasn't changed since you read it. Retry on conflict.

C MVCC — let both reads see a consistent snapshot, rely on the database to detect write conflicts at commit time.

D Serializable isolation — set the transaction isolation level to `SERIALIZABLE` and let the database handle it.

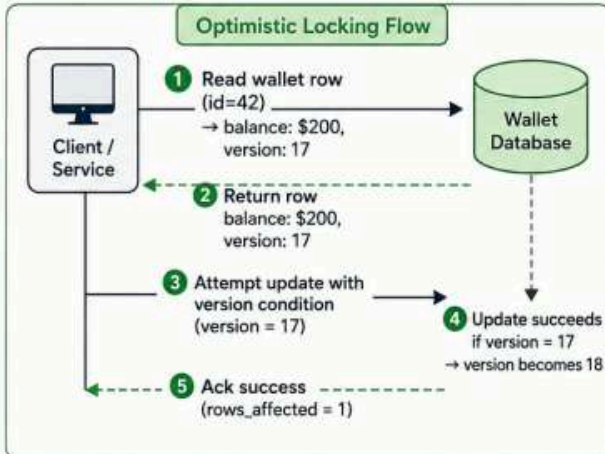
Answer 1

✓ CORRECT ANSWER

B Why B wins (optimistic locking):



You read the wallet row, grab the version number. When you write, you include a **WHERE version = :read_version** condition. If another transaction already updated the row, your **WHERE** matches **zero rows** — conflict detected, retry.



Update Query

```

UPDATE wallets
SET balance = balance - 150, version = version + 1
WHERE id = :wallet_id AND version = :read_version;
  
```



If rows_affected = 0 → conflict → retry.



No locks held during the read. At **10K TPS** with **low conflict rates**, this is significantly faster than pessimistic locking — you're only paying for retry cost on actual conflicts, not lock acquisition on every read.



The catch:

under **HIGH contention** (same hot wallet hit repeatedly), **retry storms** eat you alive.



Redis INCRBY with atomic decrement

OR



Queue in front of the wallet update

Why B wins (optimistic locking):

You read the wallet row, grab the version number. When you write, you include a **WHERE version = :read_version** condition. If another transaction already updated the row, your **WHERE** matches zero rows — conflict detected, retry.

```
UPDATE wallets
```

```
SET balance = balance - 150, version = version + 1
```

```
WHERE id = :wallet_id AND version = :read_version;
```

If rows_affected = 0 → conflict → retry. No locks held during the read. At 10K TPS with low conflict rates, this is significantly faster than pessimistic locking — you're only paying for retry cost on actual conflicts, not lock acquisition on every read.

The catch: under HIGH contention (same hot wallet hit repeatedly), retry storms eat you alive. Hot wallets need Redis INCRBY with atomic decrement, or a queue in front of the wallet update.

Answer 2

Why A is the trap answer (pessimistic locking):

SELECT FOR UPDATE works. It's correct. But it holds a row-level lock for the entire transaction — every concurrent write queues up.

At 10K TPS on a popular wallet, you've serialized all writes to a single queue. Throughput collapses. Lock wait timeouts cascade — failed transactions retry, adding more contention. You've built a self-reinforcing bottleneck.

Use pessimistic locking for low-concurrency paths (admin ops, batch jobs) where conflict probability is near 100%. Not for high-throughput payment writes.

Answer 3

Why C is the silent failure (MVCC + default isolation):

This is the dangerous one. PostgreSQL defaults to READ COMMITTED. MVCC gives each transaction a consistent snapshot — but at READ COMMITTED, that snapshot refreshes per statement, not per transaction.

Two concurrent transactions both read \$200, both pass the "sufficient funds" check, both commit. No conflict detected. **Double spend in production.**

MVCC prevents dirty reads. It does NOT prevent lost updates at READ COMMITTED. Most engineers assume MVCC = safe from concurrent writes. It doesn't.

Answer 4

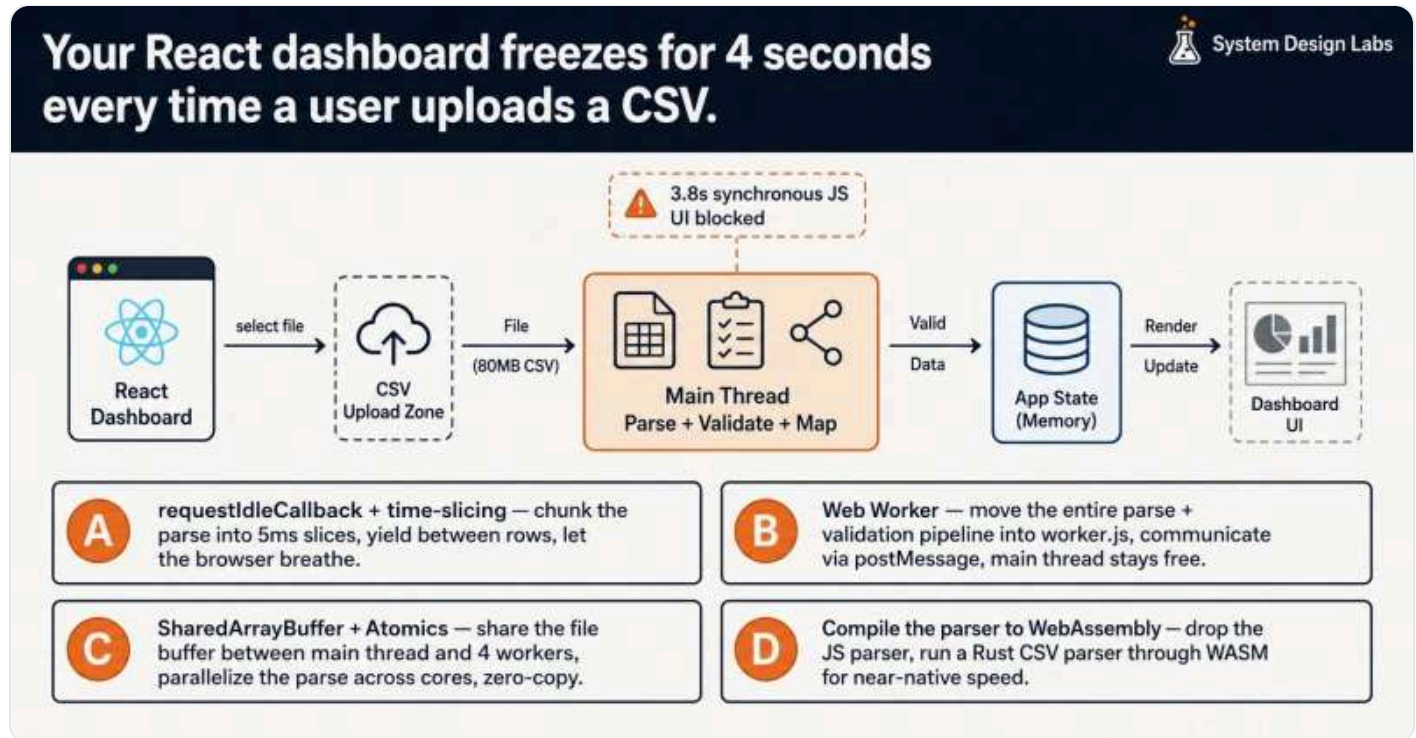
Why D is almost always the wrong default (SERIALIZABLE):

Theoretically correct — the database guarantees transactions behave as if they ran serially. No anomalies.

The problem: PostgreSQL's SSI tracks read-write dependencies across transactions. Under high concurrency, serialization failures spike — transactions abort and retry even when there was no actual conflict. Real throughput penalty at scale.

Right for complex financial workflows where correctness > throughput (end-of-day reconciliation). Wrong default for a high-throughput per-transaction API.

Offloading Heavy Work Off the Main Thread



Your React dashboard freezes for 4 seconds every time a user uploads a CSV.

Not a network freeze. A *browser* freeze. Tab unresponsive. Animations stuck. The "Page Unresponsive" dialog popping up on Chrome.

Here's what's happening in prod:

- User drops a 80MB CSV (200k rows, 40 columns) into the upload zone
- Your code: `Papa.parse(file, { complete: ... })` on the main thread
- Parsing + validation + schema mapping = 3.8s of synchronous JS
- During those 3.8s: scroll dies, the loading spinner stops spinning, input lag goes infinite
- Your INP score on the perf dashboard is now red. Lighthouse is screaming.

Product wants this fixed before the enterprise demo on Friday. Four options on the table:

- A** `requestIdleCallback` + time-slicing — chunk the parse into 5ms slices, yield between rows, let the browser breathe.

B Web Worker — move the entire parse + validation pipeline into `worker.js`, communicate via `postMessage`, main thread stays free.

C `SharedArrayBuffer` + `Atomics` — share the file buffer between main thread and 4 workers, parallelize the parse across cores, zero-copy.

D Compile the parser to WebAssembly — drop the JS parser, run a Rust CSV parser through WASM for near-native speed.

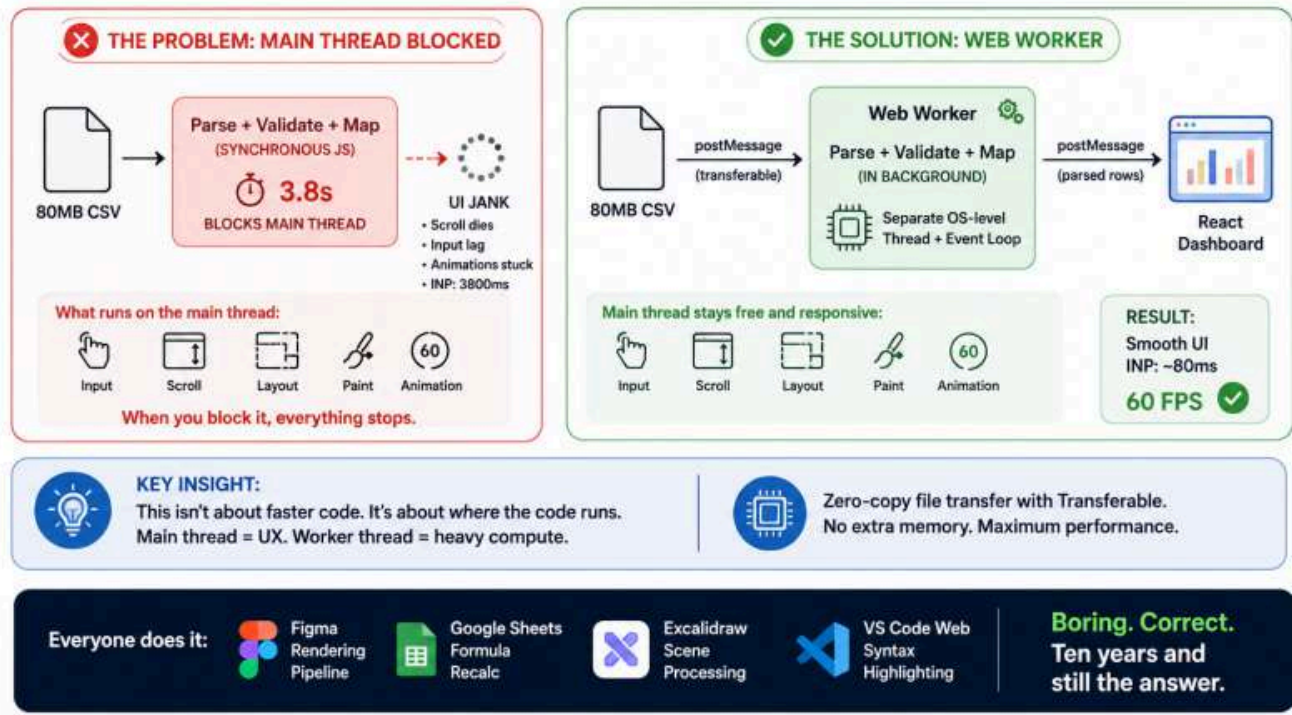
Answer 1

✓ CORRECT ANSWER

Why B Wins: Web Worker



Move the heavy CSV parse + validation off the main thread.
Keep the UI smooth. Always.



Why B wins:

The actual problem isn't *speed*. It's *where the work runs*. The main thread is the same thread that handles scroll, input, layout, paint, and animation. Anything synchronous on it = jank. Period.

A Web Worker is a separate OS-level thread with its own event loop. You ship the file (transferable via `postMessage` with a `Transferable` so it's zero-copy), parse it over there, send back the parsed rows. Main thread stays at 60fps the entire time. INP drops from 3800ms to ~80ms. Done.

This is what Figma does for its rendering pipeline, what Google Sheets does for formula recalc, what Excalidraw does for its scene processing, what VS Code Web does for syntax highlighting. Every serious browser-based product runs heavy compute in a worker. It is the boring, correct, ten-year-old answer.

Answer 2

Why A is wrong (the "clever" trap):

requestIdleCallback + time-slicing keeps the UI *technically* responsive but stretches the parse from 3.8s to ~6s of wall-clock time, because you're constantly yielding. Worse: idle callbacks are deprioritized — under load, they don't fire at all. React 18's concurrent rendering uses time-slicing for *rendering*, not for blocking IO-shaped work. Wrong tool.

Answer 3

Why C is wrong (the SENIOR ENGINEER TRAP):

SharedArrayBuffer + Atomics is the answer that sounds smart in a design review. "We'll parallelize across cores! Zero-copy! Lock-free!" Then you discover:

- It requires COOP + COEP headers site-wide, which break Stripe Elements, YouTube embeds, Intercom, and half your analytics scripts.
- A single 80MB CSV parse is *embarrassingly serializable* — splitting rows across 4 cores saves you ~2s while costing you weeks of cross-origin isolation hell.
- You now own a custom thread pool in JavaScript. In 2026.

SAB is the right answer for *exactly two* problems: real-time audio/video processing (where you need lock-free ring buffers between threads at 48kHz), and shared simulation state (think multiplayer game engines in the browser). A one-shot CSV upload is neither.

Answer 4

Why D is wrong:

WASM makes the parser *faster*. It doesn't move it *off the main thread*. A WASM parser running synchronously on the main thread still freezes the UI — just for 1.2s instead of 3.8s. You'd still need to put it inside a worker. So D isn't an answer; it's a possible optimization *inside* B. And honestly, Papa Parse in a worker is fast enough that you don't need it.

Moving from Batch to Real-Time Streaming



Your data team just got a new SLA: surface fraud signals within 500ms of a transaction.

Right now you're running nightly Spark batch jobs. The business wants "real-time." Your team knows Spark. Someone already opened a PR adding Spark Structured Streaming.

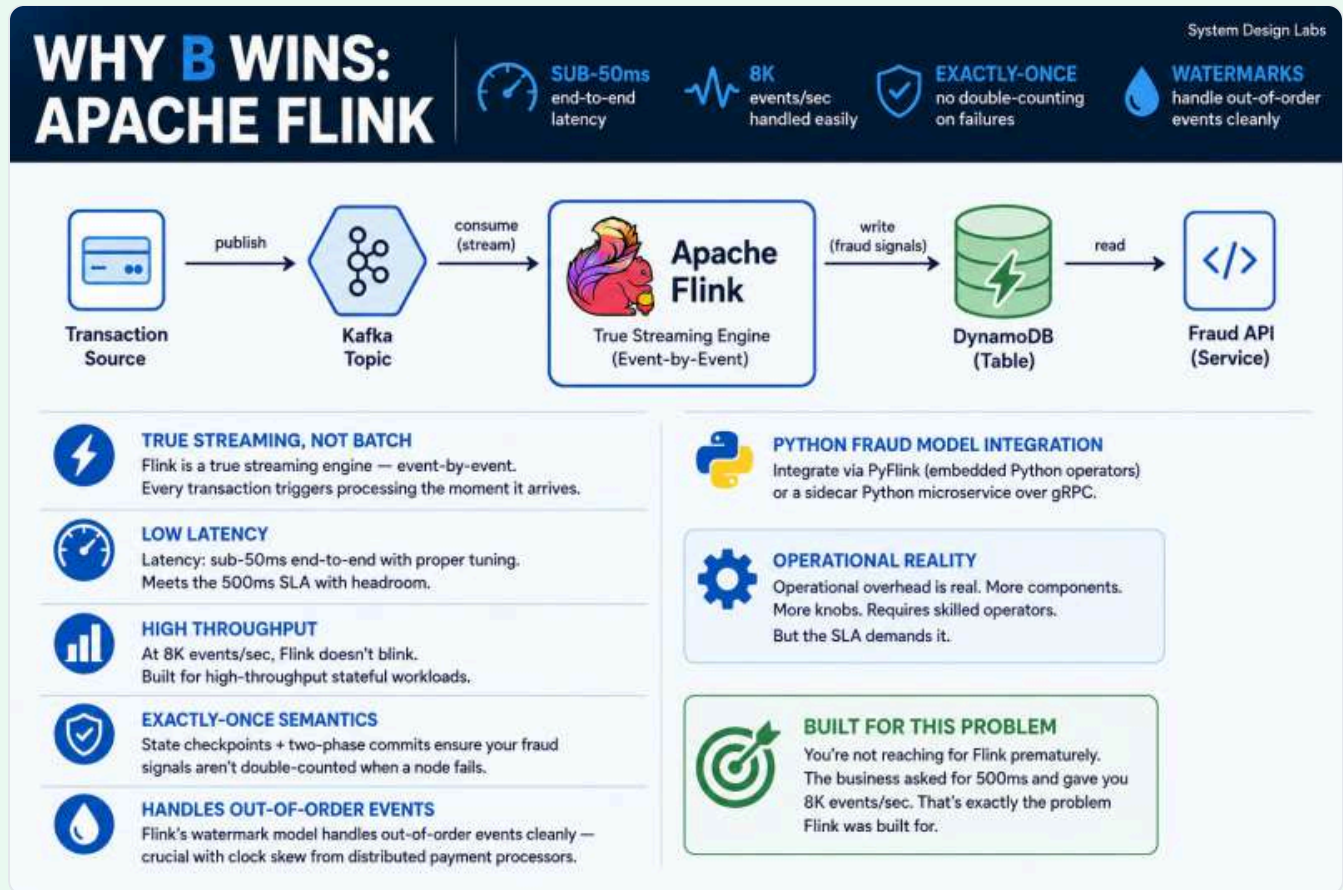
The transaction volume: 8,000 events/sec peak. You're on AWS. The fraud model runs in Python. The output feeds a DynamoDB table the API reads from.

You need to redesign the pipeline. What do you pick?

- A) Kafka Streams — event-by-event processing, stateful operators, sub-10ms latency. Lives inside your app JVM.
- B) Apache Flink — true streaming engine, exactly-once semantics, built for high-throughput stateful processing.
- C) Spark Structured Streaming — micro-batch under the hood, 100ms–5s windows, same API your team already knows.
- D) Keep the batch job, drop the window to 1 minute — "near real-time" at zero migration cost.

Answer 1

✓ CORRECT ANSWER



Why B wins (Apache Flink):

Flink is a true streaming engine — event-by-event, not batch. Every transaction triggers processing the moment it arrives. Latency: sub-50ms end-to-end with proper tuning. At 8K events/sec, Flink doesn't blink. Exactly-once semantics mean your fraud signals aren't double-counted when a node fails. The Python fraud model integrates via PyFlink or a sidecar microservice. Flink's watermark model handles out-of-order events cleanly — which matters when transactions arrive from distributed payment processors with clock skew.

Operational overhead is real. But the SLA demands it. You're not reaching for Flink prematurely here — the business asked for 500ms and gave you 8K events/sec. That's exactly the problem Flink was built for.

Answer 2

Why A is close but not the cleanest fit (Kafka Streams):

Kafka Streams is event-by-event too — sub-10ms achievable. The constraint: it runs inside your application JVM. Your fraud model is Python. That means Kafka Streams handles the stream topology, but the model scoring lives in a separate Python service, adding a network hop. Flink handles that boundary more cleanly with PyFlink. Kafka Streams is the right call if the fraud logic is stateful JVM-native aggregation. Wrong default when the model is a Python service.

Answer 3

Why C is the senior engineer trap (Spark Structured Streaming):

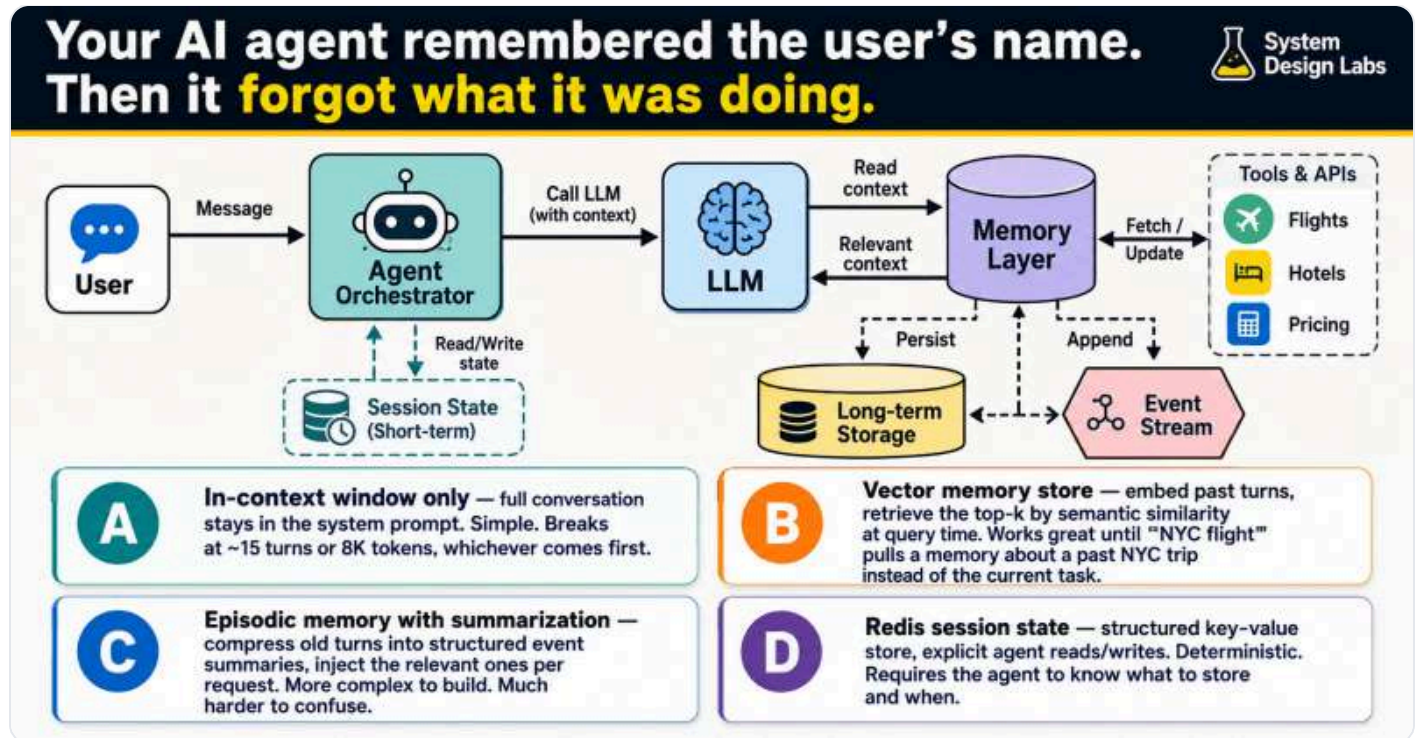
The PR is already open. The API looks familiar. But Structured Streaming is micro-batch — it collects events for a trigger interval (practically 100ms–5s under production load), then processes them as a mini-batch. You'll tune it, it'll pass staging, and it'll breach SLA the first time peak traffic hits. Mistaking "Streaming" in the name for actual event-by-event processing is the trap.

Answer 4

Why D is eliminated immediately (batch, 1-minute windows):

Minimum latency = the window size. At 60 seconds you're 120x over SLA before the first line of fraud logic runs. "Near real-time" is not a SLA.

Designing Memory for an AI Agent



Your AI agent remembered the user's name.

Then it forgot what it was doing.

Here's the setup:

User asks the agent: book the cheapest flight to NYC, search hotels under \$150/night, then compare total trip cost.

By step 3, the agent calls the LLM with 8,000 tokens of raw conversation history — and still answers as if it's turn 1.

You need a memory architecture before this ships. Which one do you pick?

A) In-context window only — full conversation stays in the system prompt. Simple. Breaks at ~15 turns or 8K tokens, whichever comes first.

B) Vector memory store — embed past turns, retrieve the top-k by semantic similarity at query time. Works great until "NYC flight" pulls a memory about a past NYC trip instead of the current task.

C) Episodic memory with summarization — compress old turns into structured event summaries, inject the relevant ones per request. More complex to build. Much harder to confuse.

D) Redis session state — structured key-value store, explicit agent reads/writes. Deterministic. Requires the agent to know what to store and when.

Answer 1

✓ CORRECT ANSWER



C — Episodic memory with summarization

System Design Labs

Right answer for task-oriented agents.



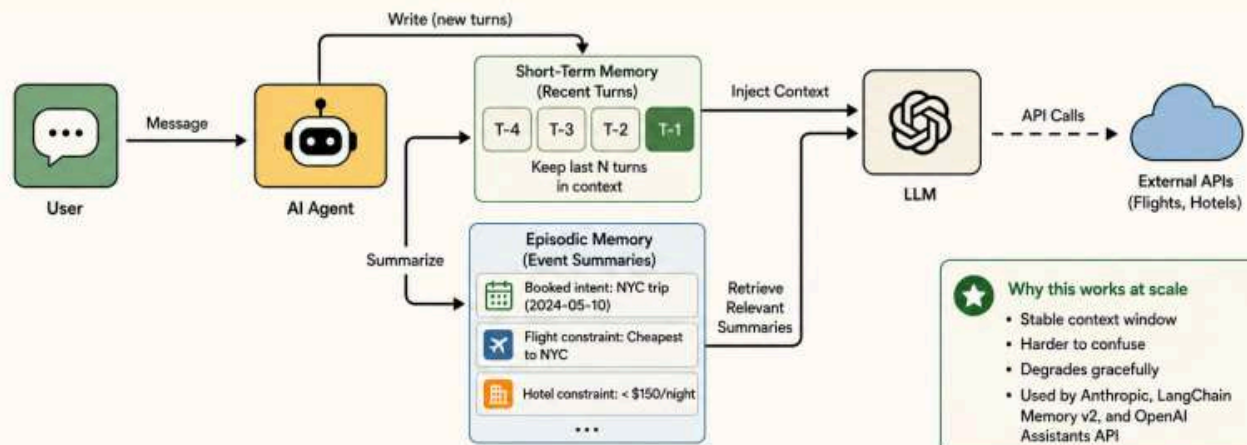
→ Keep the last N turns in context (short-term)



→ Compress older turns into structured event summaries (episodic)



→ Inject only relevant summaries per new request



Why this works at scale

- Stable context window
- Harder to confuse
- Degrades gracefully
- Used by Anthropic, LangChain Memory v2, and OpenAI Assistants API



A In-context window only — full conversation stays in the system prompt. Simple. Breaks at ~15 turns or 8K tokens, whichever comes first.



B Vector memory store — embed past turns, retrieve the top-k by semantic similarity at query time. Works great until "NYC flight" pulls a memory about a past NYC trip instead of the current task.



C Episodic memory with summarization — compress old turns into structured event summaries, inject the relevant ones per request. More complex to build. Much harder to confuse.



D Redis session state — structured key-value store, explicit agent reads/writes. Deterministic. Requires the agent to know what to store and when.

C — Episodic memory with summarization

Right answer for task-oriented agents. The pattern:

- Keep the last N turns in context (short-term)
- Compress older turns into structured event summaries (episodic)
- Inject only relevant summaries per new request

It's how production agents at Anthropic, LangChain Memory v2, and OpenAI Assistants API all converge. More engineering upfront — but the only pattern that degrades gracefully at scale.

Answer 2

A — In-context window only

Collapses fast. At ~15 turns or 8K tokens you're either truncating early context or hitting the model's limit. Works for demos. Fails in production agents with multi-step tasks. The "just increase context length" argument ignores cost and the lost-in-the-middle problem — LLMs reliably ignore content buried in the middle of long prompts.

Answer 3

B — Vector memory store

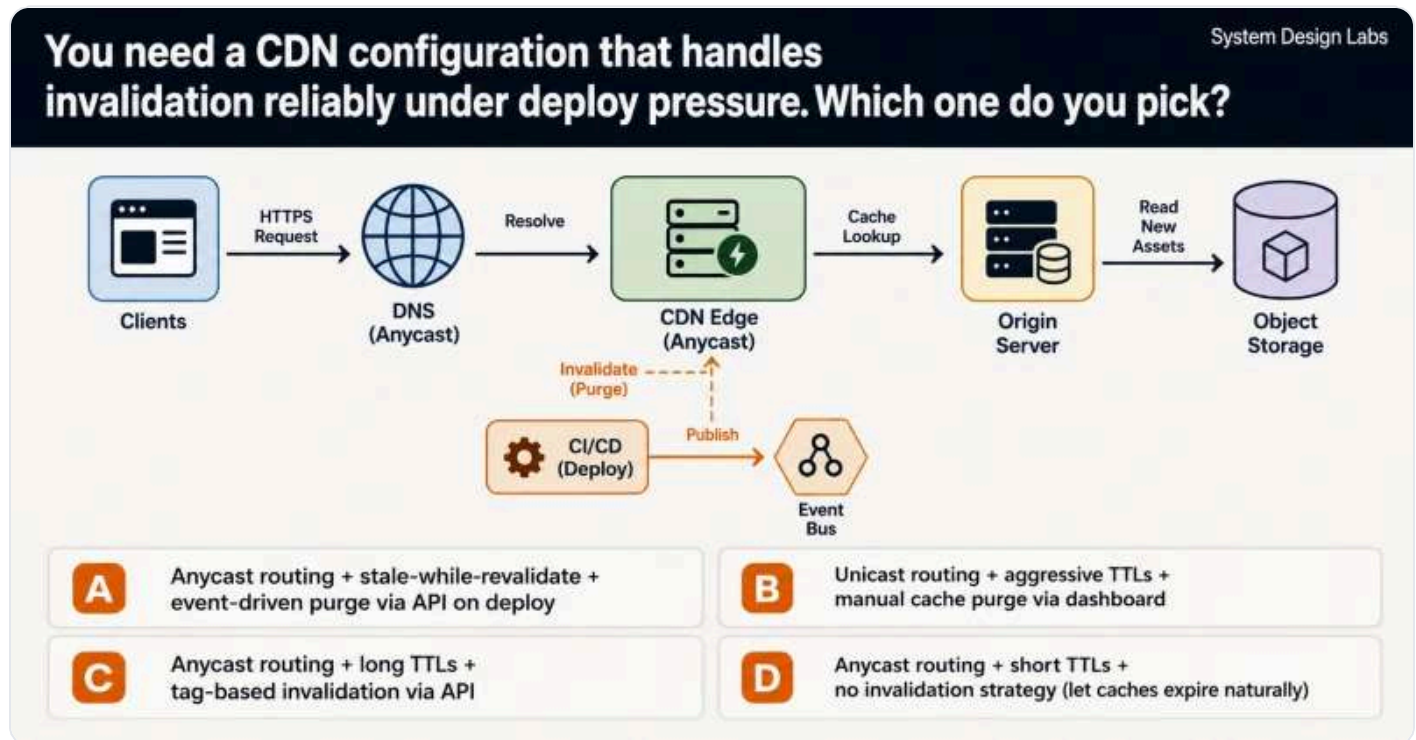
The retrieval problem is nastier than it looks. Semantic similarity isn't the same as task relevance. In a multi-step booking task, embedding "find hotels under \$150" might retrieve a memory from a different user session — or a past conversation about a different city entirely. Great for knowledge retrieval. Unreliable for procedural task state.

Answer 4

D — Redis session state

Solid and deterministic. The catch: the agent has to explicitly decide what to write to state and when to read it. That's a non-trivial design problem — you're building a working memory protocol on top of your tool calls. Most teams underestimate the schema design this requires.

Reliable CDN Cache Invalidation on Deploy



Your CDN saved you from explaining a 400ms load time to your VP.

Then your deploy shipped. And nobody got the fix.

Here's what happened:

Your edge nodes were still serving the old JS bundle — TTL hadn't expired. You triggered a cache purge manually. But you hit the wrong environment. Prod kept serving the stale version for 6 more hours.

You need a CDN configuration that handles invalidation reliably under deploy pressure. Which one do you pick?

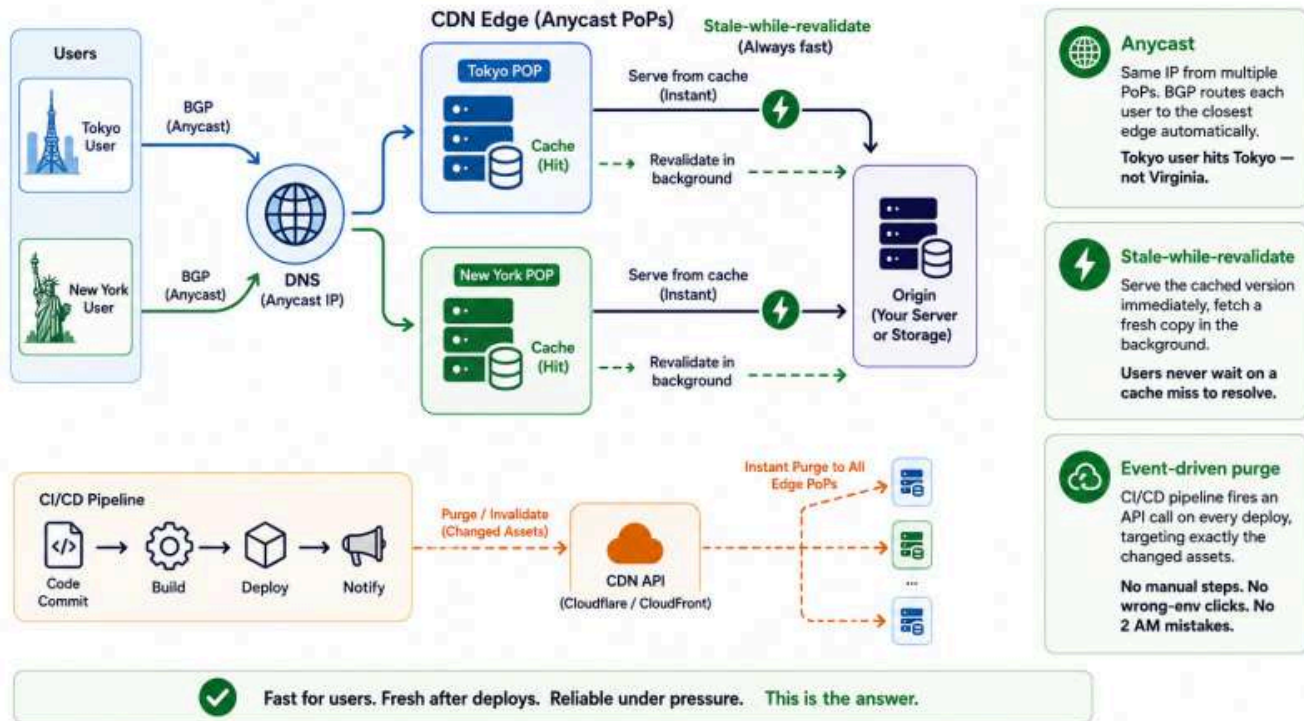
- A) Anycast routing + stale-while-revalidate + event-driven purge via API on deploy
- B) Unicast routing + aggressive TTLs + manual cache purge via dashboard
- C) Anycast routing + long TTLs + tag-based invalidation via API
- D) Anycast routing + short TTLs + no invalidation strategy (let caches expire naturally)

Answer 1

✓ CORRECT ANSWER

A Anycast + stale-while-revalidate + event-driven purge via API on deploy

System Design Labs



A — Anycast + stale-while-revalidate + event-driven purge — the right answer

Anycast advertises the same IP from multiple PoPs. BGP routes each user to the closest edge automatically. Tokyo user hits Tokyo — not Virginia.

Stale-while-revalidate: serve the cached version immediately, fetch a fresh copy in the background. Users never wait on a cache miss to resolve.

Event-driven purge: your CI/CD pipeline fires a Cloudflare or CloudFront API call on every deploy, targeting exactly the changed assets. No manual steps. No wrong-env clicks. No 2 AM mistakes.

Answer 2

C — Anycast + long TTLs + tag-based invalidation — solid, but has a trap

Tag-based invalidation (Cloudflare Cache-Tag, Fastly surrogate keys) lets you group objects and purge entire logical groups in one API call. New product images? Invalidate tag:product-images. Clean and surgical.

The trap: long TTLs work perfectly when your deploy pipeline is robust. But if your invalidation API call fails silently during an incident — when you're moving the fastest — you're serving the wrong version for hours with no TTL fallback to save you.

Option A's stale-while-revalidate degrades more gracefully when your pipeline gets messy.

Answer 3

B — Unicast + aggressive TTLs + manual purge

Unicast means one edge region. Your Tokyo users are routing to Virginia. You've deployed a CDN and you're routing past it.

Manual dashboard purges during a deploy: someone will forget, click the wrong environment, or skip it entirely at 2 AM. Aggressive TTLs shrink the damage window — they don't make invalidation reliable.

Answer 4

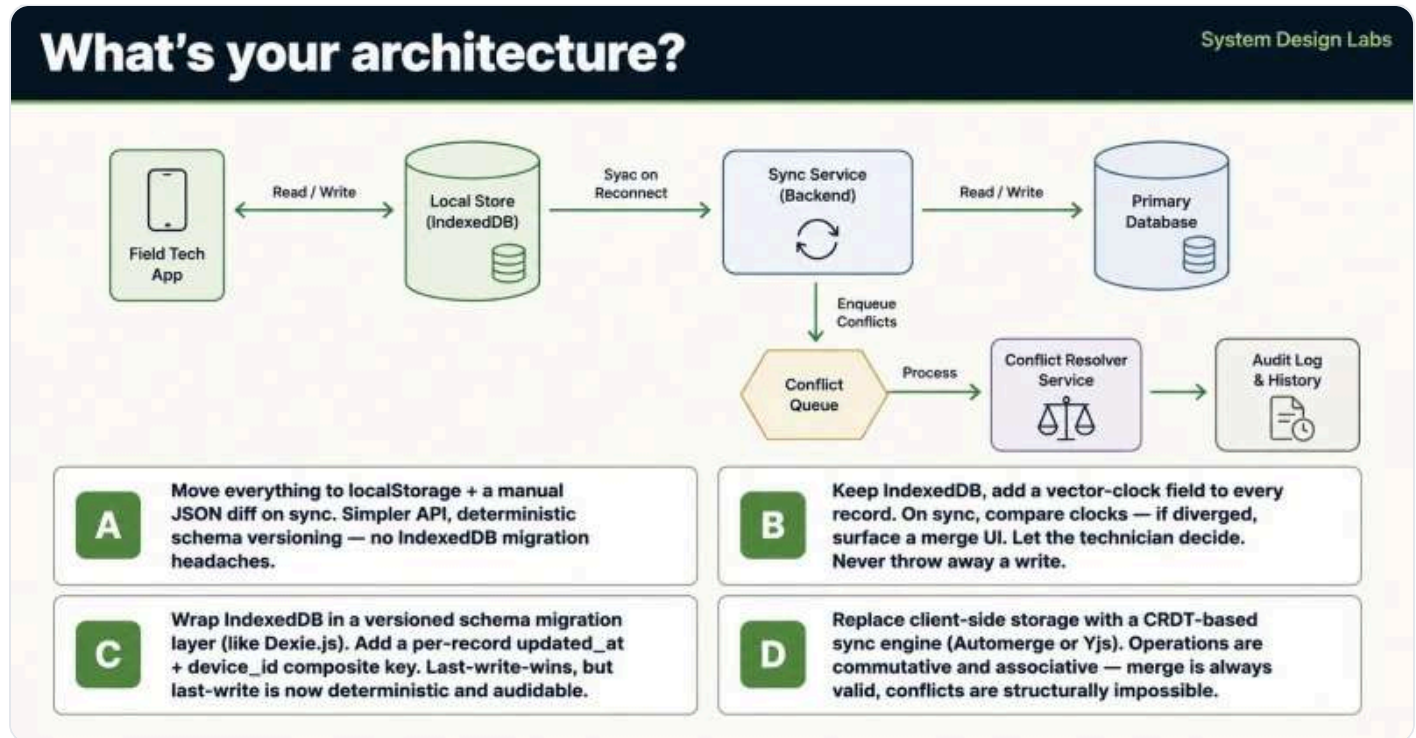
D — Short TTLs, no invalidation

"Short TTLs = always fresh" is the most expensive misconception in CDN config.

Every TTL expiry is a potential cache stampede. 200k RPS at peak, TTL expires on a hot endpoint, every PoP fires a simultaneous request to your origin. You go from 0 to 200k RPS in 2 seconds.

And you have zero control for hotfixes. You just wait for the TTLs to drain.

Syncing Offline Edits Without Data Loss



Your team just shipped "offline mode" for your field-service app.

Technicians work in basements, tunnels, plant floors — connectivity is unreliable.

The demo looked great. [Navigator.onLine](#) said false, the app kept working, the sync button pulsed green. You shipped to 400 users.

Then the incident reports started.

Technician in Munich finishes a repair job offline, syncs when she gets signal. Her updates are gone — overwritten by a colleague who edited the same record online 12 minutes earlier. Last-write-wins. Her write lost.

Technician in São Paulo opens the app, goes offline, edits three assets. Comes back online. App throws an unhandled promise rejection and crashes. IndexedDB schema was on version 2. The update shipped version 3. The migration never ran because he'd never opened the app while online.

You're now asked to actually fix offline-first — not demo it.

Here's the setup:

- 400 field technicians, avg offline window of 40 minutes

- Write conflicts happen ~3x per day, always on the same 8 "hot" assets
- Sync currently runs on reconnect via a single bulk POST
- IndexedDB is being used but schema migrations are undocumented
- You need conflict resolution, migration safety, and sync reliability

What's your architecture?

A) Move everything to localStorage + a manual JSON diff on sync. Simpler API, deterministic schema versioning — no IndexedDB migration headaches.

B) Keep IndexedDB, add a vector-clock field to every record. On sync, compare clocks — if diverged, surface a merge UI. Let the technician decide. Never throw away a write.

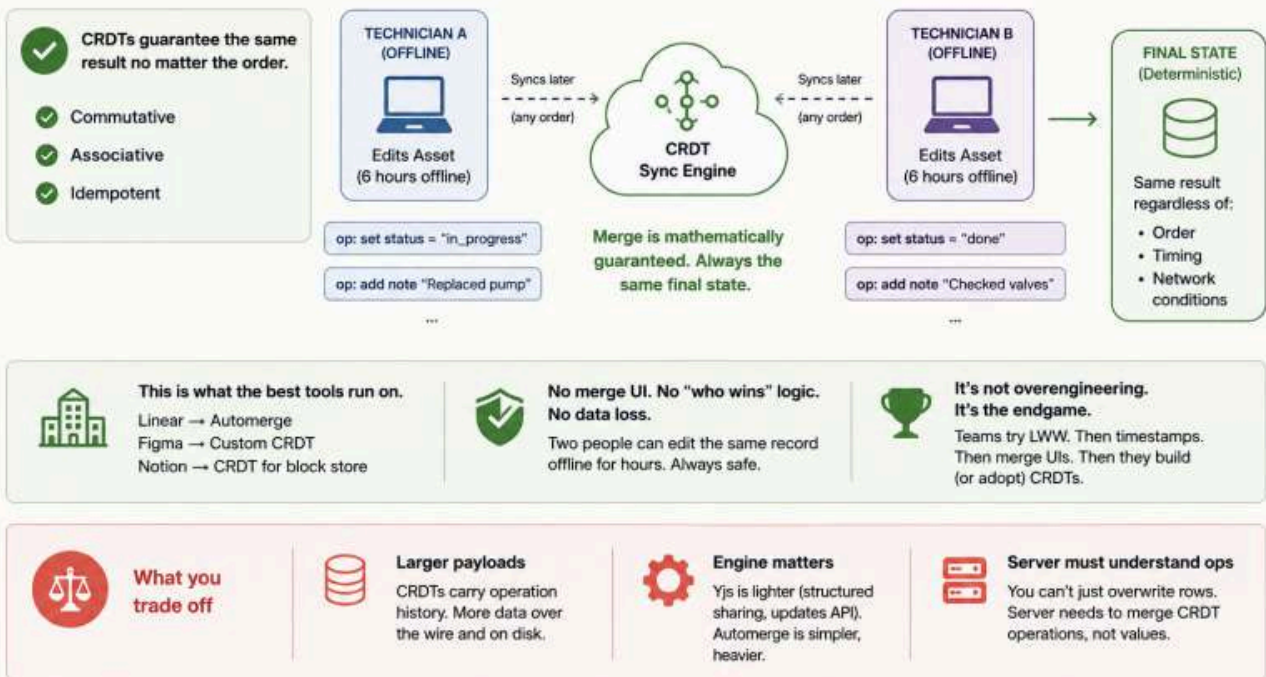
C) Wrap IndexedDB in a versioned schema migration layer (like Dexie.js). Add a per-record updated_at + device_id composite key. Last-write-wins, but last-write is now deterministic and auditable.

D) Replace client-side storage with a CRDT-based sync engine (Automerge or Yjs). Operations are commutative and associative — merge is always valid, conflicts are structurally impossible.

Answer 1

✓ CORRECT ANSWER

Why D wins: CRDT Engine — Automerge / Yjs



Bottom line: If you need offline + multi-writer + zero data loss, CRDTs are the only approach with a mathematical guarantee.

Why D wins (CRDT engine — Automerge / Yjs):

CRDTs (Conflict-free Replicated Data Types) are the only approach where merge is mathematically guaranteed to produce the same result regardless of the order operations arrive. They're commutative, associative, and idempotent.

Two technicians can edit the same asset record offline for 6 hours, sync in any order — and the final state is always deterministic. No merge UI. No "who wins" logic. No data loss.

Linear uses Automerge. Figma uses a custom CRDT for multiplayer. Notion runs a similar model for their block store. This isn't overengineering — it's what teams reach after trying everything else.

Tradeoff: CRDT payloads are larger (they carry operation history). Yjs is lighter than Automerge for most cases. Server-side merge logic needs to understand CRDT ops, not just overwrite rows.

Answer 2

Why B is the correct second choice (Vector Clocks + Merge UI):

Vector clocks detect *exactly* when a conflict occurred — which device wrote what and in what order. When diverged, you surface a merge UI and let the technician decide.

Right answer when humans must stay in the loop. A technician marking an asset "inspected" vs another marking it "needs repair" — that's not a conflict an algorithm should auto-resolve.

Tradeoff: every conflict requires user action. At 3/day it's manageable. At 300/day it's a support nightmare.

Answer 3

Why C is good but not enough (Versioned IndexedDB + LWW):

Dexie.js makes IndexedDB migrations genuinely manageable. LWW with a deterministic composite key (updated_at + device_id) is production-grade.

But LWW throws away real writes. The Munich technician *always loses* to anyone who touched the same record online in the interim. In a field-service app where "inspected" vs "not inspected" has regulatory consequences — that's data loss you can't accept.

LWW works when the cost of losing a write is low (a UI preference). It breaks when every write has operational meaning.

Answer 4

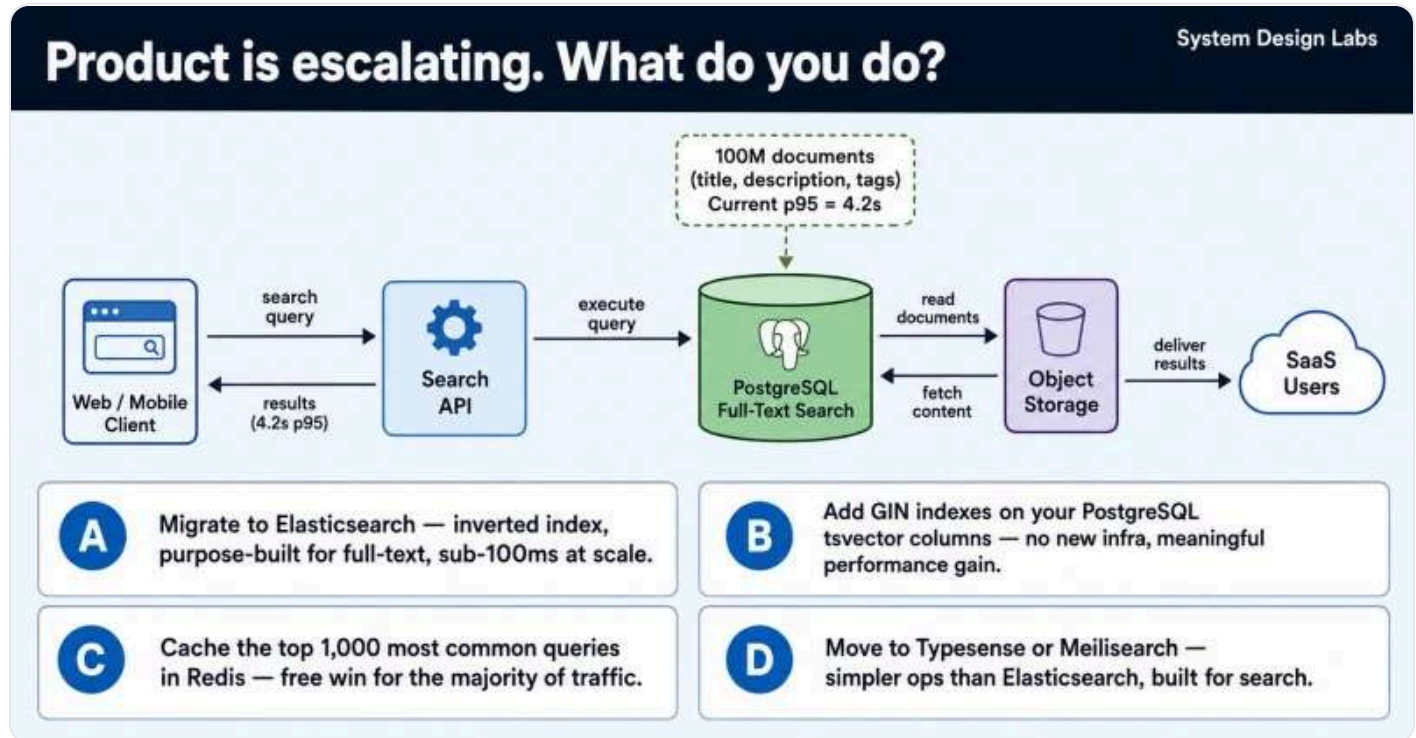
Why A fails (localStorage + JSON diff):

localStorage is hard-capped at ~5MB. At 200 records with attachment metadata, you'll hit it — silently, with no useful error in older Chrome. No indexing. No range queries. In private/incognito mode, it's scoped per-tab and wiped on close.

JSON diff breaks on nested arrays, partial field updates, deletions. You'd end up building a mini-CRDT by hand without the guarantees.

Works in a weekend prototype. Corrupts silently in production.

Scaling Full-Text Search



Your search box returns results in 80ms.

Then you add 50M more documents. Now it's 4 seconds.

Your DBA says "add an index." Your search engineer says "that's not how this works."

They're both right — about different things.

Here's the setup:

Platform: SaaS product, 100M documents

Search fields: title, description, tags, partial phrases

Current stack: PostgreSQL full-text search

Current perf: p95 = 4.2s

Target: sub-200ms at p99

Product is escalating. What do you do?

- A** Migrate to Elasticsearch — inverted index, purpose-built for full-text, sub-100ms at scale.

B Add GIN indexes on your PostgreSQL tsvector columns — no new infra, meaningful performance gain.

C Cache the top 1,000 most common queries in Redis — free win for the majority of traffic.

D Move to Typesense or Meilisearch — simpler ops than Elasticsearch, built for search.

Answer 1

✓ CORRECT ANSWER

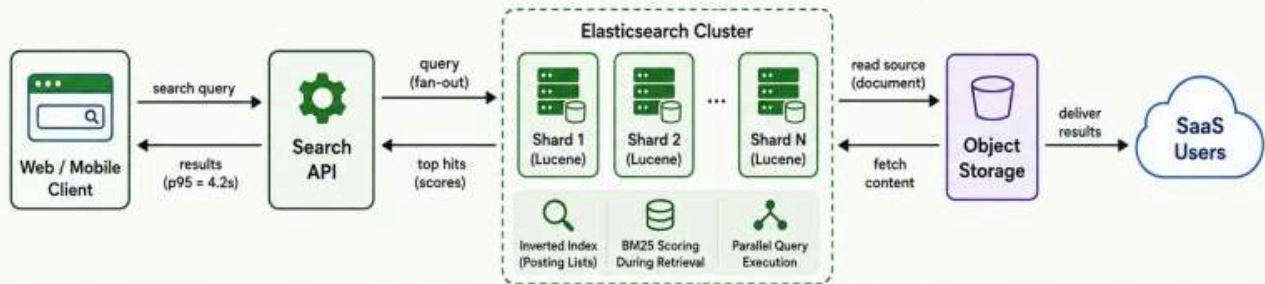
**ANSWER: A – Elasticsearch with inverted indexes**

System Design Labs

WHY A WINS

Elasticsearch uses inverted indexes. Queries do fast posting-list intersections instead of table scans. Scales to 100M+ docs with horizontal sharding and parallel query fan-out for sub-100ms at p99.

Target:
sub-200ms at p99

**A**

Migrate to Elasticsearch — inverted index, purpose-built for full-text, sub-100ms at scale.

B

Add GIN indexes on your PostgreSQL tsvector columns — no new infra, meaningful performance gain.

C

Cache the top 1,000 most common queries in Redis — free win for the majority of traffic.

D

Move to Typesense or Meilisearch — simpler ops than Elasticsearch, built for search.

Why A wins (Elasticsearch):

Elasticsearch is built on an inverted index. When you index a document, Elasticsearch tokenizes the text and writes posting lists: "database" → [doc_1, doc_5, doc_23...], "index" → [doc_2, doc_5, doc_18...]. A query for "database index" does a fast posting-list intersection — not a table scan.

At 100M documents, that intersection still runs in milliseconds because posting lists are stored in delta-compressed sorted form, Lucene segments are immutable (no row locking, no MVCC overhead), horizontal sharding is native (queries fan out in parallel), and BM25 scoring runs during retrieval — not after it.

Result: sub-100ms at p99, even at 100M+ docs, with proper shard sizing. Netflix, Uber, and GitHub all run some form of this at their search layer.

The tradeoffs you accept: new infrastructure to operate (index lifecycle, shard rebalancing), write amplification from segment merging, and eventual consistency — by default, newly indexed docs aren't searchable for ~1 second.

Answer 2

Why B is the trap answer (GIN indexes on PostgreSQL):

GIN indexes on tsvector are genuinely good. For 1M–5M documents, this is the right call — no new infra, no ops burden, real improvement.

At 100M documents, you hit three walls: index size balloons (GIN is large — easily 40–80GB), PostgreSQL full-text has no native horizontal read scaling, and BM25 relevance scoring is bolted on, not native. You get from 4.2s down to maybe 500ms. You need 200ms. Close, but the wrong tool at this scale.

GIN is what you reach for before you need a dedicated search engine, not instead of one.

Answer 3

Why C misses the point (Redis cache):

Caching top queries is a real optimization — but it's not a search architecture. Your top 1,000 queries might cover 40% of traffic. The other 60% hits the slow path. Worse: every document update invalidates your cache logic. You've added complexity without touching the tail latency product is actually complaining about.

C is a band-aid. The p99 problem is still there the moment any user types something that isn't in the cache.

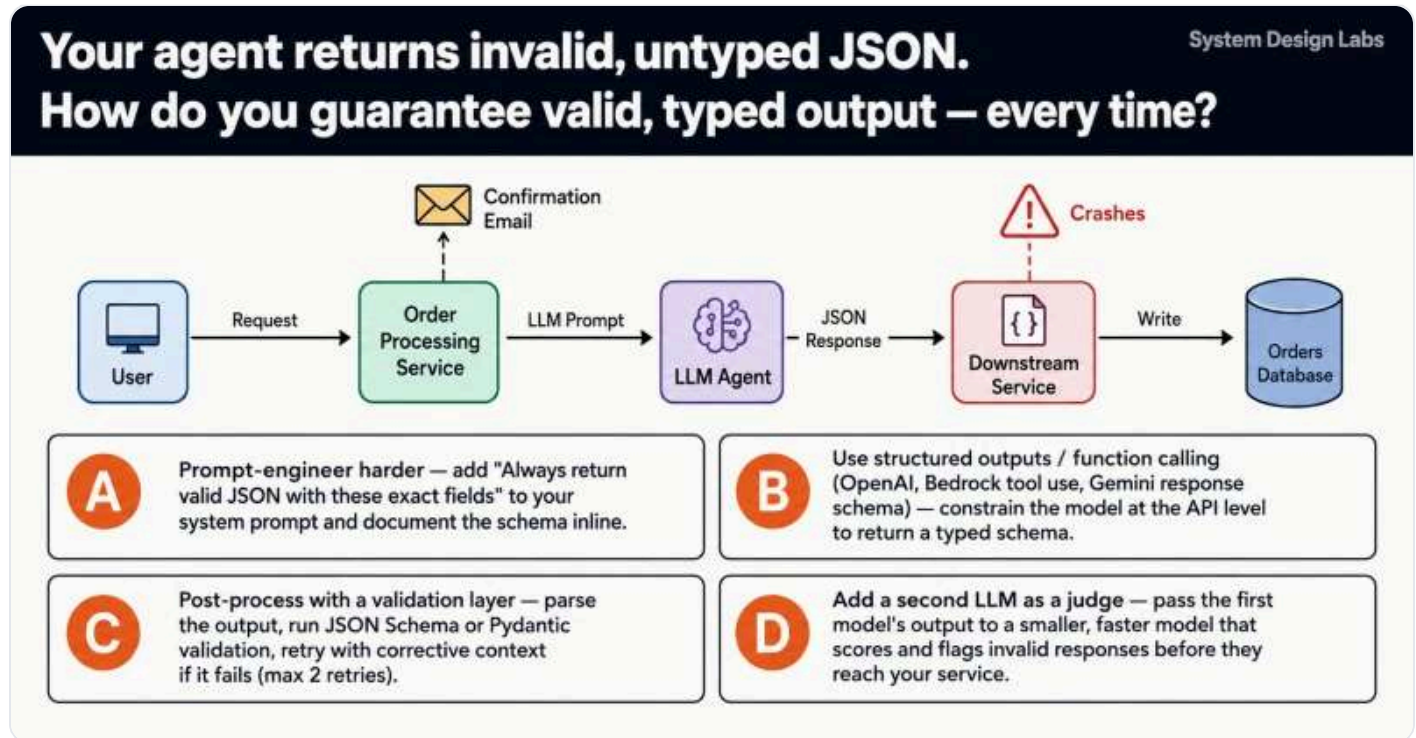
Answer 4

Why D is tempting but limited (Typesense / Meilisearch):

Typesense and Meilisearch are excellent — simpler ops than Elasticsearch, great developer experience, built-in typo tolerance. For 10M–30M documents, D is a legitimate answer that wins on simplicity.

At 100M documents with strict p99 requirements and complex relevance tuning, you start hitting their limits: distributed indexing is less mature, fine-grained scoring customization is constrained, and operational tooling at scale is thinner than Elasticsearch's ecosystem. For a smaller product, D is the right call. At 100M with a p99 SLA, Elasticsearch is where you end up.

Enforcing Structured Output from an LLM



Your production LLM agent just returned this JSON to your order processing service:

```
{
  "action": "refund",
  "amount": "fifty dollars",
  "order_id": null,
  "confidence": "pretty high"
}
```

Your downstream service crashes. The retry hits the same model. Same broken output. The refund never fires — but the user got a confirmation email.

You need your agent to return valid, typed, structured output — every time. What do you do?

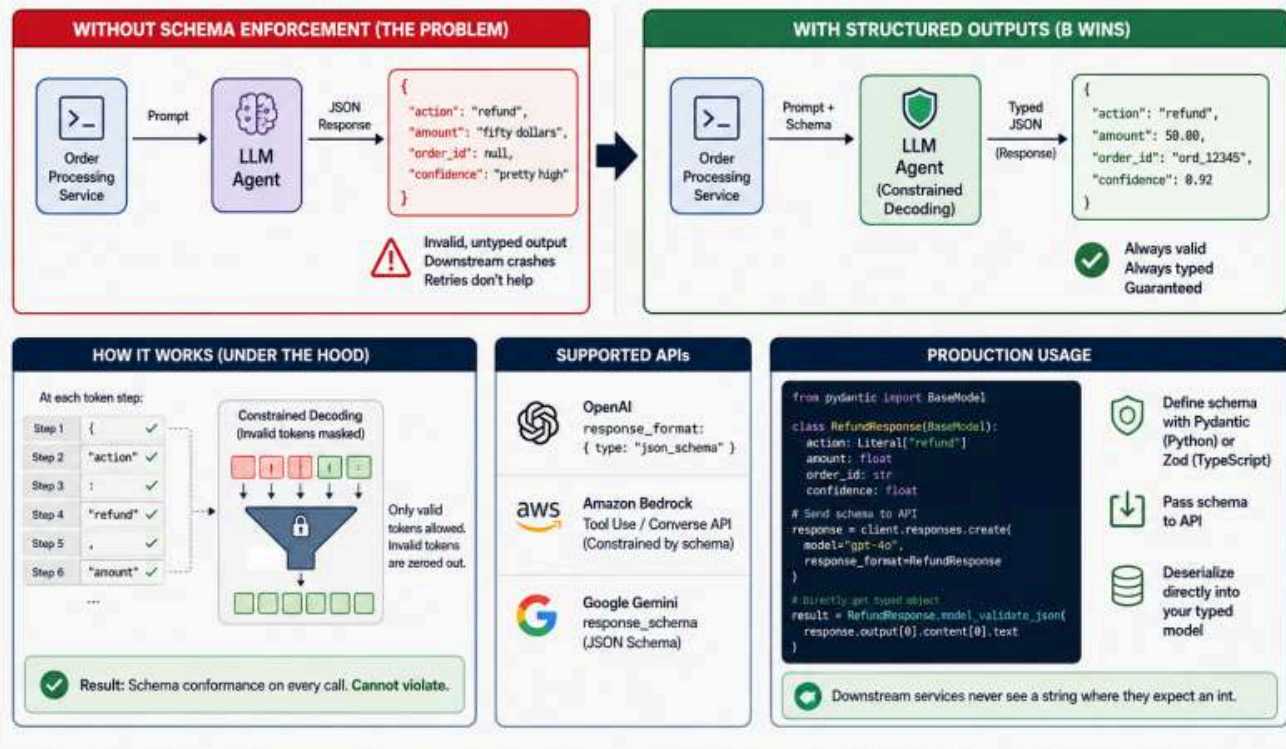
- A)** Prompt-engineer harder — add "Always return valid JSON with these exact fields" to your system prompt and document the schema inline.
- B)** Use structured outputs / function calling (OpenAI, Bedrock tool use, Gemini response schema) — constrain the model at the API level to return a typed schema.
- C)** Post-process with a validation layer — parse the output, run JSON Schema or Pydantic validation, retry with corrective context if it fails (max 2 retries).
- D)** Add a second LLM as a judge — pass the first model's output to a smaller, faster model that scores and flags invalid responses before they reach your service.

Answer 1 ✓ CORRECT ANSWER



B WINS: Structured Outputs / API-Level Schema Enforcement

The only option that eliminates the problem at the source.



Why B wins (Structured outputs / API-level schema enforcement):

This is the only option that eliminates the problem at the source. When you use OpenAI's `response_format: { type: "json_schema" }`, Bedrock's tool use / converse API, or Gemini's `response_schema`, the model's token sampling is *constrained* — it cannot produce output that violates the schema. Not "it's less likely." Cannot.

Under the hood, these APIs use constrained decoding: the model's output probability distribution is masked so invalid tokens are zeroed out at each step. The result is guaranteed schema conformance on every call. No retries. No parsing. No "it worked 98% of the time."

In production: use Pydantic (Python) or Zod (TypeScript) to define your schema, pass it to the API, deserialize directly into your typed model. Your downstream service never sees a string where it expects an int.

Answer 2

Why A is wishful thinking (Prompt engineering):

"Always return valid JSON" is not a contract. It's a suggestion.

LLMs are probabilistic text predictors. Under load, edge-case prompts, or slightly different phrasing, they drift. You'll get valid JSON 97% of the time and a markdown code block the other 3%. At 10k requests/day that's 300 failures. At 100k, it's 3,000.

Prompt engineering is a first step when you're prototyping. It is not an output validation strategy for production. Teams that rely on this eventually write 50-line system prompts trying to enumerate every failure mode. The model still escapes.

Answer 3

Why C is correct but incomplete (Post-process + retry loop):

Validation layer + retry with corrective context is a legitimate, widely-used pattern — especially when you can't use structured outputs (e.g., open-source models like Llama, Mistral, or your own fine-tune via Ollama/vLLM).

The implementation: parse → validate with Pydantic/JSON Schema → on failure, re-inject the original prompt + the bad output + a correction instruction ("Your previous output had these errors: [list]. Try again.") → retry, max 2 attempts → fail hard and surface to human review.

But: this is your fallback, not your primary. Every retry is latency + cost. Under high load, retry storms can spike your API spend 3–4x. Corrective context also doesn't help when the model fundamentally can't produce what you're asking — you'll just retry into the same failure.

Use C when B isn't available. Use B when it is....

Answer 4

Why D is a trap (LLM-as-judge for validation):

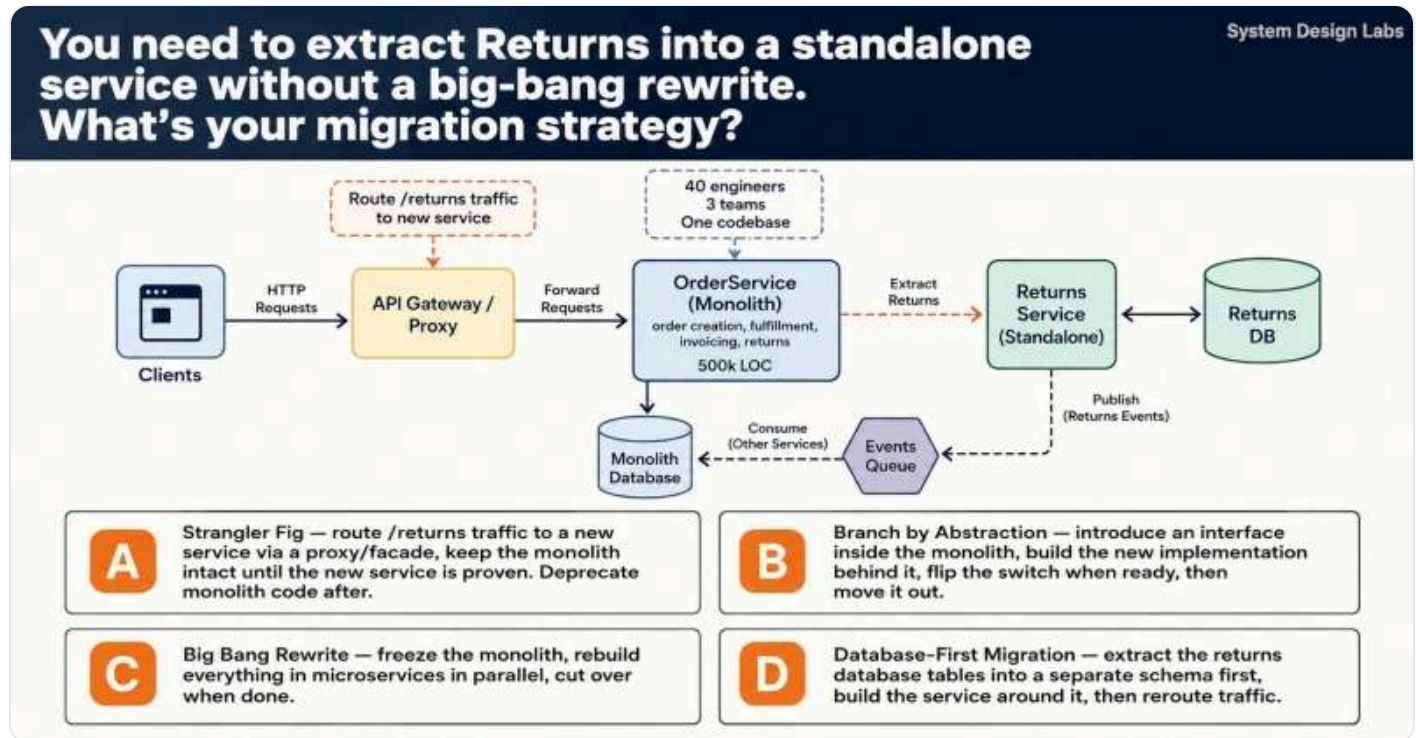
LLM-as-judge is a real eval pattern — it's used to score *quality*, tone, safety, and correctness in offline evaluation pipelines and red-teaming.

For *structural* output validation in a hot path? It's expensive, slow (~200–500ms extra per request), and unreliable for schema enforcement. A judge model can tell you "this response seems wrong" — it cannot guarantee a valid typed object. You've added latency and cost without actually solving the problem.

LLM-as-judge belongs in your eval harness (nightly runs, CI) — not in your request/response loop for data integrity.

The worse failure mode: teams add a judge *in addition to* bad structured prompting, feel safer, and ship it. Both models can hallucinate in sync. The judge agrees the broken output looks fine. Silent corruption ships to prod.

Incrementally Strangling a Monolith



Your monolith is 6 years old.

500k lines of code. One deploy every 3 weeks. One bad migration takes down the whole thing.

Leadership says: "rewrite it." You've seen what happens when teams do that. 18 months. Budget overruns. Half the features never make it back. The new system launches and nobody trusts it.

There's a better way. You don't rewrite — you strangle.

Here's your system:

- OrderService (monolith) handles: order creation, fulfillment, invoicing, returns
- 40 engineers. 3 teams. All deploying to the same codebase.
- Returns processing is a bottleneck. The business wants it extracted first.
- You can't freeze feature work during migration.

You need to extract Returns into a standalone service without a big-bang rewrite.

What's your migration strategy?

- A) Strangler Fig** — route /returns traffic to a new service via a proxy/facade, keep the monolith intact until the new service is proven. Deprecate monolith code after.
- B) Branch by Abstraction** — introduce an interface inside the monolith, build the new implementation behind it, flip the switch when ready, then move it out.
- C) Big Bang Rewrite** — freeze the monolith, rebuild everything in microservices in parallel, cut over when done.
- D) Database-First Migration** — extract the returns database tables into a separate schema first, build the service around it, then reroute traffic.

Answer 1

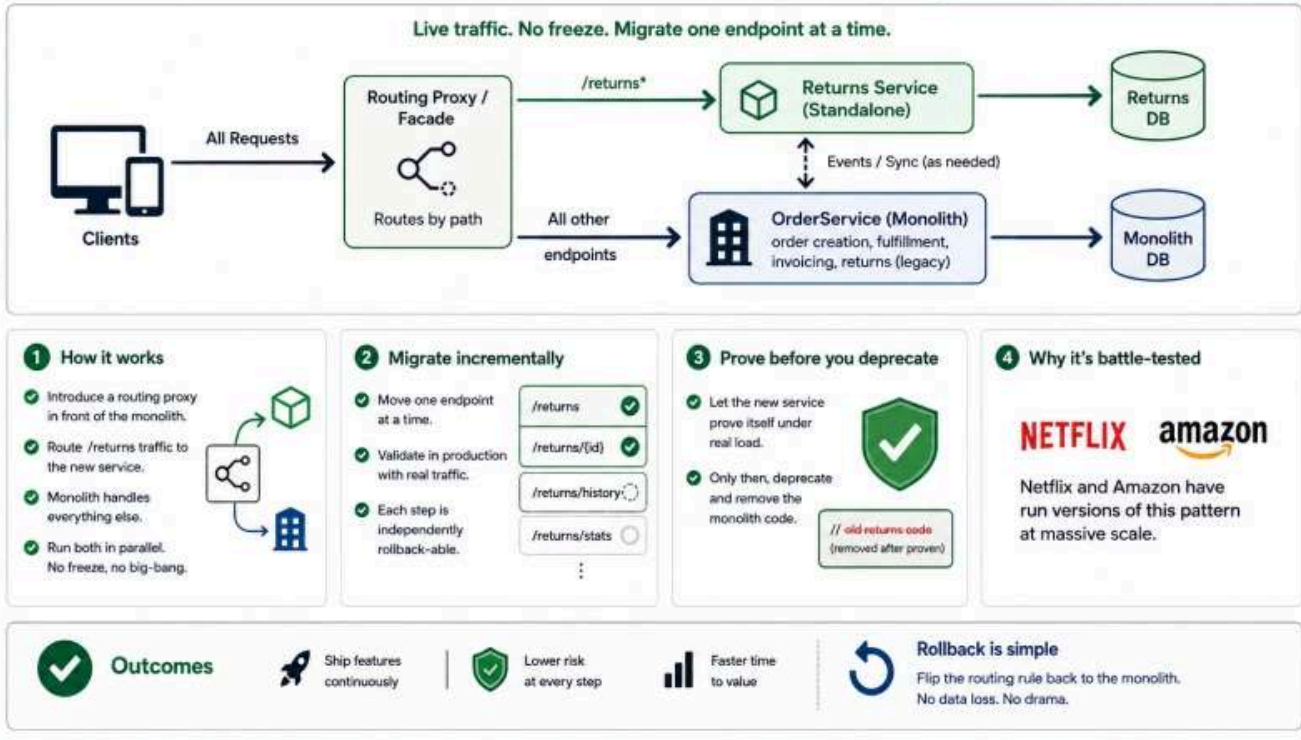
✓ CORRECT ANSWER

A**Strangler Fig – the right way to migrate**

Route `/returns` to a new service via a proxy, keep the monolith intact, and deprecate old code only after the new service proves itself.

**Why A wins**

- ✓ Migrate live, no freeze
- ✓ Rollback at every step
- ✓ Proven at scale (Netflix, Amazon)

**Why A wins:**

You introduce a routing proxy in front of the monolith. `/returns` traffic routes to the new service. The monolith handles everything else. You run both in parallel — no freeze, no big-bang. Migrate one endpoint at a time, each step independently rollback-able. Only deprecate monolith code after the new service has proven itself under real load. Netflix and Amazon have run versions of this at scale.

Answer 2**Why B is close but different (Branch by Abstraction):**

Real pattern — but it lives *inside* the codebase. You wrap the logic behind an interface, build the new implementation behind a flag, then flip it. It's a refactoring tool, not a deployment strategy. Often used *inside* a Strangler Fig migration to prepare internal structure. But on its own it doesn't get you to an independently deployed service.

Answer 3

Why C is wrong (Big Bang Rewrite):

The monolith doesn't freeze while you rebuild. Business keeps shipping features. Your parallel rewrite immediately diverges from reality. "Done" keeps moving. The cutover is a single high-stakes event with no incremental validation. Joel Spolsky called this "the single worst strategic mistake any software company can make" — because every hack in the old codebase was a real bug fix you've now forgotten.

Answer 4

Why D is more dangerous than it looks (Database-First):

Extracting the DB tables first sounds logical but breaks the monolith's ORM immediately. You've created a distributed data problem *before* building the distributed system to handle it. Data migration is the *last* step — extract service logic first, keep data co-located temporarily, then migrate with dual-write. Doing it in reverse order compounds the risk at every step.

How an Agent Chooses Which Tools to Call



Your AI agent just got a user message: "Book me a flight to Dubai next Friday."

The LLM has access to 12 tools: `search_flights`, `get_user_preferences`, `check_calendar`, `book_flight`, `send_confirmation`, `get_weather`...

How does the agent decide which tools to call, in what order, and when to stop?

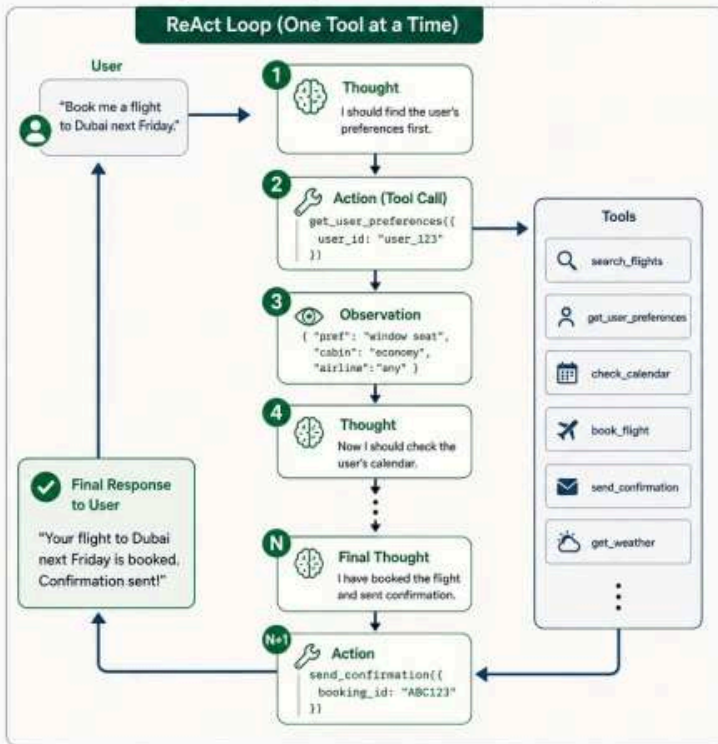
- A) ReAct loop** — model reasons step-by-step, emits a "thought" then picks one tool at a time, observes output, repeats until it self-decides it's done
- B) Parallel tool calling** — model emits ALL required tool calls in a single response, executes them concurrently, feeds all results back in one context update
- C) Forced function schema** — you lock the model into a strict JSON schema per turn; it can't produce free text, only structured tool calls you defined
- D) Planner-executor split** — a lightweight planner LLM creates a tool call DAG upfront, a separate executor runs the graph, results flow back to planner only at checkpoints

Answer 1

✓ CORRECT ANSWER

A**ReAct loop** ✓ Correct for most cases

ReAct (Reason + Act) is the dominant production pattern for a reason.



The model emits a "Thought:" explaining what it's doing, then an "Action:" with a single tool call, then observes the result before deciding what's next.

Why it works:

- Each step is observable and debuggable
- The model can bail out, retry, or change direction after each tool result
- Works even with tools that have side effects — you can gate dangerous calls
- Supported natively by OpenAI, Anthropic, LangChain, LlamaIndex out of the box

⚠ The trap:

ReAct is sequential. If you need `get_user_preferences` AND `check_calendar` AND `search_flights` all before `book_flight`, you're making 3 round trips when you could make 1.

**A — ReAct loop** Correct for most cases

ReAct (Reason + Act) is the dominant production pattern for a reason.

The model emits a "Thought:" explaining what it's doing, then an "Action:" with a single tool call, then observes the result before deciding what's next.

Why it works:

- Each step is observable and debuggable
- The model can bail out, retry, or change direction after each tool result
- Works even with tools that have side effects — you can gate dangerous calls
- Supported natively by OpenAI, Anthropic, LangChain, LlamaIndex out of the box

The trap: ReAct is sequential. If you need `get_user_preferences` AND `check_calendar` AND `search_flights` all before `book_flight`, you're making 3 round trips when you could make 1.

Answer 2

B — Parallel tool calling Right when dependencies allow it

Modern LLMs (GPT-4o, Claude 3.5+) can emit multiple tool calls in a single response.

If the tools are independent — run them in parallel. Massive latency win.

Example: `get_user_preferences` + `check_calendar` have no dependency on each other. Call both at once. Then feed both results back before calling `search_flights`.

The trap: not all tools are independent. If Tool B needs the output of Tool A, parallel calling breaks your context. The model doesn't "know" B failed because A hadn't run yet — it just gets a bad result and may hallucinate a fix.

Production pattern: use parallel calling for the fan-out phase, ReAct for the sequential decision phase.

Answer 3

C — Forced function schema The senior engineer trap

Looks like control. Actually limits you.

When you force strict JSON schema mode, you lose the model's ability to reason before calling.

This breaks down fast when:

- The user's intent is ambiguous (should the agent call `search_flights` or `clarify_intent`?)
- A tool fails and the model needs to decide whether to retry or give up
- You need conditional logic between steps

Use schema enforcement for output validation, not tool dispatch. Validate what comes back from the model — don't handcuff the model going in....

Answer 4

D — Planner-executor split 🏗️ Right at scale, overkill for most

This is the architecture behind production agentic pipelines like Devin, AutoGPT-style systems, and enterprise agent frameworks.

The planner builds a DAG of tool calls upfront. The executor runs it. The planner only re-engages at checkpoints or on failure.

Benefits:

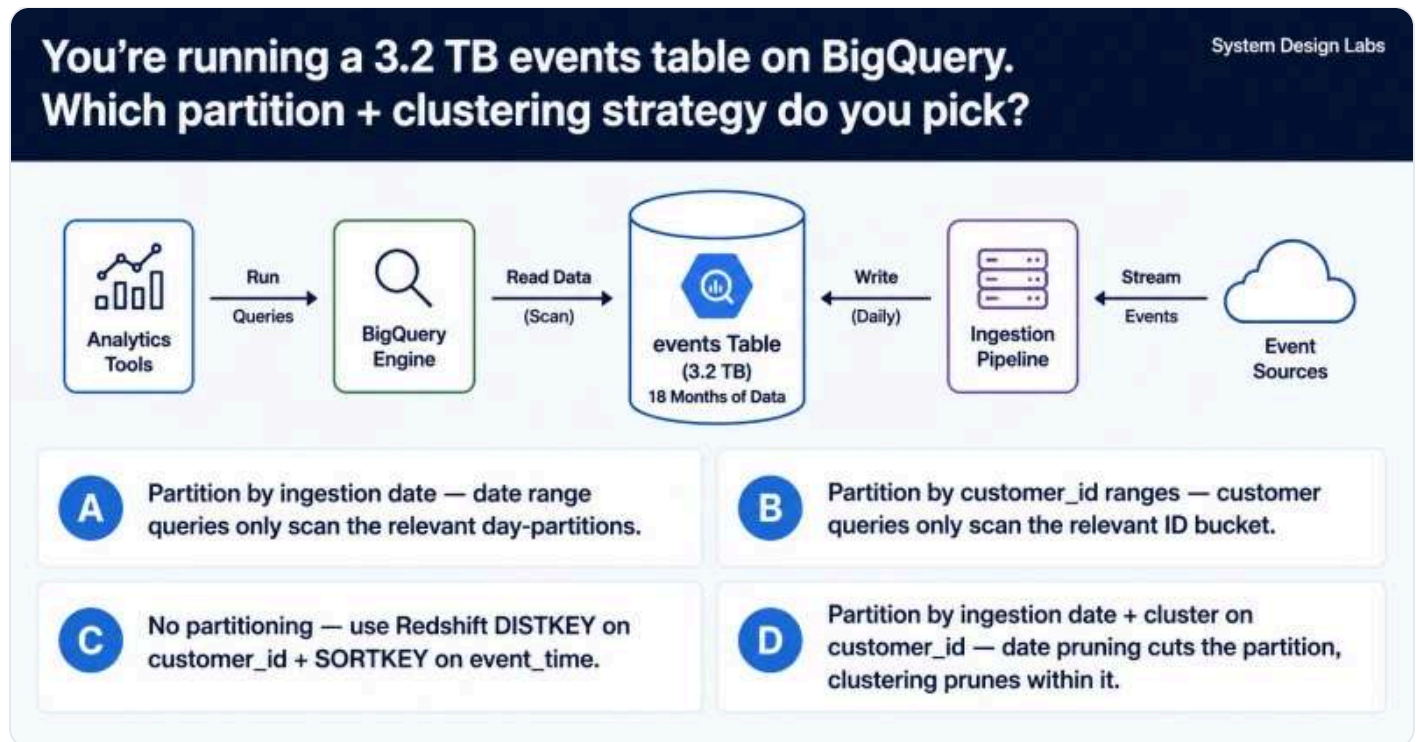
- Parallelism by default (the DAG has clear dependency edges)
- Cost-efficient (the powerful planner model runs less)
- Auditable — you can inspect the plan before running it

The trap: the planner has to get the full plan right before execution starts. If the world changes mid-run (API returns unexpected data, user updates their request), you need replanning. That's a whole second problem.

Most teams reach for this too early. Start with ReAct. Add parallel tool calling when latency hurts.

Only build planner-executor when you have complex multi-step workflows with 10+ tools and real parallelism requirements....

Controlling Data-Warehouse Query Costs



Your data team just opened a \$4,200 BigQuery bill.

For a single month. One analyst. 12 queries.

The queries weren't wrong. They weren't inefficient SQL. They were reasonable analytics queries — "give me last 30 days of events for customer X." The problem was that every single one scanned the full 3.2 TB table. No partition pruning. No cost control. Just full scans, every time.

This is the most expensive silent bug in data engineering. You write a query. It looks fast. It returns results. And every run quietly eats through terabytes you're paying per-byte to scan.

The fix is partition strategy — but picking the wrong one doesn't just fail to help, it actively makes things worse.

Here's the setup:

You're running a 3.2 TB events table on BigQuery. 18 months of data. Ingested daily. Analyst queries almost always filter on two things: a date range ("last 30 days") and a customer_id ("for customer X").

Which partition + clustering strategy do you pick?

A) Partition by ingestion date — date range queries only scan the relevant day-partitions.

- B)** Partition by `customer_id` ranges — customer queries only scan the relevant ID bucket.
- C)** No partitioning — use Redshift `DISTKEY` on `customer_id` + `SORTKEY` on `event_time`.
- D)** Partition by ingestion date + cluster on `customer_id` — date pruning cuts the partition, clustering prunes within it.

Answer 1

✓ CORRECT ANSWER

D**Why D wins: Partition by ingestion date + cluster on customer_id****BigQuery**
Partition + Cluster**1. Partition by date**

BigQuery skips every partition outside the date range before scanning a single row.

For a "last 30 days" query on 18 months of data, you immediately eliminate ~94% of the table.

**2. Cluster by customer_id**

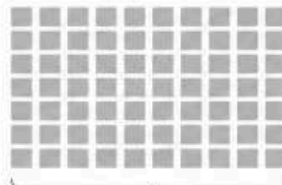
Within each scanned partition, BigQuery sorts the rows by customer_id into storage blocks.

A filter on customer_id now skips 90–95% of the remaining rows.

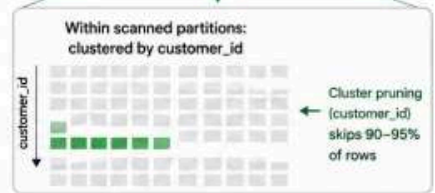
**Combined impact**

A "last 30 days for customer X" query goes from scanning 3.2 TB → ~12 GB.

Bill drops from \$4,200 → ~\$70/month.

WITHOUT PARTITIONING + CLUSTERING18 MONTHS OF DATA
3.2 TBQuery: last 30 days
for customer XScans 3.2 TB
\$4,200/month**WITH PARTITION BY DATE + CLUSTER BY CUSTOMER_ID**18 MONTHS OF DATA
3.2 TB

Partition pruning (date)

Cluster pruning
(customer_id)
skips 90–95%
of rowsScans ~12 GB
\$70/month✓ Same queries.
Same data.
Right strategy.

This pattern works across every major warehouse:

**BigQuery**
Partition + Cluster**Snowflake**
Micro-partitioning +
Cluster Keys**Redshift**
Partition Key +
SORTKEY Compound**Why D wins:**

Your analyst queries always filter on two columns: a date range and a customer_id. A single-key strategy protects one and ignores the other.

Here's what D does in BigQuery:

- **Partition by date** → BigQuery skips every partition outside the date range before scanning a single row. For a "last 30 days" query on 18 months of data, you immediately eliminate ~94% of the table.
- **Cluster by customer_id** → Within each scanned partition, BigQuery sorts the rows by customer_id into storage blocks. A filter on customer_id now skips 90–95% of the remaining rows.

Combined: a "last 30 days for customer X" query goes from scanning 3.2 TB → ~12 GB. Bill drops from \$4,200 → ~\$70/month. Same queries. Same data. Right strategy.

This pattern works across every major warehouse: BigQuery (partition + cluster), Snowflake (micro-partitioning + cluster keys), Redshift (partition key + SORTKEY compound).

Answer 2

Why A is incomplete (date partition only):

Date partition alone is a real improvement — date range filters prune the irrelevant partitions immediately. Your bill probably drops 60–70%.

But your analyst queries also filter by `customer_id`. Inside each date partition, all customer rows are interleaved across storage blocks. BigQuery has to read the entire partition to find customer X's rows. You've cut the scan from 3.2 TB → ~500 GB (the last 30 days), but you're still scanning every customer inside that window.

A is the obvious first step. D is A taken all the way.

Answer 3

Why B is the senior engineer trap (`customer_id` range partitioning):

This one gets smart people. Your analysts always filter by `customer_id` — why not partition on that?

Because real `customer_id` distributions are never uniform. Your enterprise clients (IDs 1–10,000) generate 80% of all events. Every analytics query against active customers hits the same partition. No pruning benefit — the entire interesting data is in one bucket.

Worse: your queries also filter by date. BigQuery now has to scan the full customer partition to find rows from the last 30 days. You've traded a full table scan for a full-partition scan on your hottest partition. Same cost, different framing.

Hot partitions on high-traffic ID ranges is the most common expensive mistake in data warehouse design. Looks right in theory. Never works in production.

Answer 4

Why C fails at this scale (DISTKEY + SORTKEY, no partitioning):

DISTKEY distributes rows across Redshift nodes by `customer_id` hash — optimizes joins. SORTKEY on `event_time` sorts rows on disk within each slice — helps date range scans read contiguous blocks.

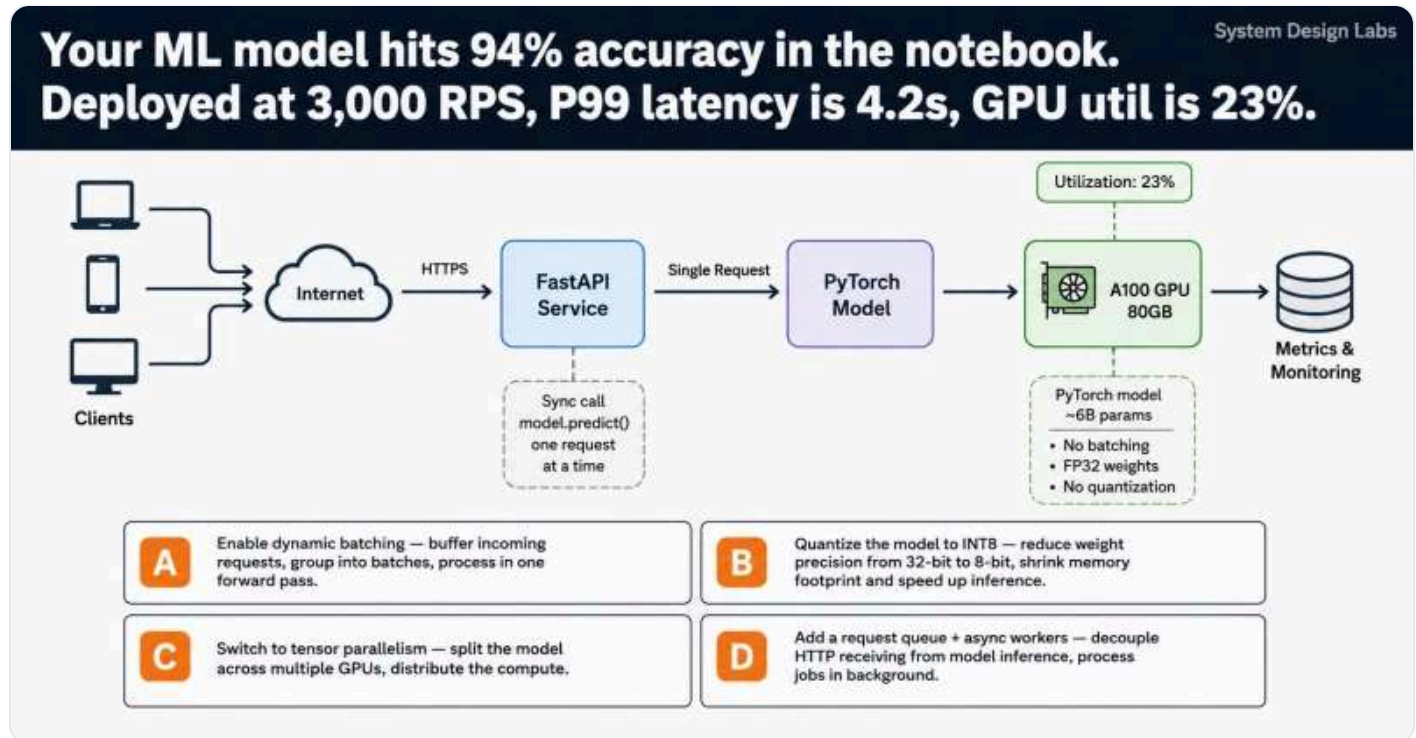
Both are real optimizations. Neither replaces partition pruning at 3.2 TB.

Without partitioning, every query triggers a full table scan across all nodes. If your DISTKEY distribution is skewed — which it will be when enterprise customers dominate — one Redshift node holds 60–70% of your data. Every expensive query hammers that node.

You've turned a cost problem into a latency + cost problem.

DISTKEY + SORTKEY is solid for join optimization and smaller tables. It's not a substitute for partition pruning at terabyte scale.

Serving ML Models at High Throughput



Your ML team trained a model that hits 94% accuracy in the notebook.

Then you deploy it.

Peak traffic: 3,000 inference requests/second.

P99 latency shoots to 4.2 seconds.

GPU utilization: 23%.

The model works. The serving layer is the bottleneck.

Here's the setup:

- PyTorch model, ~6B params
- Single A100 GPU, 80GB VRAM
- FastAPI wrapper calling `model.predict()` one request at a time
- No batching, FP32 weights, no quantization

Users are hitting timeouts. GPUs are sitting mostly idle. Your infra bill is climbing.

What do you fix first?

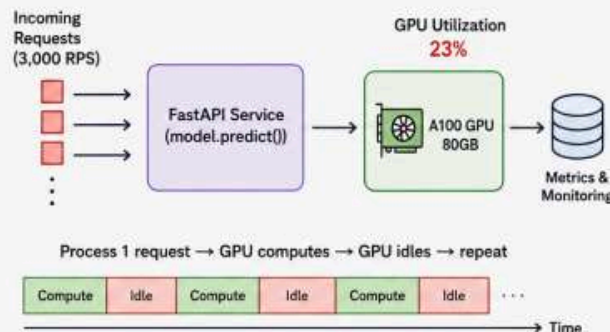
- A)** Enable dynamic batching — buffer incoming requests, group into batches, process in one forward pass.
- B)** Quantize the model to INT8 — reduce weight precision from 32-bit to 8-bit, shrink memory footprint and speed up inference.
- C)** Switch to tensor parallelism — split the model across multiple GPUs, distribute the compute.
- D)** Add a request queue + async workers — decouple HTTP receiving from model inference, process jobs in background.

Answer 1 ✓ CORRECT ANSWER

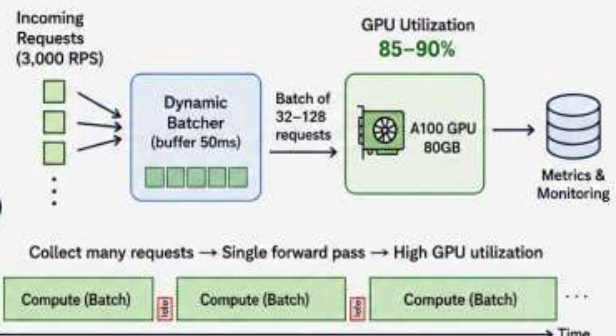
**Answer: A — Dynamic Batching**

System Design Labs

Batch requests for a few milliseconds, run one forward pass, and keep the GPU fed.

**BEFORE:** No Batching (One Request at a Time)

Symptoms: 3K RPS, P99 = 4.2s, GPU Util = 23%
GPU is starving. High overhead, low throughput.

**AFTER:** Dynamic Batching (Many Requests per Forward Pass)

Result: Higher throughput, lower P99 latency, happier users,
and dramatically better GPU utilization.

**Why A Wins**

- ✓ The GPU is underutilized (23%) → it's not the compute, it's the request pattern.
- ✓ Dynamic batching turns many small, expensive calls into fewer, efficient batch calls.
- ✓ Massively improves throughput and latency without changing the model.
- ✓ This is the first lever to pull for low GPU util + high RPS.

Why Others Don't Fix This First

- B** Quantization helps throughput, but doesn't fix low GPU utilization.
- C** Tensor parallelism helps if one GPU is maxed out. It's not.
- D** Queues/async help resilience, not GPU utilization or latency.

Answer: A — Dynamic Batching ✓

Here's the breakdown:

Why A wins (Dynamic Batching):

The smoking gun is the combination: 3K RPS, P99 = 4.2s, GPU utilization = 23%.

GPU utilization at 23% tells you the GPU is starving. It's not overloaded — it's bored. Every request is arriving one at a time, paying the full GPU kernel launch overhead, then idling while the next request trickles in. GPUs are built for parallelism. They want 256 inputs to process simultaneously, not 1.

Dynamic batching buffers requests for a few milliseconds (say, 50ms), collects 32–128 of them, and fires a single forward pass. That same GPU that was at 23% utilization can jump to 85–90%. Latency improves because batch throughput dramatically outpaces sequential per-request calls, even with the small buffer wait.

This is exactly how NVIDIA Triton Inference Server, TorchServe, and vLLM (for LLMs) work by default. Batching is the first thing you reach for when GPU util is low and RPS is high.

Answer 2

Why B is the trap answer (INT8 Quantization):

Quantization is legitimate and valuable — but it solves a *different* problem. It reduces model size (~4x smaller) and speeds up individual forward passes. If you were memory-constrained or couldn't fit the model at all, quantization is critical. But this model fits comfortably in 80GB VRAM and the GPU isn't saturated.

INT8 quantization also introduces precision loss you need to validate carefully, and the speedup on a modern A100 for a 6B param model in isolation is modest — maybe 1.5–2x. That doesn't fix 3K RPS with sequential inference. You'll still have sequential execution, just slightly faster per call.

The right order: batch first, quantize second (for memory or cost, not latency emergencies).

Answer 3

Why C is wrong here (Tensor Parallelism):

Tensor parallelism splits model layers across multiple GPUs using NVLink or interconnects. It reduces per-request latency for *very large* models that don't fit on one device, or when you need sub-100ms latency on a single huge model.

But this is a 6B param model on an 80GB A100 — it fits. Tensor parallelism adds coordination overhead and complexity, and it doesn't help when the bottleneck is sequential request processing. You'd also need 2+ GPUs, which inflates cost before you've even tried the cheap fix.

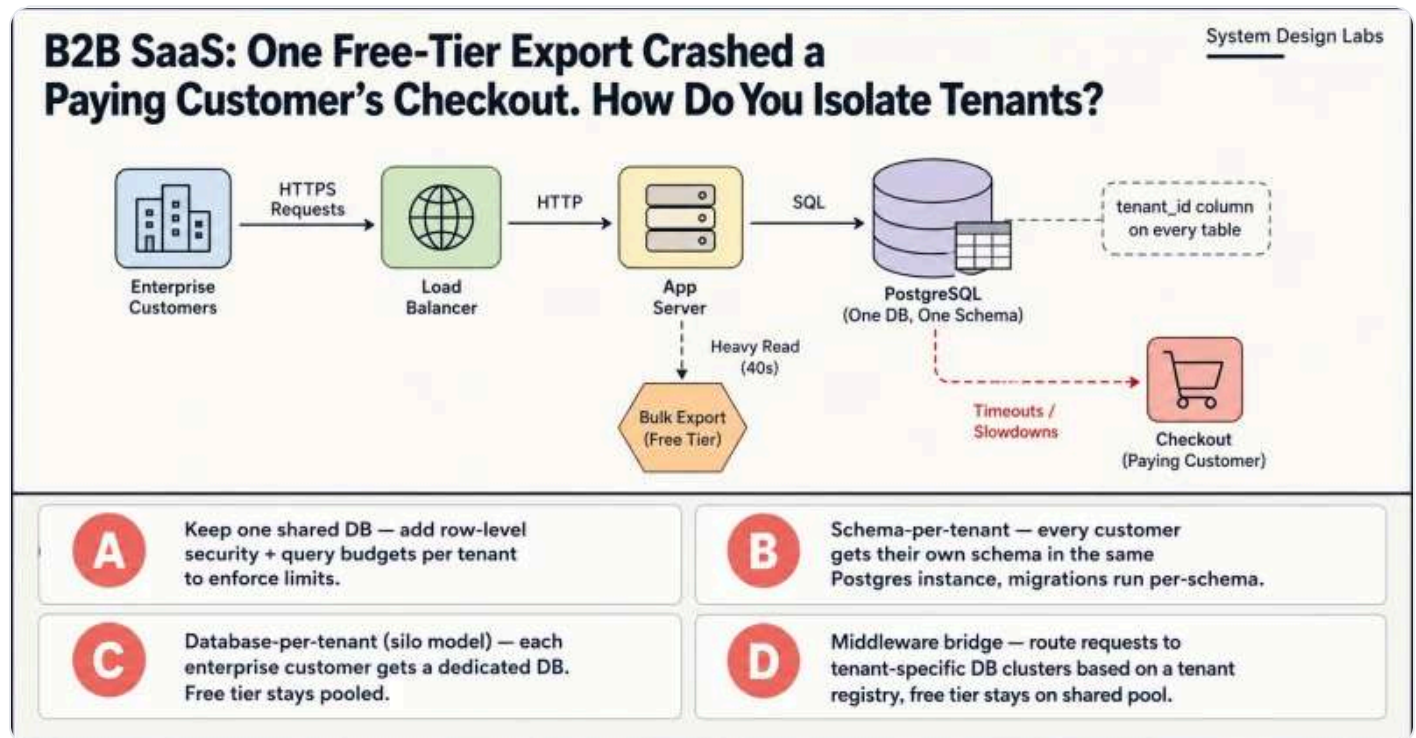
Answer 4

Why D is wrong (Async Queue + Workers):

An async queue decouples HTTP from inference and prevents timeout failures — that's a reliability win. But it doesn't improve GPU utilization or actual throughput. You're still processing one request at a time, just asynchronously. Users now get job IDs instead of timeouts, but the wall-clock wait is the same or worse.

Queuing without batching is adding a lobby to a slow restaurant. It hides the congestion — it doesn't fix it.

Isolating Noisy Tenants in Multi-Tenant SaaS



You're building a B2B SaaS product. 50 enterprise customers. Each one wants their data isolated. Some are on free plans. A few are paying \$50k/year and demanding SLA guarantees.

Your current setup:

→ One database. One schema. A `tenant_id` column on every table.

→ One app server handling all traffic.

→ A free-tier customer running a badly-written bulk export just hammered your DB for 40 seconds. A paying enterprise customer's checkout flow timed out.

Your investors are not happy. Neither is that enterprise customer.

The engineering question: how do you isolate tenants without rebuilding the whole product?

A) Keep one shared DB — add row-level security + query budgets per tenant to enforce limits.

B) Schema-per-tenant — every customer gets their own schema in the same Postgres instance, migrations run per-schema.

C) Database-per-tenant (silo model) — each enterprise customer gets a dedicated DB. Free tier stays pooled.

D) Middleware bridge — route requests to tenant-specific DB clusters based on a tenant registry, free tier stays on shared pool.

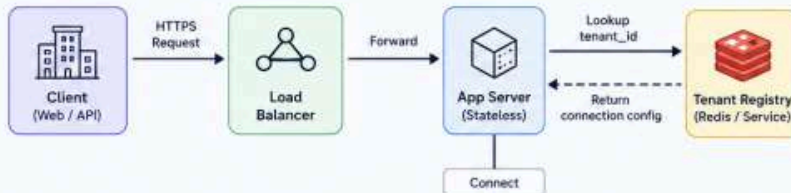
Answer 1

✓ CORRECT ANSWER

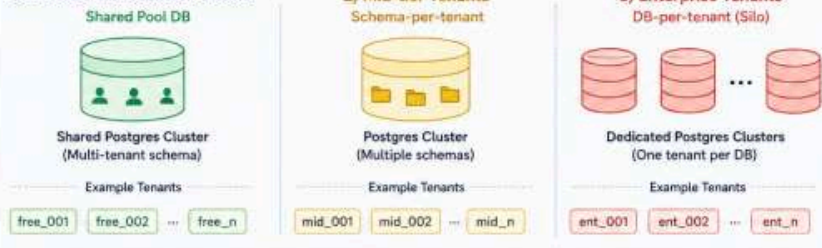
Why D wins: Middleware Bridge — Hybrid Model

Route each tenant to the right data home. Isolate where it matters. Scale where it's cheap.

REQUEST FLOW



DATA TIER (THREE ISOLATION LEVELS)



- Free / Small (Shared)
- Mid-tier (Schema-per-tenant)
- Enterprise (DB-per-tenant)

THE PAYOFF

- Noisy neighbor is contained.**
A runaway free-tier export can only impact other free-tier tenants.
- Enterprise customers can't touch each other. Period.**
Dedicated DBs provide strong isolation and SLA guarantees.
- Grow tenants up the isolation ladder without code changes.**
Move a tenant between tiers by updating the registry only.
- GDPR deletion is a drop, not a cascade.**
One DB drop for enterprise tenants. Clean, fast, verifiable.



How it works

At request time, the app server asks the Tenant Registry where a tenant lives. It gets back a connection config (host, port, db, schema) and connects. The application code stays the same for every tenant.



Why D wins (Middleware bridge — hybrid model):

This is the pattern Salesforce, HubSpot, and every mature B2B SaaS converges on eventually.

The architecture:

- Free/small tenants → shared pool DB (cost-efficient, acceptable risk)
- Mid-tier tenants → schema-per-tenant in a shared Postgres cluster
- Enterprise tenants → dedicated DB cluster (full isolation, SLA-able)

A tenant registry (Redis or a lightweight service) maps `tenant_id` → connection config at request time. Your app server doesn't care which model a tenant uses — it asks the registry and gets a connection string back.

The payoff:

- Noisy neighbor is contained. A runaway free-tier export can only hurt other free-tier tenants.
- Enterprise customers can't touch each other. Period.

- You can migrate a tenant up the isolation ladder as they grow — without touching app code.
- GDPR deletion is one DB drop for enterprise tenants, not a cascade delete across shared tables.

Answer 2

Why A fails (Row-level security + query budgets):

RLS works. Query budgets can work. But they share the same DB process. A tenant that opens 500 connections, triggers a table lock, or fills the WAL — that bleeds into every other tenant on the instance. You can throttle CPU, but you can't throttle I/O blast radius once it's in the buffer pool.

This is the band-aid answer. Fine at 5 tenants. Falls apart at 50.

Answer 3

Why B is dangerous (Schema-per-tenant):

Schema-per-tenant sounds clean. It is, until:

→ 300 tenants need a migration. That's 300 sequential DDL operations on one Postgres instance. Migration windows become a multi-hour risk event.

→ At 500 tenants you're burning connection slots faster than you can PgBounce your way out.

→ One long-running query in schema_tenant_247 still holds locks on shared system tables. Noisy neighbor hasn't gone away — you just can't see it as clearly.

Schema-per-tenant is a local maximum. You hit Postgres scaling walls before you hit the point where it pays off.

Answer 4

Why C is too expensive too early (Full silo):

The silo model is the correct isolation model for enterprise customers. But applying it to *all* tenants — including a free tier with 1,000 accounts — means 1,000 Postgres instances. That's \$40k/month in RDS costs before you've made a dollar.

Full silo for everyone is the right endgame for the top 5% of your customer base. Applying it everywhere is overengineering.

Versioning an API Without Breaking Clients



Your API just shipped a breaking change.

/users now returns fullName instead of first_name + last_name. 3 mobile clients broke. 1 partner integration went down. Your on-call is not happy.

You had a versioning strategy. It just wasn't the right one.

There are 4 ways to version an API. Here's what actually happens when you pick each one in production:

A — URL path versioning (/v1/users, /v2/users)

Simple. Explicit. Every request makes the version visible in logs and caches. But now you're maintaining 2 full route trees. A bugfix in the business logic layer has to be patched in both. Teams quietly let v1 rot.

B — Header versioning (API-Version: 2)

Clean URLs. Version negotiation in the transport layer, not the path. Harder to test in a browser, invisible in logs unless you instrument for it, and clients forget to send the header — defaulting to whatever your server decides "latest" means.

C — Query param versioning (/users?version=2)

Fast to implement. Zero client SDK changes. Cache-unfriendly — every CDN layer treats ?version=1 and ?version=2 as separate cache keys. Works until you have 40 endpoints and version drift becomes untrackable.

D — Content negotiation (Accept: application/vnd.api.v2+json)

The REST-purist approach. Semantically correct — you're asking for a *representation*, not a route. Almost nobody implements it right. Client library support is inconsistent. One wrong Accept header and you get a 406.

Answer 1

✓ CORRECT ANSWER

System Design Labs

D Content negotiation: correct spec, wrong reality

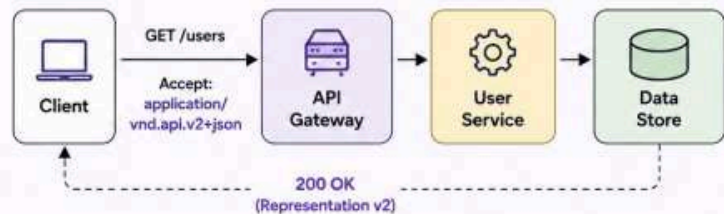
`Accept: application/vnd.api.v2+json`

is what the HTTP spec intended.

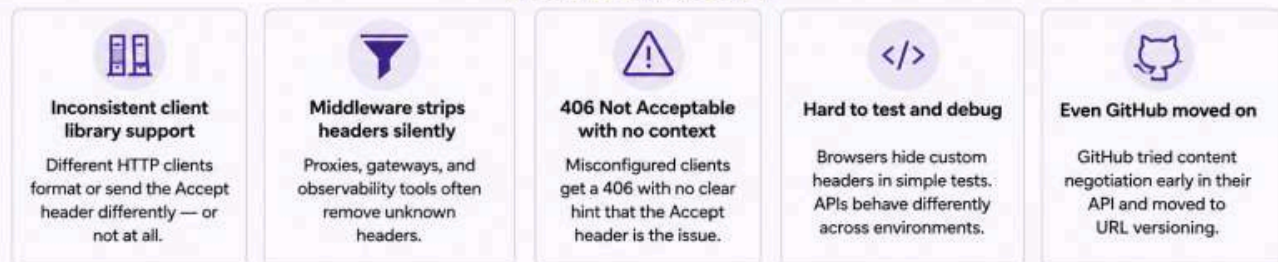
You're asking for a specific representation of a resource, not a different route.

Semantically, it's the most correct.

HOW CONTENT NEGOTIATION SHOULD WORK



THE REALITY IN PRODUCTION



Bottom line:

The most correct on paper, but the least reliable in practice. Great in theory. Painful in production.

`Accept: application/vnd.api.v2+json`

→ 406 Not Acceptable

D — Content negotiation: correct spec, wrong reality

`Accept: application/vnd.api.v2+json` is what the HTTP spec intended. You're asking for a specific *representation* of a resource, not a different route. Semantically, it's the most correct.

In practice: Accept header parsing is inconsistent across client libraries. Middleware strips headers silently. A misconfigured client gets a 406 Not Acceptable with no obvious error message pointing to the version field. GitHub tried content negotiation early in their API and moved to URL versioning. That tells you something.

Answer 2

A — URL path versioning: the pragmatic default (not wrong, but has a cost)

This is what Stripe, Twilio, and GitHub use. Not because it's theoretically pure — it isn't — but because it's operationally obvious.

Every proxy, log aggregator, and CDN understands it without configuration. `/v1/` vs `/v2/` is visible everywhere without instrumentation.

The cost is real: you're running parallel route trees, and v1 maintenance debt compounds fast. A bugfix in shared business logic has to be patched in both versions. Stripe's answer was to pin clients to a version at the API key level and deprecate slowly over years. Most teams don't have that discipline — so v1 quietly rots while v2 gets all the attention.

Answer 3

B — Header versioning: good theory, rough edges

Clean architecture. The URL represents a resource, the header represents the negotiation. This is how internal microservices often version — when you control both client and server, it works well.

The problem: invisible in browser tabs, impossible to share as a curl command without extra flags, and "default to latest" on a missing header is a footgun. In year two, a legacy client quietly drops the header. Your server defaults it to v2. The client breaks. Nobody knows why — it's not in the URL, it's not in the logs unless you specifically instrumented for it.

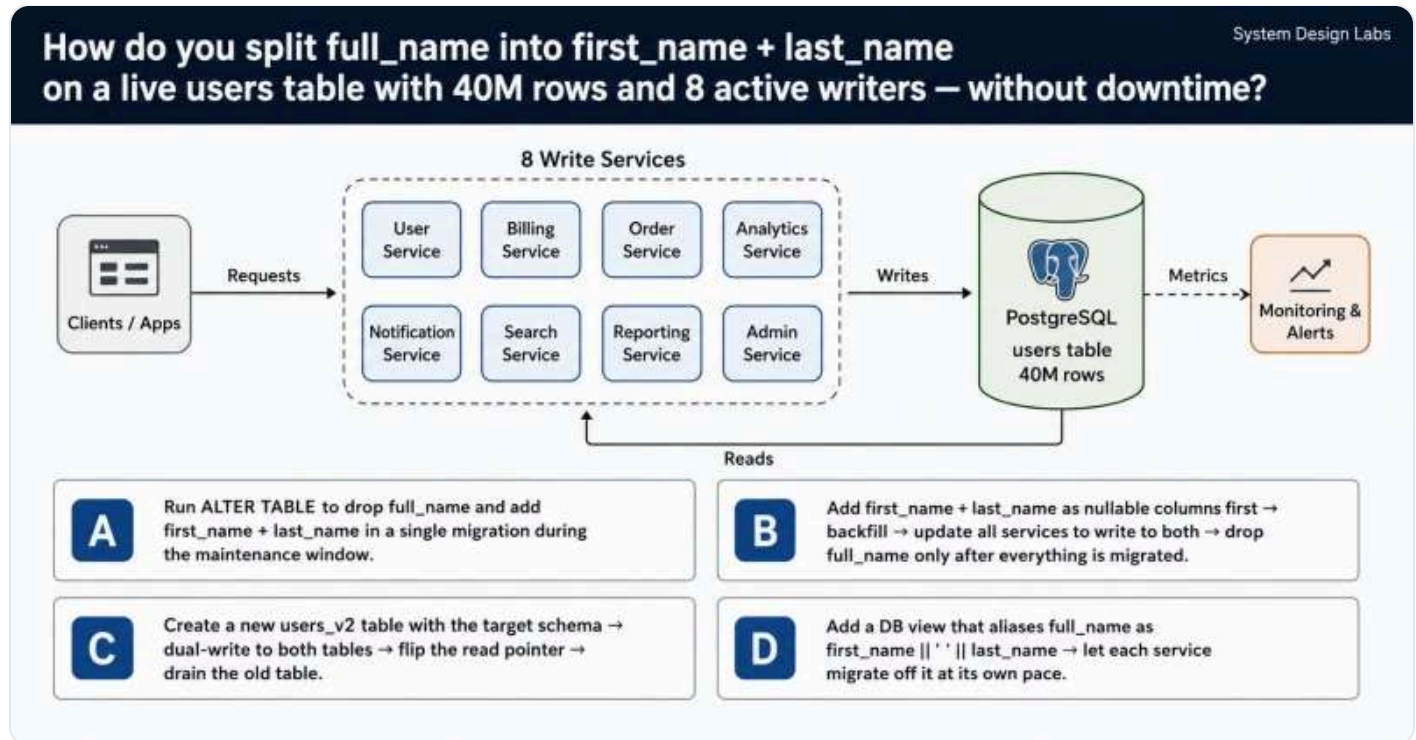
Answer 4

C — Query params: the quick fix that stays forever

Fast to implement. Zero client SDK changes. Works immediately.

Cache invalidation gets messy at scale — every CDN layer treats `?version=1` and `?version=2` as separate cache keys, so your cache hit rate tanks. The bigger problem is version drift: with no forcing function, clients scatter across v1/v2/v3 with no deprecation pressure. Good for internal tools where you control all the clients. Gets expensive for public APIs.

Zero-Downtime Schema Migrations



Your migration ran fine in staging.

Then you ran it in production.

The app went down.

Not because the SQL was wrong. Because you ran it on a live table with 40 million rows while 8 services were actively writing to it.

Your setup:

→ PostgreSQL. users table. 40M rows. Active writes from 8 services.

→ Product request: split full_name into first_name + last_name.

→ You have a 2-hour maintenance window tonight.

The engineering question: how do you ship this without downtime?

A) Run ALTER TABLE to drop full_name and add first_name + last_name in a single migration during the maintenance window.

B) Add first_name + last_name as nullable columns first → backfill → update all services to write to both → drop full_name only after everything is migrated.

- C)** Create a new users_v2 table with the target schema → dual-write to both tables → flip the read pointer → drain the old table.
- D)** Add a DB view that aliases full_name as first_name || ' ' || last_name → let each service migrate off it at its own pace.

Answer 1 ✓ CORRECT ANSWER

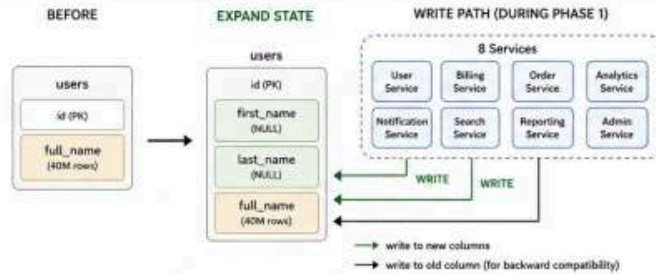
Why **B** is correct (expand/contract)

System Design Labs

✓ Two phases. Expand safely. Then contract when everything is ready. Every step is reversible.

PHASE 1
EXPAND

- 1 Add first_name and last_name as nullable columns.
- 2 Backfill in small batches (rate-limited, no row lock escalation).
- 3 Update services to write to both columns.

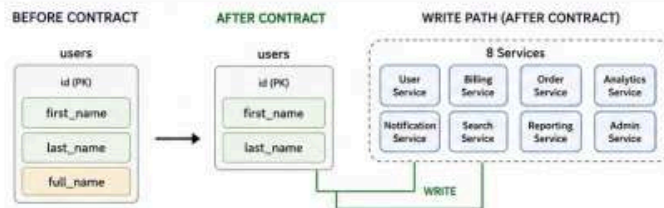


✓ WHY THIS IS SAFE

- Adding a nullable column with no default in PostgreSQL is a metadata-only operation — near-instant, no lock.
- Backfill in small batches keeps the system responsive.
- Old column remains source of truth until you're ready.
- Every step is independently reversible.

PHASE 2
CONTRACT

- 1 Once all services write to the new columns and reads are fully migrated...
- 2 Drop full_name.
- 3 You control the timing.



✓ RESULT

- full_name is dropped only when it's safe.
- You control the exact timing.
- Minimal risk. Maximum control.

Slower than A? Yes.
3am pages? Zero.

Why B is correct

Two phases.

Phase 1 — Expand: add first_name and last_name as nullable columns. On PostgreSQL, adding a nullable column with no default is a metadata-only operation — near-instant, no lock. Backfill in small batches (rate-limited, no row lock escalation). Update services to write to both columns simultaneously.

Phase 2 — Contract: once every service is writing to the new columns and reads are fully migrated, drop full_name. You control the timing.

The key property: every step is independently reversible. If something breaks after Phase 1, you stop — the old column still exists, old services still work.

Slower than A? Yes. 3am pages? Zero.

Answer 2

Why A will get you paged

ALTER TABLE on a live table acquires an ACCESS EXCLUSIVE lock in PostgreSQL. It blocks every read and write for the duration. On a 40M row table with active traffic, that lock holds for minutes — not seconds. Your 8 services time out. The queue backs up. Alerts fire.

CONCURRENTLY helps for index creation. It doesn't help for column drops or renames. The maintenance window doesn't save you — the lock is the problem, not the timing.

Answer 3

Why C is overkill

Shadow table + dual-write works. But it requires a dual-write layer in every service, an atomic read-pointer flip, and a drain/cleanup phase for the old table.

Worth it for genuine structural rewrites — new primary key, re-partitioning, changing storage engines. For a column split on a single table? That's 3 weeks of engineering for a 3-day problem.

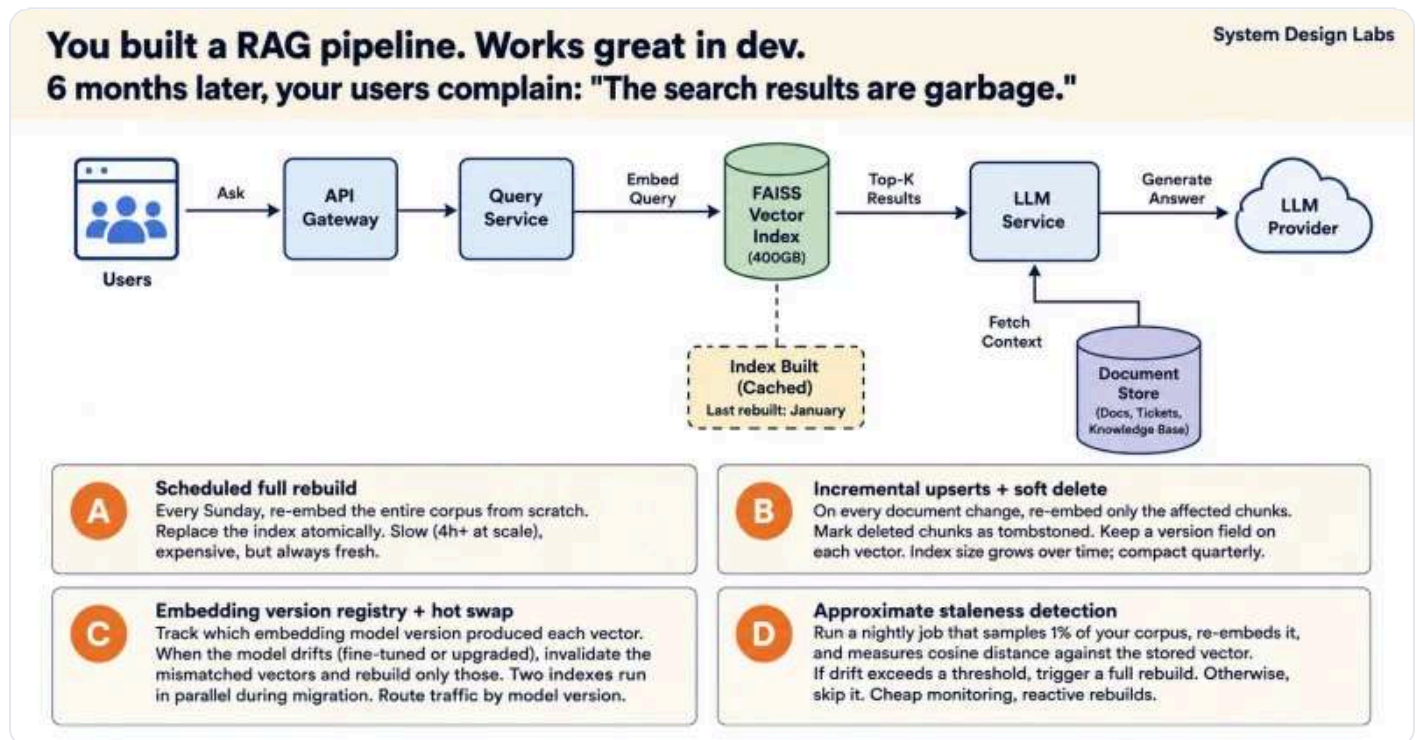
Answer 4

Why D defers the problem

A view aliasing full_name buys time. Useful as a transition layer inside Option B. But PostgreSQL write-through views only work on simple single-table views with no joins or aggregates. You still need to backfill the real columns eventually.

It defers the migration. It doesn't replace it.

Handling Embedding Drift in RAG



You built a RAG pipeline. Works great in dev.

6 months later, your users complain: "The search results are garbage."

You haven't changed a line of code.

Here's what happened:

Your product evolved. New features, new docs, new support tickets. The data drifted — but your embedding index didn't.

Now you're serving a 400GB FAISS index that was last rebuilt in January. Your chunks are stale. Your nearest-neighbor results point to deprecated docs. Your LLM is confidently hallucinating from outdated context.

You need to fix this. 4 engineers each propose a solution:

A) Scheduled full rebuild

Every Sunday, re-embed the entire corpus from scratch. Replace the index atomically. Slow (4h+ at scale), expensive, but always fresh.

B) Incremental upserts + soft delete

On every document change, re-embed only the affected chunks. Mark deleted chunks as tombstoned. Keep a version field on each vector. Index size grows over time; compact quarterly.

C) Embedding version registry + hot swap

Track which embedding model version produced each vector. When the model drifts (fine-tuned or upgraded), invalidate the mismatched vectors and rebuild only those. Two indexes run in parallel during migration. Route traffic by model version.

D) Approximate staleness detection

Run a nightly job that samples 1% of your corpus, re-embeds it, and measures cosine distance against the stored vector. If drift exceeds a threshold, trigger a full rebuild. Otherwise, skip it. Cheap monitoring, reactive rebuilds.

Real constraint: your corpus is 50M chunks. Full rebuild = 4 hours + ~\$800 in embedding API cost. You deploy model updates every 6 weeks.

Answer 1

✓ CORRECT ANSWER

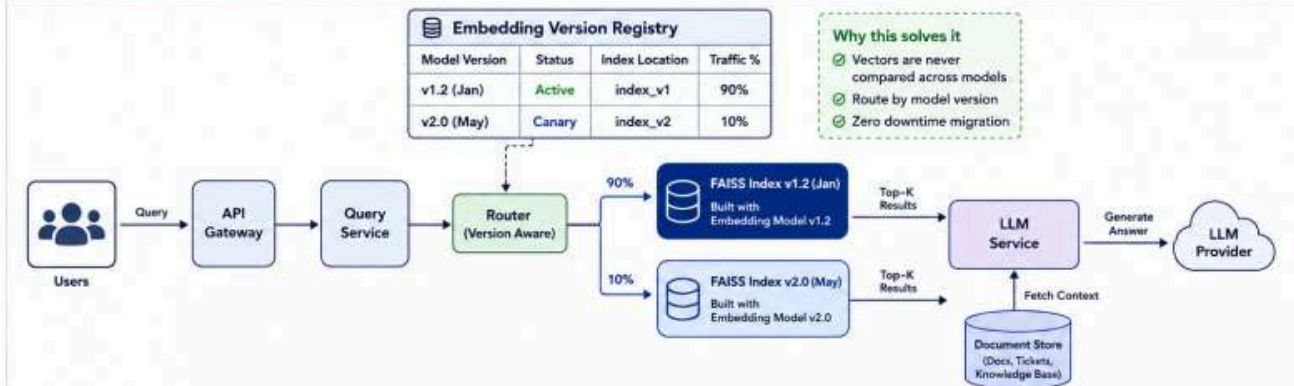


C is the right answer — Embedding Version Registry + Hot Swap

This is the only solution that handles the real problem: your embedding model changed.



C tracks which model produced which vector and routes queries to the correct index version during migration. Two indexes in parallel during transition = zero downtime, zero mixed-space comparisons.



PRODUCTION WIN



Gradual Migration
Migrate 10% of traffic to the new index, validate retrieval quality, then increase confidently.



Zero Downtime
Two indexes run in parallel. No big bang. No 4-hour Sunday rebuild blocking your team.



Safe Validation
Compare results, monitor quality, roll back instantly if needed.



Correctness
No mixed embedding spaces. Cosine similarity math stays valid.

WHY A, B, AND D FAIL



Scheduled Full Rebuild
Still creates a mixed index during migration if the model changed. Big bang, expensive, risky.



Incremental Upserts + Soft Delete
Index ends up with vectors from multiple models. Cosine similarity breaks across embedding spaces.



Approximate Staleness Detection
Detects drift reactively, but rebuild is still a big bang and causes downtime.



The Real Problem
Different embedding models produce different vector spaces. You can't compare them.

C is the right answer — Embedding Version Registry + Hot Swap

This is the only solution that handles the *real* problem: your embedding model changed. When you fine-tune or upgrade your model, Option A and B still serve a mixed index — some vectors from model v1, some from v2. Your cosine similarity math breaks because you're comparing vectors from different embedding spaces. They're not comparable. C is the only option that tracks *which model produced which vector* and routes queries to the correct index version during migration. Two indexes in parallel during transition = zero downtime, zero mixed-space comparisons.

The production win: you can migrate 10% of traffic to the new index, validate retrieval quality, then cut over. No big bang. No 4-hour Sunday rebuild blocking your team.

Answer 2

B is the senior engineer trap

Incremental upserts sound smart — you're only re-embedding what changed. But here's the silent killer: model drift. You update your embedding model every 6 weeks. Now your index has vectors from 4 different model versions. Chunk A was embedded with text-embedding-ada-002. Chunk B with your fine-tuned v2. Nearest neighbor search across a mixed-space index returns garbage — not because the data is stale, but because the *geometry* is inconsistent. B handles data drift. It doesn't handle model drift. At 6-week update cycles, model drift is your actual problem.

Answer 3

A is brute-force ops debt

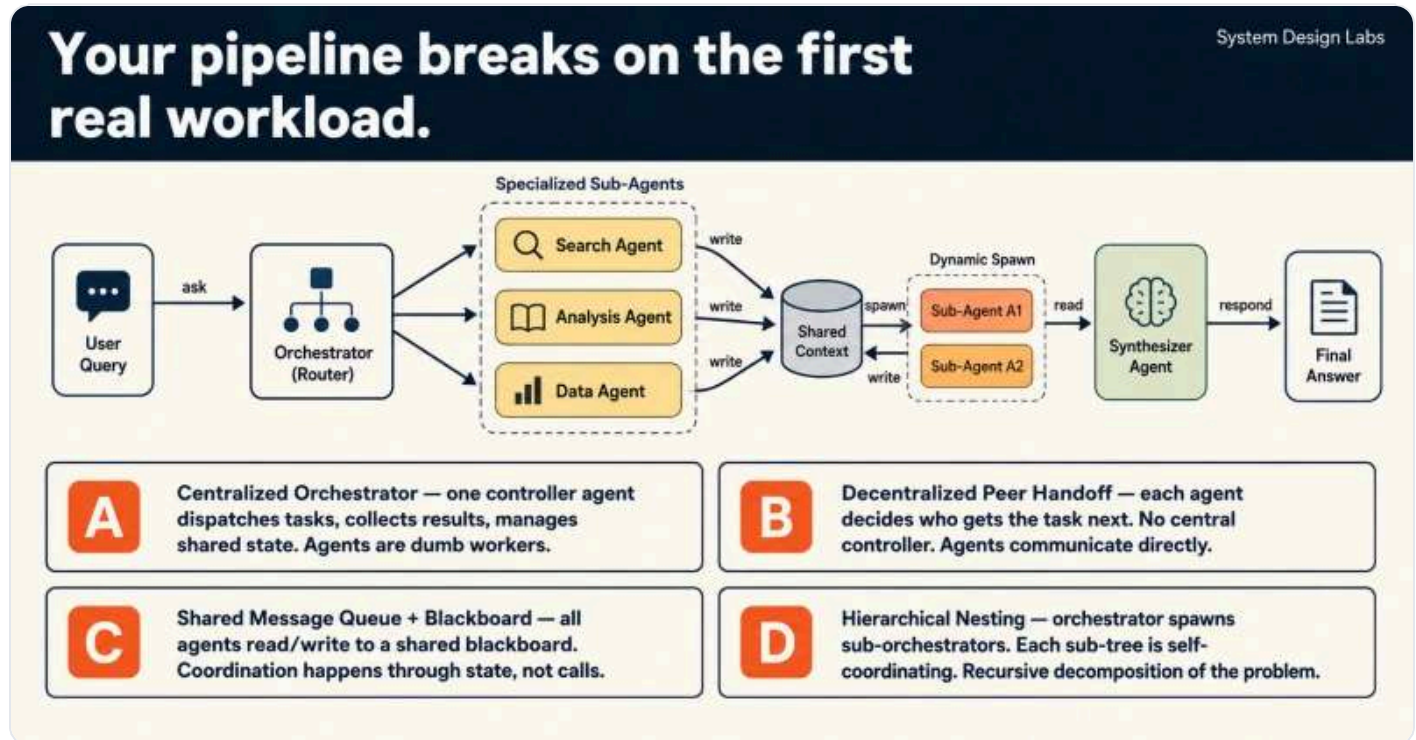
\$800 every Sunday whether you need it or not. At 6-week model updates, most rebuilds are wasted spend on data that hasn't meaningfully changed. It works. It's just expensive and blunt. Fine for a startup at 1M chunks. Painful at 50M.

Answer 4

D is monitoring, not a fix

Staleness detection is genuinely useful — but as a signal, not a strategy. The 1% sample tells you *that* you have drift, not *which* vectors are affected or *why*. You still need a rebuild strategy. D is a trigger, not an architecture. Combine it with C for a complete system.

Parallel Agents with Shared State



You built an agent. It works.

Now you need 5 of them running in parallel, sharing state, and handing off work to each other.

Your pipeline breaks on the first real workload.

Here's the setup:

You're building a research agent system. A user asks a complex question. You need to:

- Fan out to 3 specialized sub-agents simultaneously
- One agent might spawn 2 more based on what it finds
- They all write back to shared context
- A final agent synthesizes everything

Classic multi-agent orchestration. You have 4 options for how agents coordinate.

A) Centralized Orchestrator — one controller agent dispatches tasks, collects results, manages shared state. Agents are dumb workers.

B) Decentralized Peer Handoff — each agent decides who gets the task next. No central controller. Agents communicate directly.

C) Shared Message Queue + Blackboard — all agents read/write to a shared blackboard. Coordination happens through state, not calls.

D) Hierarchical Nesting — orchestrator spawns sub-orchestrators. Each sub-tree is self-coordinating. Recursive decomposition of the problem.

Answer 1

✓ CORRECT ANSWER

A**CENTRALIZED ORCHESTRATOR**

One controller agent owns the plan, dispatches tasks, collects results, manages state.

System Design Labs

**WHY A WINS****SINGLE SOURCE OF TRUTH**

The orchestrator owns the plan and state. Everyone else executes.

**EASY TO DEBUG & OBSERVE**

Failures are centralized. Logs, traces, and metrics are complete.

**RESILIENT BY DESIGN**

Sub-agents can fail or restart without corrupting global state.

**DETERMINISTIC & REPLAYABLE**

You can replay an orchestrator run from its state log end-to-end.

**PRODUCTION PROVEN**

LangGraph, CrewAI, and production systems default to this model.

**KEY TAKEAWAY**

Centralize control, distribute execution. The orchestrator plans, dispatches, collects, and synthesizes. Sub-agents stay simple, stateless, and reliable.

- ✓ Clear ownership
- ✓ Better reliability
- ✓ Easier debugging
- ✓ Scales with confidence

Answer: A — Centralized Orchestrator

Here's the full breakdown — and where each pattern actually belongs:

Why A wins (Centralized Orchestrator):

One agent owns the plan. It dispatches, collects, retries, and synthesizes. Every sub-agent is stateless and dumb on purpose — they receive a task, return a result, done.

This isn't a limitation. It's a feature.

When something fails, you know exactly where to look: the orchestrator. It holds the retry logic, the timeout budgets, the fallback paths. Sub-agents can die and restart without corrupting global state because the orchestrator is the only one maintaining it.

LangGraph, CrewAI, and every production-grade agentic framework defaults to this model for a reason. It's debuggable, observable, and deterministic. You can replay an orchestrator run from its state log. Try doing that with peer-to-peer handoffs.

Answer 2

Why B is the trap (Decentralized Peer Handoff):

This sounds elegant in theory. Agents self-organize, no bottleneck at the top.

The problem: who resolves conflicts? If Agent A hands off to Agent B, and Agent B decides the task actually belongs to Agent C, which also hands it back to Agent A — you have a cycle with no circuit breaker.

Peer handoff works in closed, well-defined pipelines (like a Git commit hook chain). It breaks the moment you have conditional routing, error recovery, or parallel fan-out.

Answer 3

Why C is partial credit (Shared Blackboard):

A shared blackboard is real and useful — but as a *data layer*, not a coordination mechanism.

Agents writing to and reading from shared state works for passing context. What it can't do cleanly is orchestrate *who runs next, when, and with what inputs*. You end up building an implicit orchestrator via polling — same thing, no visibility into it.

Use a blackboard for state sharing. Use an orchestrator for task routing. They're complementary, not alternatives.

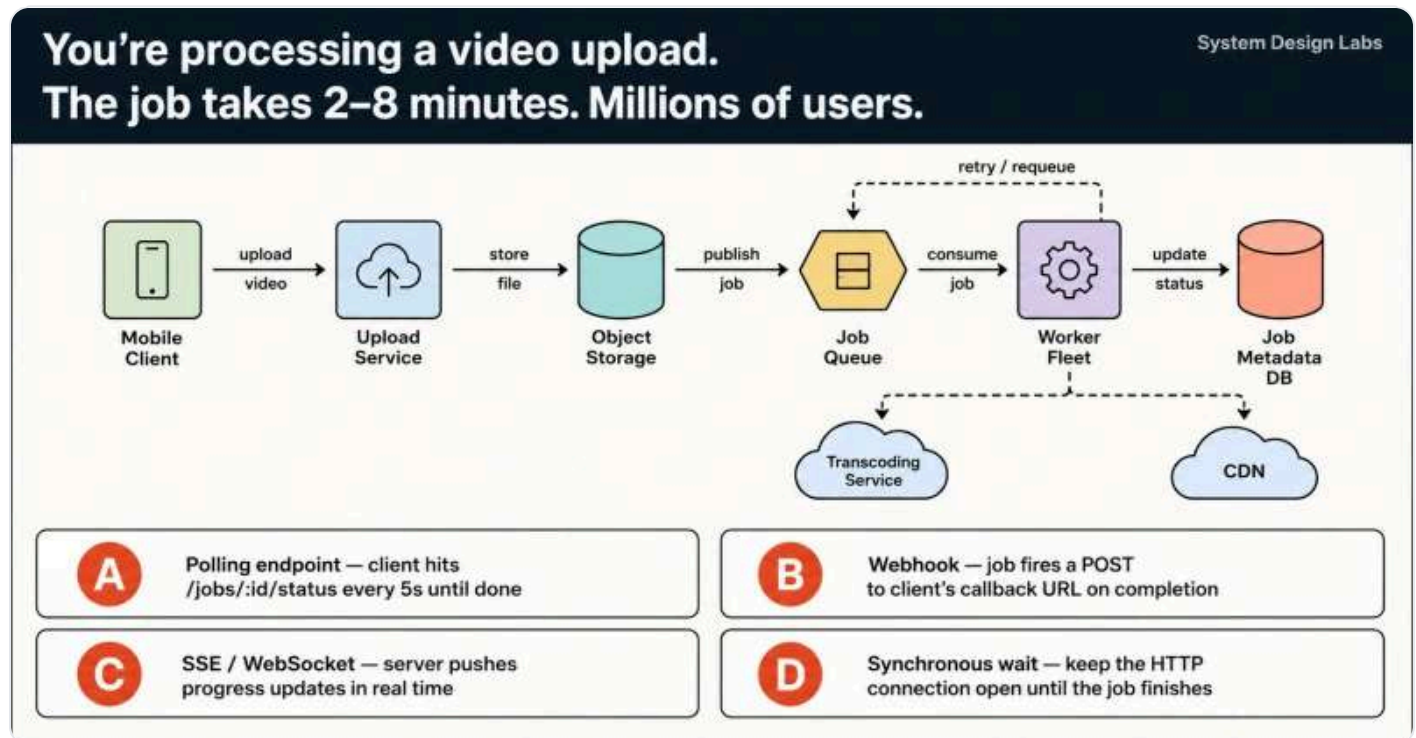
Answer 4

Why D fits a specific problem (Hierarchical Nesting):

Hierarchical orchestration is the right model when the problem itself is hierarchically decomposable — like a pipeline where "summarize AI papers" fans out to 10 sub-tasks, each of which fans out further.

The catch: recursion adds latency and debugging complexity at every layer. If your problem doesn't naturally decompose into clean sub-trees, you're adding indirection for no gain. Default to flat centralized orchestration first. Reach for hierarchical only when the problem demands it.

Tracking Long-Running Async Jobs



Your background job ran for 4 minutes and nobody knows if it finished.

That's not a job queue problem. That's a missing design problem.

Long-running jobs break every assumption you built for synchronous APIs. Your load balancer times out after 30s. Your mobile client doesn't know whether to retry. Your retry logic re-runs a job that already half-completed.

Here's the real scenario:

You're processing a video upload. The job takes 2–8 minutes. Millions of users.

What do you expose to the client?

- A) Polling endpoint — client hits `/jobs/:id/status` every 5s until done
- B) Webhook — job fires a POST to client's callback URL on completion
- C) SSE / WebSocket — server pushes progress updates in real time
- D) Synchronous wait — keep the HTTP connection open until the job finishes

One scales to millions without coupling your infrastructure to client uptime.

The others have hard production failure modes most teams don't discover until 3 AM.

The deeper problem isn't transport — it's these 4 things nobody gets right the first time:

→ **Idempotency.** Every job must be safe to re-run. If your retry logic can double-charge, double-send, or double-process — you don't have retries, you have bugs waiting.

→ **Progress granularity.** "0% → 100%" is useless for a 6-minute job. You need intermediate states: queued, processing, transcoding, uploading, complete. Clients need something to show users.

→ **Timeout vs failure.** A job that stops responding isn't the same as a job that failed. Dead workers, OOM kills, spot instance evictions — your queue needs a heartbeat or a visibility timeout, not just a try/catch.

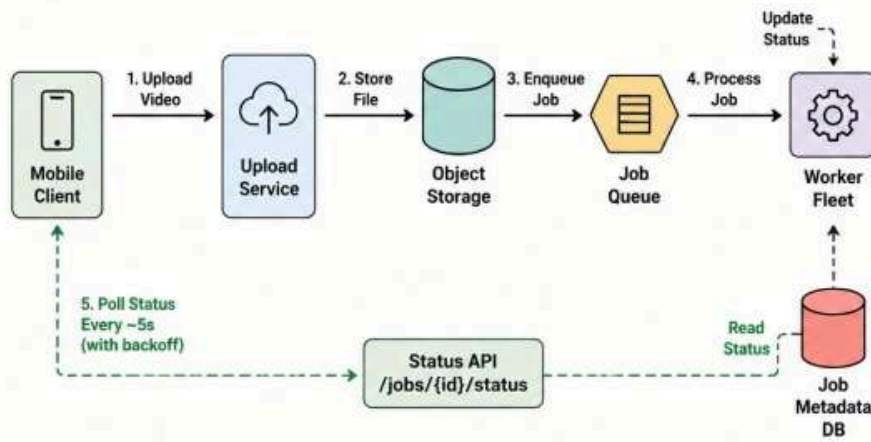
→ **Deduplication.** The client will retry. Your queue will redeliver. You need a dedup key scoped to the original request — not the job run.

Answer 1

✓ CORRECT ANSWER

A) POLLING — CORRECT FOR MOST CASES

System Design Labs

**Why Polling Wins**

Client-controlled, stateless, scales independently. The server doesn't care if the client disconnects, retries, or crashes — the job runs and the status endpoint just answers queries.

5s polling intervals on a job that takes 2–8 minutes is trivially cheap.

LinkedIn, YouTube, and S3 multipart uploads all use polling for async job status.

THE KEY RULES**Stable & Idempotent Job IDs**

Make job IDs stable and idempotent so retries return the same job.

**TTL on Job Records**

Set a TTL on job records so you don't accumulate state forever.

**Exponential Backoff**

Use exponential backoff instead of fixed intervals to reduce load.

**Stateless Status Endpoint**

The status endpoint should be fast, cacheable, and read-only.

A) Polling — Correct for most cases

Client-controlled, stateless, scales independently. The server doesn't care if the client disconnects, retries, or crashes — the job runs and the status endpoint just answers queries. 5s polling intervals on a job that takes 2–8 minutes is trivially cheap. The key rules: make job IDs stable and idempotent, set a TTL on job records so you don't accumulate state forever, and use exponential backoff not fixed intervals. LinkedIn, YouTube, and S3 multipart uploads all use polling for async job status.

Answer 2**B) Webhook — Right idea, wrong default**

Webhooks are great for server-to-server flows where the receiver has a stable HTTPS endpoint. They fall apart for mobile clients (no public URL), in environments with NAT/firewall, and when the receiver is down at delivery time. You'd need a retry queue, delivery guarantees, and signature verification just to make it reliable. Webhooks work well as a supplementary delivery mechanism for platform integrations — not as the primary client notification path for end users.

Answer 3

C) SSE / WebSocket — Expensive for this use case

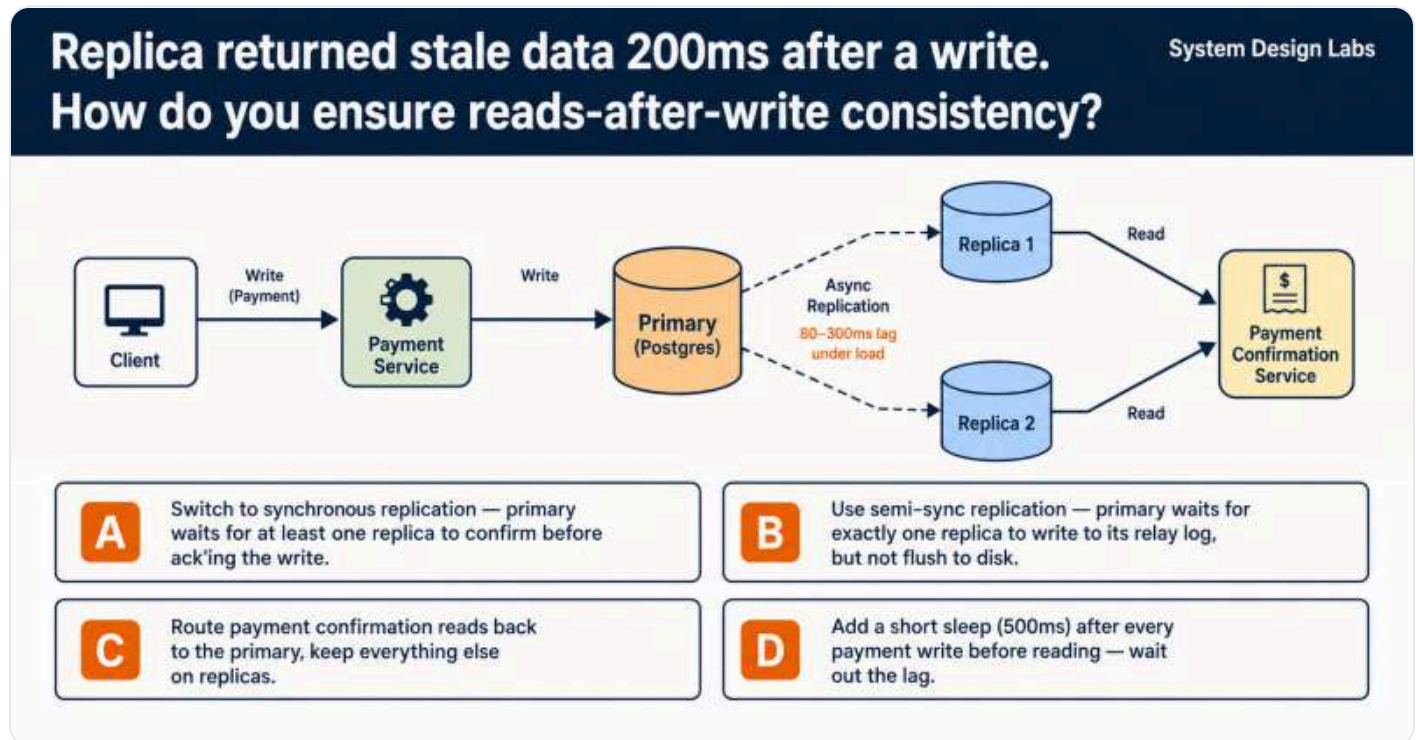
Real-time push is fantastic for chat, live dashboards, and collaborative editing. For a job that takes 2–8 minutes and is triggered once? You're holding an open connection, burning a file descriptor, and adding connection-management complexity for maybe 3 meaningful state changes. SSE makes sense when you genuinely need sub-second updates or continuous streaming. For async job progress, polling is cheaper and simpler.

Answer 4

D) Synchronous wait — Production antipattern

Keeping the HTTP connection open for 2–8 minutes kills your load balancer (most timeout at 30–60s), ties up a server thread or process, and gives the client no recovery path if the connection drops mid-job. This is how systems get "stuck" — jobs complete but the client never hears about it because the connection dropped at minute 3. Never block on long-running work. Always return immediately with a job ID.

Consistency for Payment-Confirmation Reads



Your write traffic just hit 50k writes/sec. Reads are 10x that.

You split your Postgres into one primary + two replicas. Reads go to replicas. Now you're shipping a payment confirmation flow — and a replica returned stale data 200ms after a write. The user saw "payment pending" when it already succeeded.

Here's the setup:

- Primary handles all writes
- Two read replicas, async replication
- Payment confirmation reads from a replica
- Replication lag: 80–300ms under load

You need reads-after-write consistency on payment flows without tanking write throughput. What do you do?

- A** Switch to synchronous replication — primary waits for at least one replica to confirm before ack'ing the write.

B Use semi-sync replication — primary waits for exactly one replica to write to its relay log, but not flush to disk.

C Route payment confirmation reads back to the primary, keep everything else on replicas.

D Add a short sleep (500ms) after every payment write before reading — wait out the lag.

Answer 1

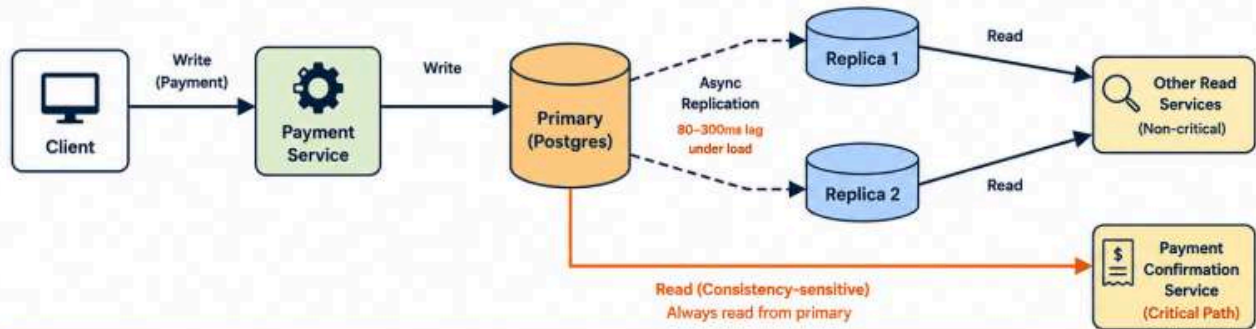
✓ CORRECT ANSWER

Why C wins (primary reads for critical paths):

System Design Labs



Payment confirmations are a tiny slice of your read traffic. You're not routing ALL reads to primary — just the reads where stale data causes **visible user harm**.



This pattern is called **read-your-writes consistency**. Stripe, Shopify, every serious fintech does this. You add a flag or middleware: "if this is a consistency-sensitive flow, read from primary." Everything else keeps hitting replicas.



Minimal impact

Payment confirmations are maybe 0.1% of your reads. Primary read traffic increases slightly.



Surgical precision

Only critical flows read from primary. Everything else stays on replicas.



No new failure modes

No topology change. No new components. No added complexity.



Maximum consistency where it matters

Users never see "payment pending" after a successful payment again.



Bottom line: Read from primary for consistency-sensitive flows. Fast, safe, and battle-tested.

Why C wins (primary reads for critical paths):

Payment confirmations are a tiny slice of your read traffic. You're not routing ALL reads to primary — just the reads where stale data causes visible user harm.

This pattern is called **read-your-writes consistency**. Stripe, Shopify, every serious fintech does this. You add a flag or middleware: "if this is a consistency-sensitive flow, read from primary." Everything else keeps hitting replicas.

Primary read traffic increases slightly — but a "just confirmed payment" flow is maybe 0.1% of your reads. Negligible. No topology change, no new failure modes, surgical precision.

Answer 2

Why A is tempting but the wrong trade (synchronous replication):

Postgres `synchronous_commit = on` — primary waits for a replica to confirm before acking the write. Zero lag. Reads always consistent.

The cost: every write now takes `primary_write_time` + network roundtrip to replica. 1–5ms normally. Under load with a flaky replica? Your write latency is hostage to your slowest replica.

If that sync replica goes down → you either halt writes or fall back to async and break your guarantee. That choice gets made at 3am under pressure.

Sync replication is a serious architectural commitment for the whole DB. Routing payment reads to primary solves the same problem, surgically.

Answer 3

Why B is the sneaky wrong answer (semi-sync replication):

Semi-sync (`rpl_semi_sync` in MySQL) — primary waits for one replica to write to its relay log. Sounds like a middle ground. It's not.

The replica acknowledged receipt of the bytes — not that the data is applied and readable. The relay log is the *incoming queue*, not the applied state. You can still read stale data immediately after a write.

Semi-sync protects you from **data loss on primary failure**. It does NOT protect you from replication lag on reads. Different problem entirely.

Senior engineers conflate "replica has the data" with "replica has applied the data." They're not the same thing.

Answer 4

Why D will destroy you at scale (sleep-based lag compensation):

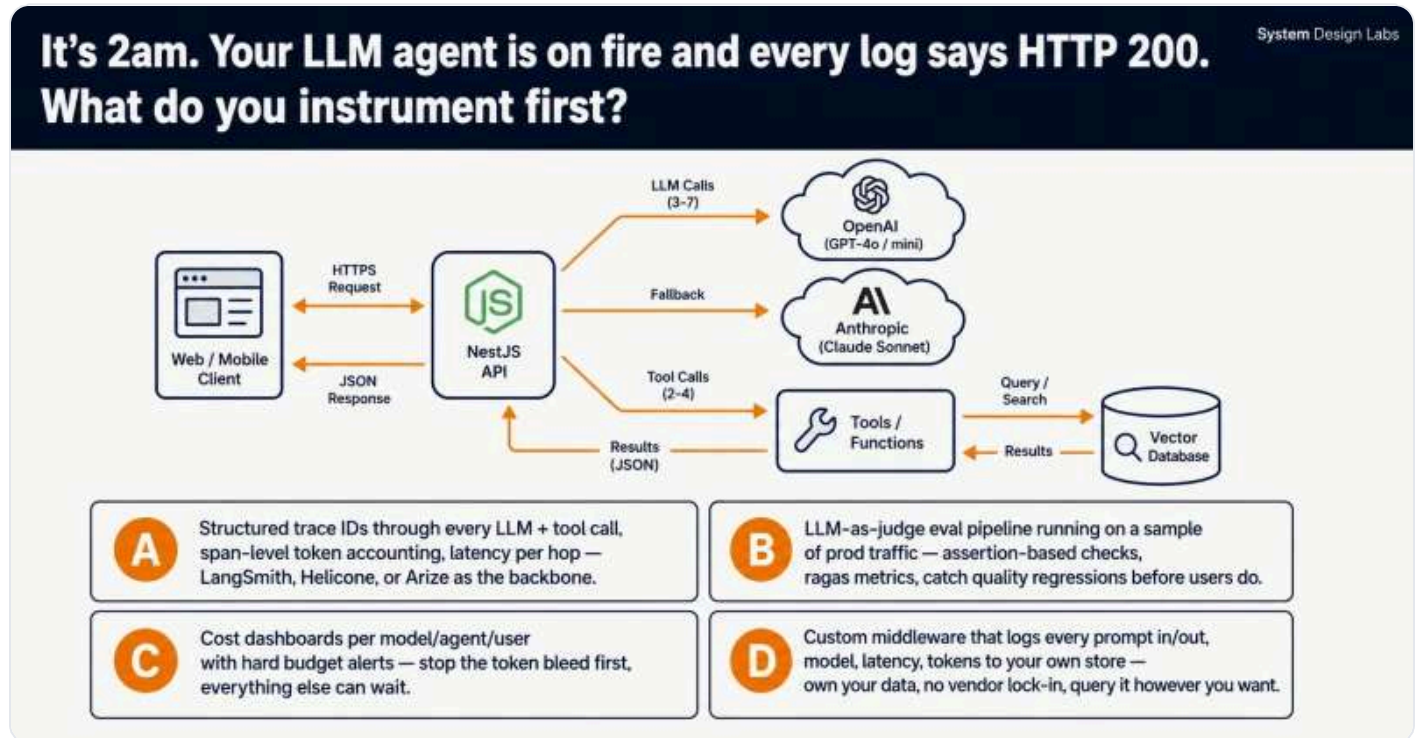
Sounds pragmatic. Works for about 2 weeks.

Replication lag isn't a fixed constant. It's 50ms now, 300ms during a write spike, 2s during a vacuum. Hardcode a sleep and you've built a system that's either too slow (`sleep > lag`) or still broken (`lag > sleep`).

Every payment confirmation path is now artificially 500ms slower. And when the sleep isn't enough during a spike — you're back to the original bug, with worse latency.

Sleep-based consistency is always a smell.

Observability for LLM Agent Pipelines



It's 2am. Your LLM agent is on fire and every log says HTTP 200.

You shipped an AI-powered support agent 6 weeks ago. It calls tools, chains 4-5 prompts per request, hits vector search, calls back into your API. Users are complaining. Answers are wrong. Token bills are 3x what they were last week.

You open your logs. Everything is green. You open your traces. You see one span called POST /chat that took 14 seconds. That's it. That's your entire visibility.

Here's the setup:

NestJS API → OpenAI (GPT-4o + GPT-4o-mini) + Anthropic (Claude Sonnet fallback)

~8,000 agent runs/day, each chains 3-7 LLM calls + 2-4 tool calls

Token spend jumped from \$180/day to \$540/day this week, no idea which flow caused it

Eval suite passes 94% in CI, prod users are hitting hallucinations you can't reproduce

On-call engineer literally cannot answer "why is this run slow / expensive / wrong?"

You have one sprint to fix the visibility problem before the next incident. What do you instrument first?

- A)** Structured trace IDs through every LLM + tool call, span-level token accounting, latency per hop — LangSmith, Helicone, or Arize as the backbone.
- B)** LLM-as-judge eval pipeline running on a sample of prod traffic — assertion-based checks, ragas metrics, catch quality regressions before users do.
- C)** Cost dashboards per model/agent/user with hard budget alerts — stop the token bleed first, everything else can wait.
- D)** Custom middleware that logs every prompt in/out, model, latency, tokens to your own store — own your data, no vendor lock-in, query it however you want.

Answer 1

✓ CORRECT ANSWER



Why A wins:

You can't fix what you can't see.

System Design Labs



Slow?

Which of the 7 LLM calls was slow.



Expensive?

Which model on which step burned the tokens.



Wrong?

Which retrieval returned garbage that poisoned the next prompt.

→ None of those questions have answers without a trace. LLM traces aren't HTTP traces — you need span-level attributes: model name, prompt tokens, completion tokens, cost, tool name, retrieval doc IDs, and the actual prompt/response payload. That's what LangSmith, Helicone, Arize, and Langfuse give you out of the box.



Why A wins:

Every problem in the scenario is the same root cause — you can't see the shape of a single agent run.

Slow? → Which of the 7 LLM calls was slow.

Expensive? → Which model on which step burned the tokens.

Wrong? → Which retrieval returned garbage that poisoned the next prompt.

None of those questions have answers without a trace. And LLM traces aren't HTTP traces — you need span-level attributes: model name, prompt tokens, completion tokens, cost, tool name, retrieval doc IDs, and the actual prompt/response payload. That's what LangSmith, Helicone, Arize, and Langfuse give you out of the box.

Once you have it: the 14-second span opens into 7 sub-spans. You see the one that took 9s was a Claude fallback triggered by a rate limit. The \$540/day spike is one user hammering an agent that recursively calls itself. The hallucinations correlate with retrievals that returned 0 chunks.

Answer 2

Why B is the senior engineer trap:

Evals answer a different question: *is quality trending up or down over time?* They're not a debugging tool. When prod is on fire, "78% correctness on last 200 samples" tells you nothing about the request that broke. Worse — evals without tracing are unfalsifiable. When the score drops, you have no way to drill into failing samples to see *why*. Evals are step 2. They ride on top of traces.

Answer 3

Why C treats the symptom:

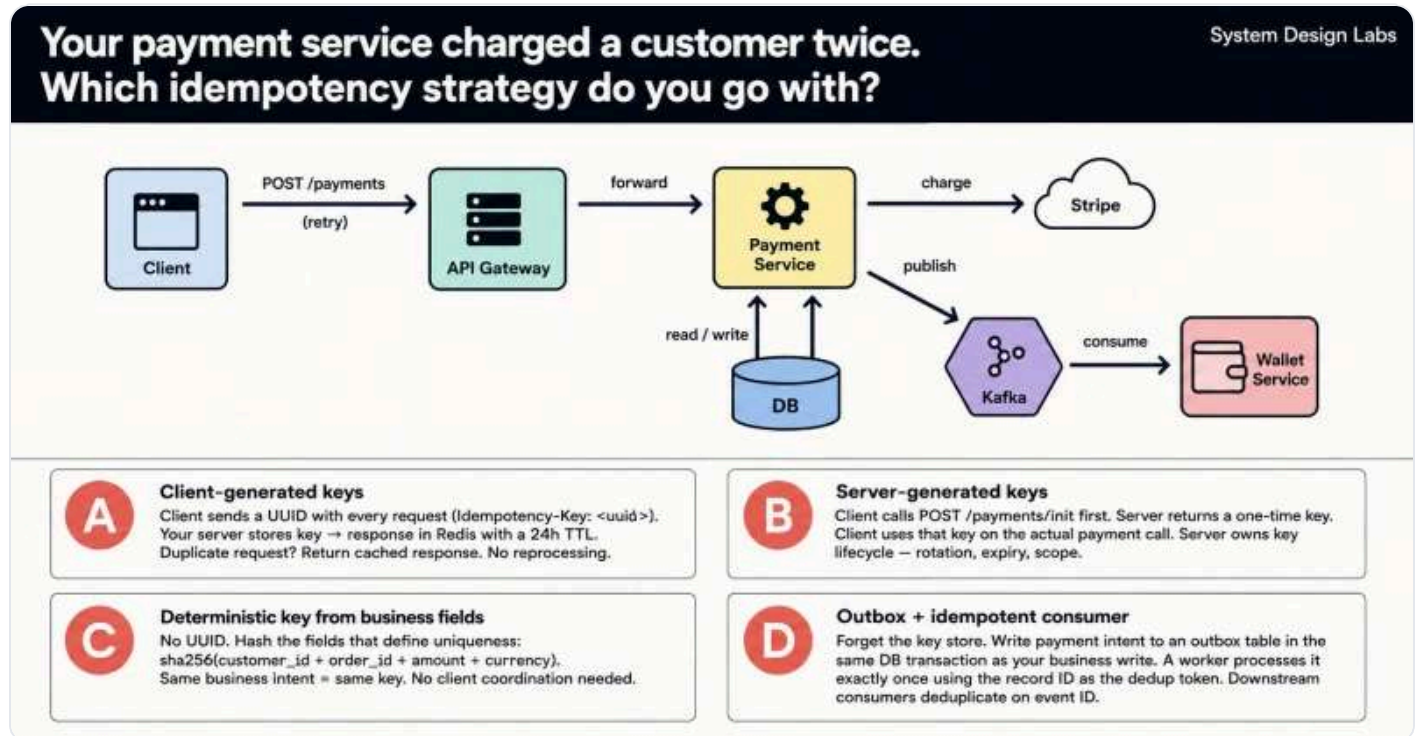
"Spend is up 3x" is the same info you already had from the bill. Aggregate cost without span-level attribution just tells you the room is on fire without telling you which wall. You can't build per-agent, per-user cost attribution without instrumenting spans first. Budget alerts: yes, ship them. As your primary investment: you're solving the CFO's problem, not the on-call engineer's.

Answer 4

Why D isn't actually pragmatic:

"Own your data, no vendor lock-in" is the sentence that has cost more teams more months than almost any other. You'd need: schema for nested agent runs, trace tree visualizer, correlation IDs across async tool calls, sampling logic, retention, PII redaction, cost rollups. You just rebuilt Langfuse badly, in your spare time, while the incident is still open. Every serious LLM observability tool emits OpenTelemetry-compatible spans — switch vendors in a week if needed. Ship the vendor now, revisit self-hosting when you've earned that problem.

Designing Idempotency Keys for Payments



Your payment service charged a customer twice.

Same request. Different response code. Client retried. You had no dedup logic.

That's not a race condition. That's a missing design decision.

And the decision isn't whether to use idempotency keys. It's how you generate them.

Here's the scenario:

Your API processes payments. Clients can and will retry on network failures. Your downstream calls (Stripe, Kafka, wallet credit) cannot be undone once triggered. A 500 or timeout on your side looks identical to a "try again" from the client's perspective.

Which idempotency strategy do you go with?

A) Client-generated keys

Client sends a UUID with every request (Idempotency-Key:). Your server stores key → response in Redis with a 24h TTL. Duplicate request? Return cached response. No reprocessing.

B) Server-generated keys

Client calls POST /payments/init first. Server returns a one-time key. Client uses that key on the actual payment call. Server owns key lifecycle — rotation, expiry, scope.

C) Deterministic key from business fields

No UUID. Hash the fields that define uniqueness: sha256(customer_id + order_id + amount + currency). Same business intent = same key. No client coordination needed.

D) Outbox + idempotent consumer

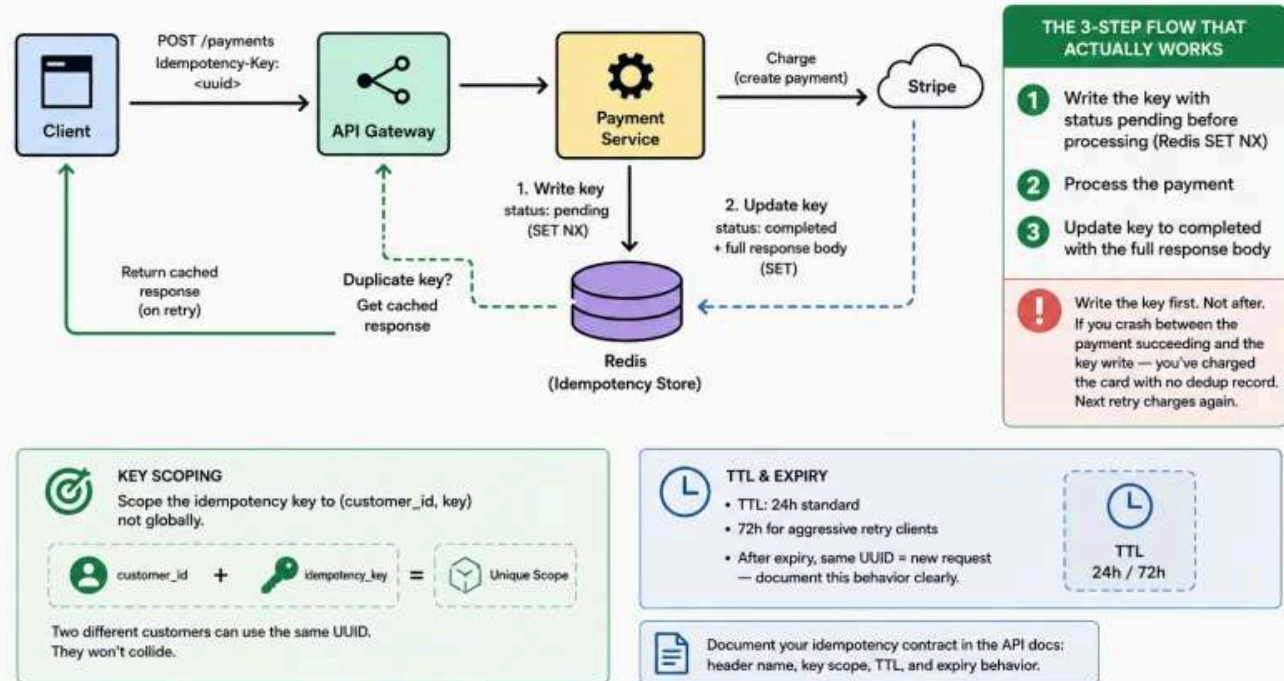
Answer 1

✓ CORRECT ANSWER

A) Client-generated keys — ✓ GOLD STANDARD FOR EXTERNAL APIs

System Design Labs

This is how Stripe does it. How Braintree does it.

Reason: the client owns the intent, so the client owns the key.

A) Client-generated keys — ✓ GOLD STANDARD FOR EXTERNAL APIs

This is how Stripe does it. How Braintree does it. Reason: the client owns the intent, so the client owns the key.

The flow that actually works in production:

1. Write the key with status pending before processing (Redis SET NX)
2. Process the payment
3. Update key to completed with the full response body

Write the key first. Not after. If you crash between the payment succeeding and the key write — you've charged the card with no dedup record. Next retry charges again.

Key scoping matters: scope to (customer_id, key) not globally. TTL: 24h standard, 72h for aggressive retry clients. After expiry, same UUID = new request — document this.

Answer 2

B) Server-generated keys — RIGHT WHEN YOU DON'T TRUST CLIENTS

Two-step feels like overhead. It's actually a feature.

The init call gives you: pre-validation before money moves, audit trail at key issuance, protection against clients sending garbage UUIDs (v1 timestamps, sequential IDs, non-unique values), and full key lifecycle control server-side.

Where it shines: internal service-to-service APIs, regulated environments, anywhere key collision is a compliance issue.

Where it hurts: mobile clients on flaky networks now have two calls that can fail independently. Retry complexity goes up, not down.

Answer 3

C) Deterministic key — THE SENIOR ENGINEER TRAP

Looks clean. Falls apart on legitimate retries.

Same customer, same amount, same item — but a fresh order after a refund. Your hash matches the completed request. You return cached success for a brand new charge that should go through.

Second failure mode: field serialization drift across SDK versions. SDK v1 orders fields differently than v2. Different hash. Your dedup window disappears silently.

Use deterministic keys for internal job dedup — cron tasks, batch pipelines, where business intent is truly unique. Never for customer-facing payment flows.

Answer 4

D) Outbox + idempotent consumer —  STRONGEST FOR DISTRIBUTED SYSTEMS

The only option that makes the payment record and downstream event atomic.

The failure in A/B/C: Redis key write, DB write, and Stripe call are three separate operations. Crash between any two = inconsistent state.

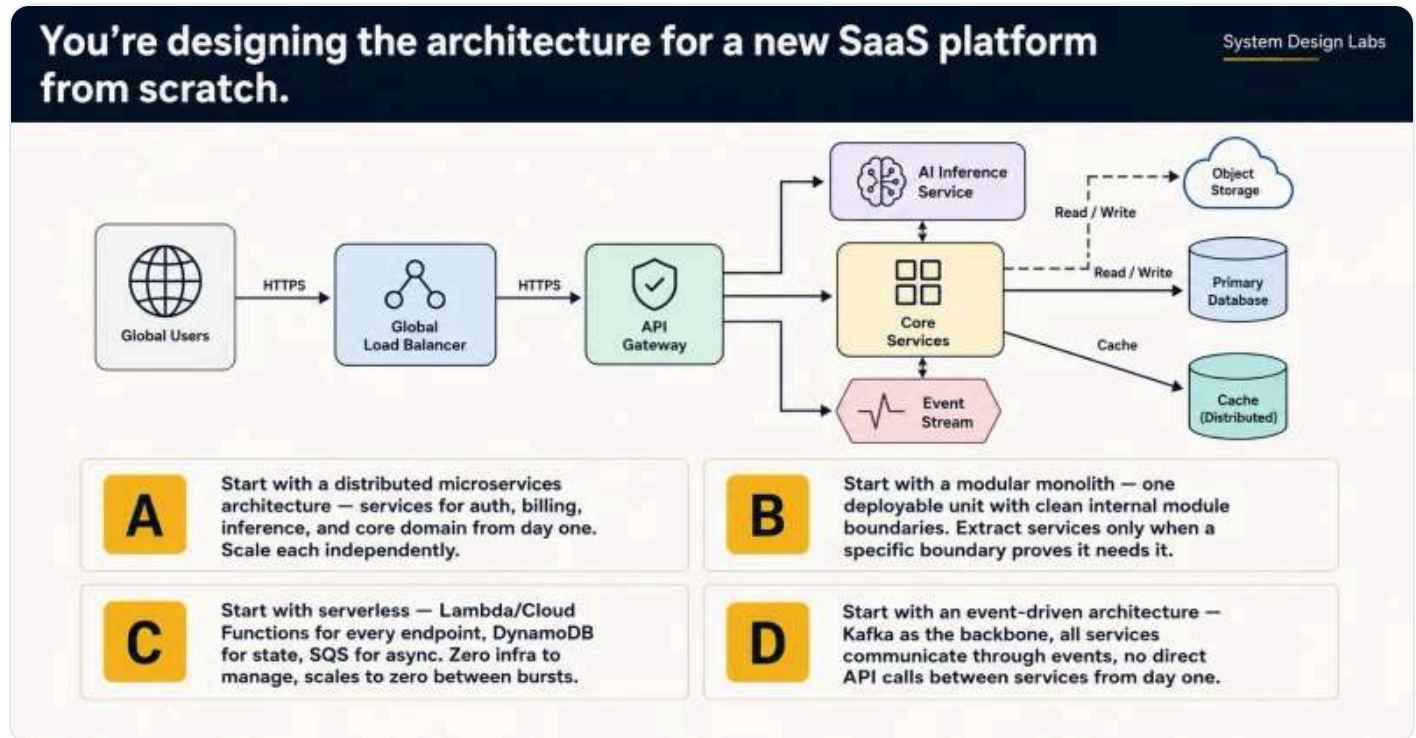
The outbox fixes it:

1. Write payment record + outbox event in one DB transaction
2. Worker processes downstream (Stripe, Kafka) using the row ID as dedup token
3. Downstream consumers deduplicate on event ID

No separate key store. No Redis dependency. Exactly-once enforced at the DB layer.

The cost: outbox table + reliable worker + idempotent consumers. Real infrastructure. Overkill at 100 payments/day. The only defensible architecture at 10K TPS with regulatory requirements.

Architecting a SaaS Platform from Scratch



You're designing the architecture for a new SaaS platform from scratch.

It needs to handle 50,000 requests per second at launch. Multi-tenant. Real-time data. Global users. AI inference built in.

Your CTO gives you 3 months and a blank slate.

You've been thinking about this for a week. You finally sit down to draw the first box.

Here's what engineers actually debate in that room:

A) Start with a distributed microservices architecture — services for auth, billing, inference, and core domain from day one. Scale each independently.

B) Start with a modular monolith — one deployable unit with clean internal module boundaries. Extract services only when a specific boundary proves it needs it.

C) Start with serverless — Lambda/Cloud Functions for every endpoint, DynamoDB for state, SQS for async. Zero infra to manage, scales to zero between bursts.

D) Start with an event-driven architecture — Kafka as the backbone, all services communicate through events, no direct API calls between services from day one.

Answer 1

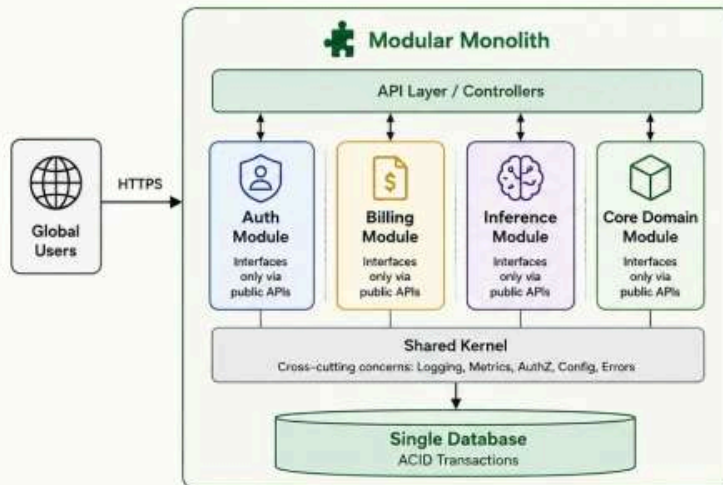
✓ CORRECT ANSWER



B) Modular Monolith — right

System Design Labs

The constraint isn't scale. It's the 3-month runway plus uncertainty about domain boundaries.



A modular monolith gives you:



Fast iteration — no network serialization overhead, no cross-service API contracts to version



Real module boundaries enforced in code (auth, billing, inference, core domain — no shared DB tables, no cross-module direct calls)

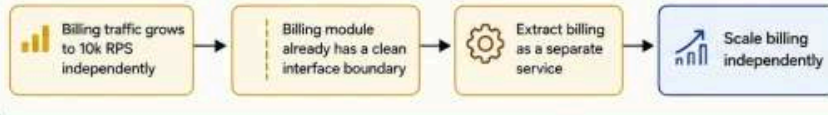


Single deploy, single observability surface, single transaction boundary



The option to extract services later — when you know exactly which boundary is real

Extraction Path Example: Billing



AI inference piece: inference latency is dominated by model I/O (50–500ms), not your application code. That's a reason to put it behind a clean internal interface — not a reason to make it a separate service on day one.



Modular now. Distributed later. Boundaries first, extraction second. Refactor, not rewrite.

B) Modular Monolith — right

The constraint isn't scale. It's the 3-month runway plus uncertainty about domain boundaries.

A modular monolith gives you:

- Fast iteration — no network serialization overhead, no cross-service API contracts to version
- Real module boundaries enforced in code (auth, billing, inference, core domain — no shared DB tables, no cross-module direct calls)
- Single deploy, single observability surface, single transaction boundary
- The option to extract services later — when you know exactly which boundary is real

When billing hits 10k RPS independently and needs separate scaling, you extract it. The boundary is already clean. The migration is a refactor, not a rewrite.

The AI inference piece specifically: inference latency is dominated by model I/O (50–500ms), not your application code. That's a reason to put it behind a clean internal interface — not a reason to make it a separate service on day one.

Answer 2

A) Microservices from day one — wrong

50,000 RPS sounds like you need microservices. It doesn't.

That's a solved load balancing problem, not a distribution problem.

What microservices from day one actually gives you:

- 6+ independently deployable services before you've found your real domain boundaries
- Distributed transactions across auth, billing, inference, and core domain
- Network hops on every request that used to be in-process calls
- 3 months of infra work instead of product

Amazon's "two-pizza teams" rule was for *existing* services with known boundaries. Not for new domains where you're still discovering what the boundaries are.

The boundary is the hard part. You don't know it yet.

Answer 3

C) Serverless — wrong (senior engineer trap)

"Scales to zero" sounds great until you price out 50k RPS sustained on Lambda.

The math:

- $50,000 \text{ req/s} \times 100\text{ms avg} = 5,000 \text{ concurrent Lambda executions}$
- At that scale you're paying more than a reserved EC2 cluster
- Cold starts are real and you promised "real-time data"
- AI inference in Lambda = cold start + memory limit + 15-min hard timeout = pain

Serverless wins for bursty, unpredictable event workloads. SaaS with consistent load and real-time requirements is not that workload.

Answer 4

D) Event-driven from day one — wrong

This one trips up engineers who've read about Kafka at Netflix and Uber.

Event-driven is the right answer — *for* Netflix and Uber, after they've already mapped their domain.

Starting with Kafka before you have a working system means:

- Every feature needs an event schema, a producer, and a consumer before it works
- Debugging "why didn't this event get processed" during week 2 of development
- Eventual consistency as the default when you don't yet know which operations need strong consistency
- Kafka operational overhead before you've validated the product

Event-driven is an optimization for a known system. Not a foundation for an unknown one.